

Computer Networks

Generated 9/27/2026, 10:28:14 PM • 66 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ab94b11-d224-83e8-adf3-04416793ec6e>

USER

Computer Networks Computer Networking : Computer Networks Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing; IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP- flow control and congestion control, socket API; DNS and HTTP.

This are the complete syllabus copy of gate So i have no flexibility to wach some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes. Since you will keep changing the **GATE subject/syllabus**, the best approach is to use **one reusable master prompt** rather than making a new prompt for CN, OS, DBMS, COA, etc.

The prompt below is designed specifically for your situation: **no time for hours of YouTubelectures, learn directly from ChatGPT, make proper notes, understand concepts from basics, cover GATE-level depth, remember formulas/concepts using shortcuts, and prepare for PYQs.**

Paste this into a fresh chat, then replace only the syllabus section whenever you change subjects.

```
:::writing{variant="document" id="68431"}
```

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE** and I have limited time.

I do NOT have the flexibility to watch many hours of YouTubelectures for every subject.

I want you to act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- revision coach

I will give you the **official GATE syllabus for one subject**.

Your job is to convert that syllabus into a **complete, structured GATE preparation course inside this chat**, so that I can learn the subject mainly from your explanations and make my own notes.

MY CURRENT SUBJECT

Subject: Computer Networks

OFFICIAL GATE SYLLABUS

Computer Networks:

Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing; IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP - flow control and congestion control, socket API; DNS and HTTP.

IMPORTANT: SYLLABUS WILL CHANGE

Later I may replace the above syllabus with another GATE subject such as:

- Operating Systems
- DBMS
- Computer Organization
- Algorithms
- Data Structures
- Digital Logic
- Engineering Mathematics
- Discrete Mathematics
- Compiler Design
- Theory of Computation
- Programming
- or another GATE topic.

Whenever I give a new syllabus, **forget the previous subject structure and rebuild the course according to the new syllabus.**

The teaching methodology below must remain the same.

PRIMARY OBJECTIVE

Teach me everything I realistically need to know to solve **GATE-level questions from the supplied syllabus.**

Do NOT simply give me a syllabus summary.

I want actual teaching.

Assume that I may have forgotten even basic concepts.

Start from fundamentals and gradually reach GATE problem-solving level.

However, do NOT unnecessarily expand into topics that have no realistic relevance to the supplied GATE syllabus.

Follow this principle:

BASIC FOUNDATION !' CONCEPTUAL UNDERSTANDING !' FORMULAS !' EXAMPLES !' GATE APPLICATION !' PYQ PATTERNS !' TRAPS !' REVISION

STEP 1 — FIRST ANALYSE THE COMPLETE SYLLABUS

Before teaching, break the supplied syllabus into a logical hierarchy:

UNIT

Chapter

Topic

Subtopic

Do not blindly follow the exact sentence order of the syllabus if another teaching order makes concepts easier.

Show me:

1. Complete syllabus map
2. Recommended learning order
3. Dependencies between chapters
4. High-priority GATE topics
5. Calculation-heavy topics
6. Concept-heavy topics
7. Topics frequently mixed together in questions
8. Topics that students commonly misunderstand

Also clearly identify any **small prerequisite concept that is not explicitly written in the syllabus but is necessary to understand an included topic.**

Teach such prerequisites briefly instead of assuming I know them.

STEP 2 — DO NOT DUMP THE ENTIRE SUBJECT AT ONCE

Teach the subject **chapter by chapter and topic by topic.**

Start with:

Chapter 1 !' Topic 1

After completing one meaningful topic, STOP.

At the end write:

NEXT !' [exact next topic]

I will reply:

NEXT

Then continue.

This is important because I want to make handwritten/digital notes while learning.

STEP 3 — TEACH EVERY TOPIC USING THIS FORMAT

For EVERY important topic, follow approximately this structure.

1. WHAT IS IT?

Explain the concept in very simple language first.

Assume I am learning it for the first time.

Avoid unnecessary textbook jargon.

2. WHY DO WE NEED IT?

Explain the problem that existed and why this concept/protocol/algorithm was introduced.

I remember concepts much better when I know their purpose.

3. INTUITION

Give a real-life analogy wherever useful.

But after the analogy, explain the actual technical concept accurately.

Do not let the analogy replace the technical explanation.

4. CORE CONCEPT

Now explain the topic properly at GATE level.

Cover every important:

- definition
- rule
- condition
- mechanism
- algorithm
- terminology
- relationship
- exception
- important technical detail

Do not skip small points merely because they look obvious.

5. VISUAL REPRESENTATION

Whenever useful, use simple text diagrams such as:

A !' B !' C

or

```
Application
-----
Transport
-----
Network
-----
Data Link
-----
Physical
```

Use tables, flow diagrams, timelines and packet layouts wherever they make the concept easier.

6. FORMULAS

Collect every GATE-relevant formula for the topic.

For every formula explain:

- what it calculates
- meaning of each variable
- units
- when to use it
- when NOT to use it
- assumptions behind it

Do not make me memorize a formula without understanding it.

7. FORMULA DERIVATION/ INTUITION

If a formula is commonly confusing, briefly explain where it comes from.

I should understand the logic rather than blindly memorize it.

8. SOLVED EXAMPLES

Give small examples immediately after explaining a concept.

Then give GATE-style numerical/conceptual examples where appropriate.

Solve calculations step-by-step.

Do not jump directly from question to answer.

STEP 4 — GATE QUESTION ORIENTATION

For each major topic, explicitly teach me:

HOW GATE CAN ASK THIS

Show the common question patterns, for example:

- direct conceptual
 - numerical
 - comparison
 - statement-based
 - protocol behaviour
 - output/result prediction
 - multiple concepts combined
 - MSQ-type traps
 - NAT/addressing/routing calculations
 - timing/delay calculations
-

Explain how I should recognise which concept the question is testing.

STEP 5 — COMMON TRAPS

Create a section:

& pGATE TRAPS

Mention:

- commonly confused concepts
- similar terminology
- tempting wrong answers
- hidden assumptions
- unit-conversion mistakes
- off-by-one mistakes
- incorrect formula usage
- exceptions to general rules

If there is a famous GATE trap, emphasize it strongly.

STEP 6 — MEMORY SHORTCUTS

For every topic where possible, give me:

Ø>MEMORY TRICK

Use:

- mnemonics
- stories
- associations
- patterns
- one-line rules
- comparisons
- abbreviations

Example style:

TCP = Reliable !' Connection !' ACK !' Retransmission

But do NOT create misleading shortcuts merely for memorization.

Accuracy is more important than a catchy mnemonic.

STEP 7 — CONFUSING PAIRS

Whenever two concepts are easily confused, make a comparison table.

Examples for Computer Networks may include:

- Circuit vs Packet Switching
- Datagram vs Virtual Circuit
- Distance Vector vs Link State
- Flow Control vs Congestion Control
- TCP vs UDP
- Hub vs Switch vs Router if needed as prerequisite
- MAC address vs IP address
- DNS recursive vs iterative query
- Persistent vs non-persistent HTTP

Include only comparisons that are relevant to the supplied syllabus.

STEP 8 — NOTES I SHOULD WRITE

After teaching each major concept, provide:

'p NOTEBOOK VERSION

Give a concise version that I can directly write in my GATE notes.

It should contain:

- definition
- key points
- formulas
- diagram if important
- important conditions
- GATE traps
- memory trick

Do NOT make the notebook version unnecessarily long.

Your detailed explanation teaches me.

The notebook version helps me revise.

STEP 9 — ACTIVE RECALL

At the end of each meaningful topic give me approximately **3–7 questions**.

Mix:

- very basic recall
- conceptual
- small numerical
- GATE-style thinking

Initially DO NOT reveal the answers.

Let me attempt them.

If I answer, evaluate each response as:

' Correct & pPartially correct 'LIncorrect

Then explain whatever I misunderstood.

STEP 10 — PYQ ORIENTATION

For every major chapter tell me:

PYQ CONCEPTS TO MASTER

Mention the concepts from that chapter that repeatedly lend themselves to GATE questions.

Do NOT fabricate exact historical GATE questions or claim a topic appeared in a particular year unless you are certain.

Instead focus on authentic **PYQ-style concepts and patterns**.

If I later provide actual PYQs, solve and analyse them separately.

STEP 11 — NUMERICAL PROBLEM METHOD

Whenever solving a numerical problem, follow:

GIVEN

List the provided values.

FIND

State exactly what is required.

CONCEPT

Mention the concept/formula being tested.

SOLUTION

Solve step-by-step.

ANSWER

Give the final answer clearly.

GATE SHORTCUT

If there is a quicker exam method, show it after the normal solution.

TRAP

Mention the mistake students commonly make.

STEP 12 — BUILD CONNECTIONS

Do not teach chapters as completely isolated facts.

Whenever relevant, tell me:

CONNECTION: This concept is related to ____ because ____.

For example, in Computer Networks connect:

- Layering !” protocols
- switching !” delay
- MAC !” Ethernet
- routing !” IP
- CIDR !” routing
- TCP !” congestion/flow control
- DNS !” application layer
- HTTP !” TCP

This will help me build a mental map of the subject.

STEP 13 — PERIODIC REVISION

After every major chapter create:

5-MINUTE REVISION

Include only:

- most important concepts
- formulas
- keywords
- diagrams
- traps
- memory shortcuts

Then give:

30-SECOND REVISION

Compress the entire chapter into roughly 5–10 lines.

STEP 14 — CHAPTER COMPLETION TEST

After completing a major chapter give me:

LEVEL 1 — Basic

3 questions

LEVEL 2 — GATE Conceptual

3 questions

LEVEL 3 — GATE Numerical / Tricky

3–5 questions where appropriate

Do not reveal solutions until I attempt them or ask for them.

STEP 15 — FORMULA SHEET

Maintain an internal running formula list while teaching.

After finishing the complete subject, create a consolidated:

GATE FORMULA SHEET

Organized chapter-wise.

Include:

- formula
 - variable meaning
 - important condition
 - common trap
-

STEP 16 — FINAL SUBJECT REVISION MATERIAL

When the entire supplied syllabus has been completed, create:

A. COMPLETE SYLLABUS CHECKLIST

Mark every supplied syllabus item as:

' Covered

Nothing from the official syllabus should be silently omitted.

B. MASTER REVISION NOTES

Compress the entire subject into high-yield revision notes.

C. COMPLETE FORMULA SHEET

All formulas in one place.

D. TOP CONFUSING CONCEPTS

Give the most easily confused concepts and differences.

E. TOP GATE TRAPS

Create a consolidated trap list.

F. MEMORY SHEET

All important mnemonics and shortcuts in one place.

G. LAST-DAY REVISION

Create material that I can revise shortly before the exam.

H. SUBJECT MENTAL MAP

Show how all important chapters connect.

VERY IMPORTANT TEACHING RULES

1. Accuracy is more important than speed.
2. Do not assume that because something sounds basic, I already know it.
3. At the same time, do not waste time on irrelevant academic depth.
4. Focus specifically on understanding required for **GATE problem solving**.
5. Whenever there are calculations, verify them carefully.
6. Clearly distinguish between:
 - definition
 - intuition
 - formula
 - assumption
 - exception
7. Never silently skip a syllabus phrase.
8. Do not overwhelm me with a giant answer covering the whole subject.
9. Teach one logical chunk at a time.
10. Use simple English.
11. Technical terminology must still remain accurate.
12. Whenever useful, explain something first in extremely simple language and then in proper GATE terminology.
13. Do not tell me to simply “watch a lecture” to understand something. Teach the required concept here.
14. Do not unnecessarily recommend books/videos during the learning process unless I explicitly ask.
15. If a concept needs a prerequisite, teach the minimum prerequisite here itself.
16. When a topic has very high GATE importance, mark:

Ø=HIGH PRIORITY

17. When a fact is mainly memorization, mark:

Ø>REMEMBER

18. When something often causes wrong answers, mark:

& GATE TRAP

19. When something is useful mainly for conceptual understanding but unlikely to require detailed memorization, say so.

20. Constantly optimize for:

Maximum GATE understanding per minute of study.

SPECIAL COMMANDS I MAY USE

If I type:

NEXT !' Continue to the next logical topic.

DEEPER !' Explain the current concept in greater depth.

SIMPLE !' Explain the current concept in much simpler language.

EXAMPLE !' Give more examples.

NUMERICALS !' Give additional numerical problems.

PYQ MODE !' Focus on GATE/PYQ-style application of the current topic.

TEST ME !' Quiz me without showing answers.

ANSWERS !' Show the solutions to the previous test.

NOTES !' Give only concise notebook-ready notes for the current topic.

REVISE !' Give a fast revision of everything covered so far.

FORMULAS !' Show all formulas learned so far.

TRAPS !' Show all GATE traps learned so far.

MEMORY !' Show all mnemonics/shortcuts learned so far.

STATUS !' Show:

- syllabus completed
- syllabus remaining
- approximate progress
- next topic

CONFUSED: [concept] !' Compare and explain the confusing concepts.

NEW SYLLABUS: !' I will paste another GATE subject syllabus. Rebuild the complete learning structure for that new subject using the same methodology.

START NOW

First, do NOT immediately start explaining Layering.

Start by analysing my complete **Computer Networks GATE syllabus**.

Give me:

1. Complete topic hierarchy
2. Best learning sequence
3. Important prerequisites
4. High-priority GATE areas
5. Numerical-heavy areas
6. Concept-heavy areas
7. Likely confusing areas
8. Complete study roadmap for this syllabus

Then begin:

CHAPTER 1 — TOPIC 1

Teach only the first logical topic thoroughly.

Stop after completing that topic and wait for me to type **NEXT**. :::

How I recommend you use it

Create **one dedicated GATE learning chat** and paste this master prompt there. For Computer Networks, leave the CN syllabus already included above. When CN finishes and you want OS, DBMS, TOC, etc., simply send:

NEW SYLLABUS: followed by the exact GATE syllabus.

Then continue using **NEXT !' NEXT !' NEXT**. Don't ask ChatGPT for the entire subject in one response; topic-by-topic will give you much better explanations and much cleaner notes.

One particularly useful workflow for you would be:

Learn topic !' write NOTEBOOK VERSION !' TEST ME !' solve questions !' correct mistakes !' NEXT.

That can replace a large amount of passive video watching while still forcing you to actually understand and recall the material.

USER

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE** and I have limited time.

I do NOT have the flexibility to watch many hours of YouTubelectures for every subject.

I want you to act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor
- revision coach

I will give you the **official GATE syllabus for one subject**.

Your job is to convert that syllabus into a **complete, structured GATE preparation course inside this chat**, so that I can learn the subject mainly from your explanations and make my own notes.

MY CURRENT SUBJECT

Subject: Computer Networks

OFFICIAL GATE SYLLABUS

Computer Networks:

Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing; IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP - flow control and congestion control, socket API;

IMPORTANT: SYLLABUS WILL CHANGE

Later I may replace the above syllabus with another GATE subject such as:

- Operating Systems
- DBMS
- Computer Organization
- Algorithms
- Data Structures
- Digital Logic
- Engineering Mathematics
- Discrete Mathematics
- Compiler Design
- Theory of Computation
- Programming
- or another GATE topic.

Whenever I give a new syllabus, **forget the previous subject structure and rebuild the course according to the new syllabus.**

The teaching methodology below must remain the same.

PRIMARY OBJECTIVE

Teach me everything I realistically need to know to solve **GATE-level questions from the supplied syllabus.**

Do NOT simply give me a syllabus summary.

I want actual teaching.

Assume that I may have forgotten even basic concepts.

Start from fundamentals and gradually reach GATE problem-solving level.

However, do NOT unnecessarily expand into topics that have no realistic relevance to the supplied GATE syllabus.

Follow this principle:

BASIC FOUNDATION !' CONCEPTUAL UNDERSTANDING !' FORMULAS !' EXAMPLES !' GATE APPLICATION !' PYQ PATTERNS !' TRAPS !' REVISION

STEP 1 — FIRST ANALYSE THE COMPLETE SYLLABUS

Before teaching, break the supplied syllabus into a logical hierarchy:

UNIT

Chapter

Topic

Subtopic

Do not blindly follow the exact sentence order of the syllabus if another teaching order makes concepts easier.

Show me:

1. Complete syllabus map
2. Recommended learning order
3. Dependencies between chapters
4. High-priority GATE topics
5. Calculation-heavy topics
6. Concept-heavy topics
7. Topics frequently mixed together in questions
8. Topics that students commonly misunderstand

Also clearly identify any **small prerequisite concept that is not explicitly written in the syllabus but is necessary to understand an included topic**.

Teach such prerequisites briefly instead of assuming I know them.

STEP 2 — DO NOT DUMP THE ENTIRE SUBJECT AT ONCE

Teach the subject **chapter by chapter and topic by topic**.

Start with:

Chapter 1 !' Topic 1

After completing one meaningful topic, STOP.

At the end write:

NEXT !' [exact next topic]

I will reply:

NEXT

Then continue.

This is important because I want to make handwritten/digital notes while learning.

STEP 3 — TEACH EVERY TOPIC USING THIS FORMAT

For EVERY important topic, follow approximately this structure.

1. WHAT IS IT?

Explain the concept in very simple language first.

Assume I am learning it for the first time.

Avoid unnecessary textbook jargon.

2. WHY DO WE NEED IT?

Explain the problem that existed and why this concept/protocol/algorithm was introduced.

I remember concepts much better when I know their purpose.

3. INTUITION

Give a real-life analogy wherever useful.

But after the analogy, explain the actual technical concept accurately.

Do not let the analogy replace the technical explanation.

4. CORE CONCEPT

Now explain the topic properly at GATE level.

Cover every important:

- definition
- rule
- condition
- mechanism
- algorithm
- terminology
- relationship
- exception
- important technical detail

Do not skip small points merely because they look obvious.

5. VISUAL REPRESENTATION

Whenever useful, use simple text diagrams such as:

A !' B !' C

or

```
Application
-----
Transport
-----
Network
-----
Data Link
-----
Physical
```

Use tables, flow diagrams, timelines and packet layouts wherever they make the concept easier.

6. FORMULAS

Collect every GATE-relevant formula for the topic.

For every formula explain:

- what it calculates
- meaning of each variable
- units
- when to use it
- when NOT to use it
- assumptions behind it

Do not make me memorize a formula without understanding it.

7. FORMULA DERIVATION/ INTUITION

If a formula is commonly confusing, briefly explain where it comes from.

I should understand the logic rather than blindly memorize it.

8. SOLVED EXAMPLES

Give small examples immediately after explaining a concept.

Then give GATE-style numerical/conceptual examples where appropriate.

Solve calculations step-by-step.

Do not jump directly from question to answer.

STEP 4 — GATE QUESTION ORIENTATION

For each major topic, explicitly teach me:

HOW GATE CAN ASK THIS

Show the common question patterns, for example:

- direct conceptual
- numerical
- comparison
- statement-based
- protocol behaviour
- output/result prediction
- multiple concepts combined
- MSQ-type traps
- NAT/addressing/routing calculations
- timing/delay calculations

Explain how I should recognise which concept the question is testing.

STEP 5 — COMMON TRAPS

Create a section:

& GATE TRAPS

Mention:

- commonly confused concepts
- similar terminology
- tempting wrong answers
- hidden assumptions
- unit-conversion mistakes
- off-by-one mistakes
- incorrect formula usage
- exceptions to general rules

If there is a famous GATE trap, emphasize it strongly.

STEP 6 — MEMORY SHORTCUTS

For every topic where possible, give me:

MEMORY TRICK

Use:

- mnemonics
- stories
- associations
- patterns
- one-line rules
- comparisons
- abbreviations

Example style:

TCP = Reliable !' Connection !' ACK !' Retransmission

But do NOT create misleading shortcuts merely for memorization.

Accuracy is more important than a catchy mnemonic.

STEP 7 — CONFUSING PAIRS

Whenever two concepts are easily confused, make a comparison table.

Examples for Computer Networks may include:

- Circuit vs Packet Switching
 - Datagram vs Virtual Circuit
-

- Distance Vector vs Link State
- Flow Control vs Congestion Control
- TCP vs UDP
- Hub vs Switch vs Router if needed as prerequisite
- MAC address vs IP address
- DNS recursive vs iterative query
- Persistent vs non-persistent HTTP

Include only comparisons that are relevant to the supplied syllabus.

STEP 8 — NOTES I SHOULD WRITE

After teaching each major concept, provide:

' b NOTEBOOK VERSION

Give a concise version that I can directly write in my GATE notes.

It should contain:

- definition
- key points
- formulas
- diagram if important
- important conditions
- GATE traps
- memory trick

Do NOT make the notebook version unnecessarily long.

Your detailed explanation teaches me.

The notebook version helps me revise.

STEP 9 — ACTIVE RECALL

At the end of each meaningful topic give me approximately **3–7 questions**.

Mix:

- very basic recall
- conceptual
- small numerical
- GATE-style thinking

Initially DO NOT reveal the answers.

Let me attempt them.

If I answer, evaluate each response as:

' Correct & bPartially correct 'L Incorrect

Then explain whatever I misunderstood.

STEP 10 — PYQ ORIENTATION

For every major chapter tell me:

PYQ CONCEPTS TO MASTER

Mention the concepts from that chapter that repeatedly lend themselves to GATE questions.

Do NOT fabricate exact historical GATE questions or claim a topic appeared in a particular year unless you are certain.

Instead focus on authentic **PYQ-style concepts and patterns**.

If I later provide actual PYQs, solve and analyse them separately.

STEP 11 — NUMERICAL PROBLEM METHOD

Whenever solving a numerical problem, follow:

GIVEN

List the provided values.

FIND

State exactly what is required.

CONCEPT

Mention the concept/formula being tested.

SOLUTION

Solve step-by-step.

ANSWER

Give the final answer clearly.

GATE SHORTCUT

If there is a quicker exam method, show it after the normal solution.

TRAP

Mention the mistake students commonly make.

STEP 12 — BUILD CONNECTIONS

Do not teach chapters as completely isolated facts.

Whenever relevant, tell me:

CONNECTION: This concept is related to _____ because _____.

For example, in Computer Networks connect:

- Layering !” protocols
- switching !” delay
- MAC !” Ethernet
- routing !” IP
- CIDR !” routing
- TCP !” congestion/flow control
- DNS !” application layer
- HTTP !” TCP

This will help me build a mental map of the subject.

STEP 13 — PERIODIC REVISION

After every major chapter create:

5-MINUTE REVISION

Include only:

- most important concepts
- formulas
- keywords
- diagrams
- traps
- memory shortcuts

Then give:

30-SECOND REVISION

Compress the entire chapter into roughly 5–10 lines.

STEP 14 — CHAPTER COMPLETION TEST

After completing a major chapter give me:

LEVEL 1 — Basic

3 questions

LEVEL 2 — GATE Conceptual

3 questions

LEVEL 3 — GATE Numerical / Tricky

3–5 questions where appropriate

Do not reveal solutions until I attempt them or ask for them.

STEP 15 — FORMULA SHEET

Maintain an internal running formula list while teaching.

After finishing the complete subject, create a consolidated:

GATE FORMULA SHEET

Organized chapter-wise.

Include:

- formula
 - variable meaning
 - important condition
 - common trap
-

STEP 16 — FINAL SUBJECT REVISION MATERIAL

When the entire supplied syllabus has been completed, create:

A. COMPLETE SYLLABUS CHECKLIST

Mark every supplied syllabus item as:

' Covered

Nothing from the official syllabus should be silently omitted.

B. MASTER REVISION NOTES

Compress the entire subject into high-yield revision notes.

C. COMPLETE FORMULA SHEET

All formulas in one place.

D. TOP CONFUSING CONCEPTS

Give the most easily confused concepts and differences.

E. TOP GATE TRAPS

Create a consolidated trap list.

F. MEMORY SHEET

All important mnemonics and shortcuts in one place.

G. LAST-DAYREVISION

Create material that I can revise shortly before the exam.

H. SUBJECT MENTAL MAP

Show how all important chapters connect.

VERY IMPORTANT TEACHING RULES

1. Accuracy is more important than speed.
2. Do not assume that because something sounds basic, I already know it.
3. At the same time, do not waste time on irrelevant academic depth.
4. Focus specifically on understanding required for **GATE problem solving**.
5. Whenever there are calculations, verify them carefully.
6. Clearly distinguish between:
 - definition
 - intuition
 - formula
 - assumption
 - exception
7. Never silently skip a syllabus phrase.
8. Do not overwhelm me with a giant answer covering the whole subject.
9. Teach one logical chunk at a time.
10. Use simple English.
11. Technical terminology must still remain accurate.
12. Whenever useful, explain something first in extremely simple language and then in proper GATE terminology.
13. Do not tell me to simply “watch a lecture” to understand something. Teach the required concept here.
14. Do not unnecessarily recommend books/videos during the learning process unless I explicitly ask.
15. If a concept needs a prerequisite, teach the minimum prerequisite here itself.
16. When a topic has very high GATE importance, mark:

Ø=HIGH PRIORITY

17. When a fact is mainly memorization, mark:

Ø>REMEMBER

18. When something often causes wrong answers, mark:

& pGATE TRAP

19. When something is useful mainly for conceptual understanding but unlikely to require detailed memorization, say so.
20. Constantly optimize for:

Maximum GATE understanding per minute of study.

SPECIAL COMMANDS I MAY USE

If I type:

NEXT !' Continue to the next logical topic.

DEEPER !' Explain the current concept in greater depth.

SIMPLE !' Explain the current concept in much simpler language.

EXAMPLE !' Give more examples.

NUMERICALS !' Give additional numerical problems.

PYQ MODE !' Focus on GATE/PYQ-style application of the current topic.

TEST ME !' Quiz me without showing answers.

ANSWERS !' Show the solutions to the previous test.

NOTES !' Give only concise notebook-ready notes for the current topic.

REVISE !' Give a fast revision of everything covered so far.

FORMULAS !' Show all formulas learned so far.

TRAPS !' Show all GATE traps learned so far.

MEMORY !' Show all mnemonics/shortcuts learned so far.

STATUS!' Show:

- syllabus completed
- syllabus remaining
- approximate progress
- next topic

CONFUSED: [concept] !' Compare and explain the confusing concepts.

NEW SYLLABUS: !' I will paste another GATE subject syllabus. Rebuild the complete learning structure for that new subject using the same methodology.

START NOW

First, do NOT immediately start explaining Layering.

Start by analysing my complete **Computer Networks GATE syllabus**.

Give me:

1. Complete topic hierarchy
2. Best learning sequence
3. Important prerequisites
4. High-priority GATE areas
5. Numerical-heavy areas
6. Concept-heavy areas
7. Likely confusing areas
8. Complete study roadmap for this syllabus

Then begin:

CHAPTER 1 — TOPIC 1

Teach only the first logical topic thoroughly.

Stop after completing that topic and wait for me to type **NEXT**.

ASSISTANT

Understood. I'll use exactly this teaching system for the syllabus you supplied. I'll first build the complete Computer Networks map, then teach only **Chapter 1 !' Topic 1** in this response.

GATE COMPUTER NETWORKS — MASTER MAP

Your official syllabus is:

Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing; IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP - flow control and congestion control, socket API; DNS and HTTP.

A good learning order is not exactly the sentence order. Some small prerequisites make later chapters much easier.

1. COMPLETE TOPIC HIERARCHY

UNIT 1 — Networking Foundations

Chapter 1 — Principles of Layering

Topic 1 — Why networks use layers

- Communication problem
- Modularity
- Separation of responsibilities
- Layered architecture
- Advantages/disadvantages of layering

Topic 2 — Protocol, Service and Interface

- Protocol
- Service
- Interface
- Peer entities
- Service Access Point
- Encapsulation/de-encapsulation

Topic 3 — OSI and TCP/IP models

- OSI 7 layers
- TCP/IP stack
- Functions of each layer
- Mapping between OSI and TCP/IP

- Devices/protocols at different layers
- End-to-end vs hop-to-hop concepts

Very important foundation for almost the entire subject.

UNIT 2 — Switching and Network Performance

Chapter 2 — Performance Metrics

Topic 1 — Delay components

- Processing delay
- Queueing delay
- Transmission delay
- Propagation delay
- Total nodal delay

Topic 2 — Transmission vs propagation

- Packet length
- Bandwidth/data rate
- Distance
- Propagation speed

Topic 3 — Throughput and bandwidth

- Bottleneck link
- End-to-end throughput
- Effective throughput

Topic 4 — Packet transfer calculations

- Store-and-forward
- Multiple links
- Multiple packets
- Pipelining

Very important numerical area.

Chapter 3 — Switching

Topic 1 — Circuit Switching

- Dedicated path
- Resource reservation
- Setup phase
- Transfer phase
- Teardown
- TDM/FDM basics if required

Topic 2 — Packet Switching

- Store-and-forward
- Statistical multiplexing
- Datagram network

Topic 3 — Virtual Circuit Switching

- Setup
- VC identifiers
- Forwarding table
- Comparison with datagram networks

Topic 4 — Comparisons

- Circuit vs packet
- Datagram vs virtual circuit
- Connection-oriented vs connectionless

UNIT 3 — Data Link Layer

Chapter 4 — Data Link Layer Fundamentals

Topic 1 — Purpose of Data Link Layer

- Node-to-node delivery
- Framing
- Error control
- Medium access
- MAC addressing

Small prerequisites only where needed.

Chapter 5 — Error Detection

Topic 1 — Types of errors

- Single-bit
- Burst errors

Topic 2 — Parity

- Even parity
- Odd parity

Topic 3 — Checksum

- One's-complement arithmetic
- Sender/receiver operation

Topic 4 — CRC

- Generator polynomial
- Modulo-2 division
- XOR division

- Codeword
- Receiver checking
- Error-detection properties

Extremely important GATE numerical/concept area.

Chapter 6 — Medium Access Control

Topic 1 — Multiple-access problem

Topic 2 — ALOHA

- Pure ALOHA
- Slotted ALOHA
- Vulnerable period
- Throughput

Topic 3 — CSMA

- 1-persistent
- Non-persistent
- p-persistent

Topic 4 — CSMA/CD

- Collision detection
- Minimum frame size
- Propagation delay
- Ethernet relevance

Possible prerequisite explanations will be included only where required.

High priority.

Chapter 7 — Ethernet

Topic 1 — Ethernet fundamentals

- LAN
- MAC addressing
- Ethernet frame

Topic 2 — Ethernet frame fields

- Preamble
- Destination
- Source
- Length/type
- Data
- FCS

Topic 3 — Ethernet operation

- Shared Ethernet

- CSMA/CD
- Switched Ethernet
- Collision domain

Topic 4 — Ethernet numerical concepts

- Minimum frame size
- Maximum network diameter / propagation relationships where applicable

Ø = Important, especially when combined with CSMA/CD.

UNIT 4 — Network Layer and Routing

Chapter 8 — Routing Fundamentals

Topic 1 — Forwarding vs routing

Topic 2 — Routing tables

Topic 3 — Shortest-path idea

Chapter 9 — Distance Vector Routing

Topics

- Distance vector concept
- Bellman-Ford relation
- Exchange with neighbours
- Routing-table update
- Convergence
- Count-to-infinity
- Routing loops
- Split horizon/related concepts as useful supporting ideas

Ø = Important numerical + conceptual area.

Chapter 10 — Link State Routing

Topics

- Network topology knowledge
- Link-state advertisements
- Flooding
- Dijkstra's shortest-path algorithm
- Shortest-path tree
- Routing table construction

Comparison

- Distance Vector vs Link State
-

One of the most important GATE area.

UNIT 5 — IPv4

Chapter 11 — IPv4 Addressing Foundation

Small prerequisite because CIDR cannot be understood properly without IPv4 addressing.

Topics

- IPv4 address = 32 bits
 - Network prefix
 - Host portion
 - Dotted decimal
 - Subnet mask
 - Prefix notation
-

Chapter 12 — CIDR

Topics

- CIDR notation /n
- Prefix length
- Network address
- Broadcast address
- Address range
- Number of addresses
- Number of usable hosts where appropriate
- Longest prefix matching
- Route aggregation
- Subnet calculations

One of the most important numerical topics.

Chapter 13 — IPv4 Fragmentation

Topics

- Why fragmentation occurs
 - MTU
 - Datagram length
 - Identification field
 - DF flag
 - MF flag
 - Fragment offset
 - 8-byte units
-

- Fragment size calculations
- Reassembly

Very common numerical/trap area.

Chapter 14 — Network Address Translation

Topics

- Private vs public address motivation
 - NAT mapping
 - Basic NAT
 - Port Address Translation/ NAT
 - Translation table
 - Advantages
 - Limitations
-

UNIT 6 — Transport Layer

Chapter 15 — TCP Foundation

Needed before flow/congestion control.

Topics

- TCP characteristics
 - Connection-oriented service
 - Reliable byte stream
 - Sequence numbers
 - ACK numbers
 - TCP segments
 - Connection establishment basics where needed
 - Sender/receiver relationship
-

Chapter 16 — TCP Flow Control

Topics

- Why flow control exists
- Sender vs receiver speed
- Receiver buffer
- Advertised receiver window
- Sliding-window idea
- Effective sending limit

Very important.

Chapter 17 — TCP Congestion Control

Topics

- What congestion means
- Congestion window `cwnd`
- Receiver window `rwnd`
- Effective window
- Slow start
- Congestion avoidance
- Threshold
- AIMD
- Timeout reaction
- Duplicate ACK behaviour as relevant
- TCP variants only to the depth required for GATE-style reasoning

0-100% Important Numerical/concept area.

Chapter 18 — Socket API

Topics

- Socket concept
- Client/server communication
- TCP socket workflow
- `socket()`
- `bind()`
- `listen()`
- `accept()`
- `connect()`
- `send()/recv()`
- `close()`
- Client vs server sequence

Usually more conceptual than numerical.

UNIT 7 — Application Layer

Chapter 19 — DNS

Topics

- Need for DNS
- Domain hierarchy
- Root
- TLD
- Authoritative server

- Local resolver
- Recursive queries
- Iterative queries
- DNS caching
- Resource records
- DNS message behaviour at GATE level

Ø=7% frequently conceptual.

Chapter 20 — HTTP

Topics

- Client-server model
- Request/response
- HTTP methods at relevant level
- Non-persistent HTTP
- Persistent HTTP
- RTT calculations
- TCP relationship
- HTTP object-transfer timing
- Cookies/cache basics only if needed for GATE-style understanding

Ø=7% timing numericals are important.

2. BEST LEARNING SEQUENCE

I recommend this sequence:

Layering

!“

Network performance

!“

Switching

!“

Data Link fundamentals

!“

Error Detection

!“

MAC

!“

Ethernet

!“

Routing fundamentals

!“

Distance Vector

!“

Link State

!“

IPv4 addressing

!“

CIDR

!“

Fragmentation

!“

NAT

!“

TCP foundation

!“

Flow Control

!“

Congestion Control

!“

Socket API

!“

DNS

!“

HTTP

This sequence minimizes situations where I have to say:

"You will understand this later."

3. IMPORTANT PREREQUISITES

These aren't necessarily separate items in your GATE syllabus, but we'll learn the minimum necessary pieces.

Prerequisite	Required for
Bits/bytes and binary	CIDR, CRC, IPv4
Data rate/bandwidth	Delay calculations

Prerequisite	Required for
Basic probability	ALOHA
Basic graph concepts	Routing
Shortest path	Link State
IPv4 address structure	CIDR/NAT/fragmentation
RTT concept	TCP, HTTP
Sequence/ACK concept	TCP
Client/server idea	Socket, DNS, HTTP

I will teach these **inside the relevant topic** rather than creating hours of extra material.

4. ~~0-16~~ HIGHEST-PRIORITY GATE AREAS

A useful priority order for problem-solving preparation is:

Priority A — Master thoroughly

- Delay/transmission calculations
- CRC
- MAC / ALOHA / CSMA/CD
- Ethernet calculations
- Distance Vector
- Link State / Dijkstra
- CIDR/subnetting
- IPv4 fragmentation
- TCP flow control
- TCP congestion control
- HTTP timing

Priority B — Strong conceptual understanding

- Layering
- Switching
- NAT
- DNS
- Socket API

This does **not** mean Priority B can be skipped.

It means Priority A deserves particularly strong numerical/PYQ practice.

5. CALCULATION-HEAVY AREAS

Watch carefully when we reach these:

Network performance

$\text{d}_{\text{trans}} = \frac{L}{R}$

$\text{d}_{\text{prop}} = \frac{D}{S}$

Packet switching

Store-and-forward calculations.

CRC

Modulo-2 polynomial division.

ALOHA

Throughput relationships.

CSMA/CD

Minimum frame transmission-time relation.

Routing

Shortest-path and distance-vector calculations.

CIDR

Network ranges, prefix length, number of addresses.

Fragmentation

Fragment size, offset, MF flag.

TCP

Window / congestion-window calculations.

HTTP

RTT-based delay calculations.

These will receive especially detailed numerical treatment.

6. CONCEPT-HEAVY AREAS

Mainly understand mechanisms and differences:

- Layering
- Protocol vs service vs interface
- Circuit vs packet vs VC switching
- Ethernet operation
- Distance Vector vs Link State
- NAT
- TCP flow vs congestion control
- Socket API
- DNS hierarchy
- Recursive vs iterative DNS
- Persistent vs non-persistent HTTP

7. MOST LIKELY CONFUSING AREAS

We'll pay special attention to these pairs:

Easily confused	Key distinction
Transmission delay vs propagation delay	Sending bits vs bits travelling
Bandwidth vs throughput	Capacity vs achieved rate
Packet switching vs circuit switching	Shared resources vs reserved resources
Datagram vs virtual circuit	Independent packets vs logical connection
Error detection vs correction	Discovering vs repairing
CSMA/CD vs CSMA/CA	Collision detection vs avoidance
MAC address vs IP address	Link-layer vs network-layer identity
Distance Vector vs Link State	Neighbour information vs topology information
Routing vs forwarding	Compute route vs move packet
Subnet mask vs CIDR prefix	Two representations of prefix boundary
Packet length vs payload	Entire datagram vs data portion
Fragment offset	Units of 8 bytes, not bytes
Flow control vs congestion control	Receiver protection vs network protection
rwnd vs cwnd	Receiver capacity vs congestion limit
Sequence number vs ACK number	Sent byte position vs expected next byte
Recursive vs iterative DNS	Resolver continues vs client/resolver follows referrals
Persistent vs non-persistent HTTP	Reuse TCP vs separate TCP connections

8. COMPLETE ROADMAP

Think of Computer Networks as one story:

TEXT

APPLICATION

DNS / HTTP

!“

TRANSPORT

TCP

Flow Control

Congestion Control

!“

NETWORK

IPv4

CIDR

```
Fragmentation
NAT
Routing
!
DATA LINK
Ethernet
MAC
Error Detection
!
PHYSICAL NETWORK
Bits actually travel
```

And layering explains **why this entire separation exists**.

NOW WE START

CHAPTER 1 — PRINCIPLES OF LAYERING

TOPIC 1 — Why Do Computer Networks Use Layers?

This looks simple, but understanding it correctly makes the entire subject easier.

1. WHAT IS A COMPUTER NETWORK?

At the simplest level:

A computer network allows two or more devices to exchange information.

Example:

Your laptop opens:

```
TEXT
www.google.com
```

Eventually Google's server sends information back.

Seems simple.

But what actually has to happen?

Your browser must:

- understand what you requested
- identify Google's server
- create data
- establish communication
- break information into manageable units
- identify source/destination
- find a path
- send data through Wi-Fi/Ethernet

- convert data to signals
- detect possible corruption
- deal with lost packets
- control transmission speed
- reconstruct received information

That's a huge problem.

If one gigantic program handled everything, networking would become extremely difficult to design and maintain.

So networking uses:

LAYERING

2. WHAT IS LAYERING?

Simple definition

Layering means:

Divide the complicated task of network communication into smaller independent tasks called layers.

Each layer performs a specific responsibility.

Example:

```
TEXT
Application
    ! "
Transport
    ! "
Network
    ! "
Data Link
    ! "
Physical
```

Instead of one huge networking mechanism, responsibilities are separated.

3. REAL-LIFE INTUITION

Imagine sending a parcel from Hyderabad to Delhi.

You write the message.

Someone packs it.

The courier company determines where it should go.

Transportation carries it.

The destination courier delivers it.

The receiver opens it.

You don't personally need to know:

- which truck carried it
- which highway was used
- which warehouse sorted it

You simply use the courier service.

Networking follows a similar modular structure.

4. WHY DO WE NEED LAYERING?

Suppose there were no layers.

Your browser would need to understand:

- HTTP
- TCP reliability
- IP routing
- Ethernet
- Wi-Fi
- physical signals
- cables
- radio transmission
- error detection

Every application would need to implement everything.

That would be terrible.

Instead:

TEXT

Browser

%

% U s e s

%%

Transport layer

%

% U s e s

%%

Network layer

%

%%

Data Link

%

%%

Physical

Each layer concentrates on its job.

5. CORE PRINCIPLE

A layer:

1. Provides services to the layer above

2. Uses services provided by the layer below

This single sentence is extremely important.

Consider:

```
TEXT
+-----+
| Application Layer |
+-----+
           !' s e r v i c e
+-----+
| Transport Layer  |
+-----+
           !' s e r v i c e
+-----+
| Network Layer    |
+-----+
           !' s e r v i c e
+-----+
| Data Link Layer  |
+-----+
           !' s e r v i c e
+-----+
| Physical Layer   |
+-----+
```

For example:

The Transport layer provides communication services to the Application layer.

But Transport itself uses the Network layer.

6. PEER-TO-PEER LOGICAL COMMUNICATION

Suppose:

```
TEXT
Computer A           Computer B

A p p l i c a t i o n   !•   l o g i c a l l y   !'           A p p l i c a t i o n
                        !"                                     !'
T r a n s p o r t       !•   l o g i c a l l y   !'           T r a n s p o r t
                        !"                                     !'
N e t w o r k           !•   l o g i c a l l y   !'           N e t w o r k
                        !"                                     !'
D a t a   L i n k       !•   l o g i c a l l y   !'           D a t a   L i n k
                        !"                                     !'
Physical   =====>   Physical
```

Same-level layers conceptually communicate using a **protocol**.

For example:

TCP at sender !" TCP at receiver.

HTTP client !" HTTP server.

7. IMPORTANT DISTINCTION — LOGICAL VS ACTUAL COMMUNICATION

When we say:

```
TEXT
T r a n s p o r t   A  !"   T r a n s p o r t   B
```

it doesn't mean transport-layer data literally jumps directly between those layers.

Actual transmission goes:

```
TEXT
Sender

Application
    !"
Transport
    !"
Network
    !"
Data Link
    !"
Physical
===== NETWORK =====
Physical
    !'
Data Link
    !'
Network
    !'
Transport
    !'
Application

Receiver
```

So communication between corresponding layers is usually **logical communication**.

Actual bits are transmitted using the lower layers and physical medium.

& **⚠GATE TRAP**

Don't interpret peer-layer communication diagrams as direct physical links.

8. MAJOR ADVANTAGES OF LAYERING

Advantage 1 — Modularity

Each networking task is separated.

Example:

TCP doesn't need to know whether bits travel using:

- Ethernet cable
- Wi-Fi
- optical fibre

It can perform transport-layer functions independently.

Advantage 2 — Easier design

Engineers can concentrate on individual layers.

Instead of building:

the complete Internet system

one group can work on routing while another works on applications.

Advantage 3 — Easier maintenance

If one layer changes while maintaining the same service/interface, other layers may not need to change.

Example:

You can replace:

TEXT

Ethernet

with:

TEXT

Wi-Fi

without redesigning HTTP.

This is one of the biggest benefits.

9. ~~0>1~~ MEMORY TRICK

Remember:

Change BELOW, continue ABOVE.

Meaning:

If the lower-layer implementation changes while its interface/service remains compatible, upper layers shouldn't need to care.

10. Advantage 4 — Interoperability and standardisation

Different manufacturers can build networking products that communicate because protocols and layer responsibilities are standardized.

Example:

A Windows laptop can communicate with a Linux server even though they were built by different companies.

11. Advantage 5 — Reusability

One lower-layer mechanism can serve many applications.

For example:

TCP can support:

```
TEXT
HTTP
SMTP
FTP
...
```

Applications do not need to independently implement TCP's entire reliable transport machinery.

12. Advantage 6 — Easier debugging

Suppose Internet communication fails.

We can investigate layer by layer:

```
TEXT
Is physical connection working?
    !"
Is Ethernet/Wi-Fi working?
    !"
Do we have IP connectivity?
    !"
Is TCP working?
    !"
Is HTTP working?
```

Layering gives structure to troubleshooting.

13. DOES LAYERING HAVE DISADVANTAGES?

Yes.

Layering is extremely useful, but it isn't free.

Disadvantage 1 — Overhead

Different layers add control information.

Conceptually:

```
TEXT
Original Data
    !"
TCP adds header
    !"
IP adds header
    !"
Ethernet adds header/trailer
```

More bits must therefore be transmitted.

We will formally learn this as **encapsulation**.

Disadvantage 2 — Possible duplication

Different layers might perform related functions.

For example, error checking might exist at multiple places for different purposes.

Disadvantage 3 — Strict layering can reduce optimisation

Sometimes information available in one layer could improve decisions in another.

A completely rigid layer boundary may make such optimization difficult.

For GATE, usually remember the main advantages much more strongly than this deeper design issue.

14. LAYERING DOES NOT MEAN EVERY DEVICE USES EVERY LAYER IDENTICALLY

This is a future-important idea.

An **end host** may process all layers.

An intermediate network device may process only the layers required for its job.

Simplified:

TEXT

Host A	Router	Host B
Application		Application
Transport		Transport
Network	Network	Network
Data Link	Data Link	Data Link
Physical	Physical	Physical

A router's fundamental job is at the **network layer**.

We'll refine device-layer relationships later.

15. END-TO-END VS HOP-TO-HOP — PREVIEW

Don't memorize deeply yet.

But understand this:

Suppose:

TEXT

A ---- Router 1 ---- Router 2 ---- B

Some functions operate conceptually:

TEXT

A ===== B

which is **end-to-end**.

Other functions happen one physical/link segment at a time:

TEXT

```
A !" R 1
R 1 !" R 2
R 2 !" B
```

which is **hop-to-hop**.

This distinction becomes important when studying:

- Data Link
- IP
- TCP

16. CONNECTION TO THE REST OF YOUR SYLLABUS

Layering is not an isolated chapter.

It gives you the framework for everything else.

TEXT

```
HTTP, DNS
%
%   A p p l i c a t i o n   L a y e r
!"

TCP
%
%   T r a n s p o r t   L a y e r
!"

IPv4 + Routing + CIDR + NAT
%
%   N e t w o r k   L a y e r
!"

Ethernet + MAC + Error Detection
%
%   D a t a   L i n k   L a y e r
!"

Actual signals
```

So whenever you feel lost later, ask:

Which layer is this problem happening at?

That alone often removes confusion.

17. HOW GATE CAN ASK LAYERING

Layering questions are usually conceptual.

Pattern 1 — Identify layer responsibility

Example:

Which layer handles routing?

! Network layer.

We'll properly learn layer functions soon.

Pattern 2 — Service relationships

Question might test whether you understand:

Layer N provides services to layer N+1 and uses services from layer N-1.

Pattern 3 — Peer communication

They may ask what entities at the same layer use to communicate.

Answer concept:

Protocol

Pattern 4 — Encapsulation

They may show:

TEXT

```
Application data
!"
Transport header added
!"
Network header added
```

and ask about packet size/header overhead.

We'll cover this in Topic 2.

Pattern 5 — Which layers are processed by which devices?

Example concepts:

- host
- router
- switch

We'll cover this carefully when the layer model is introduced.

18. & pGATE TRAPS

Trap 1

Protocol "service.

We'll formally differentiate them next.

Trap 2

Don't assume:

Every layer directly communicates physically with the corresponding layer on another computer.

It is usually logical peer communication.

Trap 3

Don't memorize networking merely as:

TEXT

Application
Transport
Network
Data Link
Physical

GATE asks what the layers **do**, not simply their names.

Trap 4

Layering doesn't imply one layer is completely ignorant of everything outside itself in every real implementation.

Layering is an architectural abstraction/model.

19. MEMORY SYSTEM

Remember layering using:

P-U-S

Each layer:

Provides service upward **U**ses service downward **S**peaks logically with its peer using a protocol

Or:

TEXT

! ' P R O V I D E
[L A Y E R]
! " U S E

This one diagram is worth remembering.

20. NO IMPORTANT FORMULA HERE

This topic is primarily conceptual.

So do **not** waste your time searching for a formula.

Formulas begin becoming important when we enter:

- delay
 - throughput
 - switching
 - CRC
 - MAC
 - CIDR
 - fragmentation
 - TCP
-

' p NOTEBOOK VERSION

You can write approximately this much in your notes.

Principles of Layering

Layering: Dividing network communication into multiple modules/layers, each performing specific functions.

Principle

A layer:

1. Provides services to the layer above.
2. Uses services of the layer below.
3. Communicates logically with the corresponding peer layer through a **protocol**.

TEXT

! ' P r o v i d e s s e r v i c e

Layer N

! " U s e s s e r v i c e

Advantages

- Modularity
- Easy design
- Easy maintenance
- Standardization/interoperability
- Reusability
- Easier debugging

Disadvantages

- Header/control overhead

- Possible duplication of functionality
- Strict boundaries may reduce some optimizations

Important

Peer-layer communication is usually **logical**; actual data travels downward through sender layers, across the physical medium, and upward through receiver layers.

Memory

P-U-S

- **P**rovide upward
- **U**se downward
- **S**peak to peer through protocol

& **Trap**: Protocol, service and interface are different concepts.

21. 30-SECOND REVISION

Layering divides complex networking into smaller modules.

Each layer:

TEXT

```
P r o v i d e s ! '
U s e s ! "
```

Corresponding layers communicate logically using protocols.

Benefits:

modularity + maintenance + interoperability + reuse.

Cost:

overhead + possible duplication.

And remember:

Change BELOW, continue ABOVE, provided the service/interface remains compatible.

ACTIVE RECALL — DO NOT LOOK UP ANSWERS

Try these yourself.

Q1

What is the main purpose of layering in computer networks?

Q2

Layer N generally provides services to which layer?

A. N" B. N C. N+1 D. Physical layer only

Q3

Layer N generally uses services provided by which layer?

A. N" B. N+1 C. Application layer D. Same layer on another machine

Q4

Two corresponding layers on different computers appear to communicate with each other. Is this usually physical communication or logical communication?

Q5

Suppose Ethernet is replaced by Wi-Fi but the upper-layer service remains compatible. Why doesn't HTTP need to be redesigned? Which benefit of layering does this demonstrate?

Q6 — GATE Thinking

Consider:

TEXT

Application
Transport
Network
Data Link
Physical

If the transport layer is Layer N, what are the layers whose services it **uses** and **provides service to**?

Don't send perfect textbook answers. Answer from your understanding; I'll correct any missing part.

NEXT !' Protocol vs Service vs Interface + Encapsulation/De-encapsulation

USER

yes

ASSISTANT

Great — continuing with the next logical topic.

CHAPTER 1 — PRINCIPLES OF LAYERING

TOPIC 2 — Protocol vs Service vs Interface + Encapsulation

This topic is small but extremely important because these three words look similar and GATE can easily confuse them.

1. FIRST UNDERSTAND THE THREE WORDS

Suppose we have:

TEXT

Application
!'
Transport
!'

Focus on the Transport layer.

It interacts in two different directions:

```

TEXT
Application
  ! '
  SERVICE
  ! '
  T r a n s p o r t   ! • % %   P R O T O C O L   % % ! '   T r a n s p o r t
  ! "
  INTERFACE
  ! "
Network

```

The three ideas are:

- **Protocol** !' communication rules between peer entities
- **Service** !' what a lower layer provides to the upper layer
- **Interface** !' how the upper layer accesses that service

This distinction is worth memorizing.

2. PROTOCOL — WHAT IS IT?

Simple meaning

A **protocol** is:

A set of rules followed by communicating entities.

Example:

When two people talk on a phone, there are implicit rules:

1. One speaks
2. Other listens
3. They understand the same language
4. They follow some order of conversation

Networking also requires rules.

For example:

```

TEXT
T C P   s e n d e r   ! • % % % %   T C P   p r o t o c o l   % % % % ! '   T C P   r e c e i v e r

```

TCP defines rules about things such as:

- sequence numbers
- acknowledgements
- retransmissions

- connection establishment
- flow control
- congestion control

3. FORMAL GATE DEFINITION

A protocol specifies the rules governing communication between peer entities at the same layer.

Peer entities

Entities belonging to the **same layer** on different communicating systems.

Example:

TEXT

Host A

Host B

A p p l i c a t i o n	!	•	p r o t o c o l	!	'	A p p l i c a t i o n
T r a n s p o r t	!	•	p r o t o c o l	!	'	T r a n s p o r t
N e t w o r k	!	•	p r o t o c o l	!	'	N e t w o r k

These are logical relationships.

4. WHAT DOES A PROTOCOL DEFINE?

A protocol generally specifies three major things:

1. Syntax

What does the message look like?

Example:

TEXT

Header | Data

It determines:

- field sizes
- order of fields
- message format

2. Semantics

What does each field/value mean?

Example:

If one field is:

TEXT

ACK = 1001

what does that acknowledgement mean?

3. Timing

When should messages be sent?

For example:

- When should an ACK be sent?
- How long should a sender wait?
- What happens after timeout?

0>1 MEMORY TRICK

Protocol = **SST**

- **S**yntax
- **S**emantics
- **T**iming

Very useful exam memory rule.

5. SERVICE — WHAT IS IT?

A **service** means:

What functionality one layer provides to the layer above it.

Notice the direction:

TEXT

Upper Layer

!'

SERVICE

!'

Lower Layer

Example:

TCP may provide an application with:

Reliable byte-stream delivery.

The application does not necessarily care exactly how TCP achieves reliability.

It simply uses the service.

6. REAL-LIFE ANALOGY

Imagine a restaurant.

You are the customer.

The restaurant provides:

Food delivery to your table.

That is the **service**.

You don't need to know:

- which chef prepared it
- exact kitchen workflow
- where ingredients were stored

You only care about what is provided.

Similarly:

TEXT

Application

!'

Reliable communication service

!'

TCP

The application uses the service.

7. PROTOCOL VS SERVICE

This distinction is ~~0=imp~~ important.

Protocol asks:

How do peer entities communicate?

Service asks:

What does a lower layer provide to an upper layer?

Example:

TEXT

Application A

Application B

!'

Service

!'

T C P A ! • % % % % T C P P r o t o c o l % % % % ! ' T C P B

TCP has:

- a **protocol** with peer TCP
- a **service** provided to the application

& pGATE TRAP

Do NOT say:

A service defines communication between peer layers.

Wrong.

That is a **protocol**.

Service is a vertical relationship between adjacent layers.

Protocol is a logical horizontal relationship between peer entities.

8. INTERFACE — WHAT IS IT?

An interface tells us:

How one layer accesses the services of another layer.

Think:

TEXT

Application

%

%

%¼

I n t e r f a c e

Transport

An interface defines the operations available between adjacent layers.

9. SIMPLE ANALOGY

Think of an ATM.

The bank provides a service:

Withdraw money.

But how do you access it?

Through an interface:

- screen
- keypad
- buttons
- menu options

In networking:

- **Service** = what functionality is offered
- **Interface** = how that functionality is accessed

10. PROTOCOL VS SERVICE VS INTERFACE

This table is extremely important.

Concept	Main question	Relationship
Protocol	How do peer entities communicate?	Horizontal/logical
Service	What does one layer provide to the layer above?	Vertical
Interface	How does the upper layer access the service?	Between adjacent layers

0> MEMORY SHORTCUT

Remember:

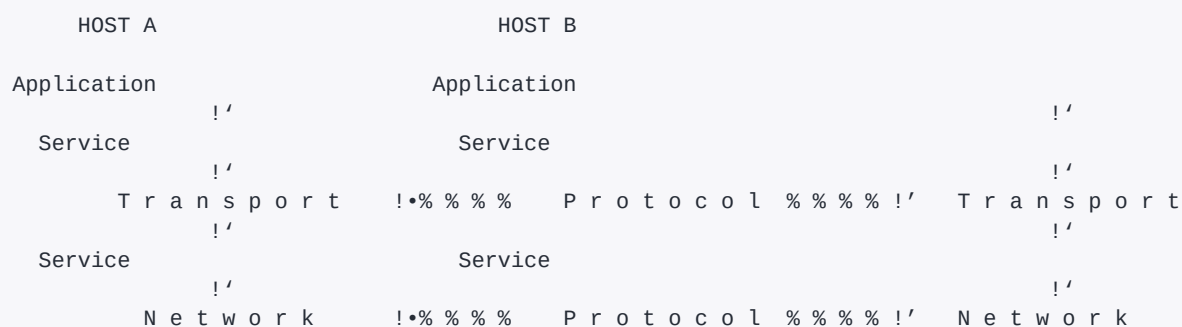
P = Peer S = Supply I = Interaction point

So:

- **Protocol !' Peer**
- **Service !' Supply functionality upward**
- **Interface !' Interaction between layers**

11. VISUAL REPRESENTATION

TEXT



Interfaces exist between adjacent layers.

12. SERVICE ACCESS POINT — SMALL PREREQUISITE

Sometimes networking terminology uses:

SAP = Service Access Point

A Service Access Point is conceptually:

A point through which an upper layer accesses services of a lower layer.

You generally don't need excessive depth here for GATE.

Remember:

```
TEXT
Layer N+1
    %
  SAP
    %
Layer N
```

13. NOW THE MOST IMPORTANT SECOND PART

ENCAPSULATION

Imagine an application creates:

```
TEXT
HELLO
```

Can those five characters directly travel across the Internet by themselves?

No.

Different layers need to add control information.

That process is called:

Encapsulation

14. WHAT IS ENCAPSULATION?

Encapsulation is the process of adding protocol control information, usually headers and sometimes trailers, as data moves downward through the protocol stack.

Example:

Start with:

```
TEXT
DATA
```

Transport adds its header:

```
TEXT
[TCP Header | DATA]
```

Network adds another header:

TEXT

[IP Header | TCP Header | DATA]

Data Link adds header/trailer:

TEXT

[Ethernet Header | IP Header | TCP Header | DATA | Trailer]

Then the Physical layer transmits bits.

15. VISUAL FLOW

Sender

TEXT

```
Application
  |
  | Data
  v
Transport
  |
  | [Transport Header | Data]
  v
Network
  |
  | [Network Header | Transport Header | Data]
  v
Data Link
  |
  | [DL Header | Network Header | Transport Header | Data | DL Trailer]
  v
Physical
  |
  v
Bits
```

Each lower layer treats the entire information received from the upper layer as its **payload**.

16. VERY IMPORTANT CONCEPT — PAYLOAD

Suppose Transport produces:

TEXT

[TCP Header | Application Data]

When Network receives it, Network doesn't treat the TCP header separately for its own processing.

The entire thing becomes its payload:

TEXT

```
IP Header | [TCP Header | Application Data]
           !'
           IP payload
```

Then Data Link sees:

```
TEXT
Ethernet Header | [IP Header | TCP Header | Data] | Trailer
                !'
                DL payload
```

This nesting is encapsulation.

17. DE-ENCAPSULATION

At the receiver, the reverse process happens.

De-encapsulation means removing layer-specific headers/trailers as data moves upward.

Example:

```
TEXT
Receiver gets frame

[Ethernet H | IP H | TCP H | DATA | Trailer]

! " R e m o v e   E t h e r n e t   i n f o r m a t i o n

[IP H | TCP H | DATA]

! " R e m o v e   I P   h e a d e r

[TCP H | DATA]

! " R e m o v e   T C P   h e a d e r

DATA
```

Finally the application receives the original data.

18. SENDER VS RECEIVER

Sender

```
TEXT
Application
! "
Encapsulation
! "
Physical transmission
```

Receiver

```
TEXT
Physical reception
! "
```

De-encapsulation
!"
Application

Easy memory:

Sender adds, Receiver removes.

19. PDU — PROTOCOL DATA UNIT

Important terminology.

A **PDU** is:

The unit of data handled by a particular protocol layer.

Common simplified naming:

Layer	PDU
Application	Data / Message
Transport	Segment for TCP
Network	Packet / IP datagram
Data Link	Frame
Physical	Bits

Easy memory:

Data !' Segment !' Packet !' Frame !' Bits

Mnemonic:

D S P F B

You can remember:

Don't Send People Fake Bills

D !' Data S !' Segment P !' Packet F !' Frame B !' Bits

& pSMALL TERMINOLOGY CAUTION

Different textbooks sometimes use terms such as:

- packet
- datagram

- segment

slightly differently depending on context.

For GATE, rely primarily on the protocol/layer context rather than blindly assuming every occurrence of the word "packet" must mean exactly one layer.

20. HEADER AND TRAILER

A **header** is control information placed before payload.

Example:

TEXT

Header | Payload

A **trailer** comes after payload:

TEXT

Header | Payload | Trailer

Ethernet, for example, includes an FCS at the end of the frame.

We'll study the exact Ethernet frame later.

21. WHY ARE HEADERS NECESSARY?

Different layers need different control information.

For example:

Transport layer may need

- port numbers
- sequence numbers
- ACK information

Network layer needs

- source IP
- destination IP

Data Link layer needs

- source MAC
- destination MAC
- error-detection information

Each layer adds information necessary for its own job.

22. ENCAPSULATION EXAMPLE

Suppose:

Application data = **1000 bytes**

TCP header = **20 bytes**

IP header = **20 bytes**

Ignore Data Link overhead for now.

After TCP:

$1000 + 20 = 1020$ bytes

After IP:

$1020 + 20 = 1040$ bytes

Therefore IP packet size:

1040 bytes

23. GATE-STYLE NUMERICAL

An application produces **1500 bytes** of data.

Transport adds a **20-byte header**.

Network adds a **20-byte header**.

Data Link adds:

- 14-byte header
- 4-byte trailer

What is the total frame size?

GIVEN

Application data:

1500B

Transport header:

20B

Network header:

20B

Data-Link overhead:

$14+4=18\text{B}$

FIND

Total transmitted frame size.

CONCEPT

Encapsulation overhead.

SOLUTION

Transport:

$$\boxed{1500+20=1520}$$

Network:

$$\boxed{1520+20=1540}$$

Data Link:

$$\boxed{1540+14+4=1558}$$

ANSWER

$$\boxed{1558 \text{ bytes}}$$

GATE shortcut

Simply add:

$$\boxed{1500+20+20+14+4}$$

$$\boxed{1558}$$

24. & pGATE TRAP — PAYLOAD VS TOTAL LENGTH

If a question says:

IP payload = 1000 bytes

do not automatically assume total IP datagram length is 1000 bytes.

If IP header is 20 bytes:

$$\boxed{\text{Total} = \text{Payload} + \text{Header}}$$

$$\boxed{1000+20=1020}$$

This becomes extremely important during IPv4 fragmentation.

25. CONNECTION TO FUTURE CHAPTERS

Encapsulation connects directly to almost everything later.

TCP

Adds TCP header.

!“

IPv4

Adds IP header.

!“

Ethernet

Creates a frame.

!“

Physical

Sends bits.

And when we study fragmentation, you'll repeatedly distinguish:

TEXT

IP Header

+

IP Payload

=

IP Datagram

So get comfortable with header/payload distinction now.

26. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Identify protocol/service/interface

Statement:

Rules governing communication between peer entities.

Answer:

Protocol

Statement:

Functionality offered by a lower layer to an upper layer.

Answer:

Service

Statement:

Mechanism through which service is accessed.

Answer:

Interface

Pattern 2 — Encapsulation order

Given:

TEXT

A p p l i c a t i o n ! ' T r a n s p o r t ! ' N e t w o r k ! ' D a t a L i n k

They may ask which headers are added and in what order.

Pattern 3 — Header overhead numerical

Example:

Payload and header sizes are provided.

Calculate:

- final packet length
- actual useful-data fraction
- protocol overhead

Pattern 4 — PDU identification

They may ask:

- Network layer !' ?
- Data Link !' ?
- Physical !' ?

Typical answer:

TEXT

N e t w o r k ! ' P a c k e t / d a t a g r a m
D a t a L i n k ! ' F r a m e
P h y s i c a l ! ' B i t s

27. EFFICIENCY / OVERHEAD FORMULA

This is a useful general relationship.

Suppose:

Payload size = (D)

Total overhead = (H)

Total transmitted size:

$[D+H]$

Payload efficiency:

$\boxed{\eta = \frac{D}{D+H}}$

Percentage efficiency:

$\boxed{\frac{D}{D+H} \times 100}$

Overhead percentage relative to total transmitted size:

$$\boxed{\frac{H}{D+H} \times 100}$$

28. EXAMPLE

Data:

$$1000B$$

Total headers:

$$100B$$

Total:

$$1100B$$

Efficiency:

$$\frac{1000}{1100} \times 100$$

$$= 90.91\%$$

So about **90.91%** of transmitted bytes represent useful original data under this simplified model.

& pGATE TRAP

Be careful whether a question asks:

Overhead as percentage of total

$$\frac{H}{D+H} \times 100$$

or:

Overhead relative to payload

$$\frac{H}{D} \times 100$$

These are **not the same**.

Read wording carefully.

29. COMPLETE COMPARISON

Feature	Protocol	Service	Interface
Meaning	Communication rules	Functionality offered	Access mechanism
Relationship	Peer-to-peer	Lower ! upper	Adjacent layers
Direction idea	Horizontal	Vertical	Vertical boundary
Example idea	TCP rules between hosts	Reliable transport	Socket/API-like access concept

& pNote: A socket API is an example of an application-facing programming interface to transport/networking services; we'll study it separately later.

30. MASTER MEMORY TRICK

Remember:

PSI

P — Protocol

Peers communicate

S — Service

Supplied upward

I — Interface

Interaction point

Then encapsulation:

Down = Add Up = Remove

And PDU:

Data !' Segment !' Packet !' Frame !' Bits

Mnemonic:

Don't Send People Fake Bills

' p NOTEBOOK VERSION

Protocol

Rules governing communication between **peer entities** at the same layer.

Defines:

- Syntax
- Semantics
- Timing

Mnemonic: **SST**

Service

Functionality provided by a lower layer to the layer above.

Service = what is provided

Interface

Defines how a layer accesses services of an adjacent lower layer.

Interface = how service is accessed

Difference

TEXT

```
Protocol !' horizontal !' peers
Service !' vertical !' lower provides to upper
Interface !' layer boundary / access point
```

Mnemonic:

P = Peer S = Supply I = Interaction

Encapsulation

Sender adds layer-specific headers/trailers while data travels downward.

TEXT

```
Data
!"
[Transport H | Data]
!"
[Network H | Transport H | Data]
!"
[DL H | Network H | Transport H | Data | Trailer]
!"
Bits
```

Receiver performs **de-encapsulation**.

Memory:

Down = Add, Up = Remove

PDU names

TEXT

```
Application !' Data
Transport !' Segment
Network !' Packet / Datagram
Data Link !' Frame
Physical !' Bits
```

Mnemonic:

Don't Send People Fake Bills

Efficiency

$$\eta = \frac{\text{Payload}}{\text{Total transmitted data}}$$

$$= \frac{D}{D+H}$$

5-MINUTE REVISION SO FAR

Layering

- divides network functionality into modules
- each layer provides services upward
- uses services downward
- logical peer communication happens through protocols

Protocol

Rules between peer entities.

Syntax + Semantics + Timing

Service

Functionality provided upward.

Interface

How the service is accessed.

Encapsulation

Headers/trailers added going downward.

De-encapsulation

Headers/trailers removed going upward.

PDU

TEXT

D a t a ! ' S e g m e n t ! ' P a c k e t ! ' F r a m e ! ' B i t s

& bTOP GATE TRAPS SO FAR

1. Protocol " service.
2. Protocol is peer-to-peer; service is between adjacent layers.
3. Peer communication is generally logical, not direct physical communication.
4. Payload " total packet length.
5. Sender encapsulates; receiver de-encapsulates.

6. Header overhead must be included when calculating transmitted size.
 7. Overhead percentage depends on what denominator the question specifies.
-

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the fundamental difference between a **protocol** and a **service**?

Q2

What three things are generally defined by a protocol?

Q3

If Layer N provides a service to Layer N+1, which layer uses that service?

Q4

Arrange these from application toward physical transmission:

- Frame
- Data
- Bits
- Packet
- Segment

Q5

An application produces 800 bytes.

Transport adds 20 bytes.

Network adds 20 bytes.

Data Link adds a 14-byte header and 4-byte trailer.

What is the final frame size?

Q6

A packet contains:

- 1200 bytes payload
- 60 bytes total header

Calculate payload efficiency as a percentage.

Q7 — GATE Conceptual

Which statement is correct?

A. A protocol specifies services offered by a layer to itself. B. A service specifies the rules used by peer entities. C. A protocol specifies rules followed by peer entities. D. An interface is communication between remote peer layers.

Q8

On the receiver side, what happens to headers while information moves upward through the layers?

NEXT !' OSI Model vs TCP/IP Model + Functions of Each Layer

USER

yes

ASSISTANT

CHAPTER 1 — PRINCIPLES OF LAYERING

TOPIC 3 — OSI MODEL vs TCP/IP MODEL + FUNCTIONS OF EACH LAYER

This topic is foundational because almost every later Computer Networks concept belongs to one of these layers.

1. WHY DO WE NEED A LAYER MODEL?

We already learned that networking is divided into layers.

But we still need a standard answer to:

How many layers should there be, and what should each layer do?

Two important models are:

1. **OSI Reference Model**
2. **TCP/IP Model**

For GATE, you should understand both, but the actual Internet is mainly based on the **TCP/IP protocol suite**.

2. OSI MODEL

OSI stands for:

Open Systems Interconnection

It was developed as a reference model for network communication.

It contains **7 layers**.

From top to bottom:

TEXT

7. Application
6. Presentation
5. Session
4. Transport
3. Network
2. Data Link
1. Physical

3. MEMORY TRICK FOR OSI LAYERS

From top to bottom:

A P S T N D P

Mnemonic:

All People Seem To Need Data Processing

Application Presentation Session Transport Network Data Link Physical

From bottom to top:

P D N T S P A

Mnemonic:

Please Do Not Throw Sausage Pizza Away

You only need one mnemonic. Use whichever feels easier.

4. BIG PICTURE

Think of the layers like this:

TEXT

A p p l i c a t i o n	!'	U s e r / n e t w o r k a p p l i c a t i o n s
P r e s e n t a t i o n	!'	D a t a r e p r e s e n t a t i o n
S e s s i o n	!'	C o n v e r s a t i o n / s e s s i o n c o n t r o l
T r a n s p o r t	!'	P r o c e s s - t o - p r o c e s s d e l i v e r y
N e t w o r k	!'	H o s t - t o - h o s t r o u t i n g
D a t a L i n k	!'	N o d e - t o - n o d e d e l i v e r y
P h y s i c a l	!'	R a w b i t s / s i g n a l s

Now we go layer by layer.

5. LAYER 7 — APPLICATION LAYER

WHAT DOES IT DO?

The Application layer provides network services to user applications.

Examples include:

- HTTP
- DNS
- SMTP
- FTP

For your GATE syllabus, the important application-layer protocols are:

- DNS

- HTTP

INTUITION

Suppose you open a browser and request a webpage.

The browser needs a protocol to communicate with a web server.

That protocol is usually:

HTTP

So:

TEXT

Browser

!"

HTTP

!"

Application Layer

IMPORTANT

& pThe Application layer is **not the application program itself**.

For example:

Chrome is an application.

HTTP is an application-layer protocol.

This distinction can be tested conceptually.

6. LAYER 6 — PRESENTATION LAYER

Its job is mainly related to:

- data representation
- translation
- encryption/decryption
- compression/decompression

Example:

Different computers may represent data differently.

The Presentation layer conceptually ensures that the receiving side can interpret the data correctly.

MEMORY

Presentation =

Format the data

Think:

Presentation = how data is presented

7. LAYER 5 — SESSION LAYER

The Session layer manages communication sessions between applications.

Functions may include:

- establishing a session
 - maintaining a session
 - terminating a session
 - dialogue control
 - synchronization/checkpoints
-

INTUITION

Think of a long conversation.

The Session layer is conceptually responsible for:

managing the conversation.

GATE NOTE

In the practical TCP/IP model, separate Presentation and Session layers are generally not explicitly present.

Their functions are usually handled by applications or other mechanisms.

This becomes important in OSI vs TCP/IP comparison.

8. LAYER 4 — TRANSPORT LAYER

~~NOT~~ VERY IMPORTANT.

The Transport layer provides:

Process-to-process communication

This means communication between application processes running on end systems.

Important protocols:

- TCP
- UDP

Your syllabus specifically includes TCP.

9. WHAT DOES TRANSPORT HANDLE?

Depending on the protocol, transport-layer functions can include:

- segmentation
- reassembly
- reliability
- error recovery
- flow control
- congestion control
- multiplexing/demultiplexing
- port addressing

For TCP, these are very important.

10. PROCESS-TO-PROCESS

Suppose your laptop is running:

- browser
- email client
- video call

All use the network.

An IP address identifies the host.

But how does arriving data reach the correct application?

Transport protocols use:

Port numbers

Conceptually:

TEXT

```
IP address !' identifies machine
Port number !' identifies process / application endpoint
```

We will revisit this during TCP and sockets.

11. LAYER 3 — NETWORK LAYER

Ø=VERY IMPORTANT.

The Network layer handles:

Delivery across multiple interconnected networks.

Its main responsibilities include:

- logical addressing
- routing
- forwarding

Important protocol:

IPv4

Your syllabus includes:

- IPv4 fragmentation
- CIDR
- NAT
- Distance Vector routing
- Link State routing

All are strongly related to this layer.

12. NETWORK-LAYER ADDRESS

The important address here is:

IP address

Example:

TEXT

192.168.1.10

Later we will learn how CIDR divides an IP address into:

- network prefix
- host part

13. ROUTING VS FORWARDING — PREVIEW

Do not memorize deeply yet.

Routing

Determining good paths through the network.

Forwarding

Actually moving a packet from an incoming interface to the correct outgoing interface.

Memory:

Routing decides; forwarding sends.

14. LAYER 2 — DATA LINK LAYER

The Data Link layer handles communication over a **single link**.

Often summarized as:

Node-to-node delivery

Main functions include:

- framing
- MAC addressing
- error detection
- medium access control

Your syllabus includes:

- error detection
- MAC
- Ethernet

All belong here.

15. DATALINK ADDRESS

The common address at this layer is:

MAC address

Example form:

TEXT

00:1A:2B:3C:4D:5E

We do not need to memorize specific addresses.

Understand:

TEXT

I P a d d r e s s ! ' n e t w o r k l a y e r
M A C a d d r e s s ! ' d a t a l i n k l a y e r

16. LAYER 1 — PHYSICAL LAYER

The Physical layer handles:

Actual transmission of raw bits over the physical medium.

It deals with things like:

- electrical signals
- optical signals
- radio signals
- bit transmission
- connectors/media

- signaling properties

PDU:

Bits

17. COMPLETE OSI SUMMARY

Layer	Main responsibility	Example concept/protocol
Application	Network services to applications	HTTP, DNS
Presentation	Representation, encryption, compression	Encoding/format
Session	Session/dialogue management	Session control
Transport	Process-to-process delivery	TCP
Network	Routing, logical addressing	IPv4
Data Link	Node-to-node delivery	Ethernet, MAC
Physical	Raw bit transmission	Signals/media

18. THREE DIFFERENT DELIVERY IDEAS

This is a very useful GATE mental model.

Transport

Process-to-process

Network

Host-to-host

Data Link

Node-to-node

Memory:

TEXT

```
T r a n s p o r t   ! '   P r o c e s s
N e t w o r k       ! '   H o s t
D a t a   L i n k   ! '   N o d e
```

19. NOW TCP/IP MODEL

The practical Internet architecture is based on the TCP/IP protocol suite.

Different textbooks may show:

- a **4-layer TCP/IP model**

or

- a **5-layer Internet model**

Both are common.

For GATE, understand the mapping.

20. 4-LAYER TCP/IP MODEL

TEXT

Application
Transport
Internet
Network Access

Where:

- Application combines OSI Application + Presentation + Session
 - Internet corresponds roughly to OSI Network
 - Network Access combines Data Link + Physical
-

21. 5-LAYER INTERNET MODEL

Many networking textbooks use:

TEXT

Application
Transport
Network
Data Link
Physical

This is especially convenient for learning.

Notice:

OSI has 7 layers.

Five-layer Internet model has 5.

The separate:

- Presentation
- Session

layers are absorbed into Application.

22. OSI vs TCP/IP MAPPING

TEXT

OSI	TCP/IP / Internet	
A p p l i c a t i o n	%	
P r e s e n t a t i o n	% % % % % % !'	A p p l i c a t i o n
S e s s i o n	%	
T r a n s p o r t	% % % % % % !'	T r a n s p o r t
N e t w o r k	% % % % % % !'	I n t e r n e t / N e t w o r k
D a t a L i n k	%	
P h y s i c a l	% % % % % % !'	N e t w o r k A c c e s s

In the 5-layer view:

TEXT

OSI	5-layer model	
A p p l i c a t i o n	%	
P r e s e n t a t i o n	% % % !'	A p p l i c a t i o n
S e s s i o n	%	
T r a n s p o r t	% % % % % % !'	T r a n s p o r t
N e t w o r k	% % % % % % % !'	N e t w o r k
D a t a L i n k	% % % % % % !'	D a t a L i n k
P h y s i c a l	% % % % % % !'	P h y s i c a l

23. OSI vs TCP/IP — IMPORTANT DIFFERENCES

OSI	TCP/IP
7-layer reference model	Practical Internet protocol architecture
Presentation layer separate	Usually part of Application
Session layer separate	Usually part of Application
Distinguishes services, interfaces and protocols clearly	Less strict distinction historically
Model developed independently of specific protocol suite	Closely tied to TCP/IP protocols
Mainly reference/teaching architecture today	Foundation of the Internet

24. MEMORY SHORTCUT

OSI:

7 layers

TCP/IP:

Think Application + Transport + Internet/Network + Link

The biggest difference to remember:

OSI has separate Session and Presentation layers; TCP/IP does not.

25. END SYSTEMS AND INTERMEDIATE DEVICES

Consider:

TEXT

Host A ---- Switch ---- Router ---- Host B

Different devices process different layers.

26. END HOST

A host generally uses all layers.

TEXT

Application
Transport
Network
Data Link
Physical

Examples:

- laptop
- phone
- server

27. ROUTER

A router's main functional layer is:

Network layer

Simplified processing:

TEXT

Network
Data Link
Physical

Routers examine network-layer information such as the destination IP address to forward packets.

28. LAYER-2 SWITCH

A typical Ethernet switch primarily operates at:

Data Link layer

It uses:

MAC addresses

Simplified:

TEXT

Data Link
Physical

29. HUB

A traditional hub is essentially a:

Physical-layer device

It does not make forwarding decisions based on MAC or IP addresses.

It basically repeats signals/bits.

30. DEVICE-LAYER TABLE

Device	Main layer
Hub / repeater	Physical
Bridge / Layer-2 switch	Data Link
Router	Network
End host	All layers

Ø=Very useful.

31. & pGATE TRAP — SWITCH

The word **switch** can sometimes refer to devices working at different layers, such as Layer-3 switches.

But unless stated otherwise in basic networking questions:

Ethernet switch generally means Layer 2.

Read the question context.

32. ADDRESSING AT DIFFERENT LAYERS

Another high-value memory table:

Layer	Address / identifier
Application	Names, e.g. domain name depending on protocol
Transport	Port number
Network	IP address
Data Link	MAC address

Memory:

Port !' Process IP !' Host/network location MAC !' Local-link interface

33. EXAMPLE — OPENING A WEBSITE

Suppose you open:

TEXT

www.example.com

Very simplified flow:

Application

HTTP creates a request.

Transport

TCP provides reliable process-to-process communication.

Network

IP decides how packets are addressed and forwarded across networks.

Data Link

Ethernet/Wi-Fi handles the local link.

Physical

Bits become electrical/radio/optical signals.

34. CONNECTION TO YOUR COMPLETE GATE SYLLABUS

Now we can place almost everything.

Application

- DNS
- HTTP

Transport

- TCP
- flow control
- congestion control
- socket API interaction

Network

- routing
- IPv4
- CIDR
- fragmentation
- NAT

Data Link

- error detection
- MAC
- Ethernet

That is why this layer model matters.

35. ENCAPSULATION USING LAYERS

Suppose application creates data.

```
TEXT
Application
  Data
    !"

Transport
  [TCP H | Data]
    !"

Network
  [IP H | TCP H | Data]
    !"

Data Link
  [Ethernet H | IP H | TCP H | Data | Trailer]
    !"

Physical
  Bits
```

At receiver:

reverse process.

36. WHAT HAPPENS AT A ROUTER?

Very important conceptual idea.

Suppose:

TEXT

Host A ! Router ! Host B

A router receives a frame.

It removes the incoming data-link framing to inspect the network-layer packet.

Then it chooses an outgoing interface and creates a **new data-link frame** suitable for the next link.

Conceptually:

TEXT

Incoming Frame

!"

Remove L2 header/trailer

!"

IP Packet

!"

Routing/forwarding decision

!"

Add new L2 header/trailer

!"

Outgoing Frame

37. VERY IMPORTANT CONSEQUENCE

Across routers:

IP packet

Conceptually continues end-to-end, though some IP-header fields may change.

Data-link frame

Is generally recreated on every link/hop.

This becomes very important later.

38. MAC ADDRESS vs IP ADDRESS — PREVIEW

Consider:

TEXT

A ! R 1 ! R 2 ! B

The IP addresses represent end-system/network-layer addressing.

But MAC addresses are relevant to each local link.

So the frame used from:

```
TEXT
A !' R 1
```

has different link-layer addressing than:

```
TEXT
R 1 !' R 2
```

This is why you should not think:

Destination MAC always equals the final remote host's MAC.

For a remote destination, the sender typically sends the local frame toward its next hop, such as the default gateway.

We'll handle detailed address resolution only if needed for the syllabus context.

39. END-TO-END vs HOP-TO-HOP

Now we can understand this more properly.

Transport

Primarily end-to-end between hosts/processes.

```
TEXT
Host A ===== Host B
```

Data Link

Hop-to-hop.

```
TEXT
H o s t   A   ! "   R o u t e r   1   ! "   R o u t e r   2   ! "   H o s t   B
```

Network

Provides packet delivery across multiple hops and is processed by routers along the path.

40. HOW GATE CAN ASK THIS

Pattern 1 — Layer identification

Which layer performs routing?

Network

Which layer performs framing?

Data Link

Which layer sends raw bits?

Physical

41. Pattern 2 — Protocol placement

TCP belongs to?

Transport layer

HTTP belongs to?

Application layer

IP belongs to?

Network layer

Ethernet belongs mainly to?

Data Link + Physical specifications, but frame/MAC concepts are Data Link.

42. Pattern 3 — Device layers

A router processes primarily up to which layer?

Network layer

Traditional Ethernet switch?

Data Link

Hub?

Physical

43. Pattern 4 — Address identification

MAC?

Data Link

IP?

Network

Port?

Transport

44. Pattern 5 — OSI/TCP-IP mapping

They may ask which OSI layers are merged into TCP/IP Application layer.

Answer:

- Application
- Presentation
- Session

45. Pattern 6 — PDU identification

Typical mapping:

TEXT

```
T r a n s p o r t   ! '   S e g m e n t
N e t w o r k       ! '   P a c k e t   /   d a t a g r a m
D a t a   L i n k   ! '   F r a m e
P h y s i c a l     ! '   B i t s
```

46. & pGATE TRAPS

Trap 1

Application layer "`application program.

HTTP is a protocol.

Chrome is an application.

Trap 2

Do not confuse:

Transport = process-to-process

with:

Trap 3

Router does not normally process application data just to perform ordinary IP forwarding.

Its fundamental forwarding decision is network-layer based.

Trap 4

A switch uses MAC addresses, not destination IP addresses, in basic Layer-2 forwarding.

Trap 5

A hub doesn't understand MAC addresses.

It operates at physical layer.

Trap 6

OSI and TCP/IP layer counts differ.

OSI:

\[7 \]

TCP/IP commonly:

\[4 \]

or educational Internet stack:

\[5 \]

Be guided by the model specified in the question.

Trap 7

Presentation and Session layers exist explicitly in OSI, not as separate layers in the typical TCP/IP model.

47. MEMORY SHEET

OSI order

A P S T N D P

All People Seem To Need Data Processing

Main layer jobs

TEXT

```
Application    !'  Application services
Presentation   !'  Presentation / format
Session        !'  Session
Transport      !'  Process
Network        !'  Route
Data Link      !'  Frame / local link
Physical       !'  Bits
```

Addresses

TEXT

```
Transport !'  Port
Network   !'  IP
Data Link !'  MAC
```

Mnemonic:

P-I-M

Port !' IP !' MAC

as we move downward:

TEXT

```
Transport
Network
Data Link
```

Devices

TEXT

```
Hub      !'  L 1
Switch   !'  L 2
Router   !'  L 3
```

Very easy mnemonic:

H-S-R = 1-2-3

Hub = 1 Switch = 2 Router = 3

Remember this.

48. ' p NOTEBOOK VERSION

OSI Model

7 layers:

TEXT

```
Application
Presentation
Session
Transport
Network
Data Link
Physical
```

Mnemonic:

All People Seem To Need Data Processing

Functions

- Application !' network services to applications
- Presentation !' translation, encryption, compression
- Session !' session/dialogue management
- Transport !' process-to-process delivery
- Network !' routing + IP addressing
- Data Link !' framing + MAC + error detection + medium access
- Physical !' raw-bit transmission

TCP/IP Mapping

TEXT

```
O S I   A   +   P   +   S   !'   T C P / I P   A p p l i c a t i o n
O S I   T r a n s p o r t   !'   T r a n s p o r t
O S I   N e t w o r k       !'   I n t e r n e t
O S I   D L   +   P H Y       !'   N e t w o r k   A c c e s s
```

5-layer educational model:

TEXT

```
Application
Transport
Network
Data Link
Physical
```

Address Mapping

TEXT

```
T r a n s p o r t   !'   P o r t
N e t w o r k       !'   I P
D a t a   L i n k   !'   M A C
```

Device Mapping

TEXT

```
H u b           !'   L a y e r   1
S w i t c h     !'   L a y e r   2
R o u t e r     !'   L a y e r   3
H o s t         !'   A l l   l a y e r s
```

Mnemonic:

HSR = 1, 2, 3

Delivery Mapping

TEXT

```
T r a n s p o r t !'   P r o c e s s - t o - p r o c e s s
N e t w o r k     !'   H o s t - t o - h o s t
D a t a   L i n k !'   N o d e - t o - n o d e
```

49. 5-MINUTE REVISION — CHAPTER 1

You have now covered the main foundation of layering.

Layering

Each layer:

- provides service upward
- uses service downward
- communicates logically with peers through protocols

Protocol

Rules between peer entities.

Defines:

Syntax, Semantics, Timing

Service

What functionality is provided upward.

Interface

How service is accessed.

Encapsulation

TEXT

D a t a ! ' S e g m e n t ! ' P a c k e t ! ' F r a m e ! ' B i t s

Sender adds headers.

Receiver removes them.

OSI

TEXT

Application
Presentation
Session
Transport
Network
Data Link
Physical

Layer jobs

Transport !' process Network !' routing/IP Data Link !' local link/frame Physical !' bits

Devices

Hub !' L1 Switch !' L2 Router !' L3

Addresses

Port !' Transport IP !' Network MAC !' Data Link

50. 30-SECOND REVISION

Remember only this:

TEXT

A p p l i c a t i o n ! ' H T T P , D N S
T r a n s p o r t ! ' T C P , P o r t s , P r o c e s s - t o - p r o c e s s
N e t w o r k ! ' I P , R o u t i n g
D a t a L i n k ! ' E t h e r n e t , M A C , F r a m e s
P h y s i c a l ! ' B i t s

And:

Hub 1 !' Switch 2 !' Router 3

OSI adds two extra conceptual layers above Transport:

Session + Presentation

CHAPTER 1 COMPLETION TEST

Do not look up the answers yet.

LEVEL 1 — BASIC

Q1

Write the seven OSI layers from top to bottom.

Q2

Which layer is responsible for routing?

Q3

Which address belongs to the Data Link layer?

A. IP address B. Port number C. MAC address D. Domain name

LEVEL 2 — GATE CONCEPTUAL

Q4

Which statement is correct?

A. Transport layer provides node-to-node delivery. B. Network layer provides process-to-process delivery. C. Transport layer provides process-to-process communication. D. Physical layer performs routing.

Q5

Match:

TEXT

Hub
Switch
Router

with:

TEXT

Network layer
Physical layer
Data Link layer

Q6

In the TCP/IP model, which three OSI layers are generally combined into the Application layer?

LEVEL 3 — TRICKY

Q7

Suppose:

TEXT

Host A !' Router R !' Host B

Which of these is normally recreated separately on the link A!' R and R!' B ?

A. Application message B. TCP connection C. Data-link frame D. HTTP request

Q8

A packet travels through three routers between sender and receiver.

Which device type normally examines the destination IP address to make a forwarding decision?

Q9

For communication with a remote host, is the destination MAC address in the sender's first Ethernet frame necessarily the MAC address of the final remote host? Explain in one sentence.

Q10 — Mental Map

Place each syllabus item in the correct broad layer:

- DNS
- TCP congestion control
- CIDR
- Ethernet
- IPv4 fragmentation
- HTTP
- MAC
- Distance Vector routing

You have now completed the **Principles of Layering foundation**.

NEXT !' Network Performance Metrics: Transmission Delay, Propagation Delay, Processing Delay, Queueing Delay and Total Nodal Delay

USER

yes

ASSISTANT

CHAPTER 2 — NETWORK PERFORMANCE METRICS

TOPIC 1 — Transmission Delay, Propagation Delay, Processing Delay, Queueing Delay & Total Nodal Delay

Ø=HIGH PRIORITY FOR GATE

This is one of the most important numerical foundations in Computer Networks.

A large number of networking questions become easy once you clearly distinguish:

Transmission delay vs Propagation delay

These two are frequently confused.

1. WHAT IS DELAY IN A NETWORK?

Suppose Host A sends a packet to Host B:

TEXT

Host A ----- Router ----- Host B

The packet does not reach instantly.

It experiences different kinds of delay.

At a node/link, the important delays are:

1. Processing delay
2. Queueing delay
3. Transmission delay
4. Propagation delay

So:

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

This is the first major formula to remember.

2. FIRST BUILD THE INTUITION

Imagine a toll gate on a highway.

A car arrives.

Processing delay

The toll operator checks the car/payment.

Queueing delay

The car waits behind other cars.

Transmission delay

Think of putting an entire convoy of cars onto the road one by one.

Propagation delay

Once a car is on the road, how long it takes to physically travel to the next toll gate.

This analogy is not perfect, but it helps separate the concepts.

3. PROCESSING DELAY

What is it?

Processing delay is:

Time taken by a node to examine/process the packet.

A router may need to:

- examine packet header
- check for errors
- inspect destination address
- determine outgoing link

Symbol:

$[d_{\text{proc}}]$

Usually a question directly gives this delay if it matters.

There is no standard universal formula for it.

4. QUEUEING DELAY

Suppose many packets arrive at the same router.

The outgoing link can transmit only one packet at a time.

Packets may wait in a buffer:

TEXT

Packets arriving

P1

P2

P3

P4

!"

+-----+

| QUEUE | -----> Link

+-----+

The time spent waiting is:

Queueing delay

Symbol:

$[d_{\text{queue}}]$

5. IMPORTANT PROPERTY OF QUEUEING DELAY

Unlike transmission and propagation delay, queueing delay may be highly variable.

It depends on:

- traffic load
- number of packets already waiting
- packet lengths
- transmission rate
- scheduling

It can range roughly from:

almost 0 !' very large

If the buffer becomes full, newly arriving packets may be dropped.

6. TRANSMISSION DELAY

d_{trans} is extremely important.

Suppose you have a packet of:

L { bits }

and a link transmits at:

R { bits/second }

How long does it take to push all L bits onto the link?

$$d_{trans} = \frac{L}{R}$$

Where:

- L = packet length in **bits**
- R = transmission rate in **bits/s**

Unit:

{ seconds }

7. WHY L/R ?

Suppose:

$R = 1000$ { bits/s }

Meaning:

The link can put 1000 bits onto the medium every second.

If packet length:

$L = 2000$ { bits }

then:

$$d_{trans} = \frac{2000}{1000}$$

$$\boxed{=2\text{ s}}$$

It takes 2 seconds to place the entire packet onto the link.

8. VERY IMPORTANT — TRANSMISSION DELAY DEPENDS ON WHAT?

$$\boxed{d_{\text{trans}} = \frac{L}{R}}$$

Therefore transmission delay depends on:

- packet size (L)
- transmission rate (R)

It does **NOT** depend on distance.

& pGATE TRAP

A 1 km link and a 1000 km link with the same packet size and transmission rate have the same **transmission delay**.

Distance affects propagation delay instead.

9. PROPAGATION DELAY

Once a bit is placed onto the link, it must physically travel through:

- copper
- fibre
- radio medium

The time taken to travel is propagation delay.

Formula:

$$\boxed{d_{\text{prop}} = \frac{D}{S}}$$

Where:

- (D) = distance travelled
- (S) = propagation speed in medium

10. EXAMPLE

Distance:

$$\boxed{D = 2000\text{ km}}$$

Convert:

$$\boxed{2000\text{ km} = 2 \times 10^6\text{ m}}$$

Propagation speed:

$$\boxed{S = 2 \times 10^8\text{ m/s}}$$

Then:

$$\boxed{d_{\text{prop}} = \frac{2 \times 10^6}{2 \times 10^8}}$$

$$\boxed{= 10^{-2}\text{ s}}$$

$$\boxed{\frac{D}{S}}$$

11. PROPAGATION DELAY DEPENDS ON WHAT?

$$d_{\text{prop}} = \frac{D}{S}$$

Depends on:

- distance
- propagation speed

It does **NOT** depend on packet size.

& #x2190; GATE TRAP.

A 100-bit packet and a 1,000,000-bit packet travelling over the same physical path have the same propagation delay.

12. TRANSMISSION vs PROPAGATION— MASTER COMPARISON

Feature	Transmission Delay	Propagation Delay
Meaning	Time to push packet onto link	Time for signal to travel
Formula	$\frac{L}{R}$	$\frac{D}{S}$
Depends on packet size?	Yes	No
Depends on bandwidth/rate?	Yes	No
Depends on distance?	No	Yes
Depends on propagation speed?	No	Yes

Remember this table conceptually.

13. MEMORY TRICK

Think:

Transmission = Loading

You're loading bits onto the wire.

So:

$$\boxed{\frac{L}{R}}$$

Propagation = Travelling

Bits are travelling through the wire.

So:

$$\boxed{\frac{D}{S}}$$

Memory sentence:

Load using rate; travel using speed.

14. BANDWIDTH / TRANSMISSION RATE vs PROPAGATION SPEED

This is another major confusion.

Suppose:

$R = 100 \text{ Mbps}$

This is transmission rate.

It means:

how many bits can be inserted onto the link per second.

Propagation speed could be around:

$2 \times 10^8 \text{ m/s}$

This describes:

how quickly the signal travels physically through the medium.

They measure completely different things.

15. SIMPLE ANALOGY

Imagine a road.

Transmission rate

How many cars per second can enter the road.

Propagation speed

How fast an individual car moves along the road.

Increasing road entry capacity doesn't necessarily make each car physically travel faster.

Same with networking.

16. WHEN DOES THE FIRST BIT ARRIVE?

Suppose Host A starts sending at $t=0$.

Propagation delay:

d_{prop}

The **first bit** reaches destination after approximately:

$\boxed{d_{\text{prop}}}$

provided we ignore processing/other delays before the link.

17. WHEN DOES THE LAST BIT ARRIVE?

Sender needs:

d_{trans}

to place all bits onto link.

Then the last bit needs:

d_{prop}

to travel.

So:

$d_{\text{lastbit}} = d_{\text{trans}} + d_{\text{prop}}$

for a single link, ignoring processing and queueing.

Important.

18. VISUAL TIMELINE

Suppose transmission delay = 4 ms.

Propagation delay = 6 ms.

TEXT

Sender

Receiver

```
t=0      first bit leaves
|----->
|
t=4ms     last bit leaves
|----->

t=6ms     first bit arrives

t=10ms    last bit arrives
```

So:

$4 + 6 = 10 \text{ ms}$

19. GATE-STYLE EXAMPLE 1

A packet contains:

1500 bytes

Link rate:

10 Mbps

Find transmission delay.

GIVEN

$$[L=1500\text{ bytes}]$$

Convert to bits:

$$[1500 \times 8 = 12000\text{ bits}]$$

$$[R=10 \times 10^6\text{ bits/s}]$$

FIND

Transmission delay.

CONCEPT

$$[d_{\text{trans}} = \frac{L}{R}]$$

SOLUTION

$$[d_{\text{trans}} = \frac{12000}{10 \times 10^6}]$$

$$[= 1.2 \times 10^{-3}]$$

$$[= \boxed{1.2\text{ ms}}]$$

20. & pGATE TRAP — BYTES vs BITS

If packet size is given in bytes and rate is given in bits/s:

Convert bytes to bits.

$$[1\text{ byte} = 8\text{ bits}]$$

This is one of the easiest ways to lose a mark.

21. GATE-STYLE EXAMPLE 2

Distance:

$$[3000\text{ km}]$$

Propagation speed:

$$[2 \times 10^8\text{ m/s}]$$

Find propagation delay.

GIVEN

$$[D=3000\text{ km}]$$

$$[= 3 \times 10^6\text{ m}]$$

$$[S=2 \times 10^8\text{ m/s}]$$

SOLUTION

$$d_{\text{prop}} = \frac{3 \times 10^6}{2 \times 10^8}$$

$$= 1.5 \times 10^{-2}$$

$$= \boxed{15 \text{ ms}}$$

22. GATE-STYLE EXAMPLE 3 — TOTAL LINK DELAY

Packet:

$$1000 \text{ bytes}$$

Rate:

$$1 \text{ Mbps}$$

Distance:

$$2000 \text{ km}$$

Propagation speed:

$$2 \times 10^8 \text{ m/s}$$

Assume processing and queueing delay = 0.

Find time until complete packet reaches receiver.

Step 1 — Transmission delay

$$L = 1000 \times 8 = 8000 \text{ bits}$$

$$d_{\text{trans}} = \frac{8000}{10^6}$$

$$= 0.008 \text{ s} = 8 \text{ ms}$$

Step 2 — Propagation delay

$$D = 2 \times 10^6 \text{ m}$$

$$d_{\text{prop}} = \frac{2 \times 10^6}{2 \times 10^8}$$

$$= 0.01 \text{ s} = 10 \text{ ms}$$

Step 3 — Total

$$8 + 10 = 18 \text{ ms}$$

ANSWER

$$\boxed{18 \text{ ms}}$$

23. PROCESSING + QUEUEING + TRANSMISSION + PROPAGATION

At a router:

TEXT

Packet arrives
!"

```
Processing
! "
Queue
! "
Transmission onto outgoing link
! "
Propagation across link
```

Therefore:

$$\boxed{d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}}$$

24. EXAMPLE WITH ALL FOUR DELAYS

Suppose:

$$d_{\text{proc}} = 1\text{ms}$$

$$d_{\text{queue}} = 4\text{ms}$$

Packet length:

$$L = 1000\text{B} = 8000\text{b}$$

Rate:

$$R = 2\text{Mbps}$$

Distance:

$$D = 1000\text{km} = 10^6\text{m}$$

Propagation speed:

$$S = 2 \times 10^8\text{m/s}$$

Transmission

$$d_{\text{trans}} = \frac{8000}{2 \times 10^6}$$

$$= 4\text{ms}$$

Propagation

$$d_{\text{prop}} = \frac{10^6}{2 \times 10^8}$$

$$= 5\text{ms}$$

Total

$$1 + 4 + 4 + 5$$

$$= \boxed{14\text{ms}}$$

25. QUEUEING DELAY — IMPORTANT INTUITION

Suppose transmission time for each packet is:

$$2\text{ms}$$

When your packet arrives, three packets are already waiting.

Ignoring the packet currently being transmitted:

$$d_{\text{queue}} = 3 \times 2$$

$$= \boxed{6\text{ms}}$$

But a GATE question might also say:

One packet is currently halfway through transmission.

Then you must include its remaining transmission time.

So queueing problems depend very carefully on wording.

26. TRAFFIC INTENSITY — USEFUL GATE CONCEPT

A common network-load measure is:

$$\rho = \frac{L_a}{R}$$

Where:

- L = average packet length in bits
- a = average packet arrival rate in packets/s
- R = link transmission rate in bits/s

Interpretation:

$$L_a$$

is the average incoming traffic in bits/s.

So:

$$\frac{L_a}{R}$$

compares incoming traffic to link capacity.

27. INTERPRETING TRAFFIC INTENSITY

If

$$\rho \ll 1$$

link is lightly loaded.

Queueing delay tends to be small.

If

$$\rho \rightarrow 1$$

queueing delay can become very large.

If average offered load exceeds service capacity

$$\rho > 1$$

the queue cannot remain stable indefinitely if arrivals continue at that rate and buffer assumptions permit buildup.

Packets will eventually face very large delays or loss in finite buffers.

28. & DON'T MEMORIZE A RANDOM QUEUEING FORMULA

GATE may sometimes provide a specific model/formula.

Do not universally assume a simple expression such as:

$$\left[\frac{\rho}{1-\rho} \right]$$

without knowing the queueing model and units.

For now, the important concept is:

As traffic intensity approaches 1, queueing becomes severe.

29. BANDWIDTH-DELAY PRODUCT — IMPORTANT CONNECTION

Consider a link of rate:

$$[R]$$

and propagation delay:

$$[d_{\text{prop}}]$$

Then:

$$[\boxed{ R \times d_{\text{prop}} }]$$

gives the number of bits that can be "in flight" on the link under continuous transmission.

This is called:

Bandwidth-delay product

Units:

$$[\frac{\text{bits}}{\text{s}} \times \text{s} = \text{bits}]$$

30. INTUITION

Imagine a pipe:

TEXT

Sender ===== Receiver

Bandwidth corresponds roughly to pipe capacity/rate.

Propagation delay corresponds to pipe length/travel time.

Bandwidth-delay product tells you roughly:

How many bits can occupy the link simultaneously.

31. EXAMPLE

Rate:

$[10\text{Mbps}]$

Propagation delay:

$[20\text{ms}]$

Convert:

$[20\text{ms}=0.02\text{s}]$

Then:

$[\text{BDP} = 10 \times 10^6 \times 0.02]$

$[=200000\text{bits}]$

$[=\boxed{200\text{kb}}]$

In bytes:

$[\frac{200000}{8} = 25000\text{B}]$

$[=25\text{KB}]$

approximately in decimal units.

32. CONNECTION TO FUTURE TCP

~~Ø=0~~ This concept becomes very important later.

TCP performance depends on keeping enough data "in flight."

So bandwidth-delay product will connect to:

- TCP window size
- flow control
- congestion control

Remember it now; we'll reuse it later.

33. ONE-WAY DELAY vs RTT

One-way delay

Time from sender to receiver.

RTT

Round Trip Time

Time for information to go from sender to receiver and a response/ACK to come back.

In a symmetric simplified path:

$[\text{RTT} \approx 2 \times \text{one-way delay}]$

But do NOT automatically use this unless assumptions justify it.

Paths/delays can differ in real networks.

34. EXAMPLE

One-way propagation delay:

$[25\text{ms}]$

If everything is symmetric and other delays ignored:

$[\text{RTT} = 2 \times 25]$

$[\boxed{50\text{ms}}]$

RTT will become extremely important in:

- TCP
- HTTP

35. PACKET SIZE AND EACH DELAY

Suppose packet size doubles.

What changes?

Transmission delay

$[d_{\text{trans}} = \frac{L}{R}]$

Doubles.

Propagation delay

$[d_{\text{prop}} = \frac{D}{S}]$

Unchanged.

This is a classic conceptual test.

36. LINK RATE DOUBLES

If:

$[R \rightarrow 2R]$

then:

$[d_{\text{trans}} = \frac{L}{2R}]$

Transmission delay becomes half.

Propagation delay remains unchanged.

37. DISTANCE DOUBLES

If:

$[D \rightarrow 2D]$

then:

$$d_{\text{prop}} = \frac{2D}{S}$$

Propagation delay doubles.

Transmission delay stays unchanged.

38. PROPAGATION SPEED DOUBLES

$$d_{\text{prop}} = \frac{D}{2S}$$

Propagation delay halves.

Transmission delay does not change.

39. HOW GATE CAN ASK THIS

Pattern 1 — Direct numerical

Given:

- packet size
- bandwidth

Find transmission delay.

Pattern 2 — Propagation numerical

Given:

- distance
- signal speed

Find propagation delay.

Pattern 3 — Total delay

Given several delay components.

Calculate:

$$d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

Pattern 4 — Conceptual comparison

Ask what changes if:

- packet length doubles
 - distance doubles
 - bandwidth doubles
-

Pattern 5 — First-bit/last-bit timing

May ask:

When does first bit arrive?

or:

When does complete packet arrive?

These are different.

Pattern 6 — Bandwidth-delay product

Given rate + propagation/RTT.

Calculate bits in flight.

Pattern 7 — Queueing

Several packets waiting for same link.

Calculate waiting time.

40. & pGATE TRAPS

Trap 1 — Bytes vs bits

$[1\text{B}=8\text{b}]$

Always check units.

Trap 2 — Mbps

Usually:

$[1\text{Mbps}=10^6\text{ bits/s}]$

unless question explicitly defines otherwise.

Similarly:

$[1\text{Gbps}=10^9\text{ bits/s}]$

Trap 3 — ms vs seconds

$[1\text{ms}=10^{-3}\text{s}]$

$[1\mu\text{s}=10^{-6}\text{s}]$

Trap 4 — km vs metres

$\lceil 1\text{km}=1000\text{m} \rceil$

Trap 5 — Transmission "propagation

Transmission:

$\lceil L/R \rceil$

Propagation:

$\lceil D/S \rceil$

Trap 6 — First bit vs complete packet

First bit arrival:

approximately

$\lceil d_{\text{prop}} \rceil$

Complete packet arrival:

$\lceil d_{\text{trans}}+d_{\text{prop}} \rceil$

for one link, if other delays are ignored.

Trap 7 — Don't add transmission delay to every bit separately

Transmission delay already accounts for pushing the **entire packet** onto the link.

41. MASTER MEMORY TRICKS

Four delays:

P Q T P

- Processing
- Queueing
- Transmission
- Propagation

Memory sentence:

Packets Queue Then Propagate

Not perfect because processing comes first, but easy to recall:

Process !' Queue !' Transmit !' Propagate

Transmission

$\lceil \boxed{L/R} \rceil$

Memory:

Length / Rate

Propagation

$\lceil \boxed{D/S} \rceil$

Memory:

Distance / Speed

Core distinction

Transmission = putting bits **ON** the link.

Propagation = moving bits **ALONG** the link.

42. CONNECTION TO SWITCHING

Soon we will study:

- circuit switching
- packet switching
- store-and-forward
- multiple links
- multiple packets

All those questions depend on:

$\lceil \frac{L}{R} \rceil$

If you master today's topic, packet-switching numericals become much easier.

' p NOTEBOOK VERSION

Network Delay

Total nodal delay:

$\lceil \boxed{d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}} \rceil$

1. Processing Delay

Time to examine/process packet.

2. Queueing Delay

Time packet waits in router/link queue.

Depends on traffic/load.

3. Transmission Delay

Time to push all packet bits onto link.

$$\boxed{d_{\text{trans}} = \frac{L}{R}}$$

- L : packet length in bits
- R : transmission rate in bits/s

Depends on packet size and link rate.

Does **not** depend on distance.

4. Propagation Delay

Time for signal to travel through medium.

$$\boxed{d_{\text{prop}} = \frac{D}{S}}$$

- D : distance
- S : propagation speed

Depends on distance and medium.

Does **not** depend on packet size.

One Link

Ignoring processing/queueing:

First bit arrives after:

$$d_{\text{prop}}$$

Last bit arrives after:

$$\boxed{d_{\text{trans}} + d_{\text{prop}}}$$

Bandwidth-Delay Product

$$\boxed{\text{BDP} = R \times d_{\text{prop}}}$$

Approximate number of bits in flight.

Traffic Intensity

$$\boxed{\rho = \frac{\lambda}{R}}$$

As:

$$\rho \rightarrow 1$$

queueing becomes severe.

43. 30-SECOND REVISION

Remember only these:

$$\boxed{\text{Transmission} = L/R}$$

$$\boxed{\text{Propagation} = D/S}$$

$$\boxed{\text{Total} = \text{Processing} + \text{Queueing} + \text{Transmission} + \text{Propagation}}$$

And:

Transmission depends on bits and bandwidth.

Propagation depends on distance and speed.

First bit !' propagation.

Last bit !' transmission + propagation.

ACTIVE RECALL — DON'T LOOK BACK

Q1

A 2000-byte packet is transmitted over a 4 Mbps link.

Find transmission delay.

Q2

A link is 4000 km long. Propagation speed is:

$$2 \times 10^8 \text{ m/s}$$

Find propagation delay.

Q3

Which delay changes when packet size increases?

A. Propagation only B. Transmission only C. Both necessarily D. Neither

Q4

Which delay changes when physical distance increases?

A. Transmission B. Propagation C. Processing necessarily D. None

Q5 — GATE Thinking

Packet length:

\[1000B \]

Rate:

\[2Mbps \]

Distance:

\[1000km \]

Propagation speed:

\[2 \times 10^8 \text{m/s} \]

Ignoring queueing and processing, calculate when the **complete packet** reaches the receiver.

Q6

Why does increasing bandwidth reduce transmission delay but not propagation delay?

Answer in your own words.

Q7

A link has:

\[R=20\text{Mbps} \]

and:

\[d_{\text{prop}}=10\text{ms} \]

Find the bandwidth-delay product in bits.

Q8 — Trap

A packet takes 4 ms to be transmitted onto a link and the propagation delay is 8 ms.

At what time after transmission begins does:

1. the first bit reach the receiver?
 2. the last bit reach the receiver?
-

NEXT !' Store-and-Forward Packet Switching: Multiple Links, Multiple Packets & Pipelining Numericals

USER

next

ASSISTANT

CHAPTER 2 — NETWORK PERFORMANCE METRICS

TOPIC 2 — Store-and-Forward Packet Switching, Multiple Links, Multiple Packets & Pipelining

VERY HIGH PRIORITY FOR GATE NUMERICALS

You already know the most important formula:

$$d_{\text{trans}} = \frac{L}{R}$$

Now GATE makes the problem more interesting:

What if a packet must travel through multiple links and routers?

And then:

What if there are multiple packets?

This is where **store-and-forward** and **pipelining** become important.

1. FIRST — WHAT IS PACKET SWITCHING?

Suppose Host A wants to send a large amount of data to Host B.

Instead of reserving an entire dedicated path, the data is generally divided into smaller units called:

Packets

These packets travel through network devices such as routers.

Example:

```
TEXT
Host A
|
Packet
v
Router 1
|
v
Router 2
|
v
Host B
```

Different communications can share the same network resources.

That is a key idea of:

Packet Switching

We will later compare packet switching properly with:

- Circuit switching
- Virtual circuit switching

For now, we focus on its **performance calculations**.

2. WHAT IS STORE-AND-FORWARD?

Consider:

```
TEXT
A ----- R ----- B
```

A sends a packet to router R.

A typical packet-switched router using **store-and-forward** must receive the **entire packet** before it can start transmitting that packet onto the next link.

That's the key rule.

Receive complete packet !' then forward it.

3. WHY IS THIS IMPORTANT?

Suppose packet transmission takes:

\[5ms \]

from A to R.

Router R cannot normally start forwarding that packet at:

\[1ms \]

or:

\[2ms \]

under store-and-forward assumptions.

It waits until the full packet has arrived.

Then it starts transmission onto the next link.

4. VISUAL INTUITION

Suppose one packet is:

```
TEXT
[===== PACKET =====]
```

Router receives:

```
TEXT
Beginning of packet
!"
[=====

More arrives
!"
```

```
[=====

Entire packet arrives
!"
[=====]

NOW router forwards
```

This is **store !' then forward**.

5. ONE PACKET, ONE LINK

This is what we already know.

Packet length:

$[L]$

Link rate:

$[R]$

Transmission delay:

$[\frac{L}{R}]$

If propagation delay is (d_p) :

$[\frac{L}{R} + d_p]$

for the complete packet to reach the other end, ignoring other delays.

6. ONE PACKET, TWO LINKS

Now consider:

```
TEXT
A ----- R ----- B
      Link 1      Link 2
```

Assume:

- same rate (R)
- packet size (L)
- zero propagation
- zero processing
- zero queueing

On Link 1:

$[\frac{L}{R}]$

Router receives the complete packet.

Then Link 2:

$[\frac{L}{R}]$

Therefore:

$$\boxed{\text{Total} = \frac{L}{R} + \frac{L}{R}}$$

$$\boxed{\text{Total} = 2 \frac{L}{R}}$$

7. ONE PACKET, THREE LINKS

TEXT

A ---- R1 ---- R2 ---- B

Three links.

Each requires:

$$\frac{L}{R}$$

Therefore:

$$\boxed{\text{Total} = 3 \frac{L}{R}}$$

8. GENERAL FORMULA — ONE PACKET

If a packet crosses:

$$N$$

links, all having the same transmission rate R :

$$\boxed{d_{\text{trans,total}} = N \frac{L}{R}}$$

assuming store-and-forward and ignoring other delays.

Important.

9. COUNT LINKS, NOT ROUTERS

Consider:

TEXT

A ---- R1 ---- R2 ---- B

How many routers?

$$2$$

How many links?

$$3$$

For transmission-delay calculation:

$$3 \frac{L}{R}$$

not:

$$2 \frac{L}{R}$$

Memory trick

With:

K routers

between two hosts, there are usually:

$K+1$ links

if they form a simple path.

10. ADD PROPAGATION DELAY

Suppose every link has propagation delay:

d_p

and there are N links.

Then one packet experiences:

Transmission

$N \frac{L}{R}$

Propagation

$N d_p$

Therefore:

$$D = N \left(\frac{L}{R} + d_p \right)$$

assuming identical links and ignoring processing/queueing.

11. MORE GENERAL FORM

If links have different:

- transmission rates R_i
- propagation delays d_{p_i}

then for **one packet**:

$$D = \sum_{i=1}^N \frac{L}{R_i} + \sum_{i=1}^N d_{p_i}$$

plus processing and queueing delays if specified.

12. GATE EXAMPLE — ONE PACKET

A 1000-byte packet travels through **3 links**, each of rate:

2Mbps

Ignore propagation and processing.

Find total transmission delay.

GIVEN

$L = 1000B = 8000 \text{ bits}$

$$[R=2\times10^6\text{bps}]$$

$$[N=3]$$

CONCEPT

$$[D=N\frac{L}{R}]$$

SOLUTION

One link:

$$[\frac{8000}{2\times10^6} = 0.004\text{s} = 4\text{ms}]$$

Three links:

$$[3\times4 = 12\text{ms}]$$

ANSWER

$$[\boxed{12\text{ms}}]$$

13. NOW THE IMPORTANT PART — MULTIPLE PACKETS

Suppose Host A has **3 packets**:

TEXT

P1
P2
P3

and the path has:

TEXT

A ---- R ---- B

Two links.

Suppose transmission delay of each packet per link is:

$$[T=\frac{L}{R}]$$

A common mistake is:

$$[3\text{ packets}\times2\text{ links}\times T = 6T]$$

'L That is usually **wrong**.

Why?

Because of:

PIPELINING

14. WHAT IS PIPELINING?

While router R is sending Packet 1 toward B, Host A can simultaneously transmit Packet 2 toward R.

Different links can work at the same time.

Example:

TEXT

A ----- R ----- B

At some instant:

TEXT

A -- P2 --> R -- P1 --> B

Two different packets are simultaneously being transmitted over different links.

That's the pipeline.

15. VISUAL TIMELINE — 3 PACKETS, 2 LINKS

Let:

$T = \frac{L}{R}$

Time 0 !' T

TEXT

A -- P1 --> R

At (T) :

P1 has reached R.

Time T !' 2T

Simultaneously:

TEXT

A -- P2 --> R
R -- P1 --> B

At $(2T)$:

- P1 reaches B
- P2 reaches R

Time 2T !' 3T

TEXT

A -- P3 --> R
R -- P2 --> B

At $(3T)$:

- P2 reaches B
- P3 reaches R

Time 3T !' 4T

TEXT

R -- P3 --> B

At $(4T)$:

P3 reaches B.

Therefore total time:

$\boxed{4T}$

not $(6T)$.

16. GENERAL FORMULA — MULTIPLE PACKETS

Suppose:

- (P) packets
- (N) links
- all packets equal size (L)
- every link has same rate (R)
- store-and-forward
- no propagation, processing or queueing

Then:

$\boxed{D = (N+P-1)\frac{L}{R}}$

MEMORIZE WITH UNDERSTANDING

17. WHY $(N+P-1)$?

First packet needs:

$N\frac{L}{R}$

to cross all (N) links.

After the pipeline fills, each additional packet reaches the destination one transmission time later:

$\frac{L}{R}$

There are:

$(P-1)$

additional packets.

Therefore:

$D = N\frac{L}{R} + (P-1)\frac{L}{R}$

$= (N+P-1)\frac{L}{R}$

18. MEMORY TRICK

For equal links:

Links + Packets - 1

$$\boxed{(N+P-1)\frac{L}{R}}$$

Think:

First packet crosses all links; every extra packet adds one slot.

19. GATE EXAMPLE — MULTIPLE PACKETS

Host A sends **5 packets** through **3 links**.

Each packet:

$$1000B$$

Each link:

$$1Mbps$$

Ignore propagation.

GIVEN

$$P=5$$

$$N=3$$

$$L=1000B=8000bits$$

$$R=10^6bps$$

Step 1 — Per-link transmission time

$$T = \frac{8000}{10^6} = 0.008s = 8ms$$

Step 2 — Pipeline formula

$$D = (N+P-1)T$$

$$= (3+5-1)(8)$$

$$= 7 \times 8$$

$$\boxed{56ms}$$

20. WITHOUT PIPELINING — WHAT WOULD HAPPEN?

If every packet had to completely reach B before A started the next packet:

Each packet:

$$\lceil 3T \rceil$$

Five packets:

$$\lceil 15T \rceil$$

But with pipelining:

$$\lceil 7T \rceil$$

Huge improvement.

This illustrates why packet networks can use links efficiently.

21. MULTIPLE PACKETS + PROPAGATION

Suppose each link has propagation delay:

$$d_p$$

For the first packet:

$$N \left(\frac{L}{R} + d_p \right)$$

But once the pipeline is active, each additional equal packet under identical-link assumptions follows one transmission-time slot later:

$$\frac{L}{R}$$

Therefore:

$$D = N d_p + (N+P-1) \frac{L}{R}$$

assuming:

- identical links
- equal packet size
- store-and-forward
- no extra queueing/processing.

Important nuance:

Propagation delay occurs on every link, but you **do not multiply the full propagation delay by the number of packets** when calculating completion time of the pipelined stream.

22. GATE EXAMPLE WITH PROPAGATION

Suppose:

- 4 packets
- 2 links
- transmission time = 5 ms per packet per link
- propagation delay = 3 ms per link

Then:

$$D = N d_p + (N+P-1)T$$

$$= 2(3) + (2+4-1)(5)$$

$$\lfloor 6+25 \rfloor$$

$$\lfloor \boxed{31\text{ms}} \rfloor$$

23. WHY NOT $(4 \times 2 \times 3\text{ms})$ PROPAGATION?

Because packets are travelling in a pipeline simultaneously.

Total completion time is not equal to:

Sum of every individual packet's independent delay.

That would ignore overlap.

This is one of the central ideas in pipelining problems.

24. DIFFERENT LINK RATES

Now consider:

TEXT

```
A ---- R1 ---- R2 ---- B
R1rate  R2rate  R3rate
```

Suppose:

$$\lfloor R_1, R_2, R_3 \rfloor$$

are different.

Transmission times are:

$$\lfloor T_1 = \frac{L}{R_1} \rfloor$$

$$\lfloor T_2 = \frac{L}{R_2} \rfloor$$

$$\lfloor T_3 = \frac{L}{R_3} \rfloor$$

For **one packet**:

$$\lfloor \boxed{D = T_1 + T_2 + T_3} \rfloor$$

ignoring propagation.

Easy.

25. MULTIPLE PACKETS WITH DIFFERENT LINK RATES

Now things become more interesting.

The slowest link determines how quickly completed packets can eventually flow through the pipeline.

The slowest transmission stage has:

$$\lfloor \boxed{T_{\text{bottleneck}} = \max(T_1, T_2, \dots, T_N)} \rfloor$$

Since:

$$T_i = \frac{L}{R_i}$$

the maximum transmission time corresponds to the **smallest link rate**.

26. GENERAL DIFFERENT-RATE FORMULA

For (P) equal-sized packets over (N) store-and-forward links:

$$D = \sum_{i=1}^N T_i + (P-1) \max(T_i)$$

where:

$$T_i = \frac{L}{R_i}$$

ignoring propagation/processing effects.

If propagation delays are included:

$$D = \sum_{i=1}^N (T_i + d_{p_i}) + (P-1) \max(T_i)$$

under the standard pipeline assumptions.

$\emptyset = 70\%$ is a very useful GATE extension.

27. WHY DOES THE SLOWEST LINK MATTER?

Imagine three workers:

TEXT

Worker	1	!	1	sec / item
Worker	2	!	5	sec / item
Worker	3	!	2	sec / item

Worker 2 is slowest.

Even if Workers 1 and 3 are fast, the system cannot ultimately produce one finished item every second.

The bottleneck controls the pipeline rate:

$$5 \text{ sec/item}$$

Network links behave similarly.

28. GATE EXAMPLE — DIFFERENT RATES

Three links have transmission times per packet:

$$T_1 = 2 \text{ ms}$$

$$T_2 = 5 \text{ ms}$$

$$T_3 = 3 \text{ ms}$$

There are 4 packets.

First packet

$$2 + 5 + 3 = 10 \text{ ms}$$

Bottleneck

$$\lceil \max(2, 5, 3) \rceil = 5 \text{ms}$$

Three additional packets:

$$\lceil (P-1) \times 5 \rceil = 3 \times 5 = 15 \text{ms}$$

Total:

$$\lceil 10 + 15 \rceil = \boxed{25 \text{ms}}$$

29. BOTTLENECK LINK

The link with the smallest transmission rate:

$$\lceil R_{\min} \rceil$$

is usually the bottleneck for equal packet sizes.

End-to-end steady-state throughput cannot exceed:

$$\lceil \boxed{R_{\min}} \rceil$$

This connects directly to our next performance metric:

Throughput

30. PACKETIZATION— LARGE MESSAGE DIVIDED INTO PACKETS

~~Not~~ other classic GATE idea.

Suppose a message has:

$$\lceil M \text{ bits} \rceil$$

You could send the whole thing as one giant packet.

Or divide it into:

$$\lceil P \rceil$$

smaller packets.

Ignoring headers:

$$\lceil L = \frac{M}{P} \rceil$$

per packet.

Packetization permits pipelining.

31. EXAMPLE — NO PACKETIZATION

Message size:

$$\lceil 8000 \text{bits} \rceil$$

Two links:

TEXT

A ---- R ---- B

Rate:

$[1000\text{bps}]$

If the entire 8000 bits are one packet:

$[T = \frac{8000}{1000} = 8\text{s}]$

Two links:

$[2 \times 8 = 16\text{s}]$

Total:

$[\boxed{16\text{s}}]$

32. DIVIDE INTO 4 PACKETS

Each packet:

$[\frac{8000}{4} = 2000\text{bits}]$

Transmission time:

$[T = \frac{2000}{1000} = 2\text{s}]$

Now:

$[N=2, \text{quad } P=4]$

Using:

$[(N+P-1)T]$

$[= (2+4-1)(2)]$

$[= 5 \times 2]$

$[\boxed{10\text{s}}]$

So packetization reduces completion time:

$[16\text{s} \rightarrow 10\text{s}]$

because of pipelining.

33. WHY NOT DIVIDE INTO INFINITELY MANY TINY PACKETS?

Interesting question.

In our simplified model, smaller packets improve pipelining.

But real packets require headers.

Suppose every packet gets:

- TCP header
- IP header
- Ethernet header etc.

If packets become extremely small:

Header overhead becomes very large.

There are also:

- processing costs
- minimum/maximum packet constraints
- queueing effects

So real networking balances packet size and overhead.

34. MESSAGE SIZE + HEADER OVERHEAD

Suppose message payload:

M

is divided into P packets.

Each packet carries payload:

$\frac{M}{P}$

and header of:

H

Therefore actual packet size:

$L = \frac{M}{P} + H$

The total transmitted bits across each link become larger than the original message because headers are repeated for every packet.

Important if GATE gives packet header size.

35. EXAMPLE WITH HEADER

Message:

8000bits

Split into 4 packets.

Header:

200bits

Payload per packet:

2000bits

Actual packet length:

$2000 + 200 = 2200\text{bits}$

When calculating:

$d_{\text{trans}} = \frac{L}{R}$

use:

$\lfloor 2200 \text{ bits} \rfloor$

not 2000 bits.

& p**GATE TRAP**

36. STORE-AND-FORWARD vs CUT-THROUGH— SMALL NOTE

Store-and-forward:

Router receives complete frame/packet before forwarding it.

Some technologies can use forms of **cut-through forwarding**, where forwarding may begin before the entire unit arrives.

But unless a GATE question explicitly says otherwise:

Use the forwarding model stated or implied by the problem.

For standard packet-switching delay questions, store-and-forward is common.

37. PROCESSING DELAY AT ROUTERS

Suppose:

TEXT

A ---- R1 ---- R2 ---- B

There are:

$\lfloor 2 \rfloor$

routers.

If each router has processing delay:

$\lfloor d_{\text{proc}} \rfloor$

then for one packet, total router processing delay is:

$\lfloor \boxed{2d_{\text{proc}}} \rfloor$

Notice:

- 3 links
- 2 intermediate routers

Again, count carefully.

38. GENERAL ONE-PACKET END-TO-END DELAY

Suppose there are:

- $\lfloor N \rfloor$ links
- $\lfloor N-1 \rfloor$ intermediate routers

For one packet:

$$\boxed{D = \sum_{i=1}^N \left(\frac{L}{R_i} + d_{\text{prop}_i} \right) + \sum_{j=1}^{N-1} (d_{\text{proc}_j} + d_{\text{queue}_j})}$$

This is a very general formula.

Don't blindly memorize the giant expression.

Understand:

Add every delay the packet actually experiences.

39. HOW GATE CAN ASK THIS

Pattern 1 — One packet, several links

Use:

$$N \frac{L}{R}$$

plus other delays if given.

Pattern 2 — Several packets, equal links

Use:

$$(N+P-1) \frac{L}{R}$$

Pattern 3 — Add propagation

Use:

$$N d_p + (N+P-1) \frac{L}{R}$$

for identical links.

Pattern 4 — Different link rates

First packet:

$$\sum T_i$$

Additional packets spaced by:

$$\max(T_i)$$

So:

$$\sum T_i + (P-1) \max(T_i)$$

Pattern 5 — Message divided into packets

First calculate:

$$L = \frac{M}{P}$$

or:

$$L = \frac{M}{P} + H$$

if header is added per packet.

Then apply pipeline formula.

Pattern 6 — Count routers vs links

If there are K intermediate routers:

$$\boxed{\text{Links} = K + 1}$$

for a simple linear path.

40. & pTOP GATE TRAPS

Trap 1 — Multiplying packets × links

Do NOT automatically calculate:

$$P \times N \times \frac{L}{R}$$

because transmissions can overlap.

Trap 2 — Forgetting store-and-forward

A router cannot start normal store-and-forward transmission until the complete packet is received.

Trap 3 — Routers vs links

TEXT

A - R1 - R2 - B

has:

- 2 routers
- 3 links

Trap 4 — Payload vs packet size

If each packet has a header:

$$\text{Packet} = \text{Payload} + \text{Header}$$

Use the total packet size for transmission delay.

Trap 5 — Bottleneck

For different rates, the fastest link does not control steady pipeline output.

The **slowest link** does.

Trap 6 — Propagation does not disappear

Pipelining overlaps packet transmissions, but each signal still physically propagates along every link.

Trap 7 — Don't use $(N+P-1)$ blindly

That simple formula assumes:

- equal packet sizes
- equal transmission times on links
- store-and-forward
- no complicated extra queueing

If rates differ, use the more general bottleneck method.

41. MASTER MEMORY SHEET

One packet + N equal links

$$\boxed{N \frac{L}{R}}$$

Memory:

Every link must transmit it once.

P packets + N equal links

$$\boxed{(N+P-1) \frac{L}{R}}$$

Memory:

Links + Packets " 1

With propagation

$$\boxed{Nd_p + (N+P-1) \frac{L}{R}}$$

Different rates

$$T_i = \frac{L}{R_i}$$

$$\boxed{D = \sum T_i + (P-1) \max(T_i)}$$

Bottleneck

$$\boxed{R_{\text{bottleneck}} = \min(R_i)}$$

Slowest rate !' longest transmission time.

' p NOTEBOOK VERSION

Store-and-Forward

Router receives the **complete packet** before forwarding it to the next link.

One Packet

For (N) equal-rate links:

$$\boxed{D = N \frac{L}{R}}$$

With propagation (d_p) per link:

$$\boxed{D = N \left(\frac{L}{R} + d_p \right)}$$

Multiple Packets — Pipelining

Different links can transmit different packets simultaneously.

For:

- (P) packets
- (N) equal-rate links

$$\boxed{D = (N+P-1) \frac{L}{R}}$$

With propagation:

$$\boxed{D = Nd_p + (N+P-1) \frac{L}{R}}$$

Mnemonic:

Links + Packets " 1

Different Link Rates

$$T_i = \frac{L}{R_i}$$

First packet:

$$\sum T_i$$

Each additional packet:

$$\max(T_i)$$

Therefore:

$$\boxed{D = \sum T_i + (P-1) \max(T_i)}$$

Packetization

If message size M is divided into P packets:

$$\text{Payload/packet} = \frac{M}{P}$$

With header H :

$$L = \frac{M}{P} + H$$

42. 30-SECOND REVISION

Remember:

Store entire packet !' then forward.

One packet, N links:

$$N \frac{L}{R}$$

P packets, N equal links:

$$(N+P-1) \frac{L}{R}$$

Different rates:

First packet = sum of stage times.

Next packets = bottleneck stage time.

And:

2 routers means 3 links on a simple sender-to-receiver chain.

ACTIVE RECALL — DON'T LOOK BACK

Q1

A 1200-byte packet crosses 4 links, each running at:

$$4 \text{ Mbps}$$

Ignore all other delays.

Find the time until the complete packet crosses all four links.

Q2

There are:

- 6 packets
- 3 equal links
- transmission time = 2 ms per packet per link

Using store-and-forward pipelining, find total time.

Q3

Two intermediate routers lie between A and B:

TEXT

A !' R 1 !' R 2 !' B

How many transmission delays does **one packet** experience?

Q4 — GATE Trap

Five packets travel across two equal links.

Each packet requires 4 ms transmission per link.

Which is correct?

A. 40 ms B. 24 ms C. 20 ms D. 8 ms

Q5

A message of 12,000 bits is divided into 4 equal packets.

Each packet gets a 500-bit header.

What packet length should be used when calculating transmission delay?

Q6 — Different Rates

Three links require respectively:

$\sqrt[2]{2\text{ms}}$, $\sqrt[4]{6\text{ms}}$, $\sqrt[3]{3\text{ms}}$

to transmit one packet.

There are 5 packets.

Ignoring propagation, calculate total completion time.

Q7 — Conceptual

Why isn't the total time for $\sqrt{(P)}$ packets over $\sqrt{(N)}$ links simply:

$\sqrt{PN\frac{L}{R}}$?

Explain in one sentence.

Q8 — Challenging GATE Style

There are 5 equal packets travelling over 4 identical links.

For every link:

$\sqrt{d_{\text{trans}}=3\text{ms}}$

$\sqrt{d_{\text{prop}}=2\text{ms}}$

Ignoring processing and extra queueing, find the time from the beginning of transmission of Packet 1 until the last bit of Packet 5 reaches the destination.

NEXT !' Throughput, Bottleneck Link, Bandwidth vs Throughput & Performance Numericals

USER

next

ASSISTANT

CHAPTER 2 — NETWORK PERFORMANCE METRICS

TOPIC 3 — Throughput, Bottleneck Link, Bandwidth vs Throughput

Ø=Ý%
HIGH PRIORITY FOR GATE

A very common mistake is to think:

Bandwidth = Throughput

They are related, but they are **not the same thing**.

1. WHAT IS BANDWIDTH / LINK RATE?

Suppose a link is advertised as:

$[100 \text{ Mbps}]$

This means its transmission rate/capacity is approximately:

$[100 \times 10^6 \text{ bits/s}]$

In many GATE networking numericals, terms such as:

- bandwidth
- link capacity
- transmission rate
- data rate

may refer to this (R) .

2. WHAT IS THROUGHPUT?

Throughput means:

The actual rate at which data is successfully transferred from sender to receiver.

If:

20 Mbits

are successfully delivered in:

2 seconds

then:

$\text{Throughput} = \frac{\text{Data delivered}}{\text{Time}}$

$= \frac{20 \text{ Mbits}}{2 \text{ s}}$

$= 10 \text{ Mbps}$

3. BANDWIDTH vs THROUGHPUT

Suppose:

TEXT

Link capacity = 100 Mbps

but because of:

- congestion
- slower links elsewhere
- protocol overhead
- losses/retransmissions
- competing traffic

you actually obtain:

TEXT

70 Mbps

Then:

- **Bandwidth/link rate = 100 Mbps**
- **Throughput = 70 Mbps**

4. MASTER DIFFERENCE

Bandwidth / capacity	Throughput
Maximum/link transmission capability	Actual achieved transfer rate
Property of link/path component	Performance observed during transfer
Usually upper bound	Usually "d limiting capacity
Example: 100 Mbps link	Example: actual 70 Mbps

Memory:

Bandwidth = what you CAN carry. Throughput = what you ACTUALLY carry.

5. BOTTLENECK LINK

Very important.

Suppose:

TEXT

A --10 Mbps-- R1 --5 Mbps-- R2 --20 Mbps-- B

There are three link rates:

$R_1=10\text{Mbps}$

$R_2=5\text{Mbps}$

$R_3=20\text{Mbps}$

Which link limits sustained end-to-end data transfer?

The:

5Mbps

link.

Therefore:

$R_{\text{bottleneck}} = \min(R_1, R_2, \dots, R_n)$

Here:

$R_{\text{bottleneck}} = 5\text{Mbps}$

6. WHY DOES THE SLOWEST LINK MATTER?

Imagine three pipes:

TEXT

Wide pipe !' Narrow pipe !' Wide pipe

Even if the first and third pipes carry water very quickly, everything must eventually pass through the narrow middle pipe.

Similarly:

TEXT

100 Mbps !' 10 Mbps !' 1 Gbps

cannot provide a sustained end-to-end flow greater than roughly:

10Mbps

under the simple single-flow model.

7. END-TO-END THROUGHPUT

For a simple path with no other limiting factors:

$$\boxed{\text{Throughput} \leq \min(R_1, R_2, \dots, R_n)}$$

In ideal steady state:

$$\boxed{\text{Throughput} \approx \min(R_i)}$$

8. TWO-LINK EXAMPLE

Suppose:

TEXT

Server -----10 Mbps----- Router -----2 Mbps----- Client

Then:

$$\text{Throughput} = \min(10, 2)$$

$$\boxed{2\text{Mbps}}$$

The 2 Mbps link is the bottleneck.

9. GATE-STYLE EXAMPLE

A server is connected to a client through links of:

$$100\text{Mbps}, \quad 40\text{Mbps}, \quad 80\text{Mbps}$$

Assume no competing traffic.

Find maximum steady-state throughput.

GIVEN

$$R_1 = 100\text{Mbps}$$

$$R_2 = 40\text{Mbps}$$

$$R_3 = 80\text{Mbps}$$

CONCEPT

$$\text{Throughput} = \min(R_i)$$

SOLUTION

$$\min(100, 40, 80) = 40$$

ANSWER

$$\boxed{40\text{Mbps}}$$

10. & pGATE TRAP — DON'T TAKE THE AVERAGE

Wrong:

$$\left[\frac{100+40+80}{3} \right]$$

The path is not limited by the average.

It is limited by the bottleneck.

Correct:

$$\left[\min(R_i) \right]$$

11. FILE TRANSFER TIME USING THROUGHPUT

Suppose a file contains:

$$F \text{ (bits)}$$

and achieved throughput is:

$$T \text{ (bits/s)}$$

Then approximately:

$$\boxed{\text{Transfer Time} = \frac{F}{T}}$$

12. EXAMPLE

File size:

$$100\text{MB}$$

Path throughput:

$$20\text{Mbps}$$

First convert the file to bits.

Using decimal networking units unless specified otherwise:

$$100\text{MB} = 100 \times 10^6 \text{ (bytes)}$$

$$= 800 \times 10^6 \text{ (bits)}$$

Then:

$$\text{Time} = \frac{800 \times 10^6}{20 \times 10^6}$$

$$= \boxed{40\text{s}}$$

13. MB vs Mb

This is extremely important.

MB

Megabytes

Mb

Megabits

Since:

$1\text{Byte}=8\text{bits}$

then:

$100\text{MB}=800\text{Mb}$

under the same decimal-prefix convention.

Memory:

Capital B = Byte Small b = bit

14. THROUGHPUT WITH SHARED LINKS

Now suppose one bottleneck link of:

$R=12\text{Mbps}$

is fairly shared by 3 simultaneous identical flows.

Simplified equal-sharing assumption:

Throughput per flow = $\frac{12}{3}$

$=\boxed{4\text{Mbps}}$

15. IMPORTANT — DON'T ASSUME FAIR SHARING UNLESS JUSTIFIED

A real network may not divide capacity perfectly equally.

If a GATE question says:

- equally shared
- fair sharing
- each connection gets equal bandwidth

then divide appropriately.

Otherwise, do not invent equal sharing.

16. ACCESS LINK BOTTLENECK EXAMPLE

Suppose:

TEXT

Server --1 Gbps-- Internet --100 Mbps-- Home Router --20 Mbps-- Laptop

Assuming the listed links dominate performance:

Throughput = $\min(1000,100,20)$

$=\boxed{20\text{Mbps}}$

Even though the server has a 1 Gbps connection, the user's 20 Mbps access link limits the flow.

17. INSTANTANEOUS vs AVERAGE THROUGHPUT

Instantaneous throughput

Transfer rate at a particular moment.

Average throughput

Total successfully delivered data divided by total time.

$$\boxed{\text{Average Throughput} = \frac{\text{Total delivered bits}}{\text{Total transfer time}}}$$

For GATE, always read which one the question is asking.

18. GOODPUT — USEFUL DISTINCTION

You may encounter the term **goodput**.

Throughput can count successfully transmitted protocol data, depending on definition/context.

Goodput specifically focuses on:

Useful application payload delivered per unit time.

Headers and retransmitted information do not contribute useful application payload.

So generally:

$$\text{Goodput} \leq \text{Throughput} \leq \text{Link Capacity}$$

under the common interpretation.

19. EXAMPLE — OVERHEAD AFFECTS GOODPUT

Suppose a link transmits:

$$1\text{Mbps}$$

Each packet contains:

- 800 bits payload
- 200 bits header

Total packet:

$$1000\text{bits}$$

Payload fraction:

$$\frac{800}{1000} = 0.8$$

If the link is fully utilized and there are no retransmissions:

$$\text{Goodput} = 1\text{Mbps} \times 0.8$$

$$\boxed{0.8\text{Mbps}}$$

20. CONNECTION TO ENCAPSULATION

Earlier we learned:

$$\text{Efficiency} = \frac{\text{Payload}}{\text{Payload} + \text{Overhead}}$$

So if raw throughput/link transmission rate is R :

$$\text{Useful payload rate}$$

may be estimated as:

$$R \times \text{Efficiency}$$

under simplified assumptions.

This is the connection between:

Encapsulation overhead ! "goodput"

21. THROUGHPUT vs LATENCY

Another common confusion:

A network may have:

- very high throughput
- but high latency

or:

- low throughput
- but low latency

They are different performance dimensions.

22. EXAMPLE

A satellite link might offer:

$$100\text{Mbps}$$

throughput/capacity but still have large propagation delay due to long distance.

So downloading a huge file may be fast once the pipeline is full, but interactive request-response communication can still feel delayed.

23. BANDWIDTH-DELAY PRODUCT REVISITED

We learned:

$$BDP = R \times d_{\text{prop}}$$

More commonly for end-to-end protocols such as TCP, we often care about:

$$BDP \approx \text{Throughput} \times RTT$$

depending on context.

This estimates how much data must be outstanding to fully utilize a path.

24. EXAMPLE

Path rate:

$$\backslash[50\text{Mbps} \backslash]$$

RTT:

$$\backslash[100\text{ms}=0.1\text{s} \backslash]$$

Then:

$$\backslash[\text{BDP}=50\times 10^6\times 0.1 \backslash]$$

$$\backslash[=5\times 10^6\text{bits} \backslash]$$

$$\backslash[=5\text{Mbits} \backslash]$$

In bytes:

$$\backslash[\frac{5\times 10^6}{8} = 625000\text{B} \backslash]$$

$$\backslash[\approx 625\text{KB} \backslash]$$

So roughly 625 KB of unacknowledged data may need to be in flight to fully utilize a 50 Mbps path with 100 ms RTT under the simplified model.

We'll use this idea again in TCP.

25. THROUGHPUT WITH STOP-AND-WAIT — PREVIEW

Suppose a sender sends one packet and then waits for its ACK before sending the next.

Even if the link has huge bandwidth, long RTT can drastically reduce throughput.

Very simplified:

$$\backslash[\text{Throughput} \approx \frac{\text{Packet Size}}{\text{Packet transmission time} + \text{RTT}} \backslash]$$

depending on assumptions.

This becomes important later in flow-control protocols.

For now, just understand:

High bandwidth alone does not guarantee high achieved throughput.

26. UTILIZATION

Utilization tells us what fraction of a resource's capacity is being used.

If:

$$\backslash[R=100\text{Mbps} \backslash]$$

but actual transmission rate is:

$$\backslash[60\text{Mbps} \backslash]$$

then:

$$\text{Utilization} = \frac{60}{100}$$

$$= 0.6$$

or:

$$\boxed{60\%}$$

General formula:

$$\boxed{\text{Utilization} = \frac{\text{Actual used rate}}{\text{Available capacity}}}$$

when that interpretation fits the problem.

27. GATE EXAMPLE — BOTTLENECK + FILE TIME

A file of:

$$250\text{MB}$$

travels through links of:

$$100\text{Mbps}, \quad 25\text{Mbps}, \quad 50\text{Mbps}$$

Assume ideal steady-state transfer and ignore startup/other delays.

Find:

1. Bottleneck rate
2. Maximum throughput
3. Approximate file transfer time

Step 1 — Bottleneck

$$\min(100, 25, 50) = 25\text{Mbps}$$

Step 2 — Throughput

$$\boxed{25\text{Mbps}}$$

Step 3 — Convert file

$$250\text{MB} = 250 \times 8 = 2000\text{Mb}$$

Step 4 — Time

$$\frac{2000\text{Mb}}{25\text{Mbps}}$$

$$= \boxed{80\text{s}}$$

28. & μIMPORTANT ASSUMPTION

That calculation assumes a large enough transfer and ideal steady-state conditions such that:

- pipeline startup delay is negligible
- no retransmissions
- no additional contention

- no protocol limitation
- throughput is bottleneck-limited

If GATE explicitly includes propagation, packetization, RTT, or protocol behavior, include them.

29. MULTIPLE FLOWS THROUGH A BOTTLENECK

Suppose:

TEXT

```
Flow A % % %
          % % % 20 Mbps bottleneck
Flow B % % %
```

If capacity is shared equally:

$\lfloor \text{Throughput}_A = \text{Throughput}_B = 10\text{Mbps} \rfloor$

But suppose Flow A's own access link supports only:

$\lfloor 6\text{Mbps} \rfloor$

Then A cannot use 10 Mbps.

Its throughput is at most:

$\lfloor 6\text{Mbps} \rfloor$

The remainder may potentially be available to other flows depending on assumptions.

30. GENERAL THINKING METHOD

Whenever you see a throughput question:

Step 1

List every possible rate limitation.

Example:

- source access rate
- intermediate links
- destination access rate
- shared-link allocation
- protocol/window limitation

Step 2

Find the smallest effective limit.

Step 3

Use that as the achievable throughput under the problem's assumptions.

31. NETWORK PERFORMANCE METRICS — HOW THEY CONNECT

You now know:

Transmission delay

$$\boxed{L/R}$$

Time to push a packet onto a link.

Propagation delay

$$\boxed{D/S}$$

Time for signals to travel.

Throughput

$$\boxed{\text{Data/Time}}$$

Rate of successful data delivery.

Bottleneck

$$\boxed{\min(R_i)}$$

Slowest link limits steady flow.

BDP

$$\boxed{\text{Rate} \times \text{Delay}}$$

Data that can be in flight.

These concepts are related, but not interchangeable.

32. VERY IMPORTANT CONCEPTUAL EXAMPLE

Suppose a 1000-byte packet is sent over a:

$$\boxed{1\text{Gbps}}$$

link that is 10,000 km long.

Increasing link rate from:

$$\boxed{1\text{Gbps} \rightarrow 10\text{Gbps}}$$

reduces:

Transmission delay

Yes.

Because:

$$\boxed{L/R}$$

becomes smaller.

Propagation delay

No.

Because:

$\lfloor D/S \rfloor$

doesn't contain $\lfloor R \rfloor$.

Therefore extremely fast bandwidth cannot eliminate physical propagation latency.

$\emptyset = 0\%$ Classic networking idea.

33. HOW GATE CAN ASK THIS

Pattern 1 — Identify bottleneck

Given several link rates.

Use:

$\lfloor \min(R_i) \rfloor$

Pattern 2 — File transfer time

$\lfloor \text{Time} = \frac{\text{File size}}{\text{Throughput}} \rfloor$

Pattern 3 — Shared bandwidth

Divide capacity according to the sharing condition.

Pattern 4 — Bandwidth vs throughput conceptual

Ask whether increasing one necessarily guarantees equal increase in the other.

Answer:

No, because other bottlenecks may exist.

Pattern 5 — Goodput

Payload fraction plus link rate.

Pattern 6 — Utilization

$\lfloor \text{Actual/Capacity} \rfloor$

Pattern 7 — BDP

$\lfloor \text{Rate} \times \text{Propagation delay} \rfloor$

or sometimes:

$\lfloor \text{Rate} \times \text{RTT} \rfloor$

depending on what the question defines.

34. & pGATE TRAPS

Trap 1

Bandwidth and throughput are not automatically equal.

Trap 2

End-to-end throughput isn't the average of link rates.

Use the bottleneck.

Trap 3

Do not use the fastest link.

Use:

$\min(R_i)$

Trap 4

MB and Mb differ by factor 8.

Trap 5

Do not blindly assume $1\text{MB}=2^{20}$ bytes.

In networking problems, decimal prefixes are commonly used unless the question specifies otherwise. Follow the definitions supplied by the question.

Trap 6

High link rate doesn't mean low propagation delay.

Trap 7

Throughput and goodput may differ because of:

- headers
 - retransmissions
 - control packets
-

35. MEMORY TRICKS

Bandwidth

Capacity

Throughput

Actually transferred

Bottleneck

Smallest rate wins

$$\boxed{\min(R_i)}$$

Transfer time

Work / Speed

$$\boxed{\text{Data/Throughput}}$$

' b NOTEBOOK VERSION

Bandwidth / Link Rate

Maximum transmission capability of a link.

Unit:

$$\boxed{\text{bps}}$$

Throughput

Actual successful data-transfer rate.

$$\boxed{\text{Throughput} = \frac{\text{Data delivered}}{\text{Time}}}$$

Bottleneck

For a simple path:

$$\boxed{R_b = \min(R_1, R_2, \dots, R_n)}$$

Ideal steady-state throughput:

$$\boxed{\text{Throughput} \approx R_b}$$

File Transfer

$$\boxed{\text{Time} = \frac{\text{File size (bits)}}{\text{Throughput (bits/s)}}}$$

Goodput

Useful application data delivered per second.

Typically:

$\backslash [\text{Goodput} \leq \text{Throughput}]$

Utilization

$\backslash [\boxed{ U = \frac{\text{Actual rate}}{\text{Capacity}} }]$

BDP

$\backslash [\boxed{ \text{BDP} = \text{Rate} \times \text{Delay} }]$

Represents data that can be in flight.

36. 5-MINUTE REVISION — PERFORMANCE METRICS

You now know three major performance ideas.

Delay

$\backslash [d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}]$

Transmission

$\backslash [\frac{L}{R}]$

Propagation

$\backslash [\frac{D}{S}]$

Store-and-forward

One packet, $\backslash (N)$ equal links:

$\backslash [N \frac{LR}{R}]$

$\backslash (P)$ packets:

$\backslash [(N+P-1) \frac{LR}{R}]$

Throughput

$\backslash [\frac{\text{Data}}{\text{Time}}]$

Bottleneck

$\backslash [\min(R_i)]$

BDP

$\backslash [\text{Rate} \times \text{Delay}]$

37. 30-SECOND REVISION

Remember this chain:

TEXT

L/R

!"

Transmission delay

D/S

!"

Propagation delay

Smallest link rate

!"

Bottleneck

Data / Time

!"

Throughput

Rate $\times$ Delay

!"

Bits in flight

ACTIVE RECALL — DON'T LOOK BACK

Q1

A path has link rates:

[100Mbps, 20Mbps, 50Mbps]

What is its ideal maximum steady-state throughput?

Q2

A:

[200MB]

file is transferred at:

[40Mbps]

Approximately how long does it take under ideal conditions?

Q3

A 60 Mbps bottleneck is equally shared among 5 flows.

What throughput does each flow receive under equal-sharing assumptions?

Q4

A link has capacity:

[100Mbps]

Actual achieved throughput:

[75Mbps]

Calculate utilization.

Q5 — Conceptual

Can a network have very high bandwidth but high latency?

Explain why in one sentence.

Q6

Each packet contains:

- 900 bytes payload
- 100 bytes overhead

If packets are sent at an aggregate frame/packet rate equivalent to:

\[10Mbps \]

with no losses, what is the useful payload goodput?

Q7 — GATE Trap

A path contains:

\[10Mbps, \ 100Mbps, \ 1Gbps \]

links.

Which value determines ideal steady-state end-to-end throughput?

A. 1 Gbps B. 100 Mbps C. 10 Mbps D. Average of all three

Q8

A path supports:

\[80Mbps \]

and has an RTT of:

\[50ms \]

Using $\text{rate} \times \text{RTT}$, find the amount of data in flight required to fill the path, in:

1. bits
 2. bytes
-

NEXT !' Circuit Switching vs Packet Switching vs Virtual Circuit Switching — Concepts, Differences & GATE Traps

USER

next

CHAPTER 3 — SWITCHING

TOPIC 1 — Circuit Switching vs Packet Switching vs Virtual Circuit Switching

TOPIC 1 — HIGH PRIORITY FOR GATE

This topic is concept-heavy and frequently appears through comparisons and tricky statements.

The three ideas are:

1. Circuit Switching
2. Packet Switching — Datagram
3. Virtual Circuit Switching

The easiest way to remember them is:

Circuit = Reserve first Datagram = Decide for every packet Virtual Circuit = Set path first, then send packets

1. WHAT IS SWITCHING?

Suppose Host A wants to communicate with Host B through intermediate switches/routers.

TEXT

A ---- S1 ---- S2 ---- S3 ---- B

There may be many possible paths.

A switching technique determines:

How network resources and intermediate devices are used to carry information from source to destination.

2. WHY DO WE NEED SWITCHING?

It is impractical to have a dedicated physical cable between every pair of devices.

For n devices, a fully connected network would require:

$$\frac{n(n-1)}{2}$$

links.

For large networks, this becomes enormous.

Instead, users share communication infrastructure through switches/routers.

PART A — CIRCUIT SWITCHING

3. WHAT IS CIRCUIT SWITCHING?

In circuit switching:

A dedicated communication path and required resources are reserved before actual data transmission begins.

Think of a traditional telephone call.

Before conversation:

TEXT

```
Setup connection
! "
Reserve resources
! "
Transfer data
! "
Release connection
```

4. THREE PHASES OF CIRCUIT SWITCHING

Phase 1 — Connection Setup

A path is established.

Example:

TEXT

```
A !' S 1 !' S 3 !' S 5 !' B
```

Resources may be reserved along this path.

Phase 2 — Data Transfer

Once setup finishes, data flows through the established circuit.

The same reserved path is used for the connection.

Phase 3 — Teardown

When communication finishes, the reserved resources are released.

So remember:

Setup !' Transfer !' Teardown

Ø> Mnemonic:

S-T-T

5. MAIN PROPERTY — RESOURCE RESERVATION

Suppose each link has:

1 Mbps

and circuit switching divides it among 4 users.

Each may receive:

$\frac{1\text{ Mbps}}{4} = 250\text{ kbps}$

during its circuit, depending on the multiplexing scheme.

This gives predictable bandwidth.

But there is a problem.

If a user is silent:

Their reserved capacity may remain unused.

This leads to inefficient utilization for bursty computer traffic.

6. REAL-LIFE ANALOGY

Imagine reserving an entire table in a restaurant for two hours.

Even if you spend 30 minutes away from the table:

Nobody else can use your reserved table.

Circuit switching behaves similarly.

7. ADVANTAGES OF CIRCUIT SWITCHING

Once the circuit is established:

- reserved bandwidth is available
- predictable performance
- little/no queueing caused by competing connections for the reserved circuit capacity
- packets/data generally follow the same established circuit
- useful for continuous traffic

Historically suitable for voice telephone systems.

8. DISADVANTAGES

- setup time before communication
- reserved capacity may be wasted during idle periods
- inefficient for bursty traffic
- call/setup may fail if sufficient resources are unavailable

9. CIRCUIT SWITCHING AND MULTIPLEXING

Two important ways to divide link resources are:

- **FDM — Frequency Division Multiplexing**
- **TDM — Time Division Multiplexing**

We only need the basic idea here.

10. FDM

Frequency spectrum is divided among users.

Conceptually:

TEXT

Frequency

%

% U s e r 4

% U s e r 3

% U s e r 2

% U s e r 1

% % % % % % % % % % %

Each connection gets a separate frequency band.

Memory:

FDM !' Frequency is divided.

11. TDM

Time is divided into repeating slots.

Example:

TEXT

| A | B | C | A | B | C | A | B | C |
T i m e !'

Each user gets specific time slots.

Memory:

TDM !' Time is divided.

12. SMALL NUMERICAL

A:

\[2Mbps \]

link uses TDM for 8 equal circuits.

Rate per circuit:

$\lfloor \frac{2\text{Mbps}}{8} \rfloor$

$\lfloor \boxed{250\text{kbps}} \rfloor$

PART B — PACKET SWITCHING

13. WHAT IS PACKET SWITCHING?

In packet switching:

Data is divided into packets, and network resources are shared dynamically among users.

Resources are generally not permanently reserved for one communication.

Suppose:

TEXT

User A packets: A1 A2 A3

User B packets: B1 B2

User C packets: C1

A shared link might transmit:

TEXT

A1 | B1 | A2 | C1 | B2 | A3

depending on packet arrivals and scheduling.

This is called:

Statistical multiplexing

14. WHY IS PACKET SWITCHING GOOD FOR COMPUTER NETWORKS?

Computer traffic is often bursty.

Example:

You browse a webpage.

For a moment:

Lots of data.

Then:

No data.

Then another click:

More data.

If bandwidth were reserved continuously, much of it would be wasted.

Packet switching allows unused capacity to be used by other users.

15. STATISTICAL MULTIPLEXING

Unlike fixed TDM slots:

Users transmit when they actually have packets.

This improves efficiency.

But there is a cost:

If many users send simultaneously:

TEXT

P a c k e t s ! ' R o u t e r ! ' Q u e u e

then:

- queueing occurs
- delay varies
- packets may be dropped if buffer fills

So packet switching offers efficiency at the cost of less predictable delay.

16. PACKET SWITCHING — DATAGRAM APPROACH

The Internet's IP layer fundamentally uses the:

Datagram model

In a datagram network:

Each packet is treated independently.

Packets belonging to the same communication may theoretically take different paths.

Example:

TEXT

A ! ' R 1 ! ' R 2 ! ' R 4 ! ' ! ' B
! ' R 3 ! ' R 5 ! '

Packet 1 might use:

TEXT

A !' R 1 !' R 2 !' R 4 !' B

Packet 2 might use:

TEXT

A !' R 1 !' R 3 !' R 5 !' B

depending on routing conditions.

17. DATAGRAM CHARACTERISTICS

Important properties:

- no connection setup required at network layer
- each packet carries destination addressing information
- packets are independently forwarded
- routing decision can effectively be made packet-by-packet
- packets may follow different paths
- packets may arrive out of order
- network does not reserve a dedicated end-to-end circuit

Ø=important.

18. WHY CAN PACKETS ARRIVE OUT OF ORDER?

Suppose:

Packet 1 chooses a congested route.

Packet 2 chooses a faster route.

Then:

TEXT

Send:

P 1 !' P 2

Receive:

P 2 !' P 1

This is possible in a datagram network.

Higher layers such as TCP can handle ordering when reliable ordered delivery is required.

PART C — VIRTUAL CIRCUIT SWITCHING

19. WHAT IS A VIRTUAL CIRCUIT?

Virtual circuit switching combines ideas from both circuit and packet switching.

Before data transmission:

A logical path is established.

Then packets follow that path.

But unlike classic circuit switching:

A dedicated physical circuit with permanently reserved bandwidth is not necessarily created.

This is why it is called:

Virtual circuit.

20. BASIC OPERATION

Three phases again:

1. Setup

Choose logical route.

TEXT

A !' S 1 !' S 3 !' S 5 !' B

Intermediate switches create forwarding-table entries.

2. Data transfer

Packets follow that established logical route.

3. Teardown

Virtual circuit state is removed.

21. VIRTUAL CIRCUIT IDENTIFIER

Instead of carrying a complete destination address for every forwarding decision, packets can carry a shorter:

VC identifier / label

The switch uses the VC identifier to determine:

- outgoing link
- outgoing VC identifier

22. VC TABLE EXAMPLE

Suppose switch S receives:

TEXT

```
Incoming interface = 1
VC ID = 12
```

Its table may say:

Incoming interface	Incoming VC	Outgoing interface	Outgoing VC
1	12	3	7

Meaning:

TEXT

```
(Interface 1, VC 12)
!
Switch
!
(Interface 3, VC 7)
```

Notice:

VC identifier can change from one link to another.

Ø=Important GATE point.

23. WHY CAN VC NUMBER CHANGE?

A VC identifier generally needs to be unique only on a particular link/interface context.

So:

TEXT

```
A --VC 5--> S1 --VC 11--> S2 --VC 4--> B
```

can represent the same end-to-end virtual circuit.

Do not assume one VC number is globally fixed across the entire path.

24. DATAGRAM vs VIRTUAL CIRCUIT — CORE DIFFERENCE

Datagram

Every packet is independently routed/forwarded.

Virtual Circuit

A route is established first; packets then follow that logical path.

Memory:

Datagram !' Decide repeatedly

25. CIRCUIT vs VIRTUAL CIRCUIT

These sound similar but are NOT the same.

Circuit switching

Typically reserves dedicated resources.

Virtual circuit

Establishes logical connection/state, but resource reservation is not inherently the same as dedicated circuit reservation.

This is one of the most important distinctions.

26. MASTER COMPARISON TABLE

Feature	Circuit Switching	Datagram Packet Switching	Virtual Circuit
Setup required?	Yes	Usually no	Yes
Dedicated path/resources?	Typically yes	No	Logical path; reservation depends on design
Packets independent?	Not the main packet model	Yes	No, belong to VC
Same route for communication?	Yes	Not guaranteed	Yes while VC exists
Full destination info per packet?	Not required in same way after setup	Typically yes	Usually VC identifier
Router/switch maintains connection state?	Circuit state	Usually no per-flow VC state	Yes
Queueing possible?	Less due to reserved capacity, depending on system	Yes	Yes unless resources are reserved
Out-of-order arrival?	Normally no due to same circuit	Possible	Normally same path preserves route order, though other factors can matter
Best suited conceptually for	Continuous traffic	Bursty data	Connection-oriented packet transport

27. CONNECTION-ORIENTED vs CONNECTIONLESS

These two words are useful.

Connection-oriented

Communication requires setup before data.

Examples conceptually:

- circuit switching
- virtual circuit network
- TCP at transport layer

Connectionless

No prior connection setup for each communication.

Example:

- IP datagram network

28. & bGATE TRAP — TCP vs VIRTUAL CIRCUIT

TCP is:

Connection-oriented at the transport layer

But IP is:

Connectionless datagram service at the network layer

So the Internet can have:

TEXT

TCP

!' c o n n e c t i o n - o r i e n t e d t r a n s p o r t

on top of

IP

!' c o n n e c t i o n l e s s n e t w o r k l a y e r

Do not conclude:

"Because TCP is connection-oriented, IP must also use virtual circuits."

Wrong.

Very important.

29. CIRCUIT SWITCHING DELAY

Suppose:

- setup delay = $\backslash(T_s\backslash)$
- data transmission time = $\backslash(T_d\backslash)$
- propagation delay = $\backslash(T_p\backslash)$

Simplified total:

$$T_{\text{total}} = T_s + T_d + T_p$$

depending on how the question defines these quantities.

Once the circuit is established, data can often flow continuously without per-packet store-and-forward delays at every intermediate switch in the same way as packet switching.

30. PACKET SWITCHING DELAY

Packet switching may involve at each hop:

- transmission delay
- propagation delay
- processing
- queueing
- store-and-forward

So delay can vary.

This variability is called:

Jitter when referring to variation in packet delay.

You don't need deep study of jitter here, but understand the term.

31. WHEN IS CIRCUIT SWITCHING BETTER?

Conceptually useful when:

- traffic is continuous
- predictable rate is required
- resources can be reserved
- delay variation should be low

32. WHEN IS PACKET SWITCHING BETTER?

Useful when:

- traffic is bursty
- many users share network
- high resource utilization is important
- variable delay is acceptable

This is why packet switching is central to computer networks.

33. RESOURCE UTILIZATION EXAMPLE

Suppose there are 10 users.

Each user needs:

1 Mbps

when active.

But each is active only 10% of the time.

Circuit switching

To guarantee simultaneous service for all 10:

$10 \times 1 \text{ Mbps} = 10 \text{ Mbps}$

would be reserved.

But much of it may sit idle.

Packet switching

Because not every user is active at the same time, they can statistically share the link.

This allows more efficient utilization.

However:

If many become active simultaneously, queueing can occur.

This is the fundamental tradeoff.

34. PROBABILISTIC MULTIPLEXING — INTUITION

Packet switching effectively relies on:

Not everyone needing peak bandwidth simultaneously.

This is called:

Statistical multiplexing.

It gives efficiency but not the guaranteed capacity of circuit switching.

35. GATE-STYLE CONCEPTUAL QUESTION

Which switching technique is generally most efficient for bursty traffic?

A. Circuit switching B. Packet switching C. Dedicated TDM only D. Dedicated FDM only

Correct concept:

$\boxed{\text{Packet switching}}$

because resources are dynamically shared.

36. GATE-STYLE QUESTION — PATH

Suppose three packets belong to the same communication.

Which system can naturally allow:

TEXT

P 1 : A !' R 1 !' R 3 !' B

P 2 : A !' R 2 !' R 4 !' B

?

Answer:

Datagram packet switching.

In a virtual circuit, packets normally follow the pre-established VC route.

37. GATE-STYLE QUESTION — ROUTER STATE

Which network requires intermediate nodes to maintain per-connection VC state?

Virtual circuit network.

Datagram routers generally maintain forwarding/routing information, but not a separate VC entry for every end-to-end datagram flow.

38. DATAGRAM FORWARDING TABLE

A datagram router can have entries like:

Destination prefix	Outgoing interface
10.0.0.0/8	1
172.16.0.0/12	2
Default	3

It uses destination address/prefix to decide where the packet goes.

This will connect later to:

CIDR + longest-prefix matching.

39. VC FORWARDING TABLE

A VC switch might use:

TEXT

Incoming interface + Incoming VC

to obtain:

TEXT

Outgoing interface + Outgoing VC

That is fundamentally different from destination-address lookup.

40. FAILURE BEHAVIOR — CONCEPTUAL

Suppose a router/switch on a VC path fails.

The established VC may be disrupted and need to be re-established.

In a datagram network:

Routing can potentially adapt and later packets may take an alternate path once routing tables update.

This shows why datagram networking can be flexible.

41. ADDRESS INFORMATION

Another comparison:

Datagram

Each packet generally needs enough destination address information for independent forwarding.

Virtual Circuit

After setup, packets can carry a shorter VC identifier.

Circuit

Once physical/logical circuit is established, intermediate forwarding doesn't operate like independent destination-based packet routing.

42. FDM vs TDM — QUICK COMPARISON

FDM	TDM
Divide frequency	Divide time
Users have separate frequency bands	Users have separate time slots
Simultaneous frequency usage	Repeating turns in time
Traditional analog-style intuition	Digital time-slot intuition

Memory:

F !' Frequency T !' Time

43. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Direct comparison

Circuit vs packet switching.

Pattern 2 — Identify switching type

Statement:

A route is established before sending packets, and packets carry a short identifier.

Answer:

Virtual circuit.

Pattern 3 — Out-of-order packets

Most naturally possible in:

Datagram network.

Pattern 4 — Resource reservation

Strong characteristic of:

Circuit switching.

Pattern 5 — Bursty traffic

Better utilization:

Packet switching.

Pattern 6 — Router/switch state

Per-connection state:

VC switching.

Pattern 7 — FDM/TDM numerical

Given link bandwidth and number of equal circuits.

Find:

$$\left[\frac{\text{Total Rate}}{\text{Number of circuits}} \right]$$

44. & pGATE TRAPS

Trap 1

Virtual circuit "`` physical dedicated circuit.

Trap 2

A virtual circuit network still uses packets.

Trap 3

Datagram packets from the same flow do **not** have to follow the same route.

Trap 4

Circuit switching can waste bandwidth when users are idle.

Trap 5

Packet switching offers better statistical utilization but can suffer:

- queueing
 - variable delay
 - packet loss
-

Trap 6

TCP being connection-oriented does not mean the underlying IP network is virtual-circuit based.

Trap 7

"Connection-oriented" does not automatically imply:

dedicated physical resources.

TCP is connection-oriented but doesn't reserve a physical circuit.

Trap 8

VC identifiers may change at each hop.

Do not assume VC number is end-to-end globally constant.

45. MASTER MEMORY TRICK

Use:

C-D-V

Circuit

Claim resources first

Datagram

Decide each packet

Virtual Circuit

Verify/establish route first

Or even simpler:

TEXT

```
Circuit = RESERVE  
Datagram = ROUTE EACH  
VC      = SETUP PATH
```

46. STORY TO REMEMBER ALL THREE

Imagine travelling from Hyderabad to Delhi.

Circuit switching

You reserve an entire private road lane for your journey.

Dedicated resources.

Datagram

Every vehicle independently checks directions at every junction.

Different vehicles may choose different routes.

Independent packets.

Virtual Circuit

Before travelling, a specific route is decided:

TEXT

H y d e r a b a d ! ' N a g p u r ! ' B h o p a l ! ' D e l h i

Every vehicle follows that route.

But the roads are still shared with others.

Logical connection, shared infrastructure.

This analogy captures the core difference very well.

' p NOTEBOOK VERSION

Circuit Switching

Dedicated path/resources established before communication.

Phases:

TEXT

S e t u p ! ' D a t a T r a n s f e r ! ' T e a r d o w n

Properties:

- resource reservation
- predictable rate
- low queueing due to reserved capacity
- inefficient if user is idle
- good for continuous traffic

Multiplexing:

- FDM !' divide frequency
- TDM !' divide time

Datagram Packet Switching

Each packet treated independently.

Properties:

- no setup
- no dedicated resource reservation

- destination address carried in packets
- different packets can follow different paths
- packets may arrive out of order
- queueing/loss possible
- efficient for bursty traffic

Example:

IP

Virtual Circuit

Logical connection established before packets are sent.

TEXT

S e t u p ! ' T r a n s f e r ! ' T e a r d o w n

Properties:

- packets follow established route
- intermediate switches maintain VC state
- packet contains VC identifier
- VC ID may change per link
- uses packets
- dedicated physical resources are not necessarily reserved

Comparison Shortcut

TEXT

C i r c u i t ! ' R e s e r v e r e s o u r c e s
D a t a g r a m ! ' R o u t e e a c h p a c k e t
V C ! ' E s t a b l i s h r o u t e f i r s t

47. CHAPTER 3 — 5-MINUTE REVISION

Circuit

- setup required
- dedicated/reserved capacity
- predictable
- inefficient for bursty traffic

Packet/Datagram

- no setup
- dynamic resource sharing

- each packet independent
- variable delay
- possible packet loss
- efficient for bursty traffic

Virtual Circuit

- setup required
- logical path established
- all VC packets follow that route
- switches maintain connection state
- VC identifier used

FDM

Divide frequency.

TDM

Divide time.

48. 30-SECOND REVISION

Remember:

Circuit = reserve before sending.

Datagram = every packet independently finds its way.

VC = choose path before sending packets.

And:

Packet switching = efficient but variable.

Circuit switching = predictable but can waste resources.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Which switching technique reserves resources before actual data communication?

A. Datagram B. Circuit switching C. Pure packet switching only D. Ethernet

Q2

In which technique may packets belonging to the same communication naturally travel through different paths?

Q3

Why is packet switching generally better for bursty computer traffic than circuit switching?

Q4

A virtual circuit packet typically carries:

A. Only a complete global route B. A VC identifier C. No forwarding information whatsoever D. Only a MAC address

Q5 — GATE Trap

True or False:

A virtual circuit always means a dedicated physical communication channel is reserved.

Explain briefly.

Q6

A 12 Mbps circuit-switched link is divided equally using TDM among 6 users.

What transmission rate is allocated to each user?

Q7

Match correctly:

- FDM
- TDM
- Datagram
- Virtual Circuit

with:

- packets independently routed
 - frequency divided
 - logical path set up first
 - time divided
-

Q8 — GATE Thinking

TCP is connection-oriented, while IP is connectionless.

Is this contradictory? Explain in one or two sentences.

Q9

Which approach normally requires intermediate switches to store connection-specific VC table entries?

A. Datagram network B. Virtual circuit network C. Pure physical hub network D. Application layer only

Q10

Give one major advantage and one major disadvantage of:

1. Circuit switching
2. Datagram packet switching

NEXT !' Data Link Layer Fundamentals: Framing, Node-to-Node Delivery, MAC Addressing & Error-Control Overview

USER

next

ASSISTANT

CHAPTER 4 — DATA LINK LAYER FUNDAMENTALS

TOPIC 1 — Framing, Node-to-Node Delivery, MAC Addressing & Error-Control Overview

FOUNDATION FOR CRC, MAC AND ETHERNET

Your syllabus explicitly includes:

- Error detection
- Medium Access Control
- Ethernet

Before learning those, you need to understand what the **Data Link Layer** actually does.

1. WHERE IS THE DATA LINK LAYER?

In the 5-layer model:

TEXT

Application

Transport

Network

D a t a L i n k ! • W e a r e h e r e

Physical

Its PDU is:

Frame

Typical address associated with it:

MAC address

Its broad job:

Reliable/useful communication over a single link between directly connected nodes.

2. WHAT DOES “NODE-TO-NODE” MEAN?

Suppose:

TEXT

Host A ---- R1 ---- R2 ---- Host B

Transport-level communication may be conceptually:

TEXT

A ===== B

But Data Link communication occurs one link at a time:

TEXT

A !” R 1
R 1 !” R 2
R 2 !” B

That is why Data Link is often described as:

Node-to-node / hop-to-hop delivery

3. IMPORTANT DISTINCTION

Network Layer

Concerned with getting a packet across the whole internetwork.

TEXT

Source ===== Destination

Data Link Layer

Concerned with getting the frame across the **current link**.

TEXT

C u r r e n t n o d e !” N e x t n o d e

Memory:

IP thinks destination. MAC thinks next hop.

This will become very important later.

4. WHY DO WE NEED THE DATALINK LAYER?

The physical layer only sends:

Raw bits.

Example:

TEXT

101101001011101...

But just receiving a continuous stream of bits is not enough.

The receiver needs to know:

- where a message starts
- where it ends
- who the local sender is
- who the local receiver is
- whether bits were corrupted
- who is allowed to use a shared medium

The Data Link layer provides this structure.

5. MAIN FUNCTIONS OF DATALINK LAYER

For your GATE syllabus, remember these four especially:

1. Framing

Break bit stream into identifiable frames.

2. Physical / MAC addressing

Identify sender and receiver on a local link.

3. Error detection

Detect corrupted frames.

4. Medium Access Control

Decide who may transmit on a shared medium.

These map directly to your syllabus.

6. FRAMING

What is framing?

Framing is the process of organizing a stream of bits into identifiable blocks called frames.

Physical layer might deliver:

TEXT

101100101010011101001...

Without structure, receiver doesn't know boundaries.

Data Link converts conceptually into:

TEXT

[Frame 1][Frame 2][Frame 3]

7. WHY ARE FRAME BOUNDARIES IMPORTANT?

Imagine receiving:

TEXT

HELLOWORLDNETWORK

Were the messages:

TEXT

HELLO | WORLD | NETWORK

OR:

TEXT

HELL | OWOR | LDNETWORK

?

Without boundaries, interpretation becomes difficult.

Networking has the same issue.

The receiver needs a way to identify:

Start and end of each frame.

8. FRAME STRUCTURE

A generic frame looks like:

```
+-----+-----+-----+
| Header | Payload | Trailer |
+-----+-----+-----+
```

- destination MAC
- source MAC
- control/type information

Usually contains a Network-layer packet.

[illegible]

Often contains error-detection information.

FCS — Frame Check Sequence

9. CONNECTION TO ENCAPSULATION

```

Application Data
    !
TCP Segment
    !
IP Packet
    !
Data-Link Frame
    !
Bits

```

```
IP packet
  !"
Add L2 header/trailer
  !"
Frame
```


So:

The Network-layer packet becomes the payload of the Data-Link frame.

10. BASIC FRAMING METHODS — PREREQUISITE OVERVIEW

Your official syllabus does not explicitly say "framing techniques," so we won't over-expand.

But you should understand the common ideas:

- Character/byte count
- Byte stuffing
- Bit stuffing

especially because bit-oriented frames and error-detection questions can reference them.

11. CHARACTER / BYTE COUNT

One idea is to place frame length in a header field.

Example:

```
TEXT  
[5][A][B][C][D]
```

The first field says how many bytes belong to the frame.

Problem

If the count field itself becomes corrupted:

Receiver may lose synchronization and misinterpret later frames.

So it isn't always robust by itself.

12. BYTE STUFFING

Suppose a special byte marks:

START/END of frame.

Problem:

What if the same special byte appears naturally inside the payload?

Solution:

Add an escape byte before special data bytes.

This is called **byte stuffing**.

13. SIMPLE EXAMPLE

Suppose:

```
TEXT
FLAG = F
ESC = E
```

Payload:

```
TEXT
A F B
```

Without stuffing:

```
TEXT
F A F B F
```

Receiver may think the middle **F** ends the frame.

So sender inserts escape:

```
TEXT
F A E F B F
```

Receiver removes the inserted escape during de-stuffing.

14. BIT STUFFING

0=1 are relevant to bit-oriented protocols.

Suppose frame boundaries are indicated using:

```
TEXT
01111110
```

If the same bit pattern appears in payload, it could be mistaken for a flag.

A common solution:

After five consecutive 1s in the data, insert a 0.

Example payload pattern:

```
TEXT
11111
```

Transmitted as:

TEXT

111110

Receiver removes the stuffed 0.

15. WHY FIVE 1s?

Because the flag:

TEXT

01111110

contains six consecutive 1s.

By inserting a 0 after every five consecutive data 1s, normal data cannot accidentally create the flag pattern.

16. & pGATE TRAP — STUFFED BIT IS NOT ORIGINAL DATA

Sender:

Adds stuffed bit.

Receiver:

Removes stuffed bit.

Stuffed bits are transmission overhead, not application data.

17. MAC ADDRESSING

Now we move to another major Data-Link function.

MAC stands for:

Media Access Control

A MAC address identifies a network interface at the Data Link layer.

A common Ethernet MAC address is:

[48\text{ bits}]

or:

[6\text{ bytes}]

Example notation:

TEXT

3C:52:82:AB:14:90

18. MAC vs IP — CRITICAL DIFFERENCE

IP Address

Used at:

Network layer

Helps deliver across multiple networks.

MAC Address

Used at:

Data Link layer

Helps deliver across the current local link.

19. EXAMPLE — REMOTE COMMUNICATION

Suppose:

TEXT

Host A ---- Router R ---- Internet ---- Host B

Host A wants to send an IP packet to Host B.

The IP packet may contain:

TEXT

Source IP = Host A
Destination IP = Host B

But the first Ethernet frame only needs to reach:

Router R, the next hop.

So conceptually:

TEXT

Frame:
Destination MAC = Router R's local-interface MAC

Inside frame:
IP packet:
Destination IP = Host B

Extremely important.

20. WHY DO MAC ADDRESSES CHANGE HOP BY HOP?

Consider:

TEXT

A ----- R1 ----- R2 ----- B

First link:

TEXT

A !' R 1

Frame might contain:

TEXT

Source MAC = MAC_A
Destination MAC = MAC_R1

At R1, that frame ends.

R1 extracts the IP packet.

For next link:

TEXT

R 1 !' R 2

R1 creates a **new frame**:

TEXT

Source MAC = MAC_R1
Destination MAC = MAC_R2

The Layer-2 addressing changes.

21. WHAT ABOUT IP ADDRESSES?

In ordinary forwarding, the source/destination IP addresses generally identify the original endpoints and are not replaced at every router.

Conceptually:

TEXT

A !' R 1 !' R 2 !' B

IP:

Source = A
Destination = B

while:

TEXT

MAC :

A! ' R 1 : A / R 1
R 1! ' R 2 : R 1 / R 2
R 2! ' B : R 2 / B

There are exceptions such as NAT affecting IP-layer addressing, which we'll study later.

22. MEMORY TRICK

IP = End goal MAC = Next step

Or:

IP travels conceptually end-to-end. MAC works hop-to-hop.

23. UNICAST, BROADCAST, MULTICAST — SMALL FOUNDATION

A Data-Link destination can conceptually address:

Unicast

One interface.

Broadcast

All nodes in the local broadcast domain.

Multicast

A selected group.

For Ethernet, the broadcast MAC address is:

TEXT

FF:FF:FF:FF:FF:FF

This is a common fact worth remembering.

24. ERROR DETECTION

Physical transmission is imperfect.

Bits can change.

Example sent:

TEXT

1011001

Received:

TEXT

1010001

One bit has been corrupted.

Data Link protocols can include redundancy so that the receiver can detect many such errors.

25. ERROR DETECTION "ERROR CORRECTION

is important.

Error Detection

Receiver can determine:

Something is wrong.

Error Correction

Receiver can determine enough information to:

Recover/correct the original data.

Your syllabus specifically says:

error detection

So our main focus will be:

- Parity
- Checksum
- CRC

with CRC being especially important.

26. TYPES OF ERRORS

Single-bit error

Only one bit changes.

Example:

TEXT

Sent: 101101

Received: 101001

!'

- checksum
- CRC

29. WHY ADD REDUNDANT BITS?

Without redundancy, consider:

TEXT

Original = 1010
Received = 1110

The receiver has no way to know automatically whether 1110 is legitimate or corrupted.

Redundant information creates constraints.

If received bits violate those constraints:

corruption can be detected.

30. MEDIUM ACCESS CONTROL — WHY DO WE NEED IT?

Suppose several devices share one communication medium:

TEXT

```
A % %
B % %<% % % %   S h a r e d   m e d i u m
C % %$
D % %
```

What happens if A and B transmit at the same time?

Their transmissions may interfere/collide in some shared-medium technologies.

We need rules deciding:

Who can transmit, and when?

That is:

Medium Access Control

31. MAC SUBLAYER

Traditionally, Data Link can conceptually be divided into:

TEXT

```
Data Link
%
% % %   L L C
%
```

For your GATE syllabus, MAC is especially important because it covers:

- shared-medium access
- ALOHA
- CSMA
- CSMA/CD
- Ethernet

We don't need deep LLC study unless required for context.

32. SHARED MEDIUM EXAMPLE

Imagine everyone in a room trying to speak.

If two people speak simultaneously:

Their speech overlaps.

A communication rule could be:

- speak whenever you want
- listen before speaking
- if collision occurs, stop and retry

These roughly resemble ideas we will see in:

- ALOHA
- CSMA
- CSMA/CD

33. DATALINK LAYER AND ETHERNET

Ethernet is one of the most important Data-Link technologies.

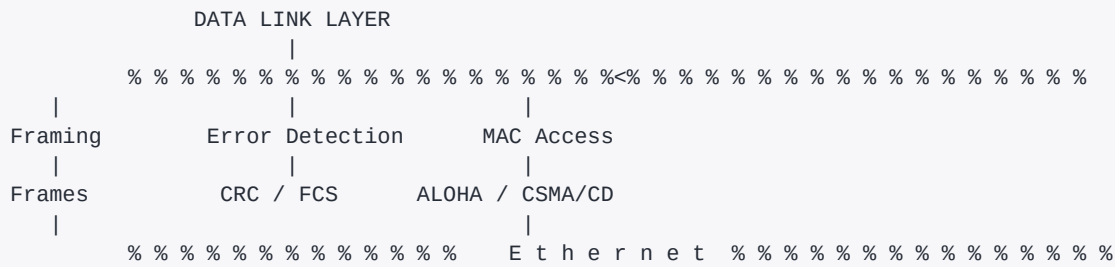
Ethernet specifies things such as:

- frame format
- MAC addressing
- medium access behaviour in shared Ethernet
- error detection using CRC/FCS
- physical-layer variants

So several parts of your syllabus eventually meet inside Ethernet.

34. MASTER CONNECTION MAP

TEXT



This is the mental map to remember.

35. FRAME vs PACKET vs SEGMENT

Another frequent confusion:

TEXT

```
Transport
  |
Segment

Network
  |
Packet / Datagram

Data Link
  |
Frame

Physical
  |
Bits
```

A frame typically **encapsulates** a network-layer packet.

36. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Identify layer responsibility

Which layer handles framing?

Data Link.

Pattern 2 — Hop-to-hop vs end-to-end

They may ask whether MAC addressing is end-to-end.

Answer:

Generally hop-to-hop/local-link.

Pattern 3 — MAC/IP changes

Given several routers, determine which addresses change at each hop.

Pattern 4 — Framing/stuffing

Calculate how many stuffed bits are inserted.

Pattern 5 — Burst-error length

Find distance from first corrupted bit to last corrupted bit.

Pattern 6 — Error detection vs correction

Choose which mechanism only detects errors.

Pattern 7 — Shared medium

Ask why MAC protocols are required.

37. GATE-STYLE BIT-STUFFING EXAMPLE

Suppose data is:

TEXT

01111110111110

Using the rule:

Insert 0 after every five consecutive 1s.

Take the first run:

TEXT

011111...

After five 1s:

TEXT

0111110...

Then continue counting consecutive 1s from zero again.

The important exam method is:

1. Scan left to right.
2. Count consecutive 1s.
3. Immediately after fifth 1, insert 0.
4. Reset count.

& Do not treat inserted 0 as part of original data.

38. ADDRESSING GATE EXAMPLE

Suppose:

TEXT

A ---- R ---- B

A and B are on different networks.

A sends to B.

On link A! R :

Destination IP

\[IP_B \]

Destination MAC

\[MAC_R \]

not:

\[MAC_B \]

because the frame needs to reach the next hop, while the IP packet is addressed to the final host.

Very important.

39. & pGATE TRAPS

Trap 1

Data Link does **not** perform end-to-end Internet routing.

That is Network-layer work.

Trap 2

MAC address and IP address do not serve the same purpose.

Trap 3

A remote host's MAC address is normally not used directly in the sender's first frame across routers.

Trap 4

Frames are local-link units.

Routers normally remove the incoming frame and create a new outgoing frame.

Trap 5

Error detection doesn't mean error correction.

Trap 6

Burst error length is not simply:

number of flipped bits.

It is:

$$\lfloor \boxed{\text{last error position} - \text{first error position} + 1} \rfloor$$

Trap 7

Byte/bit stuffing is not error detection.

It is a **framing/transparency technique**.

Trap 8

MAC has two related meanings you'll see:

- **MAC address**
- **Medium Access Control**

Do not confuse them.

They are related to the same Data-Link area but are not the same object.

40. MEMORY SHORTCUTS

Main Data-Link responsibilities

Use:

FAME

- **F**raming
- **A**ccess control
- **M**AC addressing
- **E**rror detection

That's a very useful mnemonic.

Addressing

IP = final destination idea MAC = next-hop/local delivery

Error concepts

Detect = know it's wrong Correct = know how to fix it

Bit stuffing

Five 1s !' Stuff 0

' p NOTEBOOK VERSION

Data Link Layer

Main responsibility:

Node-to-node / hop-to-hop communication over a link.

PDU:

Frame

Main functions — **FAME**

- Framing
- Access control
- MAC addressing
- Error detection

Framing

Organizes bit stream into frames.

Generic frame:

TEXT

Header | Payload | Trailer

Network-layer packet becomes Data-Link payload.

Framing techniques may include:

- byte count
- byte stuffing

- bit stuffing

Bit stuffing:

Insert 0 after five consecutive 1s in applicable bit-oriented framing.

MAC vs IP

TEXT

```
MAC !' Data Link !' local / next - hop
IP !' Network !' end destination across networks
```

Across routers:

MAC addresses generally change hop-by-hop.

Error Types

Single-bit error

One bit altered.

Burst error

Burst length:

$\lfloor \boxed{\text{last corrupted} \dots \text{position-first corrupted} \dots \text{position}+1} \rfloor$

Error Detection

Adds redundancy so receiver can detect corruption.

Important methods:

- Parity
- Checksum
- CRC

MAC — Medium Access Control

Controls access to a shared medium.

Important upcoming protocols:

- ALOHA
- CSMA
- CSMA/CD

41. 30-SECOND REVISION

Remember:

TEXT

Data Link = FRAME

and:

FAME

Framing Access control MAC addressing Error detection

Then:

MAC = local hop IP = network destination

And:

Five 1s !' stuff 0

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the PDU of the Data Link layer?

A. Segment B. Packet C. Frame D. Bit only

Q2

Why is framing required?

Answer in one sentence.

Q3

Which address is primarily used for forwarding over a local Ethernet link?

A. Port B. MAC C. IP only D. Domain name

Q4 — GATE Thinking

Suppose:

TEXT

A ---- R1 ---- R2 ---- B

How many different Data-Link frame transmissions occur for one IP packet travelling from A to B?

Q5

If the first corrupted bit is at position 4 and the last corrupted bit is at position 11, what is the burst-error length?

Q6

True or False:

A burst error of length 8 means exactly 8 bits must have changed.

Explain.

Q7

What is the difference between:

- error detection
- error correction?

Q8 — Address Trap

Host A is sending an IP packet to Host B through router R.

On the first link A → R what is the usual:

1. destination IP address?
2. destination MAC address?

Q9

Which one is a framing technique rather than an error-detection technique?

A. CRC B. Checksum C. Bit stuffing D. Parity

NEXT !' Error Detection Part 1: Parity, Two-Dimensional Parity & Checksum

USER

next

ASSISTANT

CHAPTER 5 — ERROR DETECTION

TOPIC 1 — Parity, Two-Dimensional Parity & Checksum

0-HIGH PRIORITY FOUNDATION FOR CRC

Error detection is about adding **redundant information** so the receiver can determine whether transmitted data was corrupted.

Today we will cover:

1. Single parity
2. Two-dimensional parity
3. Checksum

Then the next topic will be the most important one:

CRC

1. BASIC IDEA OF ERROR DETECTION

Suppose sender wants to transmit:

```
TEXT
1011001
```

If the receiver simply gets:

```
TEXT
1010001
```

how does it know whether an error occurred?

It cannot, unless some redundancy is added.

So sender transforms:

```
TEXT
Original data
      !"
Add checking information
      !"
Data + redundant bits
```

Receiver then checks whether the received pattern satisfies the expected rule.

PART A — SINGLE PARITY

2. WHAT IS A PARITY BIT?

A **parity bit** is one extra bit added to data so that the total number of **1**s follows a chosen rule.

Two types:

1. Even parity
2. Odd parity

3. EVEN PARITY

With even parity:

Total number of 1 s, including the parity bit, must be even.

Example:

Data:

```
TEXT
1011001
```

Count number of 1 s:

$\sqrt[4]{}$

Already even.

Therefore parity bit:

$\boxed{0}$

Transmitted code:

```
TEXT
10110010
```

depending on where parity bit is placed.

4. ANOTHER EVEN PARITY EXAMPLE

Data:

```
TEXT
1011011
```

Number of 1 s:

$\sqrt[5]{}$

Odd.

To make total number even, add:

$\sqrt[1]{}$

So parity bit:

$\boxed{1}$

Total 1s:

$\sqrt[6]{}$

even.

5. ODD PARITY

With odd parity:

Total number of 1 s, including parity bit, must be odd.

Suppose data:

TEXT
1011001

It contains:

\[4 \]

ones.

To make total odd:

\[Parity=1 \]

Now total:

\[5 \]

ones.

6. MEMORY TRICK

Don't overcomplicate parity.

For **even parity**:

If number of 1s is already even !' add 0 If number of 1s is odd !' add 1

For **odd parity**:

If number of 1s is already odd !' add 0 If number of 1s is even !' add 1

7. WHAT ERRORS CAN SINGLE PARITY DETECT?

This is the important GATE concept.

Suppose even parity was used.

If exactly **one bit** changes, parity changes from:

- even !' odd

So the receiver detects an error.

Therefore:

Single parity detects all single-bit errors.

8. WHAT ABOUT TWO BIT ERRORS?

Suppose two bits change.

Example:

TEXT

Original: 101100

Received: 011100

Two bits changed.

The total parity may remain even.

Therefore:

Simple parity may fail to detect an even number of bit errors.

9. GENERAL RULE

A single parity bit detects:

$\boxed{\text{any odd number of bit errors}}$

Why?

Each bit flip changes parity.

An odd number of flips reverses parity.

An even number of flips restores the original parity.

Not important.

10. WHAT CAN'T PARITY TELL YOU?

Parity can tell:

Something is wrong.

But generally cannot tell:

- which bit is wrong
- how to repair it

So basic parity is:

Error detection, not general error correction.

11. GATE-STYLE PARITY QUESTION

Data:

TEXT

11010110

Use even parity.

Number of 1s:

$\boxed{5}$

Therefore parity bit:

$\boxed{1}$

because:

$5+1=6$

which is even.

12. & pGATE TRAP

If question says:

even parity

don't make the **parity bit** itself necessarily even.

The rule applies to:

Total number of 1s in the whole codeword.

PART B — TWO-DIMENSIONAL PARITY

13. WHY DO WE NEED SOMETHING BETTER?

Simple parity is weak.

If two bits flip:

Error may go undetected.

A stronger approach is:

Two-Dimensional Parity

Data bits are arranged as a matrix.

Parity is calculated:

- for each row
- for each column

14. EXAMPLE MATRIX

Suppose data is:

TEXT

```
1 0 1 1
0 1 1 0
1 1 0 0
```

Using even parity, add one parity bit to every row.

Row 1

TEXT

```
1 0 1 1
```

Number of 1s:

\[3 \]

Need parity:

\[1 \]

Result:

TEXT

```
1 0 1 1 | 1
```

Row 2

TEXT

```
0 1 1 0
```

Number of 1s:

\[2 \]

Parity:

\[0 \]

Row 3

TEXT

```
1 1 0 0
```

Number of 1s:

\[2 \]

Parity:

\[0 \]

So:

TEXT

```
1 0 1 1 | 1
0 1 1 0 | 0
1 1 0 0 | 0
```

Then column parity is also computed.

15. WHY TWO DIMENSIONS HELP

Suppose one data bit flips.

Then:

- its row parity becomes wrong
- its column parity becomes wrong

So the intersection tells us which bit changed.

Conceptually:

TEXT

```
W r o n g   r o w       ! '   R o w   2
W r o n g   c o l u m n ! '   C o l u m n   3
```

Therefore error is at:

TEXT

(Row 2, Column 3)

This gives stronger detection capability.

16. CAN 2-D PARITY CORRECT A SINGLE BIT?

With the standard rectangular parity arrangement:

A single-bit error can often be located using the failed row and column.

Thus it can support correction of a single-bit error.

But your GATE syllabus specifically emphasizes error detection, so focus on its detection power.

17. WHAT ERRORS CAN 2-D PARITY MISS?

It is much stronger than simple parity, but not perfect.

For example, if four bits forming corners of a rectangle change:

TEXT

```
X . X
. . .
X . X
```

each affected row may have two flips.

Each affected column may also have two flips.

Thus all row/column parities can remain valid.

& pVery useful conceptual trap.

18. MEMORY

Single parity:

Check one direction.

2-D parity:

Check rows + columns.

PART C — CHECKSUM

19. WHAT IS A CHECKSUM?

Checksum is another error-detection technique.

The basic idea:

1. Divide data into fixed-size words.
2. Add the words.
3. Use the result to generate a checksum.
4. Send data + checksum.
5. Receiver performs the same arithmetic and checks the result.

For Internet-style checksum problems, the arithmetic is usually:

1's-complement addition

20. WHAT IS 1's COMPLEMENT?

Take every bit and invert it:

TEXT

0	!	1
1	!	0

Example:

TEXT

10110010

1's complement:

```
TEXT
01001101
```

21. WHAT IS 1's-COMPLEMENT ADDITION?

Suppose we add fixed-width words.

If an addition generates a carry beyond the most significant bit:

Wrap the carry around and add it to the least significant bit.

This is called:

End-around carry

Very important.

22. SIMPLE 4-BIT EXAMPLE

Add:

```
TEXT
1011
0110
```

Normal binary addition:

```
TEXT
  1011
+ 0110
-----
1 0001
```

We are working with 4-bit words.

The leftmost carry:

```
TEXT
1
```

must be wrapped around:

```
TEXT
0001
+   1
-----
0010
```

So 1's-complement sum:

$\boxed{0010}$

23. GENERATING CHECKSUM

Suppose sender has words:

TEXT

W1

W2

W3

Step 1:

Add using 1's-complement addition.

$S = W_1 + W_2 + W_3$

Step 2:

Take 1's complement.

$\text{Checksum} = \overline{S}$

Then transmit:

TEXT

W1 | W2 | W3 | Checksum

24. RECEIVER CHECK

Receiver adds:

TEXT

W1 + W2 + W3 + Checksum

using 1's-complement addition.

If no detectable error has occurred, the result should be:

TEXT

111...111

all ones.

Equivalent idea:

Complement of the final sum should be all zeros.

25. COMPLETE SMALL EXAMPLE

Use 4-bit words:

TEXT

W1 = 1001
W2 = 1100

We need to calculate checksum.

Step 1 — Add

TEXT

```
  1001
+ 1100
-----
1 0101
```

There is carry 1.

End-around carry:

TEXT

```
0101
+001
----
0110
```

So sum:

TEXT

0110

Step 2 — Complement

TEXT

0110

becomes:

TEXT

1001

Therefore:

$\boxed{\text{Checksum}=1001}$

26. RECEIVER VERIFICATION

Receiver has:

TEXT

1001
1100
1001

First:

```
TEXT
1001 + 1100
```

gave:

```
TEXT
0110
```

Now:

```
TEXT
0110
+1001
----
1111
```

All ones.

Therefore:

Accepted as no error detected.

27. WHY ALL ONES?

Because checksum is the complement of the data sum.

Conceptually:

$\lceil S + \overline{S} = 111\dots111 \rceil$

in 1's-complement arithmetic.

That's the core property.

28. CHECKSUM ALGORITHM — SENDER

Write this in notes:

Sender

1. Divide message into equal-sized words.
2. Add all words using 1's-complement addition.
3. Perform end-around carry.
4. Complement the sum.
5. Send checksum with data.

29. CHECKSUM ALGORITHM — RECEIVER

Receiver

1. Add all received data words + checksum.
2. Use 1's-complement addition.
3. If result is all 1s !' no error detected.
4. Otherwise !' error detected.

30. & µGATE TRAP — DON'T DISCARD THE CARRY

In ordinary fixed-width binary arithmetic, you may sometimes ignore overflow.

Not here.

In 1's-complement addition:

Carry must be wrapped around.

Example:

```
TEXT
1 0101
```

becomes:

```
TEXT
0101 + 1
```

not simply:

```
TEXT
0101
```

31. ANOTHER EXAMPLE

Use 8-bit words:

```
TEXT
W1 = 11110000
W2 = 11001100
```

Add:

```
TEXT
  11110000
+ 11001100
-----
1 10111100
```

Wrap carry:

TEXT

```
10111100
+      1
-----
10111101
```

Complement:

TEXT

01000010

Checksum:

$\boxed{01000010}$

32. CHECKSUM vs PARITY

Feature	Parity	Checksum
Added redundancy	Usually 1 bit	One checksum word
Basic operation	Count parity of 1s	1's-complement addition
Complexity	Very low	Higher
Detects single-bit errors?	Yes	Generally yes
Detects all multiple-bit errors?	No	No
Strength	Weak	Better than simple parity in many cases
Common usage idea	Simple links/memory mechanisms	Internet transport/network protocols

33. CHECKSUM vs CRC — PREVIEW

CRC, which comes next, generally provides stronger structured detection for burst errors.

Very broadly:

Checksum

Uses:

Addition

CRC

Uses:

Polynomial / modulo-2 division

Memory:

Checksum = Add CRC = Divide using XOR

34. WHERE ARE CHECKSUMS USED?

Useful context:

Internet protocols have historically used checksums in places such as:

- IP header checksum in IPv4
- TCP checksum
- UDP checksum

For your syllabus, the most relevant later connection is:

TCP checksum/error-detection context

Do not confuse this with Ethernet FCS, which uses CRC.

35. VERY IMPORTANT DISTINCTION

Ethernet

Uses:

CRC-based Frame Check Sequence.

TCP

Uses:

Checksum.

So:

TEXT

E	t	h	e	r	n	e	t	!	'	C	R	C			
T	C	P				!	'	C	h	e	c	k	s	u	m

Useful later.

36. WHAT CAN A CHECKSUM MISS?

Checksum is not mathematically guaranteed to detect every possible error pattern.

Certain changes can compensate for each other and preserve the same checksum.

So never say:

Checksum detects all errors.

Wrong.

37. HAMMING DISTANCE — SMALL USEFUL FOUNDATION

A concept useful in error detection is:

Hamming distance

It means the number of bit positions in which two bit strings differ.

Example:

```
TEXT
101101
100001
```

Compare:

```
TEXT
1 = 1
0 = 0
1 " " 0 !• d i f f e r e n c e
1 " " 0 !• d i f f e r e n c e
0 = 0
1 = 1
```

Hamming distance:

$\boxed{2}$

38. WHY HAMMING DISTANCE MATTERS

If a coding scheme has minimum Hamming distance:

$d_{\min}$

then it can detect up to:

$\boxed{d_{\min}-1}$

bit errors.

And can correct up to:

$\boxed{\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor}$

errors.

This is a broader coding-theory idea.

We won't over-expand unless needed.

39. PARITY AND HAMMING DISTANCE

A simple parity code has minimum Hamming distance:

$\lfloor d_{\min} \rfloor = 2$

So guaranteed detectable errors:

$\lfloor 2-1 \rfloor = 1$

Thus all single-bit errors are detected.

This is consistent with what we learned.

40. HOW GATE CAN ASK PARITY

Pattern 1

Calculate parity bit.

Pattern 2

Ask which error counts are guaranteed detectable.

Single parity:

Any odd number of bit errors.

Pattern 3

Ask if two-bit error can pass undetected.

Yes.

41. HOW GATE CAN ASK 2-D PARITY

Pattern 1

Given rows/columns, locate a single error.

Pattern 2

Ask whether rectangular four-bit error can be undetected.

Pattern 3

Compare with simple parity.

42. HOW GATE CAN ASK CHECKSUM

~~0=Most~~ Most likely numerically.

They may give binary words and ask:

- sum
- checksum
- receiver result
- whether an error is detected

Always remember:

End-around carry

43. GATE SHORTCUT FOR CHECKSUM

Do not repeatedly convert to decimal unless that actually helps.

For short binary words:

1. Add binary directly.
2. Wrap carry.
3. Complement.

Keep the word size clearly fixed.

44. & pCHECKSUM TRAPS

Trap 1

Forget end-around carry.

Trap 2

Use 2's complement instead of 1's complement.

Wrong.

Trap 3

Complement each word individually before adding.

Wrong for normal checksum generation.

Trap 4

At receiver, expect all zeros before complementing.

Normally the direct sum should be:

TEXT

111 . . . 111

Trap 5

Change word width midway.

If words are 8-bit, keep 8-bit arithmetic.

45. ERROR-DETECTION STRENGTH — CONCEPTUAL ORDER

Very broadly for the techniques we're discussing:

TEXT

Simple parity
!"
2-D parity / checksum
!"
CRC

But don't interpret this as an absolute comparison for every possible error pattern.

For GATE purposes:

CRC is the major powerful structured error-detection technique you need to master.

46. MEMORY SHEET

Parity

Odd flips change parity. Even flips may hide.

Even parity:

TEXT

Total 1s = even

Odd parity:

TEXT

Total 1s = odd

2-D Parity

Rows + Columns

Single error gives:

one bad row + one bad column.

Checksum

Remember:

A-W-C

- **A**dd words
- **W**rap carry

- Complement

At receiver:

Sum everything !' all ones.

' p NOTEBOOK VERSION

1. Parity

One redundant bit is added.

Even parity

Total number of 1s must be even.

Odd parity

Total number of 1s must be odd.

Single parity detects:

$\boxed{\text{all odd numbers of bit errors}}$

Therefore all single-bit errors are detected.

It may fail for an even number of bit errors.

2. Two-Dimensional Parity

Arrange bits in matrix.

Add parity for:

- each row
- each column

A single-bit error causes:

- one incorrect row
- one incorrect column

Their intersection locates the error.

Can miss some patterns such as four errors at rectangle corners.

3. Checksum

Uses:

1's-complement addition.

Sender

1. Divide data into words.

2. Add words.
3. Wrap end carry.
4. Complement sum.
5. Send result as checksum.

Receiver

Add:

$\backslash [\text{Data} + \text{Checksum}]$

If final sum is:

TEXT

111 . . . 111

!' No error detected.

Memory:

Add !' Wrap !' Complement

1's Complement

Flip every bit:

TEXT

0 !" 1

Hamming Distance

Number of differing bit positions.

If minimum distance is $\backslash (d_{\min})$:

$\backslash [\boxed{\text{Detect}} \uparrow \text{ to } d_{\min}-1]$

$\backslash [\boxed{\text{Correct}} \uparrow \text{ to } \left\lfloor \frac{d_{\min}-1}{2} \right\rfloor]$

47. 30-SECOND REVISION

Remember:

TEXT

P a r i t y !' c o u n t 1 s
C h e c k s u m !' a d d w o r d s
C R C !' d i v i d e u s i n g X O R

Parity:

odd number of flips detected.

Checksum:

Add !' wrap carry !' complement.

Receiver checksum verification:

All ones = pass.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Data:

TEXT

10110110

Using **even parity**, what parity bit should be added?

Q2

Data has 6 ones.

Using **odd parity**, what parity bit is required?

Q3

Which error is simple parity guaranteed to detect?

A. Any two-bit error B. Any odd number of bit errors C. Every burst error D. Every error pattern

Q4 — GATE Trap

If exactly 4 bits change in a code protected only by one parity bit, is the error guaranteed to be detected?

Explain.

Q5

In two-dimensional parity, one row and one column fail parity checks.

What does their intersection indicate?

Q6

Calculate the 4-bit 1's-complement checksum of:

TEXT

W1 = 1010

W2 = 0111

Show:

1. binary addition
2. end-around carry
3. checksum

Q7

Why is this wrong for a checksum?

"If there is an overflow carry, simply discard it."

Q8

At the receiver, after adding all words and checksum using 1's-complement arithmetic, the result is:

TEXT

11111111

What does this mean?

A. Definitely no bit changed B. No error was detected C. Error must have occurred D. Checksum cannot be used

Q9 — Concept

What is the difference between simple parity and two-dimensional parity?

Q10 — Future Connection

Which mechanism is typically associated with Ethernet FCS?

A. Single parity B. Internet checksum C. CRC D. TDM

NEXT !' CRC (Cyclic Redundancy Check): Generator Polynomial, Modulo-2 Division, CRC Calculation & GATE Error-Detection Properties

USER

next

ASSISTANT

CHAPTER 5 — ERROR DETECTION

TOPIC 2 — CRC (Cyclic Redundancy Check)

Ø=Ý%OVERFLOW HIGH PRIORITY FOR GATE

CRC is one of the most important Data Link Layer numericals.

If you understand these five things, most GATE CRC questions become manageable:

1. Generator polynomial
2. Appending zeros
3. Modulo-2 division
4. CRC remainder
5. Error-detection properties

The most important rule:

CRC uses XOR, not normal subtraction.

1. WHAT IS CRC?

CRC stands for:

Cyclic Redundancy Check

It is an error-detection technique in which the sender treats the data as a binary polynomial and divides it by a predefined:

Generator polynomial

The remainder from this division becomes the CRC bits.

These CRC bits are appended to the original data.

2. WHY DO WE NEED CRC?

Parity is simple but weak.

Checksum is useful, but CRC is especially effective at detecting many common transmission errors, particularly:

burst errors

That's why Ethernet uses CRC in its Frame Check Sequence.

Remember:

TEXT

```
E t h e r n e t   ! '   C R C
T C P           ! '   C h e c k s u m
```

3. CORE IDEA

Suppose original data is:

$[D]$

Generator is:

$[G]$

Sender calculates some remainder:

$$\lfloor R \rfloor$$

and transmits:

$$\lfloor T = D \text{ followed by } R \rfloor$$

The crucial property is:

$$\boxed{T \bmod G = 0}$$

Here "mod" means modulo-2 polynomial division.

So the transmitted codeword is chosen to be **exactly divisible by the generator**.

4. WHAT IS MODULO-2 ARITHMETIC?

CRC does not use ordinary arithmetic.

It uses modulo-2 arithmetic.

The addition/subtraction operation is:

XOR

Remember:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

So:

TEXT

```
0 XOR 0 = 0
0 XOR 1 = 1
1 XOR 0 = 1
1 XOR 1 = 0
```

5. NO BORROW. NO CARRY.

Very important.

Normal subtraction might involve borrowing.

CRC division does not.

For example:

TEXT

```
1101
XOR
1011
----
0110
```

That's it.

No borrow.

No negative numbers.

6. MEMORY TRICK

For CRC:

Division looks normal, subtraction becomes XOR.

Or:

CRC = Copy division, Replace subtraction with XOR.

7. GENERATOR POLYNOMIAL

Suppose generator bits are:

TEXT

```
1011
```

This corresponds to:

$x^3 + x + 1$

Why?

Positions represent powers:

TEXT

```
1 0 1 1
!"  !"  !"  !"
x3 x2 x1 x0
```

Therefore:

$1x^3 + 0x^2 + 1x + 1$

$\boxed{x^3 + x + 1}$

8. POLYNOMIAL TO BINARY

Suppose:

$$[G(x)=x^4+x+1]$$

Write powers:

$$[x^4+x^3+x^2+x+1]$$

Coefficients:

TEXT

1 0 0 1 1

Therefore generator:

$$[\boxed{10011}]$$

9. & pGATE TRAP — MISSING POWERS

If:

$$[G(x)=x^4+x^2+1]$$

do not write:

TEXT

111

Wrong.

You must preserve positions:

$$[x^4+x^3+x^2+x^1+x^0]$$

Coefficients:

TEXT

1 0 1 0 1

So:

$$[\boxed{10101}]$$

10. DEGREE OF GENERATOR

If generator has:

$$[r+1]$$

bits, its polynomial degree is:

$$[r]$$

Example:

TEXT

Generator = 1011

has 4 bits.

Therefore:

$$\lfloor r=3 \rfloor$$

CRC remainder contains:

$$\lfloor \boxed{3 \text{ bits}} \rfloor$$

◊=Fundamental rule:

Generator degree = number of CRC bits.

11. WHY DO WE APPEND ZEROS?

Suppose data is:

TEXT

1101

Generator:

TEXT

1011

Generator degree:

$$\lfloor r=3 \rfloor$$

Before division, append:

$$\lfloor 3 \rfloor$$

zeros:

TEXT

1101 000

Why?

Because we're reserving 3 positions at the end for the CRC remainder.

12. SENDER ALGORITHM

Suppose:

- Data = $\lfloor D \rfloor$
- Generator degree = $\lfloor r \rfloor$

Then:

Step 1

Append $\lfloor r \rfloor$ zeros to data.

$$\lfloor D \times 2^r \rfloor$$

in binary interpretation.

Step 2

Divide the resulting bit string by generator using modulo-2 division.

Step 3

Take the r -bit remainder.

Step 4

Replace appended zeros with that remainder.

The result is the transmitted:

codeword

13. FORMULA

Conceptually:

$$T = D \cdot 2^r + R$$

where:

- D = data
- r = generator degree
- R = CRC remainder
- T = transmitted codeword

In modulo-2 arithmetic, addition corresponds to XOR.

14. FULL CRC EXAMPLE

Let's calculate carefully.

Data:

TEXT
1101

Generator:

TEXT
1011

Generator degree:

$$r=3$$

Append 3 zeros:

TEXT
1101000

Now divide:

TEXT

1101000 ÷ 1011

using XOR.

15. LONG DIVISION — STEP 1

Take first 4 bits because generator has 4 bits:

TEXT

1101

XOR with:

TEXT

1011

TEXT

1101

1011

0110

Bring down next bit from dividend (0):

TEXT

1100

We ignore the leading zero and continue based on the current 4-bit working portion.

16. STEP 2

Current:

TEXT

1100

XOR:

TEXT

1011

Result:

TEXT

0111

Bring down next 0:

TEXT

1110

17. STEP 3

Current:

TEXT

1110

XOR:

TEXT

1011

Result:

TEXT

0101

Bring down final 0 :

TEXT

1010

18. STEP 4

Current:

TEXT

1010

XOR:

TEXT

1011

Result:

TEXT

0001

Generator degree is 3, so remainder is the final 3 bits:

TEXT

001

Therefore:

$\boxed{\text{CRC}=001}$

19. TRANSMITTED CODEWORD

Original data:

TEXT

1101

CRC:

TEXT

001

Transmit:

TEXT

1101001

So:

$\boxed{T=1101001}$

20. RECEIVER CHECK

Receiver divides the **entire received codeword** by the same generator.

So:

TEXT

$1101001 \div 1011$

If received without a detectable error:

$\boxed{\text{Remainder}=000}$

Therefore:

No error detected.

21. VERY IMPORTANT RECEIVER RULE

Sender:

Append zeros !' divide !' attach remainder.

Receiver:

Do not append zeros again.

The receiver directly divides the received codeword by the generator.

& pGATE trap.

22. WHAT IF RECEIVER GETS NON-ZERO REMAINDER?

Suppose receiver division produces:

TEXT

101

Then:

$\boxed{\text{Error detected}}$

But if remainder is zero:

It means CRC did not detect an error.

It does **not** mathematically prove that no corruption occurred, because certain error patterns can go undetected.

23. & pIMPORTANT WORDING

Correct:

Zero remainder !' No error detected

Not necessarily:

Zero remainder !' absolutely no transmission error occurred.

This distinction matters.

24. AN EASIER CRC DIVISION METHOD

Many students get confused drawing formal long division.

Use this practical method.

Suppose dividend:

TEXT

1101000

Generator:

TEXT

1011

At every step:

If current leading bit = 1

XOR with generator.

If current leading bit = 0

Effectively XOR with:

```
TEXT
0000
```

then shift/bring down next bit.

This makes the procedure systematic.

25. GATE SHORTCUT — DEGREE FIRST

Whenever you see a CRC problem, first write:

```
TEXT
Generator length = ?
Degree = length - 1
CRC bits = degree
Zeros appended = degree
```

Example:

```
TEXT
Generator = 11001
```

Length:

$\lfloor 5 \rfloor$

Degree:

$\lfloor 4 \rfloor$

Therefore:

- append 4 zeros
- remainder has 4 bits

This single step prevents many mistakes.

26. EXAMPLE — POLYNOMIAL GENERATOR

Data:

```
TEXT
101101
```

Generator:

$\lfloor G(x)=x^3+x+1 \rfloor$

Convert generator:

$$\lfloor x^3 + x + 1 \rfloor$$

Coefficients:

TEXT

1011

So:

$$\lfloor r = 3 \rfloor$$

Append:

TEXT

101101000

Then perform XOR division by:

TEXT

1011

This is the correct setup.

Even if GATE doesn't ask you to finish the division, it may ask:

- number of appended zeros
- remainder length
- transmitted frame length

27. CODEWORD LENGTH

Suppose original data has:

$$\lfloor k \rfloor$$

bits.

Generator degree:

$$\lfloor r \rfloor$$

CRC adds $\lfloor r \rfloor$ bits.

Therefore transmitted codeword length:

$$\lfloor \boxed{k+r} \rfloor$$

Example:

Data:

$$\lfloor 1000 \text{ bits} \rfloor$$

Generator degree:

$$\lfloor 32 \rfloor$$

Codeword:

$$\lfloor \boxed{1032 \text{ bits}} \rfloor$$

assuming a single CRC block and no other overhead.

28. WHY DOES CRC DETECT ERRORS?

Suppose correct transmitted codeword is:

$$[T]$$

and error pattern is:

$$[E]$$

Received word:

$$[T' = T \oplus E]$$

Since:

$$[T]$$

was constructed to be divisible by:

$$[G]$$

the receiver really detects whether:

$$[E]$$

is divisible by:

$$[G]$$

If error pattern is **not divisible** by generator:

nonzero remainder != detected.

If:

$$[E]$$

is divisible by generator:

remainder may be zero != undetected.

This is the mathematical heart of CRC.

29. ERROR POLYNOMIAL

Suppose transmitted:

TEXT

1011011

and received:

TEXT

1010011

XOR:

TEXT

1011011

XOR

1010011

0001000

The error pattern is:

TEXT

0001000

A 1 means:

that bit position flipped.

CRC detects the error unless this error polynomial is divisible by the generator polynomial.

30. SINGLE-BIT ERROR DETECTION

A single-bit error polynomial has the form:

$$E(x) = x^i$$

A valid CRC generator has at least two nonzero terms and therefore generally does not divide a pure power of x .

Hence standard CRC generators detect:

$$\boxed{\text{all single-bit errors}}$$

Important GATE property.

31. TWO-BIT ERROR

A two-bit error has:

$$E(x) = x^i + x^j$$

Factor:

$$E(x) = x^j(x^{i-j} + 1)$$

Whether CRC detects every two-bit error depends on:

- generator polynomial
- separation between erroneous bits
- frame length

So do **not** memorize:

"CRC always detects every two-bit error."

It depends on the generator.

32. ODD NUMBER OF BIT ERRORS

Very important property.

If generator polynomial contains factor:

$$(x+1)$$

then the CRC detects:

$\boxed{\text{all error patterns containing an odd number of bit errors}}$

Why?

For an odd number of erroneous bits:

$$E(1)=1$$

so $(x+1)$ does not divide that error polynomial over $GF(2)$.

You don't need the proof in detail, but remember the condition.

33. MEMORY

Generator has $(x+1)$ catches all odd-bit-count errors.

Do not say CRC automatically detects all odd errors for every generator.

The generator condition matters.

34. BURST ERROR

A burst error of length (L_b) means:

The first and last bits of the burst are erroneous, and the span between them is (L_b) bits.

Not every bit inside the burst must be corrupted.

Example:

TEXT

Error pattern:

0010100100

! ' ! '

First error to last error defines burst length.

35. CRC BURST-DETECTION PROPERTY

If generator polynomial degree is:

$$\leq r$$

then CRC can detect:

$$\text{all burst errors of length } \leq r$$

is very important.

36. EXAMPLE

Generator:

TEXT
10011

Degree:

$$4$$

Therefore it guarantees detection of every burst error of length:

$$\leq 4$$

37. WHAT ABOUT BURSTS LONGER THAN r ?

CRC can still detect many of them.

For burst errors of length:

$$\leq r+1$$

the probability of undetected error depends on the generator; under the standard random-pattern treatment, only a small fraction may be undetected.

For longer random burst errors, a commonly cited undetected probability is approximately:

$$2^{-r}$$

under suitable assumptions.

For GATE, use this only when the problem explicitly asks/provides the probabilistic model.

The guaranteed fact to remember is:

$$\text{Burst length } \leq r \rightarrow \text{detected}$$

38. GATE TRAP — GENERATOR LENGTH vs DEGREE

Generator:

TEXT
101101

has:

$\lceil 6 \rceil$ bits

but degree:

$\lceil 5 \rceil$

Therefore guaranteed burst detection length:

$\lceil 5 \rceil$

not 6.

CRC bits:

$\lceil 5 \rceil$

not 6.

39. GOOD GENERATOR POLYNOMIAL

At minimum, a CRC generator should normally have:

- highest-order coefficient = 1
- constant term = 1

So binary generator generally starts and ends with:

TEXT

1 . . . 1

Example:

TEXT

10011

This prevents certain trivial weaknesses.

40. WHY CONSTANT TERM SHOULD BE 1

If generator ended in zero, it would contain factor x .

That could reduce effectiveness against some error patterns and would be an unusual CRC generator.

For exam purposes:

Standard generator binary representation usually starts and ends with 1.

41. GATE-STYLE QUESTION — NUMBER OF REDUNDANT BITS

Generator:

$G(x) = x^5 + x^2 + 1$

What is number of CRC bits?

Highest power:

$\sqrt[5]{}$

Therefore:

$\boxed{5}$

Binary generator:

TEXT

100101

which contains 6 bits.

But CRC length is degree:

$\sqrt[5]{}$

42. GATE-STYLE QUESTION — APPENDING ZEROS

Data:

TEXT

10101101

Generator:

TEXT

1101

Generator length:

$\sqrt[4]{}$

Degree:

$\sqrt[3]{}$

So sender first forms:

TEXT

10101101 000

Then divides by:

TEXT

1101

43. GATE-STYLE QUESTION — FRAME LENGTH

Data:

$\sqrt[64]{\text{bits}}$

Generator:

$$x^8 + x^2 + x + 1$$

Degree:

$$8$$

CRC bits:

$$8$$

Frame/codeword length:

$$64 + 8$$

$$72 \text{ bits}$$

44. CRC vs CHECKSUM

CRC	Checksum
Polynomial division	1's-complement addition
XOR arithmetic	Addition + end-around carry
Strong burst-error detection	Useful general error detection
Ethernet FCS	TCP/UDP checksum
Remainder appended	Complemented sum appended

Memory:

CRC = XOR division Checksum = Add, wrap, complement

45. CRC vs PARITY

Parity	CRC
Very simple	More powerful
Usually 1 redundant bit	Multiple CRC bits
Detects all odd-count flips	Detection depends on generator properties
Weak for bursts	Strong for burst errors

46. HOW GATE CAN ASK CRC

Pattern 1 — Direct remainder calculation

Given:

- data
- generator

Find CRC.

Pattern 2 — Transmitted codeword

Find:

$\lfloor \text{Data} + \text{Remainder} \rfloor$

Pattern 3 — Receiver checking

Given received sequence and generator.

Determine whether error is detected.

Pattern 4 — Polynomial conversion

Convert:

$\lfloor x^4 + x^2 + 1 \rfloor$

to:

TEXT

10101

Pattern 5 — Redundant bits

Generator degree $\lfloor r \rfloor$.

CRC bits:

$\lfloor r \rfloor$

Pattern 6 — Burst error

Generator degree $\lfloor r \rfloor$.

All bursts of length:

$\lfloor \leq r \rfloor$

are guaranteed detectable.

Pattern 7 — Odd errors

If $\lfloor G(x) \rfloor$ contains:

$\lfloor x+1 \rfloor$

then all odd-number-of-bit error patterns are detected.

Pattern 8 — Error polynomial

They may ask whether a particular error pattern goes undetected.

Method:

Divide error polynomial by generator.

If exactly divisible:

potentially undetected.

47. EXAMPLE — WILL THIS ERROR BE DETECTED?

Suppose generator:

TEXT

1011

and error pattern:

TEXT

1011

Notice:

$$\lfloor E = G \rfloor$$

Therefore:

$$\lfloor E \bmod G = 0 \rfloor$$

So CRC would **not detect** that particular error pattern.

This demonstrates:

CRC is powerful, but not perfect.

48. ANOTHER KEY PROPERTY

If the error pattern equals:

$$\lfloor G(x) \times Q(x) \rfloor$$

for some polynomial $\lfloor Q(x) \rfloor$, then:

$$\lfloor E(x) \rfloor$$

is divisible by $\lfloor G(x) \rfloor$.

Therefore it may go undetected.

So the entire design of CRC revolves around choosing a generator whose multiples do not resemble common error patterns.

49. MANUAL DIVISION EXAM STRATEGY

When doing CRC by hand:

Step 1

Write:

$\lfloor r = |G| - 1 \rfloor$

Step 2

Append $\lfloor r \rfloor$ zeros.

Step 3

Keep generator directly below the current leading 1.

Step 4

XOR.

Step 5

Bring down next bit.

Step 6

Continue until no bits remain to bring down.

Step 7

Take exactly $\lfloor r \rfloor$ final bits as remainder.

Step 8

Pad leading zeros in remainder if needed.

Example:

If $\lfloor r = 4 \rfloor$ and mathematical remainder is:

TEXT

11

write CRC as:

TEXT

0011

0 = 1 is important.

50. & pGATE TRAP — LEADING ZEROS IN REMAINDER

Generator degree:

$\lfloor 4 \rfloor$

means CRC must contain:

$\lfloor 4 \text{ text{ bits} } \rfloor$

If remainder is:

TEXT

101

write:

TEXT

0101

Do not shorten the CRC length.

51. & pGATE TRAP — DIVISION WHEN LEADING BIT IS ZERO

Suppose current working bits begin with:

TEXT

0

Do not XOR with the generator aligned to that zero.

Conceptually XOR with all zeros and move forward until the leading bit allows division.

This is analogous to ordinary long division where divisor doesn't fit.

52. & pGATE TRAP — NORMAL BINARY DIVISION

Do not use:

- subtraction
- borrowing
- normal remainder arithmetic

CRC uses:

$\boxed{\text{XOR}}$

only.

53. & pGATE TRAP — REMAINDER SIZE

If generator:

TEXT

1001

has 4 bits:

- degree = 3
- append 3 zeros
- remainder = 3 bits

Do not use 4 CRC bits.

54. & pGATE TRAP — POLYNOMIAL DEGREE

For:

$\sqrt{x^7 + x^3 + 1}$

degree is:

$\sqrt{7}$

not:

- number of terms = 3
- binary length maybe 8

CRC bits:

$\sqrt{7}$

55. COMPLETE CRC MENTAL MAP

TEXT

```
Original Data
    !"
Generator degree = r
    !"
Append r zeros
    !"
Modulo-2 divide using XOR
    !"
Get r-bit remainder
    !"
Data + remainder
    !"
Transmit
    !"
Receiver divides by same generator
    !"
Zero remainder?
/      \
Yes     No
    !"
No error detected      Error detected
```

56. MASTER MEMORY TRICK — CRC

Remember:

A-D-X-R

A — Append zeros

Append generator degree number of zeros.

D — Divide

Divide by generator.

X — XOR

Use XOR instead of subtraction.

R — Remainder

Replace zeros with remainder.

Receiver:

Divide again !' zero remainder means pass.

' p NOTEBOOK VERSION

CRC — Cyclic Redundancy Check

CRC is an error-detection technique using:

Modulo-2 polynomial division

Modulo-2 subtraction:

$\boxed{\text{XOR}}$

No carry/borrow.

Generator

If generator has $(r+1)$ bits:

$\boxed{\text{degree}=r}$

Therefore:

- CRC bits = (r)
- append (r) zeros

Example:

TEXT

G = 1011

Degree:

$\boxed{3}$

CRC length:

$\boxed{3}$

Sender

1. Take data $\text{\(D\)}$.
2. Append $\text{\(r\)}$ zeros.
3. Divide by generator using XOR.
4. Get $\text{\(r\)}$ -bit remainder $\text{\(R\)}$.
5. Transmit:

$$\boxed{T=D||R}$$

The codeword satisfies:

$$\boxed{T \bmod G=0}$$

Receiver

Divide received codeword by same generator.

Remainder = 0

No error detected.

Remainder $\neq 0$

Error detected.

Generator polynomial

$$\boxed{x^4+x^2+1}$$

becomes:

TEXT

10101

Error Detection Properties

For generator degree $\text{\(r\)}$:

Single-bit errors

$$\boxed{\text{All detected}}$$

for standard CRC generators.

Burst errors

$$\boxed{\text{All bursts of length } \leq r \text{ detected}}$$

Odd number of errors

If:

$$\boxed{x+1}$$

is a factor of $G(x)$, then all odd-number-of-bit errors are detected.

Undetected error

Possible when:

$E(x)$ is divisible by $G(x)$

Memory

A-D-X-R

Append !' Divide !' XOR !' Remainder

57. 5-MINUTE REVISION — COMPLETE ERROR DETECTION

Parity

Count 1s.

Detects:

all odd number of bit flips.

Checksum

Add !' End-around carry !' Complement.

Receiver expects:

TEXT

111...111

CRC

Append zeros !' XOR divide !' append remainder.

Receiver expects:

TEXT

000...000 remainder

58. 30-SECOND REVISION

Remember only this:

TEXT

```
Generator length = r+1
CRC bits = r
Append r zeros
Divide using XOR
Remainder = CRC
```

Then:

All bursts of length "d are guaranteed detected.

And:

Error pattern divisible by generator !' may escape detection.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Generator:

TEXT

11001

Find:

1. generator degree
2. number of CRC bits
3. number of zeros appended to the original data

Q2

Convert:

$G(x) = x^5 + x^2 + 1$

into binary.

Q3

What arithmetic operation replaces subtraction in CRC division?

A. AND B. OR C. XOR D. 1's complement addition

Q4

A data block contains 100 bits and the generator degree is 8.

What is the transmitted codeword length?

Q5 — GATE Conceptual

A CRC generator has degree:

$\lfloor 12 \rfloor$

What is the maximum burst length for which **all** burst errors are guaranteed to be detected?

Q6

True or False:

A zero CRC remainder at the receiver mathematically guarantees that no bits changed during transmission.

Explain briefly.

Q7

If:

$\lfloor G(x) \rfloor$

contains the factor:

$\lfloor x+1 \rfloor$

what class of errors is guaranteed to be detected?

Q8 — GATE Trap

Generator:

TEXT

10101

A division produces remainder:

TEXT

11

What exact CRC bit string should be appended?

Q9

Why do we append zeros before performing CRC division at the sender?

Q10 — Numerical

Find the CRC for:

TEXT

Data = 1101

Generator = 1011

Then write the transmitted codeword.

You should now be able to verify that:

and codeword:

1101001

next

CHAPTER 6 — MEDIUM ACCESS CONTROL

HIGH PRIORITY FOR GATE NUMERICALS

$$\boxed{S=Ge^{-G}}$$

But first, understand **why** these formulas exist instead of blindly memorizing them.

1. WHAT IS THE MULTIPLE-ACCESS PROBLEM?

The question is:

Who gets to use the shared channel, and when?

That is the **multiple-access problem**.

2. WHY DO WE NEED MAC PROTOCOLS?

Suppose both A and B transmit simultaneously.

Their signals may interfere.

Conceptually:

```
TEXT
A ----->
B ----->

          ! "

COLLISION
```

The receiver may not be able to correctly recover either frame.

Therefore we need a protocol that determines:

- when a station can transmit
- what happens after collision
- when retransmission should occur

These are:

Medium Access Control protocols

3. MAC = MEDIUM ACCESS CONTROL

Remember that earlier we saw **MAC address**.

Here MAC means:

Medium Access Control

They are related to the Data Link layer, but are different concepts.

4. BROAD TYPES OF MULTIPLE ACCESS

Useful mental map:

```
TEXT
Multiple Access
|
+-- Random Access
|   |
|   +-- ALOHA
|   +-- CSMA
|   +-- CSMA/CD
|
+-- Controlled Access
```


Your syllabus specifically says:

Medium Access Control

For GATE, random-access protocols such as:

- ALOHA
- CSMA
- CSMA/CD

are especially important.

Today:

ALOHA

5. WHAT IS ALOHA?

ALOHA is one of the simplest random-access protocols.

Its philosophy is almost:

If you have a frame, transmit it.

If a collision occurs:

Wait for a random amount of time and retransmit.

There are two important versions:

1. Pure ALOHA
2. Slotted ALOHA

PART A — PURE ALOHA

6. PURE ALOHA — BASIC IDEA

In Pure ALOHA:

A station can transmit whenever it has a frame.

There are no fixed transmission slots.

Example:

TEXT

T i m e !'

Station A: [AAAAAA]

Station B: [BBBBBB]

Station C: [CCCCCC]

A and B overlap.

Therefore:

Collision.

7. WHAT HAPPENS AFTER COLLISION?

If sender determines that transmission failed:

1. It waits for a **random backoff time**.
2. It retransmits.

Why random?

Suppose A and B collide.

If both immediately retransmit after exactly 1 second:

TEXT

A !' r e t r a n s m i t
B !' r e t r a n s m i t

They collide again.

Random waiting reduces repeated synchronization of collisions.

8. FRAME TRANSMISSION TIME

Suppose every frame takes:

T

seconds to transmit.

That is:

$$T = \frac{L}{R}$$

where:

- L = frame length in bits
- R = link rate in bits/s

This T becomes extremely important in ALOHA.

9. THE MOST IMPORTANT IDEA — VULNERABLE PERIOD

Suppose Station A begins transmitting a frame at time:

$t=0$

and its frame requires:

T

seconds.

Therefore A transmits during:

TEXT

0 ----- T
A's frame

What other frames can collide with A?

Not just those starting between:

0 and T

A frame that began **before** A can also overlap with it.

10. PURE ALOHA VULNERABLE PERIOD — INTUITION

Suppose another frame begins just before A:

TEXT

Other: [-----]
A: [-----]

They overlap.

The earliest dangerous starting time for another frame is approximately:

$-T$

The latest dangerous starting time is approximately:

$+T$

Therefore vulnerable interval:

$-T \rightarrow +T$

Length:

$2T$

$\frac{2T}{2} = T$

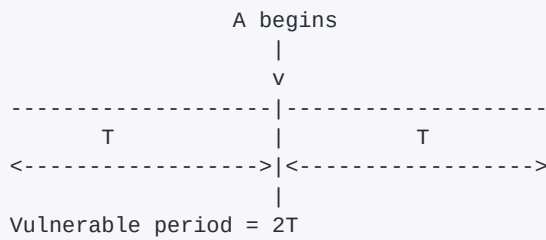
PURE ALOHA VULNERABLE PERIOD

$2T$

11. VISUAL

For A's frame:

TEXT



Any competing frame beginning in this vulnerable interval can overlap A under the simplified ALOHA model.

12. MEMORY TRICK

Pure ALOHA:

No slots !' danger on both sides

Therefore:

$\boxed{2T}$

Think:

PURE = 2

Not mathematically meaningful, but easy to remember.

13. OFFERED LOAD — G

Now we need an important variable:

G

G represents:

Average number of transmission attempts per frame-transmission time.

Importantly, attempts include:

- new frames
- retransmitted frames

So if:

$G=0.5$

it means an average of 0.5 attempts per frame time.

14. THROUGHPUT — S

$\bar{S}$ represents:

Average number of successfully transmitted frames per frame time.

Therefore:

$$\bar{S} \leq G$$

because not every attempt succeeds.

15. WHY DO WE GET AN EXPONENTIAL?

ALOHA analysis commonly assumes transmission attempts follow a:

Poisson process.

If mean attempts in an interval are:

$$\lambda$$

then probability of exactly k attempts is:

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

For successful transmission, we need:

zero other transmissions during the vulnerable period.

The probability of zero arrivals for Poisson mean λ is:

$$P(0) = e^{-\lambda}$$

16. PURE ALOHA SUCCESS PROBABILITY

Vulnerable period:

$$2T$$

If offered load per frame time is:

$$G$$

then expected interfering attempts during $(2T)$:

$$2G$$

Probability of zero interfering attempts:

$$\boxed{e^{-2G}}$$

Therefore probability that a transmission succeeds:

$$\boxed{P_{\text{success}} = e^{-2G}}$$

17. PURE ALOHA THROUGHPUT

Attempt rate:

$$[G]$$

Success probability:

$$[e^{-2G}]$$

Therefore:

$$[\boxed{S = Ge^{-2G}}]$$

$S = 0.184 \times G$ is the formula.

18. FINDING MAXIMUM PURE ALOHA THROUGHPUT

We want maximum of:

$$[S = Ge^{-2G}]$$

Differentiate:

$$[\frac{dS}{dG} = e^{-2G}(1-2G)]$$

For maximum:

$$[1-2G=0]$$

Therefore:

$$[\boxed{G = \frac{1}{2}}]$$

Substitute:

$$[S_{\max} = \frac{1}{2} e^{-1}]$$

$$[\boxed{S_{\max} = \frac{1}{2e}}]$$

Approximately:

$$[\boxed{0.184}]$$

or:

$$[\boxed{18.4\%}]$$

19. WHAT DOES 18.4% MEAN?

At its best operating point, Pure ALOHA successfully utilizes only about:

$$[18.4\%]$$

of the channel capacity in this model.

That's poor.

Why?

Because stations can start transmitting at any arbitrary time, producing a large vulnerable period.

20. GATE MEMORY

Pure ALOHA:

$$S = G e^{-2G}$$

Maximum at:

$$G = 0.5$$

Maximum throughput:

$$S \approx \frac{1}{2e} \approx 0.184$$

Vulnerable period:

$$2T$$

Remember the group:

Pure ALOHA: $S = G e^{-2G}$ at $G = 0.5$, $S \approx 0.184$

21. PURE ALOHA EXAMPLE

Suppose:

$$G = 0.5$$

Find throughput.

FORMULA

$$S = G e^{-2G}$$

Substitute:

$$S = 0.5 e^{-1}$$

Using:

$$e^{-1} \approx 0.3679$$

$$S \approx 0.5(0.3679)$$

$$S \approx 0.184$$

This is the maximum.

22. ANOTHER EXAMPLE

If:

$$G = 1$$

then:

$$S = 1 e^{-2}$$

$$e^{-2} \approx 0.1353$$

Therefore:

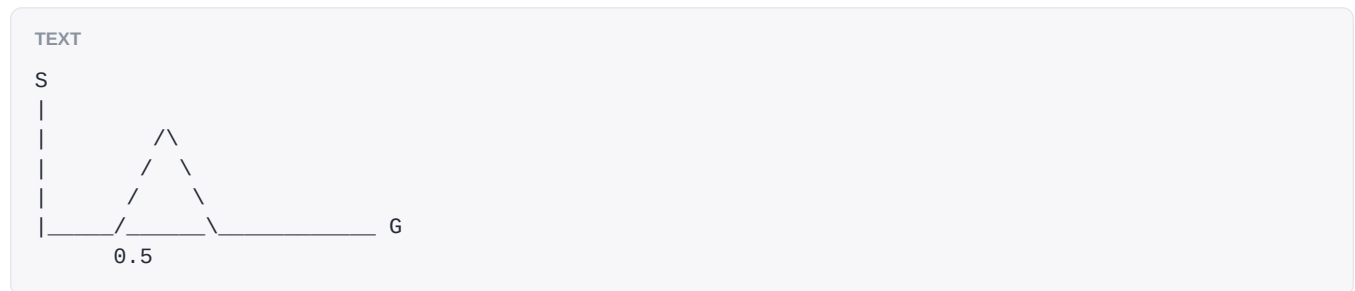
$\boxed{S \approx 0.1353}$

Interesting:

Increasing offered load beyond the optimum causes **lower throughput** because collisions increase.

23. IMPORTANT GRAPH SHAPE

Conceptually:



Initially:

More attempts !' more successful frames.

After a point:

More attempts !' too many collisions !' throughput decreases.

This shape is important conceptually.

PART B — SLOTTED ALOHA

24. WHY IMPROVE PURE ALOHA?

Pure ALOHA allows a station to transmit at any instant.

This creates vulnerable period:

$2T$

Can we reduce collisions?

Yes.

Make everybody agree:

Frames may begin transmission only at specific time boundaries.

This creates:

Slotted ALOHA

25. WHAT IS SLOTTED ALOHA?

Time is divided into equal slots.

Each slot length is one frame-transmission time:

$\boxed{T}$

Conceptually:

TEXT

| Slot 1 | Slot 2 | Slot 3 | Slot 4 |

Stations can begin transmitting only:

at the beginning of a slot.

26. EXAMPLE

Suppose a station gets a frame halfway through Slot 2:

TEXT

Slot 2

|-----|

^

frame arrives

It cannot immediately transmit.

It waits until:

beginning of Slot 3.

27. WHY DOES THIS HELP?

Suppose A starts at slot boundary:

TEXT

|----- A -----|

Another station cannot begin halfway through A's slot.

It must either:

- start at the same slot boundary !' collision
- start next slot !' no overlap

Therefore vulnerable period becomes:

$\boxed{T}$

instead of:

$2T$

Ø=Y% important.

28. PURE vs SLOTTED VULNERABLE PERIOD

Pure ALOHA

TEXT

<----- T ----->|<----- T ----->
 ^
 A starts

Vulnerable = 2T

Slotted ALOHA

TEXT

|----- one slot -----|

Vulnerable = T

Therefore:

$\boxed{\text{Slotted ALOHA halves the vulnerable period}}$

29. REQUIREMENT — SYNCHRONIZATION

There is a cost.

All stations must know:

where slot boundaries occur.

Therefore Slotted ALOHA requires:

$\boxed{\text{time synchronization}}$

Pure ALOHA doesn't require slot synchronization.

30. SLOTTED ALOHA SUCCESS PROBABILITY

Vulnerable period:

T

Mean competing attempts during this interval:

G

Probability of zero interfering transmissions:

$\boxed{e^{-G}}$

Thus:

$$\boxed{P_{\text{success}} = e^{-G}}$$

31. SLOTTED ALOHA THROUGHPUT

Attempts:

$$G$$

Success probability:

$$e^{-G}$$

Therefore:

$$\boxed{S = Ge^{-G}}$$

Essential formula.

32. MAXIMUM SLOTTED ALOHA THROUGHPUT

Differentiate:

$$S = Ge^{-G}$$

$$\frac{dS}{dG} = e^{-G}(1-G)$$

Maximum when:

$$1-G=0$$

Therefore:

$$\boxed{G=1}$$

At that point:

$$S_{\text{max}} = e^{-1}$$

$$\boxed{S_{\text{max}} = \frac{1}{e}}$$

Approximately:

$$\boxed{0.368}$$

or:

$$\boxed{36.8\%}$$

33. KEY COMPARISON

Pure ALOHA maximum:

$$18.4\%$$

Slotted ALOHA maximum:

$$36.8\%$$

Therefore Slotted ALOHA has approximately:

$$\boxed{2\times}$$

the maximum throughput of Pure ALOHA.

34. MASTER MEMORY TRICK

PURE

$$\sqrt{2T}$$

$$\sqrt{Ge^{-2G}}$$

$$\sqrt{18.4\%}$$

SLOTTED

$$\sqrt{T}$$

$$\sqrt{Ge^{-G}}$$

$$\sqrt{36.8\%}$$

Just remember:

Slotted removes the 2.

35. MASTER COMPARISON TABLE

Property	Pure ALOHA	Slotted ALOHA
Transmission start	Anytime	Slot boundary only
Synchronization required	No	Yes
Slot size	None	One frame time
Vulnerable period	$\sqrt{2T}$	$\sqrt{T}$
Success probability	$\sqrt{e^{-2G}}$	$\sqrt{e^{-G}}$
Throughput	$\sqrt{Ge^{-2G}}$	$\sqrt{Ge^{-G}}$
Optimal $\sqrt{G}$	$\sqrt{0.5}$	$\sqrt{1}$
Maximum throughput	$\sqrt{1/(2e)}$	$\sqrt{1/e}$
Approx maximum	18.4%	36.8%

This entire table is notebook-worthy.

36. GATE-STYLE NUMERICAL 1

A Pure ALOHA network has frame transmission time:

$$\sqrt{4\text{ms}}$$

Find vulnerable period.

SOLUTION

Pure ALOHA:

$$V = 2T$$

$$= 2(4\text{ms})$$

$$\boxed{8\text{ms}}$$

37. GATE-STYLE NUMERICAL 2

A Slotted ALOHA network has frame length:

$$1000\text{bits}$$

and rate:

$$1\text{Mbps}$$

Find slot length and vulnerable period.

Step 1 — Frame time

$$T = \frac{L}{R}$$

$$= \frac{1000}{10^6}$$

$$= 10^{-3}\text{s}$$

$$= \boxed{1\text{ms}}$$

Each slot:

$$\boxed{1\text{ms}}$$

Slotted vulnerable period:

$$\boxed{1\text{ms}}$$

38. GATE-STYLE NUMERICAL 3

For Slotted ALOHA:

$$G = 0.5$$

Find throughput.

$$S = G e^{-G}$$

$$= 0.5 e^{-0.5}$$

Since:

$$e^{-0.5} \approx 0.6065$$

$$S \approx 0.3033$$

Therefore:

$$\boxed{S \approx 0.303}$$

or about:

$$30.3\%$$

of one successful frame per frame-time capacity.

39. GATE-STYLE NUMERICAL 4

In Pure ALOHA, offered load is:

$$G = 0.25$$

Find successful throughput.

$$S = Ge^{-2G}$$

$$= 0.25e^{-0.5}$$

$$= 0.25(0.6065)$$

$$\boxed{S \approx 0.1516}$$

40. FROM S TO SUCCESSFUL FRAMES PER SECOND

Suppose:

$$S = 0.3$$

and frame-transmission time:

$$T = 2\text{ms}$$

One frame time is:

$$0.002\text{s}$$

Number of frame times per second:

$$\frac{1}{0.002} = 500$$

Throughput:

$$0.3 \times 500$$

$$= \boxed{150 \text{ successful frames/s}}$$

41. MORE DIRECT FORM

If normalized throughput is:

$$S$$

and each frame transmission time is T , successful frame rate is:

$$\boxed{\frac{S}{T}}$$

frames/s.

Because:

$S = \text{successful frames per frame time}$

42. SUCCESSFUL BIT RATE

If every successful frame contains:

L

bits, then:

$\boxed{\text{Successful bit rate} = \frac{S}{T} \times L}$

But:

$T = \frac{L}{R}$

Therefore:

$\frac{S}{L/R} \times L = SR$

So:

$\boxed{\text{Successful throughput in bits/s} = S \times R}$

when S is normalized throughput.

~~Useful shortcut.~~

43. EXAMPLE

Slotted ALOHA link rate:

10Mbps

Operating at maximum throughput:

$S_{\max} = \frac{1}{2} \approx 0.368$

Maximum successful bit rate:

$0.368 \times 10\text{Mbps}$

$= \boxed{3.68\text{Mbps}}$

under the idealized ALOHA model.

44. PURE ALOHA MAXIMUM BIT RATE

Same 10 Mbps channel.

Pure ALOHA maximum:

0.184×10

$= \boxed{1.84\text{Mbps}}$

Again:

Slotted approximately doubles maximum efficiency.

45. OFFERED LOAD vs THROUGHPUT

Do not confuse:

$\mathbb{G}$

Attempted traffic.

Includes failed/retransmitted attempts.

$\mathbb{S}$

Successful traffic.

Thus:

$$\mathbb{S} = \mathbb{G} \times P_{\text{success}}$$

For Pure:

$$\mathbb{S} = \mathbb{G} e^{-2\mathbb{G}}$$

For Slotted:

$$\mathbb{S} = \mathbb{G} e^{-\mathbb{G}}$$

46. & pGATE TRAP — G CAN EXCEED 1

Students sometimes think:

$$0 \leq \mathbb{G} \leq 1$$

because throughput is normalized.

Wrong.

$\mathbb{G}$ is average attempted transmissions per frame time and can exceed 1.

For example:

$$\mathbb{G} = 2$$

means on average two attempts per frame time.

But collisions will be very high.

47. WHAT HAPPENS AS $\mathbb{G} \rightarrow \infty$?

Pure:

$$\mathbb{G} e^{-2\mathbb{G}} \rightarrow 0$$

Slotted:

$$\mathbb{G} e^{-\mathbb{G}} \rightarrow 0$$

So enormous offered load does **not** produce enormous successful throughput.

Instead:

The channel collapses under collisions.

48. WHY DOES SLOTTED ALOHA STILL HAVE COLLISIONS?

Suppose three stations all have frames ready at the same slot boundary:

TEXT

Slot start

|

v

A !' F r a m e

B !' F r a m e

C !' F r a m e

All transmit simultaneously.

Collision occurs.

Slotted ALOHA only prevents:

partially overlapping frame starts.

It does not prevent multiple users choosing the same slot.

49. WHY NOT ASSIGN EACH USER A SLOT PERMANENTLY?

Then we'd essentially move toward:

Fixed TDM.

But if a user has nothing to send, its slot may be wasted.

Slotted ALOHA keeps random access:

any station may attempt in a slot.

That's why it's still statistically shared.

50. PROBABILITY INTERPRETATION

Pure ALOHA:

$\lfloor e^{-2G} \rfloor$

means:

Probability that there are no competing attempts during the $\frac{1}{2}T$ vulnerable interval.

Slotted ALOHA:

$$[e^{-G}]$$

means:

Probability of no other interfering attempts in the relevant slot interval.

This is why the formulas differ only by the factor 2.

51. CONNECTION TO CSMA

ALOHA's major weakness is:

A station transmits without first checking whether somebody else is already transmitting.

Wouldn't it be better to:

Listen before transmitting?

Yes.

That gives:

CSMA — Carrier Sense Multiple Access

Memory:

ALOHA = Send first.

CSMA = Sense first.

That's our next topic.

52. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Vulnerable period

Given frame time T .

Pure:

$$[2T]$$

Slotted:

$$[T]$$

Pattern 2 — Throughput formula

Pure:

$$S = G e^{-2G}$$

Slotted:

$$S = G e^{-G}$$

Pattern 3 — Maximum throughput

Pure:

$$S = \frac{1}{2} e^{-1}$$

Slotted:

$$S = \frac{1}{2} e^{-1}$$

Pattern 4 — Optimal offered load

Pure:

$$G = \frac{1}{2}$$

Slotted:

$$G = 1$$

Pattern 5 — Convert normalized throughput to bps

$$S \times R$$

Pattern 6 — Compare synchronization

Slotted requires synchronization.

Pure does not require slots.

Pattern 7 — Frame-time calculation

$$T = \frac{L}{R}$$

then use T or $2T$.

53. & pGATE TRAPS

Trap 1

Pure ALOHA vulnerable period is:

$$2T$$

not T .

Trap 2

Slotted ALOHA vulnerable period:

$$\boxed{T}$$

not $(2T)$.

Trap 3

Do not swap formulas:

Pure:

$$G e^{-2G}$$

Slotted:

$$G e^{-G}$$

Trap 4

Maximum Slotted ALOHA throughput is:

$$\frac{1}{e}$$

not 100%.

Slots can still collide.

Trap 5

Maximum Pure ALOHA throughput is not:

$$1/e$$

It is:

$$\boxed{1/(2e)}$$

Trap 6

Slotted ALOHA requires time synchronization.

Trap 7

G means attempted load, not successful throughput.

Trap 8

Retransmissions contribute to offered load G .

Trap 9

Use frame transmission time:

$\lfloor L/R \rfloor$

not propagation delay when defining the classic ALOHA slot/vulnerable interval.

54. MASTER MEMORY SHEET

Think of this:

TEXT

PURE

Anytime

$2T$

$G^{\wedge}(-2G)$

$G^*=0.5$

Max=18.4%

SLOTTED

Slot boundary

T

$G^{\wedge}(-G)$

$G^*=1$

Max=36.8%

Memory sentence:

Slotted halves danger and doubles maximum throughput.

' p NOTEBOOK VERSION

Multiple Access

When multiple stations share the same medium, a MAC protocol decides:

who transmits and when.

Collision occurs when incompatible simultaneous transmissions overlap.

Pure ALOHA

A station transmits whenever it has data.

On collision:

Random wait + retransmit.

Frame transmission time:

$$\boxed{T=L/R}$$

Vulnerable period:

$$\boxed{2T}$$

Throughput:

$$\boxed{S=Ge^{-2G}}$$

Maximum occurs at:

$$\boxed{G=0.5}$$

$$\boxed{S_{\max}=\frac{1}{2e}\approx 0.184}$$

Maximum efficiency:

$$\boxed{18.4\%}$$

Slotted ALOHA

Time divided into slots of length T .

A station may begin only at a slot boundary.

Requires:

synchronization.

Vulnerable period:

$$\boxed{T}$$

Throughput:

$$\boxed{S=Ge^{-G}}$$

Maximum at:

$$\boxed{G=1}$$

$$\boxed{S_{\max}=\frac{1}{e}\approx 0.368}$$

Maximum efficiency:

$$\boxed{36.8\%}$$

Comparison

Pure	Slotted
Start anytime	Start at slot boundary
$2T$	T
Ge^{-2G}	Ge^{-G}
18.4% max	36.8% max

Pure	Slotted
No slot sync	Synchronization required

Memory:

Pure = 2 Slotted removes the 2

55. 30-SECOND REVISION

Memorize this box:

$$\begin{aligned} \text{Pure \&: } S &= G e^{-2G}, \quad S_{\max} = 1/(2e) = 0.184 \\ \text{Slotted \&: } S &= G e^{-G}, \quad S_{\max} = 1/e = 0.368 \end{aligned}$$

And:

$$\text{Pure \vulnerable} = 2T, \quad \text{Slotted} = T$$

ACTIVE RECALL — DON'T LOOK BACK

Q1

A frame takes:

5ms

to transmit.

What is the vulnerable period for:

1. Pure ALOHA
2. Slotted ALOHA?

Q2

Write the throughput equations for:

1. Pure ALOHA
2. Slotted ALOHA

Q3

What are the maximum theoretical normalized throughputs of:

1. Pure ALOHA
2. Slotted ALOHA?

Q4

At what offered load G is maximum throughput achieved for:

1. Pure ALOHA
2. Slotted ALOHA?

Q5 — GATE Numerical

For Pure ALOHA:

$$G=0.5$$

Find S .

Q6 — GATE Numerical

A Slotted ALOHA channel operates at:

$$R=20\text{Mbps}$$

at its theoretical maximum normalized throughput.

Approximately what successful bit rate can it achieve?

Q7 — Conceptual

Why is Slotted ALOHA more efficient than Pure ALOHA?

Q8 — Trap

True or False:

In Slotted ALOHA, collisions are completely eliminated.

Explain.

Q9

A frame contains:

$$2000\text{bits}$$

and channel rate is:

$$2\text{Mbps}$$

Find:

1. frame-transmission time
 2. Pure ALOHA vulnerable period
 3. Slotted ALOHA vulnerable period
-

Q10 — GATE Thinking

Why does throughput eventually **decrease** when offered load G becomes very large, even though more frames are being attempted?

NEXT !' CSMA: 1-Persistent, Non-Persistent & p-Persistent CSMA + Propagation-Delay Effect

USER

next

ASSISTANT

CHAPTER 6 — MEDIUM ACCESS CONTROL

TOPIC 2 — CSMA: 1-Persistent, Non-Persistent & p-Persistent + Propagation Delay Effect

$\emptyset = \gamma\%$ HIGH PRIORITY FOR GATE

ALOHA follows:

Transmit first.

CSMA improves this by doing:

Listen first, then transmit.

CSMA stands for:

Carrier Sense Multiple Access

1. WHY WAS CSMA INTRODUCED?

In ALOHA, a station may transmit even when another station is already using the channel.

That wastes frames.

CSMA asks a simple question before transmitting:

Is the channel currently busy or idle?

If the channel is idle:

transmission may begin.

If busy:

wait according to the CSMA rule.

2. WHAT DOES “CARRIER SENSE” MEAN?

Carrier sensing means:

A station listens to the communication medium to detect whether another transmission is already present.

Conceptually:

```
TEXT
Have frame
! "
Listen to channel
! "
B u s y ? % %   Y e s ! '   W a i t
%
No
! "
Transmit
```

3. DOES CSMA COMPLETELY ELIMINATE COLLISIONS?

'LNo.

This is a major GATE concept.

Even though stations listen first, collisions can still occur because:

Signals require time to propagate.

This is the key limitation of carrier sensing.

4. WHY CAN COLLISIONS STILL HAPPEN?

Suppose stations A and B are far apart.

```
TEXT
A ----- B
```

At time $(t=0)$:

A starts transmitting.

But A's signal takes some propagation time:

$[\tau]$

to reach B.

Before A's signal reaches B, B listens.

B hears:

Idle channel.

So B also starts transmitting.

Now:

TEXT

A ----->

<----- B

COLLISION

Both correctly sensed the channel as idle locally.

But because of propagation delay, they still collided.

$\alpha = \frac{\tau}{T}$ is fundamental.

5. PROPAGATION DELAY MATTERS IN CSMA

The efficiency of CSMA depends strongly on the relationship between:

- propagation delay
- frame transmission time

Define:

$$a = \frac{d_{\text{prop}}}{d_{\text{trans}}}$$

or often:

$$a = \frac{\tau}{T}$$

where:

- τ = propagation delay
- T = frame transmission time

6. INTERPRETING a

Small a

$$a \ll 1$$

Propagation is very short compared with frame transmission.

Stations quickly learn that channel is busy.

! fewer collisions.

Large a

Propagation is significant.

Stations may incorrectly think channel is idle for longer.

! more collisions.

So:

Smaller propagation/transmission ratio ! better CSMA performance.

7. THREE IMPORTANT CSMA STRATEGIES

You need to distinguish:

1. 1-Persistent CSMA
2. Non-Persistent CSMA
3. p-Persistent CSMA

They differ mainly in:

What a station does when it finds the channel busy or idle.

PART A — 1-PERSISTENT CSMA

8. WHAT IS 1-PERSISTENT CSMA?

Algorithm:

If channel is idle:

Transmit immediately.

Probability of transmitting:

$\frac{1}{2}$

Hence:

1-persistent

If channel is busy:

Keep continuously sensing.

As soon as it becomes idle:

transmit immediately.

9. FLOW

TEXT

Frame ready

!"

Sense channel

!"

Busy? % % Yes % % !' Keep sensing continuously

%

%

No

%

becomes idle

!"

!"

T r a n s m i t i m m e d i a t e l y ! * % % % %

10. WHY CAN 1-PERSISTENT CAUSE COLLISIONS?

Suppose three stations are waiting while one station is transmitting:

TEXT

A waiting
B waiting
C waiting

All continuously sense.

The moment channel becomes idle:

TEXT

A !' t r a n s m i t
B !' t r a n s m i t
C !' t r a n s m i t

All may transmit almost simultaneously.

Result:

Collision.

This is called an aggressive strategy.

11. MEMORY

1-Persistent = I WILL transmit immediately when possible.

Or:

Busy? Keep watching. Idle? Jump immediately.

12. ADVANTAGE

Low idle-channel waste.

If only one station is waiting:

it transmits immediately once channel is free.

13. DISADVANTAGE

When several stations are waiting:

many may transmit together when channel becomes idle.

So collision probability can be significant.

PART B — NON-PERSISTENT CSMA

14. WHAT IS NON-PERSISTENT CSMA?

If channel is idle:

Transmit immediately.

If channel is busy:

Do NOT continuously listen.

Instead:

1. Wait for a random amount of time.
2. Sense again.

15. FLOW

TEXT

```
Frame ready
    ! "
Sense
    ! "
Busy?
% % % %4% % %
Yes    No
    ! "    ! "
Wait    Transmit
random
time
    ! "
Sense again
```

16. WHY DOES THIS REDUCE COLLISIONS?

Suppose A, B and C all find channel busy.

Instead of all waiting until the exact instant it becomes idle:

TEXT

```
A waits random time
B waits another random time
C waits another random time
```

They are less likely to transmit together.

Thus:

fewer collisions than aggressive 1-persistent behavior.

17. WHAT IS THE COST?

A station may be waiting randomly even though the channel has already become free.

So the channel can sit idle unnecessarily.

Therefore:

lower collision probability, but potentially higher delay / lower utilization under some conditions.

18. MEMORY

Non-persistent = Busy? Go away and come back later.

That is the easiest memory trick.

PART C — p-PERSISTENT CSMA

19. WHAT IS p-PERSISTENT CSMA?

This strategy is generally used with:

slotted channels

If the channel is idle at a slot boundary:

- transmit with probability p
- defer with probability $(1-p)$

If deferred:

wait one slot and repeat if channel remains idle.

20. ALGORITHM

Suppose station senses channel idle.

Then:

$P(\text{Transmit}) = p$

$P(\text{Wait one slot}) = 1 - p$

After waiting:

- if still idle, repeat
- if busy, wait until channel becomes idle and then resume the probabilistic rule

21. EXAMPLE

Suppose:

$\backslash p=0.25 \backslash$

Then when channel is idle:

- 25% chance !' transmit
- 75% chance !' wait one slot

This reduces the chance that many waiting stations all transmit simultaneously.

22. WHY IS IT CALLED p-PERSISTENT?

Because the station persists with a probability-based transmission rule:

$\backslash p \backslash$

instead of always transmitting immediately.

23. SPECIAL CASE

If:

$\backslash p=1 \backslash$

then p-persistent behavior becomes similar to:

1-persistent CSMA

because the station always transmits when permitted.

~~Useful~~ conceptual connection.

24. HOW DOES p AFFECT PERFORMANCE?

Large $\backslash(p\backslash)$

Stations transmit aggressively.

Pros:

- less waiting

Cons:

- more collision risk

Small $\backslash(p\backslash)$

Stations defer more often.

Pros:

- fewer simultaneous attempts

Cons:

- more idle slots/delay

So choosing p is a tradeoff.

25. MASTER COMPARISON TABLE

Feature	1-Persistent	Non-Persistent	p-Persistent
If idle	Transmit immediately	Transmit immediately	Transmit with probability p
If busy	Keep sensing	Wait random time	Wait until idle
Slotted required?	No	No	Usually yes
Collision tendency	Higher under contention	Lower	Adjustable
Idle-channel waste	Low	Can be higher	Depends on p
Main idea	Aggressive	Back off	Probabilistic

Very important table.

26. MEMORY TRICK

1-Persistent

1 = Immediate

Non-Persistent

Busy = Leave and return later

p-Persistent

Probability decides

So:

TEXT

```
1 !' I m m e d i a t e
N !' R a n d o m   w a i t
p !' P r o b a b i l i t y
```

27. CSMA vs ALOHA

ALOHA:

Doesn't listen before sending.

CSMA:

Senses channel first.

Therefore CSMA generally performs better because it avoids many obviously unnecessary collisions.

28. COMPARISON

ALOHA	CSMA
Transmit without carrier sensing	Listen before transmitting
More collisions	Fewer collisions
Simple	More intelligent
Vulnerable to already-busy channel	Avoids transmitting on detected busy channel
Propagation still matters indirectly	Propagation is central

29. & μIMPORTANT: CARRIER SENSE IS LOCAL

Suppose A senses idle.

That means:

No signal has reached A yet.

It does **not** guarantee:

Nobody anywhere in the network has begun transmission.

This explains why propagation delay causes collisions.

30. GATE-STYLE TIMELINE

Suppose:

$\tau = 5 \mu s$

A starts at:

$t = 0$

Its signal reaches B only at:

$t = 5 \mu s$

If B senses at:

$$[t = 3\mu s]$$

B may still see idle channel.

So B can transmit and cause a collision.

This is why:

$$[\tau]$$

matters.

31. THE VULNERABILITY WINDOW IDEA

After one station starts transmitting, another station may still begin transmission before the first signal reaches it.

That dangerous interval is approximately related to:

$$[\tau]$$

the propagation delay.

Unlike Pure ALOHA's $(2T)$, CSMA's collision behavior is closely tied to physical propagation.

32. WHAT HAPPENS WHEN PROPAGATION DELAY IS ZERO?

Imagine idealized:

$$[\tau = 0]$$

The instant A begins transmitting:

every station immediately knows channel is busy.

Under perfect sensing assumptions, collisions due solely to simultaneous carrier-sense ignorance would be drastically reduced.

This demonstrates why smaller propagation delay improves CSMA.

33. FRAME TRANSMISSION TIME MATTERS TOO

Recall:

$$[T = \frac{LR}{B}]$$

If (T) is very large compared with propagation delay:

$$[\frac{\tau}{T} \ll 1]$$

then the channel spends relatively little time in the uncertainty window.

Performance improves.

34. EXAMPLE — CALCULATING (a)

Suppose:

Propagation delay:

$$\tau = 10 \mu\text{s}$$

Frame length:

$$L = 1000 \text{ bits}$$

Rate:

$$R = 10 \text{ Mbps}$$

First:

$$T = \frac{L}{R} = \frac{1000}{10 \times 10^6}$$

$$T = 100 \mu\text{s}$$

Therefore:

$$a = \frac{\tau}{T} = \frac{10}{100}$$

$$\boxed{a = 0.1}$$

35. INTERPRETATION

$$a = 0.1$$

means:

propagation delay equals 10% of frame-transmission time.

A smaller a generally means more efficient carrier sensing.

36. ANOTHER EXAMPLE

Same propagation delay:

$$\tau = 10 \mu\text{s}$$

But link rate increases so frame transmission time falls to:

$$T = 20 \mu\text{s}$$

Then:

$$a = \frac{\tau}{T} = \frac{10}{20} = 0.5$$

Carrier-sense uncertainty is now large relative to frame time.

This can hurt efficiency.

Interesting lesson:

Increasing link rate without changing propagation distance can make MAC timing constraints more demanding.

This becomes extremely important in Ethernet minimum-frame-size calculations.

37. CONNECTION TO CSMA/CD

CSMA only says:

Listen before transmitting.

But what if collision still happens after transmission starts?

Wouldn't it be useful to:

Detect the collision while transmitting and stop wasting the rest of the frame?

Yes.

That gives:

CSMA/CD

Carrier Sense Multiple Access with Collision Detection.

This is the classic shared Ethernet mechanism.

38. WHY CSMA/CD IS BETTER THAN BASIC CSMA

Basic CSMA:

TEXT

S e n s e ! ' T r a n s m i t

CSMA/CD:

TEXT

Sense
!"
Transmit
!"
Keep listening while transmitting
!"
Collision?
!"
Stop + retry later

This avoids wasting an entire frame after detecting collision.

39. GATE CONNECTION — 2τ

In CSMA/CD, a sender must still be transmitting long enough for the worst-case collision signal to travel:

1. from sender to farthest station
2. back from collision point to sender

This creates a critical time:

2τ

This will directly give us:

$\boxed{T_{\text{frame}} \geq 2\tau}$

and therefore the famous Ethernet:

minimum frame size

We'll derive it properly in the next topic.

$\tau = \frac{D}{c}$ is an important upcoming numerical.

40. COLLISION SCENARIO PREVIEW

Two stations:

TEXT

A ----- B

A starts first.

Just before A's signal reaches B, B starts.

Collision occurs near B.

A discovers the collision only after the collision effect travels back.

Worst case:

$\boxed{2\tau}$

So if A finishes transmitting before then:

A could incorrectly think its transmission succeeded.

Therefore frame must be long enough.

41. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Identify protocol behavior

Statement:

When channel is busy, station continuously senses until idle and transmits immediately.

Answer:

1-persistent CSMA.

Pattern 2

Statement:

When channel is busy, station waits random time before sensing again.

Answer:

Non-persistent CSMA.

Pattern 3

Statement:

When channel is idle, transmit with probability $\backslash(p)$.

Answer:

p-persistent CSMA.

Pattern 4 — Collision despite sensing

Ask why collision is possible.

Answer:

Propagation delay.

Pattern 5 — Calculate $\backslash(a)$

$\backslash \boxed{a=\tau/T} \backslash$

Pattern 6 — Compare collision behavior

Under heavy contention:

- 1-persistent tends to be aggressive
- non-persistent reduces synchronized retransmission
- p-persistent allows probabilistic control

42. & pGATE TRAPS

Trap 1

CSMA does **not** eliminate collisions.

Propagation delay still permits them.

Trap 2

1-persistent does not mean:

wait 1 second.

It means:

$P(\text{transmit when idle})=1$

Trap 3

Non-persistent does not continuously listen while channel is busy.

It waits randomly before trying again.

Trap 4

p-persistent usually assumes slotted operation.

Trap 5

Do not confuse:

p

with collision probability.

It is:

probability of transmitting when the idle opportunity occurs.

Trap 6

Carrier sensing tells you the channel's local observed state, not instantaneous global state.

Trap 7

Smaller:

$\frac{\tau}{T}$

generally improves CSMA performance.

43. SMALL PROBABILITY EXAMPLE

Suppose two stations are both waiting and use p-persistent CSMA with:

$$\left[p=0.5 \right]$$

At an idle slot boundary:

Probability both transmit

$$\left[0.5 \times 0.5 \right]$$

$$\left[=\boxed{0.25} \right]$$

! collision.

Probability neither transmits

$$\left[(1-0.5)^2 \right]$$

$$\left[=0.25 \right]$$

! idle slot.

Probability exactly one transmits

There are two possibilities:

A transmits, B waits:

$$\left[0.5(0.5)=0.25 \right]$$

B transmits, A waits:

$$\left[0.25 \right]$$

Total:

$$\left[\boxed{0.5} \right]$$

! successful access.

44. GENERAL p-PERSISTENT PROBABILITY

Suppose:

$$\left[n \right]$$

stations are ready and each transmits independently with probability (p) .

Probability exactly one transmits:

$$\left[\boxed{P_{\text{success}} = np(1-p)^{n-1}} \right]$$

Why?

Choose which one station transmits:

$$\left[n \right]$$

That station:

$$\left[p \right]$$

Remaining $(n-1)$ do not transmit:

$$\left[(1-p)^{n-1} \right]$$

Therefore:

$$\left[np(1-p)^{n-1} \right]$$

This is a useful GATE probability connection.

45. PROBABILITY OF IDLE SLOT

All n stations defer:

$$P_{\text{idle}} = (1-p)^n$$

46. PROBABILITY OF COLLISION

Anything other than:

- zero transmissions
- exactly one transmission

means collision.

Therefore:

$$P_{\text{collision}} = 1 - (1-p)^n - np(1-p)^{n-1}$$

This is useful if GATE combines MAC with probability.

47. EXAMPLE

Suppose:

$$n=3, p=0.5$$

Successful probability:

$$3(0.5)(0.5)^2$$

$$= 3(0.125)$$

$$= \boxed{0.375}$$

Idle:

$$(0.5)^3 = 0.125$$

Collision:

$$1 - 0.125 - 0.375$$

$$= \boxed{0.5}$$

48. OPTIMAL p — USEFUL INSIGHT

For n ready stations:

$$P_{\text{success}} = np(1-p)^{n-1}$$

This is maximized when:

$$p = \frac{1}{n}$$

This makes intuitive sense:

If n stations compete, each should transmit with probability around $1/n$.

You don't need to over-memorize this unless a probability-style question appears, but it is useful.

49. CONNECTION TO SLOTTED ALOHA

This probability idea resembles slotted random-access systems.

Difference:

p-persistent CSMA first:

senses whether channel is idle.

Slotted ALOHA does not rely on carrier sensing in the same way.

So:

ALOHA chooses blindly. CSMA listens first.

50. MASTER STORY

Imagine a classroom.

ALOHA

You speak whenever you want.

1-Persistent CSMA

You listen.

If someone is speaking:

Keep waiting attentively.

The instant they stop:

Speak immediately.

Problem: everyone else may do the same.

Non-Persistent CSMA

If someone is speaking:

Go away for a random time and check again.

Less crowding, but sometimes nobody speaks even though room is free.

p-Persistent CSMA

When room becomes free:

Toss a biased coin.

Speak with probability p .

Otherwise wait another slot.

This story captures all three.

'p' NOTEBOOK VERSION

CSMA

Carrier Sense Multiple Access

Listen before transmitting.

Collision can still happen because of:

$\boxed{\text{Propagation delay}}$

Propagation ratio

$\boxed{a = \frac{\tau}{T}}$

where:

- τ = propagation delay
- $T = L/R$ = frame transmission time

Smaller a !' generally better CSMA efficiency.

1-Persistent CSMA

If idle:

Transmit immediately.

If busy:

Continuously sense until idle, then transmit immediately.

Collision risk high when many stations are waiting.

Memory:

1 = Immediate

Non-Persistent CSMA

If idle:

Transmit.

If busy:

Wait random time, then sense again.

Fewer collisions, but may waste idle time.

Memory:

Busy !' leave and return later

p-Persistent CSMA

Usually slotted.

If channel idle:

$P(\text{Transmit})=p$

$P(\text{Wait})=1-p$

Memory:

Probability decides.

Comparison

TEXT

1 - Persistent !' aggressive
Non - Persistent !' random backoff
p - Persistent !' probability

n competing stations in p-persistent access

Exactly one transmits:

$\boxed{np(1-p)^{n-1}}$

Idle:

$\boxed{(1-p)^n}$

Collision:

$$\boxed{1 - (1-p)^n - np(1-p)^{n-1}}$$

Optimal:

$$\boxed{p=1/n}$$

under the simple simultaneous-contention model.

51. 30-SECOND REVISION

Remember:

ALOHA = send first

CSMA = sense first

Then:

TEXT

```
1 - p e r s i s t e n t ! '   I d l e ?   s e n d   i m m e d i a t e l y .
N o n - p e r s i s t e n t ! '   B u s y ?   r a n d o m   w a i t .
p - p e r s i s t e n t ! '   I d l e ?   s e n d   w i t h   p r o b a b i l i t y   p .
```

And the reason collisions still occur:

$$\boxed{\text{Propagation delay}}$$

ACTIVE RECALL — DON'T LOOK BACK

Q1

What does CSMA stand for?

Q2

Why can collisions still happen in CSMA even though stations listen before transmitting?

Q3

A station continuously senses while the channel is busy and transmits immediately when it becomes idle.

Which CSMA type?

A. Non-persistent B. 1-persistent C. p-persistent D. Pure ALOHA

Q4

A station finds the channel busy, waits a random amount of time, then senses again.

Which protocol?

Q5

In p-persistent CSMA:

$$p = 0.3$$

What is the probability of deferring one slot when channel is idle?

Q6 — Numerical

Propagation delay:

$$20 \mu s$$

Frame size:

$$2000 \text{ bits}$$

Rate:

$$20 \text{ Mbps}$$

Find:

1. frame-transmission time (T)
 2. $a = \tau/T$
-

Q7 — Probability

Two stations are ready in p-persistent CSMA with:

$$p = 0.25$$

Find probability that exactly one transmits in the next idle slot.

Q8 — Conceptual

Which tends to produce more synchronized collisions under heavy contention:

A. 1-persistent B. Non-persistent

Explain why.

Q9

If propagation delay becomes smaller while frame-transmission time stays constant, does CSMA performance generally improve or worsen?

Q10 — Preview

Why must a CSMA/CD sender continue transmitting long enough to cover approximately:

$$2\tau$$

in the worst case?

NEXT !' CSMA/CD: Collision Detection, (2τ) , Minimum Frame Size, Ethernet Slot Time & GATE Numericals

USER

next

ASSISTANT

CHAPTER 6 — MEDIUM ACCESS CONTROL

TOPIC 3 — CSMA/CD: Collision Detection, 2τ , Minimum Frame Size & Ethernet Numericals

~~OVERVIEW~~ OVERVIEW HIGH PRIORITY FOR GATE

This is one of the most important numerical topics in the MAC/Ethernet portion.

The central idea is:

$$T_{\text{frame}} \geq 2\tau$$

which leads to:

$$L_{\text{min}} \geq 2\tau R$$

where:

- T_{frame} = frame transmission time
- τ = one-way maximum propagation delay
- R = transmission rate
- L_{min} = minimum frame size in bits

But first, understand **why 2τ** appears.

1. WHAT IS CSMA/CD?

CSMA/CD stands for:

Carrier Sense Multiple Access with Collision Detection

Break the name:

Carrier Sense

Listen before transmitting.

Multiple Access

Many stations share the medium.

Collision Detection

While transmitting, continue monitoring the medium to detect a collision.

So the rough procedure is:

TEXT

```
Sense channel
    ! "
Idle?
    % % % % %4% % % % %
No      Yes
! "      ! "
Wait    Transmit
        ! "
        Keep monitoring
        ! "
        Collision?
        /      \
        Yes     No
        ! "      ! "
Stop transmit Success
        ! "
Random backoff
        ! "
Retry later
```

2. WHY IS COLLISION DETECTION NEEDED?

CSMA already listens before transmitting.

But we learned:

Collisions are still possible because of propagation delay.

Suppose:

TEXT

```
A ----- B
```

A starts transmitting.

Its signal has not yet reached B.

B senses:

Idle.

So B starts.

Collision occurs.

Once a station detects collision, continuing to transmit the entire frame would waste channel capacity.

CSMA/CD therefore says:

Detect collision early !' stop transmission !' retry later.

3. CLASSIC COLLISION SCENARIO

Consider two stations at opposite ends of a shared Ethernet:

TEXT

A ----- B

Let the one-way propagation delay between them be:

τ

Suppose A starts transmitting at:

$t=0$

A's first bit reaches B at:

$t=\tau$

But imagine B starts transmitting just before A's signal arrives.

Say:

$t=\tau-\epsilon$

where ϵ is very small.

B still believes the channel is idle.

Now the transmissions collide near B.

4. WHEN DOES A LEARN ABOUT THE COLLISION?

The collision disturbance must travel back toward A.

That takes approximately another:

τ

So A may learn of the collision only at almost:

2τ

after beginning transmission.

Therefore:

$\boxed{\text{Worst-case collision detection time} \approx 2\tau}$

τ is the heart of CSMA/CD.

5. TIMELINE

TEXT

A

B

$t = 0$

A starts ----->

propagation

$t = 2\tau$

<----- B starts

```
collision indication
<-----
t  "H  2 Å
A detects collision
```

 $\sqrt{2\tau}$

6. WHY MUST THE FRAME BE LONG ENOUGH?

It finishes transmitting at:

$$\backslash [T_{\text{frame}} < 2\tau \backslash]$$

"Transmission completed successfully."

A might learn about the collision only after it has already stopped transmitting.

Therefore:

$$\boxed{T_{\text{frame}} \geq \tau}$$

7. MINIMUM FRAME SIZE DERIVATION

$$T_{\text{frame}} = \frac{L}{R}$$
$$\lfloor \frac{L_{\min}}{R} \rfloor \geq 2 \tau$$
$$\lfloor \boxed{L_{\min} \geq 2\tau_R} \rfloor$$

~~Ø=Ý%ØæŋØíøæw~~with understanding.

8. MEANING OF VARIABLES

$$\boxed{L_{\min} = 2\tau R}$$

where:

- $(L_{\min})$ = minimum frame size in **bits**
- (τ) = maximum one-way propagation delay in **seconds**
- (R) = data rate in **bits/s**

Units check:

$$\left[s \times \frac{\text{bits}}{s} = \text{bits} \right]$$

Correct.

9. GATE-STYLE EXAMPLE 1

One-way maximum propagation delay:

$$\left[\tau = 25 \mu s \right]$$

Link rate:

$$\left[R = 10 \text{ Mbps} \right]$$

Find minimum frame size.

GIVEN

$$\left[\tau = 25 \times 10^{-6} s \right]$$

$$\left[R = 10 \times 10^6 \text{ bps} \right]$$

CONCEPT

$$\left[L_{\min} = 2 \tau R \right]$$

SOLUTION

$$\left[L_{\min} = 2(25 \times 10^{-6})(10 \times 10^6) \right]$$

$$\left[= 500 \text{ bits} \right]$$

ANSWER

$$\left[\boxed{500 \text{ bits}} \right]$$

In bytes:

$$\left[\frac{500}{8} = 62.5 \text{ B} \right]$$

If an actual frame size must be an integer number of bytes and at least this large, it would need to be rounded according to the problem's constraints.

10. & pGATE TRAP — (τ) vs (2τ)

If the question gives:

one-way propagation delay

use:

$$\tau$$

If it directly gives:

round-trip propagation delay

then don't multiply by 2 again.

Example:

Round-trip propagation delay:

$$50 \mu s$$

Then:

$$L_{\min} = R(50 \mu s)$$

not:

$$R(100 \mu s)$$

Read wording carefully.

11. GATE-STYLE EXAMPLE 2 — GIVEN DISTANCE

Suppose:

- maximum cable length = 2 km
- propagation speed = $2 \times 10^8 \text{ m/s}$
- transmission rate = 10 Mbps

Find minimum frame size.

Step 1 — Propagation delay

$$D = 2000 \text{ m}$$

$$\tau = \frac{D}{S}$$

$$= \frac{2000}{2 \times 10^8}$$

$$= 10^{-5} \text{ s}$$

$$= 10 \mu s$$

Step 2 — Round-trip time

$$2\tau = 20 \mu s$$

Step 3 — Minimum frame size

$$L_{\min} = 2\tau R$$

$$= 20 \times 10^{-6} \times 10 \times 10^6$$

$$= \boxed{200 \text{ bits}}$$

12. COMBINED FORMULA USING DISTANCE

Since:

$$\tau = \frac{D}{S}$$

then:

$$L_{\min} = 2R \tau$$

So:

$$L_{\min} = \frac{2DR}{S}$$

where:

- D = maximum one-way physical distance
- S = propagation speed
- R = link rate

Very useful.

13. REARRANGING THE FORMULA

GATE may ask for any unknown.

Starting with:

$$L_{\min} = 2\tau R$$

Maximum propagation delay

$$\tau_{\max} = \frac{L_{\min}}{2R}$$

Maximum round-trip propagation delay

$$2\tau = \frac{L_{\min}}{R}$$

Maximum link rate

$$R_{\max} = \frac{L_{\min}}{2\tau}$$

Maximum distance

Using:

$$\tau = \frac{D}{S}$$

$$D_{\max} = \frac{L_{\min} S}{2R}$$

These rearrangements are common numericals.

14. EXAMPLE — FIND MAXIMUM DISTANCE

Given:

- minimum frame = 512 bits

- $\text{rate} = 10\text{Mbps}$
- $\text{propagation speed} = 2 \times 10^8 \text{m/s}$

Find maximum end-to-end one-way distance.

Use:

$$D_{\max} = \frac{L_{\min} S}{2R}$$

$$L = \frac{512(2 \times 10^8)}{2(10 \times 10^6)}$$

Cancel 2:

$$L = \frac{512 \times 10^8}{10 \times 10^6}$$

$$L = 5120 \text{m}$$

$$L = \boxed{5.12 \text{km}}$$

This is the idealized value before accounting for repeater delays and Ethernet-specific engineering margins.

15. CLASSIC ETHERNET MINIMUM FRAME SIZE

Traditional Ethernet has a minimum frame size of:

$$L = \boxed{64 \text{ bytes}}$$

which is:

$$64 \times 8 = \boxed{512 \text{ bits}}$$

Very important fact.

This was chosen in classic shared half-duplex Ethernet so that a transmitting station could still detect a worst-case collision.

16. ETHERNET SLOT TIME

Classic Ethernet defines a slot time corresponding to:

$$L = \boxed{512 \text{ bit times}}$$

For classic:

$$L = 10 \text{Mbps}$$

Ethernet:

$$\text{Slot time} = \frac{512}{10 \times 10^6}$$

$$L = 51.2 \mu\text{s}$$

So:

$$L = \boxed{51.2 \mu\text{s}}$$

is the famous 10 Mbps Ethernet slot time.

17. WHAT IS A BIT TIME?

A **bit time** is:

Time required to transmit one bit.

If rate is:

$$[R]$$

then:

$$[\boxed{\text{Bit time} = \frac{1}{R}}]$$

For:

$$[10\text{Mbps}]$$

$$[\frac{1}{10^7} = 10^{-7}\text{s}]$$

$$[= 0.1\mu\text{s}]$$

So:

$$[512 \times 0.1\mu\text{s} = 51.2\mu\text{s}]$$

18. IMPORTANT — 64-BYTE FRAME DETAILS

For classic Ethernet, the minimum Ethernet frame from:

Destination MAC through FCS

is:

$$[64\text{B}]$$

This includes:

- 14-byte Ethernet header
- minimum payload + padding
- 4-byte FCS

Minimum payload is therefore:

$$[64 - 14 - 4]$$

$$[= \boxed{46\text{B}}]$$

We'll study the full Ethernet frame properly in the next chapter.

19. PREAMBLE IS A COMMON TRAP

The standard 64-byte minimum Ethernet frame size generally does **not** count the preamble/SFD as part of that MAC frame length.

So if a GATE question discusses exact Ethernet framing, read carefully what components are included.

We'll revisit this in Ethernet.

20. COLLISION DETECTION vs COLLISION AVOIDANCE

CSMA/CD

CD = Collision Detection

Used historically in shared half-duplex Ethernet.

CSMA/CA

CA = Collision Avoidance

Associated with wireless LANs such as Wi-Fi.

Why wireless prefers avoidance?

Because reliably transmitting and detecting another weak signal at the same time is difficult in wireless environments.

For your supplied syllabus, Ethernet/CSMA-CD is the main focus.

21. WHAT HAPPENS AFTER COLLISION?

Once collision is detected:

1. Abort transmission.
2. Send/ensure a jam signal so all stations recognize the collision.
3. Wait according to a random backoff algorithm.
4. Try again.

The important backoff algorithm is:

Binary Exponential Backoff

22. BINARY EXPONENTIAL BACKOFF

After collisions, the station chooses a random waiting interval.

As repeated collisions occur:

the range of possible waiting times grows exponentially.

This reduces the chance of the same stations repeatedly colliding.

23. BASIC BACKOFF RULE

After the k -th collision, choose a random integer:

$\text{rand} [0, 2^k - 1]$

from a range related to:

$\text{rand} [0, 2^k - 1]$

subject to Ethernet's cap on the exponent.

Then wait:

$\lfloor \boxed{K \times \text{SlotTime}} \rfloor$

before retrying.

For basic GATE understanding:

Every collision roughly doubles the contention window until the specified cap.

24. EXAMPLE

After first collision:

$\lfloor k=1 \rfloor$

Possible $\lfloor(K)\rfloor$:

$\lfloor 0,1 \rfloor$

After second collision:

$\lfloor k=2 \rfloor$

Possible:

$\lfloor 0,1,2,3 \rfloor$

After third:

$\lfloor k=3 \rfloor$

Possible:

$\lfloor 0,\text{dots},7 \rfloor$

So collision window grows:

TEXT

2 !' 4 !' 8 !' 16 !' . . .

possible slot choices.

25. ~~Ø~~ MEMORY

Binary exponential backoff:

More collisions !' larger random waiting range.

This reduces synchronized retransmissions.

26. WHY BINARY?

Because the window grows using powers of:

$\lfloor 2 \rfloor$

$\lfloor 2^1, 2^2, 2^3, \text{dots} \rfloor$

Hence:

Binary exponential.

27. JAM SIGNAL — CONCEPTUAL

Upon detecting collision, a transmitting station sends a short jam indication/pattern so that:

all stations recognize that a collision has occurred.

For GATE, the main concept matters more than memorizing implementation details.

28. CSMA/CD COMPLETE PROCEDURE

You can remember:

1. Sense

Is medium idle?

2. Send

If idle, transmit.

3. Detect

Monitor while transmitting.

4. Collision?

Abort + jam.

5. Backoff

Wait random number of slot times.

6. Retry

Try again.

Mnemonic:

S-S-D-B-R

Sense !' Send !' Detect !' Backoff !' Retry.

29. WHY DOES MINIMUM FRAME SIZE DEPEND ON RATE?

Recall:

$$L_{\min} = 2\tau R$$

Suppose physical network dimensions stay the same, so τ stays constant.

If:

$[R]$

increases, then:

$[L_{\min}]$

must increase proportionally.

Why?

At a higher data rate, more bits can be transmitted during the same propagation interval.

So to stay transmitting for (2τ) , the frame needs more bits.

30. EXAMPLE — RATE DOUBLES

Suppose:

$[\tau = 25 \mu s]$

At:

$[10 \text{ Mbps}]$

$[L_{\min} = 500 \text{ bits}]$

If rate becomes:

$[20 \text{ Mbps}]$

same topology:

$[L_{\min} = 2(25 \mu s)(20 \text{ Mbps})]$

$[= 1000 \text{ bits}]$

So doubling rate doubles required minimum frame size if propagation delay is unchanged.

31. WHY FAST ETHERNET COULD NOT JUST KEEP EVERYTHING THE SAME

Classic Ethernet's collision-detection design depends on:

$[FrameTime \geq RoundTripPropagation]$

If link rate increases but frame size remains 64 bytes:

transmission time becomes shorter.

Therefore allowable collision-domain diameter must shrink, or architecture must change.

This is one reason faster Ethernet imposes stricter timing/topology constraints, and modern switched full-duplex Ethernet eliminates the shared-medium collision mechanism entirely.

32. FULL-DUPLEX SWITCHED ETHERNET

Modern Ethernet commonly uses:

- switches
 - point-to-point links
-

- full duplex

In such links:

collisions do not occur in the old shared-medium sense.

Therefore:

CSMA/CD is unnecessary in full-duplex switched Ethernet.

Ø = important conceptual fact.

33. HALF-DUPLEX vs FULL-DUPLEX

Half-Duplex

A link/device can communicate in both directions, but not simultaneously.

Classic shared Ethernet collision concepts matter.

Full-Duplex

Both ends can transmit simultaneously over dedicated point-to-point link resources.

No shared collision domain between the two ends.

Therefore:

no CSMA/CD collision contention is needed.

34. & pGATE TRAP

Do not say:

"Ethernet always uses CSMA/CD."

More accurate:

Traditional shared / half-duplex Ethernet used CSMA/CD.

Modern switched full-duplex Ethernet does not require CSMA/CD.

35. COLLISION DOMAIN — PREVIEW

A collision domain is:

A set of interfaces sharing a medium where simultaneous transmissions can collide.

Traditional hub:

TEXT

```
A \
B -- HUB -- D
C /
```

All ports belong to essentially one collision domain.

A switch:

TEXT

```
A --\
B --- SWITCH --- D
C --/
```

creates separate point-to-point collision domains per port/link.

In full-duplex operation, collisions are effectively eliminated.

We'll revisit this in Ethernet.

36. GATE-STYLE NUMERICAL 3

Minimum frame size:

$[512\text{bits}]$

Rate:

$[100\text{Mbps}]$

What maximum one-way propagation delay is allowed under:

$[T_{\text{frame}} \geq 2\tau]$

?

Frame time

$[T_{\text{frame}} = \frac{512}{100 \times 10^6}]$

$[= 5.12 \mu\text{s}]$

Since:

$[2\tau \leq 5.12 \mu\text{s}]$

$[\tau \leq 2.56 \mu\text{s}]$

ANSWER

$[\boxed{2.56 \mu\text{s}}]$

37. GATE-STYLE NUMERICAL 4 — DISTANCE

Using the previous:

$[\tau_{\text{max}} = 2.56 \mu\text{s}]$

Propagation speed:

$$\left[2 \times 10^8 \text{ m/s} \right]$$

Distance:

$$\left[D = \tau S \right]$$

$$\left[= 2.56 \times 10^{-6} \times 2 \times 10^8 \right]$$

$$\left[= 512 \text{ m} \right]$$

So idealized maximum one-way diameter:

$$\left[\boxed{512 \text{ m}} \right]$$

under those simplified assumptions.

38. GATE-STYLE NUMERICAL 5 — FRAME VALIDITY

Network:

$$\left[R = 20 \text{ Mbps} \right]$$

One-way propagation delay:

$$\left[\tau = 15 \mu \text{ s} \right]$$

Frame size:

$$\left[400 \text{ bits} \right]$$

Can CSMA/CD reliably detect worst-case collision?

Step 1 — Frame time

$$\left[T_f = \frac{400}{20 \times 10^6} \right]$$

$$\left[= 20 \mu \text{ s} \right]$$

Step 2 — Worst collision detection time

$$\left[2\tau = 30 \mu \text{ s} \right]$$

Compare:

$$\left[20 < 30 \right]$$

Therefore frame finishes too early.

ANSWER

$$\left[\boxed{\text{No}} \right]$$

Minimum required:

$$\left[L_{\min} = 2(15 \mu \text{ s})(20 \text{ Mbps}) \right]$$

$$\left[= 600 \text{ bits} \right]$$

39. ANOTHER EXAM METHOD

Instead of converting frame time first, directly compare:

$$\sqrt{L}$$

with:

$$\sqrt{2\tau R}$$

If:

$$\sqrt{L} \geq \sqrt{2\tau R}$$

!' sufficient.

If:

$$\sqrt{L} < \sqrt{2\tau R}$$

!' insufficient.

This is usually faster.

40. CONNECTION WITH BANDWIDTH-DELAY PRODUCT

Earlier:

$$\sqrt{BDP = R\tau}$$

The minimum CSMA/CD frame condition is:

$$\sqrt{L_{\min}} = \sqrt{2R\tau}$$

Therefore:

$$\sqrt{\boxed{L_{\min}} = 2 \times \text{one-way bandwidth-delay product}}$$

or equivalently the number of bits that can fit into the **round-trip propagation interval**.

Nice conceptual connection.

41. WHY "BITS IN THE CABLE" MATTER?

Suppose a network has one-way BDP:

$$\sqrt{256\text{bits}}$$

Then during one propagation delay:

$$\sqrt{256\text{bits}}$$

can be put onto the medium.

During round-trip:

$$\sqrt{512\text{bits}}$$

Thus a 512-bit minimum frame ensures the sender remains active throughout that critical collision-detection window.

42. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Minimum frame size

Given:

- rate
- propagation delay

Use:

$$\boxed{L_{\min} = 2\tau R}$$

Pattern 2 — Maximum distance

Given:

- frame size
- rate
- propagation speed

Use:

$$\boxed{D_{\max} = \frac{LS}{2R}}$$

Pattern 3 — Maximum data rate

Given:

- frame size
- propagation delay

Use:

$$\boxed{R_{\max} = \frac{L}{2\tau}}$$

Pattern 4 — Check feasibility

Compare:

$$\frac{LR}{L}$$

with:

$$2\tau$$

Pattern 5 — Ethernet facts

Classic minimum Ethernet frame:

$$64B = 512\text{bits}$$

Classic 10 Mbps slot time:

$$51.2\mu s$$

Pattern 6 — Binary exponential backoff

Given collision number k , determine contention-window possibilities.

Pattern 7 — Full duplex

Ask whether CSMA/CD is required.

Answer:

Not for normal switched full-duplex Ethernet links.

43. & pGATE TRAPS

Trap 1

Use **round-trip propagation** for collision detection.

2τ

Trap 2

If RTT propagation is already given, don't double it.

Trap 3

Frame size must be converted to **bits** before combining with bps.

Trap 4

Don't confuse minimum payload with minimum Ethernet frame size.

Classic Ethernet:

64B

frame minimum,

while minimum payload is typically:

46B

Trap 5

64 bytes is not just application data.

It includes Ethernet header and FCS.

Trap 6

Modern full-duplex Ethernet does not use collision detection in the old CSMA/CD sense.

Trap 7

CSMA/CD:

detects collisions

It does not guarantee collisions never happen.

Trap 8

Binary exponential backoff is random.

Stations do not simply wait:

$\lfloor 2^k \rfloor$

slots exactly.

They choose from a range.

44. MASTER MEMORY TRICK

For CSMA/CD remember:

L "e2 Ä R

Think:

Long enough for Round Trip

- L = frame bits
- R = rate
- 2τ = collision round-trip

Or:

Frame time $\geq$ collision round trip

$\boxed{\frac{LR}{2} \geq \tau}$

This is the concept. Everything else is algebra.

45. ETHERNET MEMORY BOX

Classic Ethernet:

$\boxed{\text{Minimum frame} = 64\text{B} = 512\text{bits}}$

At 10 Mbps:

$\boxed{\text{SlotTime} = 51.2\ \mu\text{s}}$

And:

More collisions !' exponentially larger random backoff range.

' p NOTEBOOK VERSION

CSMA/CD

Carrier Sense Multiple Access with Collision Detection

Procedure:

1. Sense channel.
 2. If idle, transmit.
 3. Continue monitoring while transmitting.
 4. If collision detected !' abort/jam.
 5. Random backoff.
 6. Retransmit.
-

Why collision is still possible

Because of:

$\boxed{\text{Propagation delay}}$

A distant station may sense idle before another station's signal reaches it.

Worst-case collision detection

If one-way propagation delay is:

τ

worst-case detection:

$\boxed{2\tau}$

Therefore sender must still be transmitting:

$\boxed{T_f \geq 2\tau}$

Since:

$T_f = \frac{L}{R}$

$\boxed{L_{\min} = 2\tau R}$

Distance form

$\tau = \frac{D}{S}$

so:

$\boxed{L_{\min} = \frac{2DR}{S}}$

and:

$\boxed{D_{\max} = \frac{L_{\min} S}{2R}}$

Classic Ethernet

Minimum frame:

$\boxed{64\text{B}=512\text{bits}}$

10 Mbps slot time:

$\boxed{51.2\mu\text{s}}$

Binary Exponential Backoff

After collision, choose random number of slot times from an increasing range.

Rough pattern:

TEXT

```
1 s t   c o l l i s i o n   ! '   0 . . 1
2 n d   ! '   0 . . 3
3 r d   ! '   0 . . 7
...
```

Memory:

More collision !' larger random window.

46. 5-MINUTE REVISION — MAC SO FAR

Pure ALOHA

$S = Ge^{-2G}$

Max:

18.4%

Vulnerable:

$2T$

Slotted ALOHA

$S = Ge^{-G}$

Max:

36.8%

Vulnerable:

T

CSMA

Sense before sending.

Types:

- 1-persistent
- non-persistent
- p-persistent

CSMA/CD

Sense + transmit + detect collision.

Critical:

$$\boxed{\frac{LR}{\geq 2\tau}}$$

47. 30-SECOND REVISION

If you remember only one thing from this topic:

$$\boxed{L_{\min} = 2\tau R}$$

Why?

Sender must still be transmitting when the worst-case collision returns.

Classic Ethernet:

$$\boxed{64B = 512\text{bits}}$$

and 10 Mbps:

$$\boxed{51.2\mu\text{s}}$$

ACTIVE RECALL — DON'T LOOK BACK

Q1

What does CSMA/CD stand for?

Q2

Why is the worst-case collision-detection time approximately:

$$\boxed{2\tau}$$

rather than τ ?

Q3

A network has:

$$\boxed{\tau = 20\mu\text{s}}$$

and:

$R=10\text{Mbps}$

Find minimum frame size in:

1. bits
 2. bytes
-

Q4

A frame contains:

800bits

and link rate is:

20Mbps

What is the maximum permitted one-way propagation delay for reliable worst-case collision detection?

Q5 — Distance Numerical

Frame:

512bits

Rate:

100Mbps

Propagation speed:

$2 \times 10^8 \text{m/s}$

Find the idealized maximum one-way network distance.

Q6 — GATE Trap

The round-trip propagation delay is explicitly given as:

$40\mu\text{s}$

and rate is:

10Mbps

What minimum frame size is required?

Do you multiply the $40\mu\text{s}$ by 2 again?

Q7

What is the minimum classic Ethernet frame size?

A. 46 bytes B. 48 bytes C. 64 bytes D. 512 bytes

Q8

Why isn't CSMA/CD normally required on modern full-duplex switched Ethernet?

Q9

After the third collision, under the basic binary exponential-backoff idea, which random slot numbers are available?

Q10 — GATE Thinking

If the transmission rate doubles but:

- maximum distance
- propagation speed
- minimum frame size

all remain unchanged, does CSMA/CD collision detection become easier or harder to satisfy? Explain using:

$$L_{\min} \geq 2\tau R$$

NEXT !' Ethernet: MAC Addresses, Ethernet Frame Format, Minimum/Maximum Frame Size, Switches & Collision Domains

USER

next

ASSISTANT

CHAPTER 7 — ETHERNET

TOPIC 1 — Ethernet Fundamentals, MAC Addresses, Frame Format, Frame Sizes, Switches & Collision Domains

VERY HIGH PRIORITY FOR GATE

This topic connects almost everything we just learned:

Framing + MAC Addressing + CRC + CSMA/CD + Switching

The most important Ethernet facts you should eventually remember are:

$$\boxed{\text{MAC address} = 48 \text{ bits} = 6 \text{ bytes}}$$

$$\boxed{\text{Minimum Ethernet frame} = 64 \text{ bytes}}$$

$$\boxed{\text{Maximum standard Ethernet frame} = 1518 \text{ bytes}}$$

and the standard frame structure:

TEXT

Destination	Source	Type/Length	Data+Padding	FCS
6 B	6 B	2 B	46–1500 B	4 B

We will build this properly rather than memorizing numbers blindly.

1. WHAT IS ETHERNET?

Ethernet is a family of technologies used primarily for:

Local Area Networks (LANs)

It defines important Data-Link and Physical-layer mechanisms.

For GATE, the main Ethernet concepts are:

- Ethernet frame
- MAC addresses
- frame sizes
- FCS/CRC
- shared vs switched Ethernet
- collision domains
- CSMA/CD
- switches and MAC-learning basics

2. WHERE DOES ETHERNET FIT?

In our network stack:

TEXT

Application
Transport
Network

D a t a L i n k ! • E t h e r n e t m a i n l y h e r e
P h y s i c a l ! • E t h e r n e t a l s o d e f i n e s p h y s i c a l v a r i a n t s

At the Data-Link level, Ethernet handles:

- framing
- MAC addressing
- error detection
- historically shared-medium access

3. SIMPLE ETHERNET NETWORK

Suppose:

TEXT

PC A ---- Ethernet Switch ---- PC B

A wants to send an IP packet to B.

The IP packet becomes the payload of an Ethernet frame:

TEXT

+-----+
| Ethernet Header |

```

+-----+
| IP Packet |
+-----+
| Ethernet FCS |
+-----+

```

The switch uses the frame's:

Destination MAC address

to decide where to forward it.

4. ETHERNET MAC ADDRESS

A normal Ethernet MAC address is:

$\boxed{48 \text{ bits}}$

Since:

$8 \text{ bits} = 1 \text{ byte}$

$48 / 8 = 6$

Therefore:

$\boxed{6 \text{ bytes}}$

5. MAC ADDRESS REPRESENTATION

A 48-bit MAC is usually written as six hexadecimal groups.

Example:

TEXT

3C : 52 : 82 : A1 : 20 : 9F

Each pair of hexadecimal digits represents:

8 bits

because one hex digit represents:

4 bits

Thus:

TEXT

3C | 52 | 82 | A1 | 20 | 9F

= 6 bytes.

6. MEMORY TRICK

MAC:

6 bytes = 12 hex digits = 48 bits

Remember:

$\boxed{6B=48b}$

7. SOURCE AND DESTINATION MAC

An Ethernet frame normally contains both:

- Destination MAC
- Source MAC

Conceptually:

TEXT

```
+-----+
| Destination MAC |
+-----+
| Source MAC      |
+-----+
| Rest of frame   |
+-----+
```

Each MAC address:

$\boxed{6B}$

So together:

$\boxed{12B}$

8. ETHERNET BROADCAST ADDRESS

~~Useful~~ fact.

Ethernet broadcast MAC:

TEXT

FF:FF:FF:FF:FF:FF

In binary, every bit is 1.

This means:

Deliver to all stations in the local Ethernet broadcast domain.

9. UNICAST, BROADCAST, MULTICAST

Unicast

One destination interface.

Broadcast

All devices in local broadcast domain.

Multicast

Selected group.

Don't confuse:

Broadcast domain

with:

Collision domain.

We will separate those shortly.

10. ETHERNET FRAME FORMAT

This is extremely important.

A common Ethernet frame format is:

TEXT

Destination	Source	Type/Length	Data + Padding	FCS
6 B	6 B	2 B	46-1500 B	4 B

The actual MAC frame from Destination through FCS therefore ranges from:

\[64B \]

to:

\[1518B \]

for standard untagged Ethernet.

11. FRAME FIELD 1 — DESTINATION ADDRESS

Size:

\[\boxed{6B} \]

Tells Ethernet switches/interfaces:

Who should receive this frame?

It may identify:

- one interface
- broadcast

- multicast group

12. FRAME FIELD 2 — SOURCE ADDRESS

Size:

$\boxed{6B}$

Identifies the transmitting Ethernet interface.

Switches also use source addresses to:

Learn which MAC addresses are reachable through which ports.

We'll see this shortly.

13. FRAME FIELD 3 — TYPE/LENGTH

Size:

$\boxed{2B}$

Depending on Ethernet framing interpretation, this field indicates either:

- payload protocol type

or

- payload length

For Ethernet II, the EtherType indicates the upper-layer protocol.

For example, conceptually:

TEXT

IPv4 payload

can be identified using the type field.

For GATE, focus primarily on:

2-byte field identifying the payload/protocol context.

14. FRAME FIELD 4 — DATA

Maximum data payload:

$\boxed{1500B}$

This is the standard Ethernet:

MTU — Maximum Transmission Unit

for the IP payload carried by a conventional Ethernet frame.

0=1% number becomes important later in:

IPv4 fragmentation.

Remember:

$\boxed{\text{Ethernet MTU}=1500\text{B}}$

15. MINIMUM DATA FIELD

Minimum Ethernet data field:

$\boxed{46\text{B}}$

Why?

Because Ethernet frame must be at least:

$\boxed{64\text{B}}$

Let's derive it.

Fixed fields:

Destination:

$\boxed{6\text{B}}$

Source:

$\boxed{6\text{B}}$

Type:

$\boxed{2\text{B}}$

FCS:

$\boxed{4\text{B}}$

Total fixed:

$\boxed{6+6+2+4=18\text{B}}$

Minimum total frame:

$\boxed{64\text{B}}$

Therefore minimum data+padding:

$\boxed{64-18}$

$\boxed{=46\text{B}}$

16. WHY IS PADDING NEEDED?

Suppose actual upper-layer data is only:

$\boxed{20\text{B}}$

Ethernet still needs data field of at least:

$\boxed{46\text{B}}$

So it adds:

$46 - 20 = 26\text{B}$

of padding.

Conceptually:

TEXT

Actual payload		Padding
20 B		26 B

Total data field:

46B

17. GATE NUMERICAL — PADDING

Suppose Ethernet receives a 30-byte network-layer payload.

Required Ethernet data field:

46B

Therefore padding:

$46 - 30$

$= 16\text{B}$

18. GATE TRAP — PADDING IS NOT ALWAYS PRESENT

If payload:

500B

then:

$500 > 46$

so no padding is required.

Padding is needed only when payload is smaller than the minimum required data field.

19. FIELD 5 — FCS

FCS stands for:

Frame Check Sequence

Size:

$4\text{B} = 32\text{bits}$

Ethernet FCS is based on:

CRC

So remember:

TEXT

E t h e r n e t ! ' C R C ! ' F C S

20. WHAT DOES FCS DO?

Sender calculates CRC from the relevant frame bits.

Receiver calculates/checks it.

If the received frame fails the CRC check:

Frame is considered corrupted and discarded.

Ethernet itself does not perform TCP-like retransmission/reliability at this layer.

Higher-layer protocols may recover if needed.

21. ERROR DETECTION vs RETRANSMISSION

Ethernet detects corrupted frames using FCS.

But don't conclude:

Ethernet guarantees reliable end-to-end delivery.

It doesn't.

TCP provides reliability at a higher layer when used.

22. STANDARD FRAME SIZE

Without VLAN tagging and excluding preamble/SFD:

Minimum:

$\boxed{64\text{B}}$

Maximum:

$\boxed{1518\text{B}}$

Why maximum?

Fixed overhead:

$\boxed{18\text{B}}$

Maximum payload:

$\boxed{1500\text{B}}$

So:

$$\boxed{1500+18 = 1518\text{B}}$$

23. FRAME SIZE FORMULA

For an ordinary untagged Ethernet MAC frame:

$$\text{Frame} = 6+6+2+\text{Data}+4$$

$$= \boxed{18+\text{Data}}$$

subject to:

$$46 \leq \text{Data} \leq 1500$$

So:

$$64 \leq \text{Frame} \leq 1518$$

bytes.

24. PREAMBLE AND SFD

Before the actual Ethernet frame, the physical transmission includes:

Preamble

$$7\text{B}$$

SFD — Start Frame Delimiter

$$1\text{B}$$

Together:

$$\boxed{8\text{B}}$$

They help the receiver synchronize and identify frame start.

25. IMPORTANT FRAME DIAGRAM

On the wire, conceptually:

TEXT

Preamble	SFD	Destination	Source	Type	Data+Pad	FCS	
7 B	1 B	6 B	6 B	2 B	46-1500	4 B	

26. & pGATE TRAP — 64 BYTES

The standard:

$$64\text{B}$$

minimum normally refers to:

TEXT

D e s t i n a t i o n ! ' F C S

It excludes:

- 7-byte preamble
- 1-byte SFD

Therefore transmitted physical bytes before considering inter-frame gap could be:

$[64 + 8 = 72\text{B}]$

for a minimum frame.

If GATE asks "Ethernet frame size," normally use 64 B minimum unless it explicitly includes preamble/SFD.

27. INTERFRAME GAP — USEFUL FACT

Ethernet also requires an idle interval between frames.

Classic Ethernet's interframe gap corresponds to:

$[96 \times \text{bit times}]$

You don't need to prioritize memorizing this unless the problem explicitly involves on-wire timing/efficiency.

But if it does:

Preamble/SFD + frame + interframe gap may all contribute to actual channel occupancy.

28. FULL ETHERNET TIMING OVERHEAD

For advanced timing questions, a transmitted frame can involve:

- Preamble + SFD = 8 B
- Ethernet MAC frame = 64–1518 B
- interframe gap = 12-byte-time equivalent

because:

$[96\text{bits} = 12\text{B}]$

So a minimum 64-byte Ethernet frame can consume a larger effective wire-time than just 64 bytes.

Only use this when explicitly relevant.

29. WHY IS MINIMUM FRAME 64 BYTES?

This connects directly to CSMA/CD.

Classic Ethernet needs:

$[T_{\text{frame}} \geq 2\tau]$

At 10 Mbps, 512 bits take:

$[\frac{512}{10^7} = 51.2\mu\text{s}]$

That provided sufficient collision-detection slot time for the designed network diameter.

Therefore:

$$\boxed{64B=512\text{bits}}$$

is not arbitrary.

30. ETHERNET MTU vs FRAME SIZE

Do not confuse:

Ethernet MTU

$$\boxed{1500B}$$

means maximum upper-layer payload in a normal Ethernet frame.

Maximum Ethernet frame

$$\boxed{1518B}$$

includes Ethernet header and FCS.

This distinction will become extremely important with IPv4 fragmentation.

31. MEMORY TRICK

Remember:

Payload 1500 !' Frame 1518

because:

$$\boxed{18B}$$

Ethernet header/FCS overhead.

And:

Payload minimum 46 !' Frame minimum 64

again:

$$\boxed{46+18=64}$$

32. COMPLETE FRAME TABLE

Field	Size
Preamble	7 B
SFD	1 B
Destination MAC	6 B

Field	Size
Source MAC	6 B
Type/Length	2 B
Data + padding	46–1500 B
FCS	4 B

MAC-frame size excluding Preamble/SFD:

$\boxed{64-1518\text{B}}$

33. NOW SWITCHED ETHERNET

Historically, Ethernet could use a shared cable or hub.

Modern Ethernet typically uses switches.

Example:

TEXT

```
A -----\
B ----- SWITCH ----- D
C -----/
```

The switch receives Ethernet frames and forwards them based on:

MAC addresses.

34. WHAT DOES A SWITCH DO?

A Layer-2 Ethernet switch maintains a table like:

MAC Address	Port
MAC_A	1
MAC_B	3
MAC_C	4

This is commonly called a:

MAC address table / forwarding table.

35. HOW DOES A SWITCH LEARN?

It's important.

When a frame arrives, the switch looks at:

Source MAC address

Suppose frame arrives on Port 2:

TEXT

Source MAC = AA

The switch learns:

TEXT

AA !' Port 2

So:

Learning uses SOURCE MAC.

36. HOW DOES A SWITCH FORWARD?

After learning from source, switch examines:

Destination MAC

Then:

Destination known

Forward only through the associated port.

Destination unknown

Flood frame out all appropriate ports except incoming port.

Broadcast

Flood to all appropriate ports except incoming port.

Ø=Yes source-vs-destination distinction is common in GATE.

37. Ø>MEMORY TRICK

Learn SOURCE, Forward DESTINATION

This is one of the best Ethernet memory lines.

38. SWITCH LEARNING EXAMPLE

Topology:

TEXT

```
A -- Port1
B -- Port2    SWITCH
C -- Port3
```

Initially table is empty.

A sends frame to B.

Frame arrives Port1.

Source:

\[MAC_A \]

Switch learns:

TEXT

```
M A C _ A ! ' P o r t 1
```

Destination B is unknown.

So switch floods:

TEXT

```
Port2
Port3
```

B accepts it.

C ignores it.

39. B REPLIES

B replies to A.

Frame enters Port2.

Switch learns:

TEXT

```
M A C _ B ! ' P o r t 2
```

Destination:

\[MAC_A \]

already known at Port1.

So switch sends frame only to:

TEXT

```
Port1
```

No flooding needed.

40. MAC TABLE OVER TIME

After these two transmissions:

MAC	Port
A	1
B	2

This is how an Ethernet switch learns network topology dynamically.

41. HUB vs SWITCH

Ø = Important comparison.

Hub

Physical-layer device.

It essentially repeats incoming signals to all other ports.

TEXT

A ! ' H U B ! ' B , C , D

It does not understand MAC addresses.

Switch

Data-Link-layer device.

It reads Ethernet frames and uses MAC table.

TEXT

A ! ' S W I T C H ! ' o n l y n e e d e d p o r t

when destination is known.

42. HUB vs SWITCH TABLE

Hub	Switch
Layer 1	Layer 2
Repeats bits	Forwards frames
No MAC table	MAC table
Shared collision domain	Separate collision domain per port/link
Usually half-duplex shared behavior	Commonly full duplex

Hub	Switch
Lower efficiency	Higher efficiency

Memory:

Hub hears nothing intelligently; switch studies MACs.

43. COLLISION DOMAIN

A collision domain is:

A network region in which simultaneous transmissions can interfere/collide under shared half-duplex operation.

44. HUB AND COLLISION DOMAINS

Suppose:

TEXT

```
A \
B -- HUB -- D
C /
```

All connected devices share essentially:

$\boxed{1 \text{ collision domain}}$

because the hub repeats signals everywhere.

45. SWITCH AND COLLISION DOMAINS

Suppose four devices connect to four switch ports:

TEXT

```
A -- \
B --- \
C ---- SWITCH
D --- /
```

Each point-to-point switch link constitutes its own collision domain conceptually.

Therefore:

$\boxed{4 \text{ switch ports/links} \rightarrow 4 \text{ collision domains}}$

in the usual basic counting model.

And with full-duplex operation:

collisions don't actually occur on those dedicated links.

46. MEMORY

Hub = one big collision room.

Switch = one separate room per port.

Very easy to remember.

47. BROADCAST DOMAIN

Do not confuse with collision domain.

A Layer-2 switch normally forwards Ethernet broadcasts to all ports in the same VLAN/broadcast domain.

Therefore:

A normal switch does not divide broadcast domains merely by having multiple ports.

A router does.

48. HUB, SWITCH, ROUTER — DOMAINS

Basic GATE view:

Device	Collision domains	Broadcast domains
Hub	One shared	One
Switch	One per port/link	Usually one per VLAN
Router	Separates them	Separates broadcast domains

Very important.

49. SIMPLE EXAMPLE

Suppose 8 PCs are attached to an 8-port switch, all in one VLAN.

Basic answer:

Collision domains:

$\boxed{8}$

Broadcast domains:

$\boxed{1}$

50. IF A ROUTER IS ADDED

TEXT

LAN A ---- Router ---- LAN B

The router separates:

Broadcast domains.

So there are at least:

\[2 \]

broadcast domains across the two LAN interfaces.

51. SWITCH FORWARDING BEHAVIOUR

Given a frame:

Case 1 — Destination MAC known on another port

Forward only to that port.

Case 2 — Destination MAC known on incoming port

No need to forward elsewhere; frame is filtered.

Case 3 — Destination unknown

Flood.

Case 4 — Broadcast MAC

Flood.

52. UNKNOWN UNICAST

Suppose destination:

TEXT

AA:BB:CC:DD:EE:FF

is not in switch table.

This is an:

Unknown unicast

Switch floods it to all appropriate ports except the incoming one.

Don't confuse flooding with broadcast addressing: unknown unicast is still addressed to one MAC, but the switch doesn't yet know where that MAC resides.

53. ETHERNET FRAME THROUGH A ROUTER

Consider:

TEXT

A ----- R ----- B

A and B are on different networks.

On first link:

TEXT

Frame:
Source MAC = A
Destination MAC = R-left-interface

Router strips the incoming Ethernet frame.

Then creates a new frame on the outgoing link:

TEXT

Source MAC = R-right-interface
Destination MAC = B

Therefore Ethernet frames are:

Hop-by-hop.

54. WHAT STAYS THE SAME?

Ignoring NAT and some special processing:

IP packet endpoints generally remain:

TEXT

Source IP = A
Destination IP = B

while Layer-2 MAC addresses change at every routed hop.

Again:

IP = end goal, MAC = next hop.

55. ETHERNET SPEEDS

Ethernet has evolved through many speeds:

- 10 Mbps Ethernet
- 100 Mbps Fast Ethernet
- 1 Gbps Gigabit Ethernet

- 10 Gbps and beyond

You don't need to memorize every physical standard for this syllabus.

Focus on how rate affects:

$d_{\text{trans}} = \frac{LR}{\text{...}}$

and classic CSMA/CD constraints.

56. FULL-DUPLEX SWITCHED ETHERNET

Modern switched Ethernet often has:

TEXT

Host A ↔ Switch

where both ends can transmit simultaneously.

Because each link is dedicated and full-duplex:

No shared-medium collision occurs.

Therefore:

$\boxed{\text{No CSMA/CD needed}}$

on those links.

57. GATE-STYLE FRAME SIZE EXAMPLE

An Ethernet frame carries:

1000B

of payload.

Calculate Ethernet MAC-frame size excluding preamble and SFD.

Fixed overhead:

$6+6+2+4=18\text{B}$

Therefore:

$1000+18 = \boxed{1018\text{B}}$

No padding because:

$1000 > 46$

58. GATE-STYLE PADDING EXAMPLE

Payload:

10B

Minimum data field:

$\lfloor 46B \rfloor$

Padding:

$\lfloor 46-10=36B \rfloor$

Frame size:

$\lfloor 6+6+2+46+4 \rfloor$

$\lfloor \boxed{64B} \rfloor$

59. GATE-STYLE MAXIMUM FRAME

Payload:

$\lfloor 1500B \rfloor$

Then:

$\lfloor \text{Frame}=1500+18 \rfloor$

$\lfloor \boxed{1518B} \rfloor$

If payload were:

$\lfloor 1600B \rfloor$

it does not fit in a standard 1500-byte Ethernet MTU without some mechanism such as fragmentation at a higher layer or use of a larger-frame technology, depending on context.

60. CONNECTION TO IPv4 FRAGMENTATION

~~Ø=Σ~~ Save this for later.

Ethernet:

$\lfloor \text{MTU}=1500B \rfloor$

Suppose an IP datagram is:

$\lfloor 4000B \rfloor$

It cannot fit into one standard Ethernet payload.

Therefore IPv4 may need to:

Fragment the datagram.

Later we'll calculate:

- fragment sizes
- offsets
- MF flags

using this exact 1500-byte MTU idea.

61. HOW GATE CAN ASK ETHERNET

Pattern 1 — Frame field sizes

Find:

- header overhead
 - payload
 - padding
 - total frame size
-

Pattern 2 — MAC address length

\[48bits \]

Pattern 3 — Minimum/maximum frame

\[64B,\quad1518B \]

Pattern 4 — Minimum Ethernet payload

\[46B \]

Pattern 5 — MTU

\[1500B \]

Pattern 6 — Switch learning

Ask:

Which address does switch use to learn table entries?

Answer:

Source MAC.

Pattern 7 — Switch forwarding

Ask:

Which address determines output port?

Answer:

Destination MAC.

Pattern 8 — Unknown destination

Answer:

Flood.

Pattern 9 — Collision domains

Hub vs switch.

Pattern 10 — Full-duplex Ethernet

Does CSMA/CD operate?

Normally no.

62. & pGATE TRAPS

Trap 1

MAC address:

\[48bits \]

not 48 bytes.

Trap 2

Ethernet MTU:

\[1500B \]

is payload limit, not complete frame size.

Trap 3

Maximum untagged standard frame:

\[1518B \]

not 1500 B.

Trap 4

Minimum Ethernet frame:

\[64B \]

not 46 B.

46 B is minimum:

Data+padding field.

Trap 5

Switch learns from:

Source MAC

not destination MAC.

Trap 6

Switch forwards based on:

Destination MAC

not source MAC.

Trap 7

Unknown unicast:

Flood.

Trap 8

Switch does not inherently break broadcast domains per port.

It breaks collision domains.

Trap 9

Router breaks broadcast domains.

Trap 10

Ethernet FCS uses CRC.

TCP checksum is a different mechanism.

Trap 11

64-byte minimum frame normally excludes:

- Preamble
 - SFD
-

Trap 12

Modern full-duplex switched Ethernet does not need CSMA/CD.

63. MASTER MEMORY SYSTEM

Remember these Ethernet numbers as:

6 - 6 - 2 - 4 6! 1 5 0 0 - 4

TEXT

Destination = 6
Source = 6
Type = 2
Data = 46 to 1500
FCS = 4

Then:

$\backslash [6+6+2+46+4=64]$

and:

$\backslash [6+6+2+1500+4=1518]$

This means you don't need to separately memorize 64 and 1518 if you understand the structure.

64. SECOND MEMORY TRICK

For switch:

Learn Source, Look at Destination

Or simply:

S !' Learn D !' Forward

65. THIRD MEMORY TRICK

Domains:

Hub = 1 collision domain

Switch = 1 collision domain per port/link

Router = breaks broadcast domains

Ethernet

Ethernet is a LAN technology mainly operating at Data Link + Physical layers.

PDU:

Frame

MAC address:

\[\boxed{48\text{bits}=6\text{B}} \]

Broadcast MAC:

TEXT

FF:FF:FF:FF:FF:FF

Ethernet Frame

TEXT

Dest	Source	Type	Data+Padding	FCS
6 B	6 B	2 B	46–1500 B	4 B

Minimum frame:

\[\boxed{64\text{B}} \]

Maximum normal untagged frame:

\[\boxed{1518\text{B}} \]

Ethernet MTU:

\[\boxed{1500\text{B}} \]

Minimum payload/data field:

\[\boxed{46\text{B}} \]

FCS:

\[\boxed{4\text{B}} \]

uses:

CRC.

Preamble/SFD

Preamble:

\[7\text{B} \]

SFD:

\[1B \]

Usually excluded from the 64–1518 byte MAC-frame size.

Switch

Switch operates mainly at:

Layer 2.

Learning

Uses:

\[\boxed{\text{Source MAC}} \]

Forwarding

Uses:

\[\boxed{\text{Destination MAC}} \]

Destination unknown:

Flood.

Broadcast:

Flood.

Collision Domains

Hub:

\[\boxed{1} \]

shared collision domain.

Switch:

one collision domain per port/link.

Router:

separates broadcast domains.

Full Duplex

Modern full-duplex switched Ethernet:

66. 5-MINUTE REVISION — DATA LINK + MAC + ETHERNET

Data Link

FAME

- Framing
- Access
- MAC addressing
- Error detection

Error Detection

Parity !' count 1s

Checksum !' add/wrap/complement

CRC !' XOR division

MAC

ALOHA !' send directly

CSMA !' sense first

CSMA/CD:

$\boxed{L_{\min}=2\tau R}$

Ethernet

TEXT

6 | 6 | 2 | 46–1500 | 4

Minimum:

$\boxed{64B}$

Maximum:

$\boxed{1518B}$

MTU:

$\boxed{1500B}$

67. 30-SECOND REVISION

Memorize this:

TEXT

Ethernet:

Dest 6

Source 6

Type 2
Data 46–1500
FCS 4

Then:

$\boxed{64B \text{ Frame} 1518B}$

Switch:

Learn source, forward destination.

Hub:

one collision domain.

Switch:

one collision domain per port.

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the size of an Ethernet MAC address in:

1. bits
2. bytes?

Q2

Write the five major Ethernet MAC-frame fields and their sizes.

Q3

Why must an Ethernet payload sometimes contain padding?

Q4

An Ethernet frame contains 25 bytes of actual upper-layer data.

How many padding bytes are needed?

Q5

What are the minimum and maximum standard untagged Ethernet MAC-frame sizes?

Q6 — GATE Trap

What is the difference between:

$1500B$

and:

\[1518B \]

in Ethernet?

Q7

Which Ethernet field uses CRC for error detection?

A. Type B. Destination MAC C. FCS D. Preamble

Q8 — Switch Learning

A frame with:

TEXT

Source MAC = A

Destination MAC = B

enters switch Port 4.

Which mapping can the switch immediately learn?

Q9

If destination MAC is not in the switch table, what does the switch normally do?

Q10

Ten computers connect individually to a 10-port Ethernet switch in a single VLAN.

Under the standard basic model, how many:

1. collision domains?
 2. broadcast domains?
-

Q11 — GATE Numerical

An Ethernet frame carries:

\[1200B \]

of payload.

Ignoring preamble/SFD, what is the total MAC-frame size?

Q12 — Future Connection

An IPv4 datagram has total size:

\[3500B \]

but the outgoing Ethernet MTU is:

\[1500B \]

Can it fit inside one ordinary Ethernet frame?

What network-layer concept will be required if fragmentation is allowed?

NEXT !' Routing Fundamentals: Forwarding vs Routing, Routing Tables, Graph Representation & Shortest-Path Foundation

USER

next

ASSISTANT

CHAPTER 8 — ROUTING FUNDAMENTALS

TOPIC 1 — Forwarding vs Routing, Routing Tables, Graph Representation & Shortest-Path Foundation

SHORT-TERM PRIORITY FOUNDATION

Your syllabus explicitly includes:

- Distance Vector Routing
- Link State Routing

Before learning those algorithms, you need to clearly understand four ideas:

1. What is **routing**?
2. What is **forwarding**?
3. What is inside a **routing/forwarding table**?
4. How is a network represented as a **graph**?

Once these are clear, Distance Vector and Link State become much easier.

1. WHY DO WE NEED ROUTING?

Suppose Host A wants to send a packet to Host B.

TEXT

```
A  ----  R1  ----  R2  ----  B
      |
      R3
      |
      R4
```

There may be many possible paths.

For example:

TEXT

```
A  !'   R 1  !'   R 2  !'   B
```

or perhaps:

TEXT

A !' R 1 !' R 3 !' R 4 !' . . . !' B

The network must determine:

Which path should packets use?

That is the fundamental routing problem.

2. WHAT IS ROUTING?

Routing is the process of determining paths through a network from source toward destination.

Routing algorithms calculate information that routers use to make forwarding decisions.

Examples from your syllabus:

- **Distance Vector**
- **Link State**

These are two different ways to compute routes.

3. WHAT IS FORWARDING?

Forwarding means:

Moving an arriving packet from a router's input toward the correct outgoing interface/link.

Suppose:

TEXT

```

      Link 2
      |
Link 1 | [ Router ] | Link 3
      |
      Link 4
      !'
      !"
```

A packet arrives.

Destination:

TEXT

172.16.5.10

Router checks its forwarding table and decides:

Send through Link 3.

That action is **forwarding**.

4. ROUTING vs FORWARDING

Ø = Extremely important distinction.

Routing

Decides paths / builds routing information.

Forwarding

Uses that information to move individual packets.

Ø > Memory:

Routing decides. Forwarding sends.

5. SIMPLE ANALOGY

Suppose you use Google Maps.

Routing

Google Maps calculates:

TEXT

H y d e r a b a d ! ' N a g p u r ! ' B h o p a l ! ' D e l h i

Forwarding

At every junction, you actually take:

the next road indicated.

So:

Routing = planning Forwarding = execution

6. CONTROL PLANE vs DATAPLANE — USEFUL FOUNDATION

You may encounter these terms.

Control Plane

Responsible for determining routes and building forwarding information.

Think:

Routing

Data Plane

Responsible for actually forwarding packets.

Think:

Forwarding

So:

TEXT

```
C o n t r o l   P l a n e   ! '   d e c i d e s  
D a t a   P l a n e           ! '   f o r w a r d s
```

You do not need to over-expand this for the current syllabus, but the distinction is useful.

7. ROUTING TABLE

A router stores routing information.

A simplified routing table may look like:

Destination	Next Hop	Cost
Network A	R2	4
Network B	R3	7
Network C	R2	2

Meaning:

To reach a particular destination, send toward a particular next-hop router.

8. WHAT IS A NEXT HOP?

Suppose best route is:

TEXT

```
R 1   ! '   R 2   ! '   R 5   ! '   R 8   ! '   D e s t i n a t i o n
```

Router R1 does **not** necessarily need to store every future router for basic hop-by-hop forwarding.

It mainly needs to know:

Next hop = R2

Then R2 makes its own forwarding decision.

So packet forwarding is typically:

hop by hop.

9. EXAMPLE

Suppose R1's table contains:

Destination	Next Hop
X	R2
Y	R3
Z	R4

Packet arrives for destination Y.

R1 forwards it to:

$\boxed{R3}$

It does not need to know the entire physical journey at that instant.

10. ROUTE vs PATH vs NEXT HOP

These can sound similar.

Path

Complete sequence:

TEXT

R 1 ! ' R 3 ! ' R 7 ! ' R 9

Next hop

From R1:

TEXT

R3

Routing table entry

Information telling R1 how to reach destination, often including:

- next hop
- outgoing interface
- metric/cost

11. WHAT IS A ROUTING METRIC?

A metric is:

A numerical measure used to compare possible routes.

Examples conceptually:

- hop count
- delay
- bandwidth-related cost
- administrative/configured link cost

For GATE graph problems, you're often given simple numeric **link costs**.

12. EXAMPLE

Two paths from A to D:

TEXT

Path 1:

A --2-- B --3-- D

Cost:

$2+3=5$

Path 2:

TEXT

A --1-- C --10-- D

Cost:

$1+10=11$

If objective is minimum total cost:

$\boxed{A \text{ to } B \text{ to } D}$

is selected.

13. GATE TRAP — FEWEST HOPS ≠ LOWEST COST

Consider:

TEXT

A ----10---- D

Direct route:

$1 \text{ hop}, \text{ cost}=10$

Alternative:

TEXT

A --2-- B --2-- C --2-- D

Three hops:

$\set{cost=6}$

If routing minimizes **cost**, second path is better despite more hops.

Always inspect the metric defined by the question.

14. NETWORK AS A GRAPH

$\emptyset=V, E$ is essential for routing algorithms.

A network can be represented mathematically as a graph:

$\set{G=(V,E)}$

where:

- $\set{V}$ = vertices/nodes
- $\set{E}$ = edges/links

Routers become:

vertices.

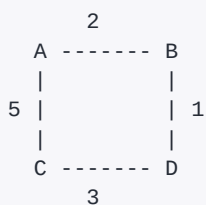
Links become:

edges.

15. GRAPH EXAMPLE

Network:

TEXT



Graph nodes:

$\set{V=\{A,B,C,D\}}$

Edges:

$\set{E=\{AB,AC,BD,CD\}}$

Costs:

$\set{c(A,B)=2}$

$$\lfloor c(A,C)=5 \rfloor$$

$$\lfloor c(B,D)=1 \rfloor$$

$$\lfloor c(C,D)=3 \rfloor$$

16. PATHCOST

For path:

TEXT

A !' B !' D

cost:

$$\lfloor 2+1 = \boxed{3} \rfloor$$

For:

TEXT

A !' C !' D

cost:

$$\lfloor 5+3 = 8 \rfloor$$

So shortest path from A to D:

$$\lfloor \boxed{A\text{to } B\text{to } D} \rfloor$$

with cost:

$$\lfloor \boxed{3} \rfloor$$

17. WHAT DOES "SHORTEST" MEAN?

& pImportant.

In routing:

Shortest path does not necessarily mean geographically shortest.

It usually means:

Path with minimum defined cost.

If each link cost is 1, then minimum cost becomes minimum hop count.

But if costs differ, shortest means minimum:

$$\lfloor \boxed{\sum \text{link costs}} \rfloor$$

18. HOP COUNT

A hop is:

One traversal from a node/router to the next node/router.

Example:

TEXT

A !' R 1 !' R 2 !' B

Number of links/hops:

\[3 \]

If a routing algorithm uses hop count as its metric:

Every link might effectively have cost 1.

19. SHORTEST-PATH TREE

Suppose router A calculates the shortest path to every router.

The collection of those routes can form a:

Shortest-path tree rooted at A

Example:

TEXT

```
  A
 / \
B   C
|
D
|
E
```

This is especially important for:

Link State routing using Dijkstra's algorithm

20. ROUTING ALGORITHMS — BIG PICTURE

Your GATE syllabus includes two major routing approaches:

Distance Vector

Each router primarily knows:

distances learned through neighbours.

Uses a Bellman-Ford-style update.

Link State

Each router learns:

network topology/link-state information.

Then independently runs:

Dijkstra's shortest-path algorithm.

21. SIMPLE MENTAL MODEL

Distance Vector

Ask neighbours:

"How far are you from destination?"

Link State

Build a map:

"What does the entire network topology look like?"

Then find shortest route yourself.

This is the biggest conceptual difference.

22. MEMORY TRICK

Distance Vector

Neighbours tell me distances.

Link State

Everyone tells me links; I build the map.

23. ROUTING INFORMATION EXCHANGE

Consider:

TEXT

A ---- B ---- C

Suppose A does not initially know how to reach C.

B tells A:

My distance to C is 1.

If A's cost to B is 1:

$\backslash$ Distance(A,C) = 1+1 = 2 $\backslash$

This is the basic intuition behind:

Distance Vector.

24. LINK-STATE INTUITION

Instead, suppose routers announce:

A says:

TEXT

I am connected to B with cost 1.

B says:

TEXT

I am connected to A and C.

C says:

TEXT

I am connected to B.

After these link-state advertisements are distributed, each router can reconstruct:

TEXT

A --1-- B --1-- C

Then calculate shortest paths.

That's the basic idea behind Link State.

25. STATIC vs DYNAMIC ROUTING — SMALL FOUNDATION

Static Routing

Routes are manually configured.

They do not automatically adapt based on routing-protocol calculations.

Dynamic Routing

Routers exchange information and update routes automatically.

Distance Vector and Link State are:

dynamic routing approaches.

26. WHY DYNAMIC ROUTING?

Suppose:

TEXT

```
A ---- B ---- C
      |
      D
```

If link:

TEXT

B-C

fails, a dynamic routing protocol can eventually:

- detect change
- exchange information
- recalculate routes

This process is called:

Convergence

27. WHAT IS CONVERGENCE?

A routing protocol has converged when:

Routers have reached a stable, mutually consistent view of reachable routes after changes propagate.

After failure:

TEXT

```
Network change
      !"
Routing updates
      !"
Temporary disagreement
      !"
New stable routes
```

The time/process needed is important.

28. WHY CONVERGENCE MATTERS

Slow convergence can temporarily cause:

- incorrect routes
- routing loops
- packet loss
- black holes

Distance Vector is especially famous for some slow-convergence problems, such as:

Count-to-infinity.

We'll study that carefully.

29. ROUTING LOOP

Suppose destination D becomes unreachable.

But routers A and B mistakenly believe the other has a route.

Then:

TEXT

A !' B !' A !' B !' A . . .

Packets can loop.

This is:

Routing loop

Distance Vector can suffer from this during convergence.

30. ROUTING TABLE vs FORWARDING TABLE

In simple GATE explanations, these may sometimes be used loosely.

More precisely:

Routing table / Routing information

Can contain multiple learned routes and protocol information.

Forwarding table

Contains the information actually used for packet forwarding.

For most foundational questions:

Don't overcomplicate the distinction unless the question specifically asks it.

31. DESTINATION-BASED FORWARDING

IP routing normally uses the packet's:

destination IP address/prefix

to select the outgoing route.

Later in CIDR, this becomes:

Longest Prefix Matching

For example:

Prefix	Interface
10.0.0.0/8	1
10.1.0.0/16	2
10.1.2.0/24	3

Destination:

TEXT

10.1.2.50

matches all three.

The most specific:

[/24]

wins.

We will study this properly under CIDR.

32. ROUTING vs SWITCHING — DON'T CONFUSE

You recently learned Ethernet switches.

Ethernet switch

Forwards frames using:

MAC addresses.

IP router

Forwards packets using:

IP destination information.

So:

TEXT

```
S w i t c h ! ' M A C ! ' L a y e r 2
R o u t e r ! ' I P ! ' L a y e r 3
```

33. FRAME CHANGES, PACKET CONTINUES

Suppose:

TEXT

```
Host A ---- R1 ---- R2 ---- Host B
```

At R1:

1. Incoming Ethernet frame arrives.
2. R1 removes Layer-2 framing.
3. Examines destination IP.
4. Chooses outgoing route/interface.
5. Places IP packet inside a new Layer-2 frame.
6. Sends to next hop.

So routing operates on:

Network-layer packet information.

34. WHAT IS ROUTE COST?

Suppose path:

TEXT

```
A ! ' B ! ' C ! ' D
```

Link costs:

$[2, 5, 1]$

Path cost:

$[2+5+1 = \boxed{8}]$

Most GATE shortest-path problems use:

additive link costs.

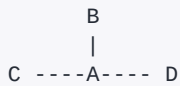
35. DIRECT NEIGHBOUR

A router's neighbour is:

A router directly connected by a link.

Example:

TEXT



Neighbours of A:

$\boxed{B,C,D}$

If E connects only to D:

TEXT



E is not A's direct neighbour.

This matters greatly in Distance Vector.

36. INITIAL DISTANCE-VECTOR KNOWLEDGE — PREVIEW

Suppose router A has neighbours B and C.

Initially A typically knows:

$D_A(A)=0$

Cost to directly connected B:

$D_A(B)=c(A,B)$

Cost to C:

$D_A(C)=c(A,C)$

Unknown destinations may initially be treated as:

∞

Then neighbours exchange vectors and improve estimates.

37. WHY INFINITY?

Suppose A has never learned any route to D.

Then logically:

$D_A(D)=\infty$

meaning:

currently unreachable/unknown.

As updates arrive, it may become finite.

This is the basis of:

Count-to-infinity.

38. SHORTEST-PATH RELAXATION — FOUNDATION

A central routing idea is:

Maybe reaching destination D through neighbour V is cheaper.

Suppose:

$\text{Cost}(A,V)=3$

and V says:

$\text{Distance}(V,D)=4$

Then route through V costs:

$3+4=7$

If A's current estimate to D is:

10

then A updates:

$10 \rightarrow 7$

This process is called:

Relaxation

39. WHY "RELAXATION"?

Because we improve an upper estimate:

Current distance

by checking whether:

Cost through another node

is smaller.

General idea:

$D(\text{destination}) = \min(\text{current}, \text{new alternative})$

This appears in both Bellman-Ford and Dijkstra in different forms.

40. BELLMAN-FORD IDEA — PREVIEW

Distance Vector uses:

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

Don't panic about the notation.

It says:

My best distance to destination y is the minimum of: cost to each neighbour v + that neighbour's distance to y .

We'll learn this thoroughly next.

41. SIMPLE BELLMAN-FORD INTUITION

A wants to reach D.

Neighbours:

B and C.

A knows:

$$c(A,B)=2$$

$$c(A,C)=5$$

B says:

$$D_B(D)=4$$

C says:

$$D_C(D)=1$$

Through B:

$$2+4=6$$

Through C:

$$5+1=6$$

So:

$$D_A(D)=6$$

Both are equally good.

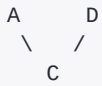
42. DIJKSTRA IDEA — PREVIEW

Link State does something different.

Router A obtains a full topology:

TEXT

B
/ \



with link costs.

Then A repeatedly chooses the nearest not-yet-finalized node and updates its neighbours.

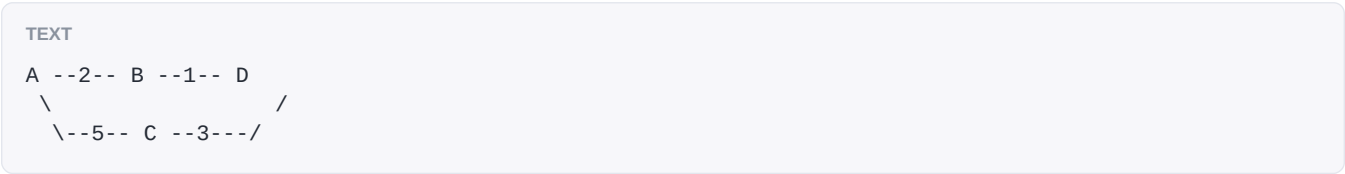
That is:

Dijkstra's algorithm.

We'll study it after Distance Vector.

43. ROUTING TABLE EXAMPLE

Consider:



Shortest from A:

To B

[2]

via B.

To C

[5]

via C.

To D

Through B:

[2+1=3]

Through C:

[5+3=8]

so:

[D_A(D)=3]

Next hop:

[B]

Routing/forwarding table:

Destination	Cost	Next Hop
B	2	B

Destination	Cost	Next Hop
C	5	C
D	3	B

44. & pGATE TRAP — DESTINATION vs NEXT HOP

For destination D:

TEXT

A !' B !' D

The next hop from A is:

$\boxed{B}$

not D.

Unless D is directly connected.

45. EQUAL-COST PATHS

Suppose:

TEXT

A --2-- B --2-- D
A --1-- C --3-- D

Both path costs:

$\boxed{4}$

So there are two equal-cost shortest paths.

A routing protocol may choose one or support equal-cost multipath depending on implementation.

For a GATE graph calculation, acknowledge ties if the question doesn't give a tie-break rule.

46. DIRECTED vs UNDIRECTED LINKS

Some graph problems assume links are symmetric:

TEXT

A !" B

same cost both directions.

Others may use directed links:

TEXT

A !' B c o s t 2
B !' A c o s t 5

& pNever assume symmetry if the question gives direction-specific costs.

47. NEGATIVE LINK COSTS?

Networking routing metrics normally don't use negative link costs in these GATE routing contexts.

Dijkstra specifically assumes non-negative edge weights.

So for Link State:

$\boxed{\text{Cost} \geq 0}$

is an important algorithmic assumption.

48. HOW GATE CAN ASK ROUTING FUNDAMENTALS

Pattern 1 — Routing vs forwarding

Routing:

compute/select routes.

Forwarding:

move packet to output interface.

Pattern 2 — Next hop

Given a shortest path, identify the next-hop router.

Pattern 3 — Path cost

Add link costs.

Pattern 4 — Shortest path

Compare candidate path costs.

Pattern 5 — Graph representation

Interpret routers as vertices, links as edges.

Pattern 6 — Convergence

Ask what happens after a topology change.

Pattern 7 — Device distinction

Switch uses MAC.

Router uses destination IP/network prefix.

49. & pGATE TRAPS

Trap 1

Routing " forwarding.

Trap 2

Shortest path " fewest hops unless every link has equal cost.

Trap 3

Next hop " final destination.

Trap 4

Distance Vector does not require every router to know the full topology.

Trap 5

Link State does not simply ask a neighbour for its final route.

It builds topology knowledge and computes shortest paths.

Trap 6

A router forwards using Network-layer information, not destination MAC across the entire Internet.

Trap 7

Do not assume link cost equals physical distance.

The problem defines the metric.

50. > MEMORY SYSTEM

Remember:

R-F

Routing = Find route Forwarding = Follow route

Distance Vector

Ask neighbours

Link State

Build map

Shortest path

Minimum cost, not minimum geography

' p NOTEBOOK VERSION

Routing Fundamentals

Routing

Process of determining paths through a network.

Routing decides.

Forwarding

Moves an arriving packet to the appropriate outgoing interface/next hop.

Forwarding sends.

Routing Table

Contains information such as:

- destination/prefix
- next hop
- outgoing interface
- metric/cost

Graph Model

$G=(V,E)$

- $\backslash(V)$ = routers/nodes
- $\backslash(E)$ = links

Path cost:

$$\backslash \boxed{ \text{Cost}(\text{path}) = \sum \text{Link} \backslash \text{Costs} } \backslash$$

Shortest path:

Minimum total metric/cost.

Hop

One link traversal.

Fewest hops is not necessarily lowest cost.

Convergence

Process of routers reaching a stable routing state after:

- startup
- link failure
- topology/cost change

Routing Algorithms

Distance Vector

Neighbour-based distance information.

Bellman-Ford idea:

$$\backslash \boxed{ D_x(y) = \min_v [c(x,v) + D_v(y)] } \backslash$$

Link State

Learn network topology and run Dijkstra.

Memory

TEXT

```
Routing      !'   Decide
Forwarding   !'   Send

DV !'   Ask neighbours
LS !'   Build map
```

51. 30-SECOND REVISION

Remember this:

TEXT

Routing

!"

Which path?

Forwarding

!"

Which output/next hop?

And:

TEXT

D i s t a n c e V e c t o r ! ' n e i g h b o u r k n o w l e d g e
L i n k S t a t e ! ' t o p o l o g y k n o w l e d g e

Shortest route means:

$\boxed{\text{minimum total cost}}$

not automatically fewest hops.

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the main difference between routing and forwarding?

Q2

Suppose the route is:

TEXT

A ! ' R 1 ! ' R 2 ! ' R 3 ! ' B

What is A's next hop?

Q3

Two paths exist:

TEXT

A ! ' B ! ' D

Costs: 3 + 4

A ! ' C ! ' E ! ' D

Costs: 1 + 2 + 2

Which is the lower-cost route?

Q4 — GATE Trap

True or False:

The route with the fewest routers must always be the shortest path.

Explain.

Q5

In graph terminology, what represent:

1. routers?
2. physical/logical links?

Q6

What does routing convergence mean?

Q7

Which broad routing approach mainly exchanges distance information with neighbours?

A. Link State B. Distance Vector C. Ethernet switching D. CRC

Q8

Which broad routing approach builds topology knowledge and runs a shortest-path algorithm locally?

Q9 — Next-Hop Question

Router A calculates the shortest route to X as:

TEXT

A !' C !' D !' X

What next-hop entry should A use for destination X?

Q10 — Foundation for Next Topic

Router A has two neighbours B and C.

$\lfloor c(A,B)=3 \rfloor$

$\lfloor c(A,C)=2 \rfloor$

B says its distance to D is:

$\lfloor 4 \rfloor$

C says its distance to D is:

$\lfloor 8 \rfloor$

Calculate A's best distance to D and the next hop.

NEXT !' Distance Vector Routing: Bellman-Ford Equation, Distance Tables, Iterative Updates & Full GATE Numericals

USER

next

ASSISTANT

CHAPTER 9 — DISTANCE VECTOR ROUTING

TOPIC 1 — Bellman-Ford Equation, Distance Vectors, Routing-Table Updates & GATE Numericals

VERY HIGH PRIORITY FOR GATE

Distance Vector questions become easy once you understand one sentence:

To reach a destination, ask every neighbour how far it is from that destination, add the cost to reach that neighbour, and choose the minimum.

Mathematically:

$$D_x(y)=\min_{v\in N(x)}\left[c(x,v)+D_v(y)\right]$$

This is the heart of Distance Vector routing.

1. WHAT IS DISTANCE VECTOR ROUTING?

Distance Vector is a distributed routing algorithm.

Every router maintains estimates of:

The minimum cost from itself to every known destination.

These costs form its:

Distance Vector

2. WHY IS IT CALLED “DISTANCE VECTOR”?

Suppose router A knows these costs:

Destination	Distance from A
A	0

Destination	Distance from A
B	2
C	5
D	6

Its distance vector can be written:

$\boxed{[0,2,5,6]}$

The entries represent A's estimated **distances** to all destinations.

Hence:

Distance Vector.

3. WHAT INFORMATION DOES A ROUTER NEED?

Router A mainly needs:

1. Cost to its directly connected neighbours.
2. Distance vectors advertised by those neighbours.

It does **not** need the complete network topology.

This is a major difference from Link State.

4. SIMPLE NETWORK

Consider:

TEXT

A ----2---- B ----3---- C

Initially:

Router A knows

$D_A(A)=0$

$D_A(B)=2$

$D_A(C)=\infty$

So:

$DV_A=[0,2,\infty]$

Router B knows

$D_B(A)=2$

$D_B(B)=0$

$D_B(C)=3$

So:

$$D_V_B = [2, 0, 3]$$

Router C knows

$$D_C(A) = \infty$$

$$D_C(B) = 3$$

$$D_C(C) = 0$$

So:

$$D_V_C = [\infty, 3, 0]$$

5. WHAT HAPPENS WHEN ROUTERS EXCHANGE VECTORS?

B tells A:

My distance to C is 3.

A already knows:

$$c(A, B) = 2$$

Therefore A calculates:

$$2 + 3 = 5$$

So A updates:

$$D_A(C) : \infty \rightarrow 5$$

Route:

TEXT

A ! B ! C

Similarly C learns A through B:

$$3 + 2 = 5$$

6. THE BELLMAN-FORD EQUATION

~~$D = \infty$~~ is the main formula.

For router x and destination y :

$$D_x(y) = \min_v \{ c(x, v) + D_v(y) \}$$

where v ranges over neighbours of x .

7. UNDERSTANDING EVERY SYMBOL

$D_x(y)$

Router x 's best-known distance to destination y .

$c(x,v)$

Direct link cost from x to neighbour v .

$D_v(y)$

Neighbour v 's advertised distance to destination y .

Therefore:

$c(x,v) + D_v(y)$

means:

Cost to go from x to neighbour v + cost from v to destination y .

Then choose:

$\min$

among all neighbours.

8. MEMORY TRICK

Distance Vector:

ME !' NEIGHBOUR !' DESTINATION

Cost:

$\boxed{ME \rightarrow \text{Neighbour} + \text{Neighbour} \rightarrow \text{Destination}}$

Then:

Take minimum.

9. SIMPLE EXAMPLE

Router A has neighbours B and C.

Given:

$c(A,B)=4$

$c(A,C)=2$

B advertises:

$D_B(D)=5$

C advertises:

$D_C(D)=8$

Calculate A's distance to D.

Through B

$$\boxed{4+5=9}$$

Through C

$$\boxed{2+8=10}$$

Therefore:

$$\boxed{D_A(D)=9}$$

Next hop:

$$\boxed{B}$$

10. VERY IMPORTANT — NEXT HOP

When A selects:

$$\boxed{4+5=9}$$

through B, routing table should contain:

Destination	Distance	Next Hop
D	9	B

A does **not** normally store only the distance.

It also needs the neighbour to which it should forward the packet.

11. INITIALIZATION

At startup, router x typically initializes:

Distance to itself

$$\boxed{D_x(x)=0}$$

Distance to directly connected neighbour v

$$\boxed{D_x(v)=c(x,v)}$$

Distance to unknown non-neighbour destinations

$$\boxed{\infty}$$

12. EXAMPLE INITIALIZATION

Network:

```
TEXT
      4
A ----- B
|
| 2
```

I
C

Router A initially knows:

Destination	Distance
A	0
B	4
C	2

If another node D exists but isn't directly known:

$D_A(D) = \infty$

until A learns a route through a neighbour.

13. DISTANCE VECTOR EXCHANGE

Routers periodically or when triggered exchange their distance vectors with:

Direct neighbours.

Example:

TEXT

A ! B ! C

A doesn't normally send its vector directly to C if C isn't a neighbour.

Instead:

TEXT

A ! B
B ! C

Updates propagate hop by hop.

14. ITERATIVE NATURE

Distance Vector does not necessarily find every route instantly.

Information spreads through successive exchanges.

Example:

TEXT

A --1-- B --1-- C --1-- D

Initially A knows:

- $A = 0$
- $B = 1$
- $C = \infty$
- $D = \infty$

After learning from B:

$$D_A(C) = 2$$

Later, after B learns D through C and advertises it:

$$D_A(D) = 3$$

So routes propagate iteratively.

15. WHY DOES IT CONVERGE?

Routers repeatedly perform:

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

Whenever a better path is learned:

Update distance and next hop.

Eventually, if the topology remains stable:

No router can improve its distances further.

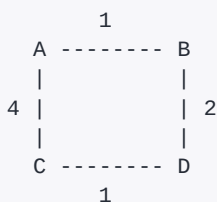
Then the network has:

Converged

16. FULL GATE EXAMPLE

Consider:

TEXT



Links:

$$A-B = 1$$

$$A-C = 4$$

$$B-D = 2$$

$$C-D = 1$$

We will calculate A's shortest distances using Distance Vector reasoning.

17. INITIAL VECTOR OF A

A knows direct links:

$$\backslash[D_A(A)=0 \backslash]$$

$$\backslash[D_A(B)=1 \backslash]$$

$$\backslash[D_A(C)=4 \backslash]$$

$$\backslash[D_A(D)=\infty \backslash]$$

So:

$$\backslash[\boxed{DV_A=[0,1,4,\infty]} \backslash]$$

18. INITIAL VECTOR OF B

B knows:

- A cost 1
- D cost 2

Therefore:

$$\backslash[DV_B=[1,0,\infty,2] \backslash]$$

19. INITIAL VECTOR OF C

C knows:

- A cost 4
- D cost 1

So:

$$\backslash[DV_C=[4,\infty,0,1] \backslash]$$

20. INITIAL VECTOR OF D

D knows:

- B cost 2
- C cost 1

So:

$$\backslash[DV_D=[\infty,2,1,0] \backslash]$$

21. A RECEIVES VECTORS FROM B AND C

A's neighbours are:

- B
- C

Let's calculate distance to D.

Through B

$$c(A,B) + D_B(D)$$

$$= 1 + 2$$

$$= 3$$

Through C

$$c(A,C) + D_C(D)$$

$$= 4 + 1$$

$$= 5$$

Therefore:

$$D_A(D) = 3$$

Next hop:

$$B$$

22. CAN A IMPROVE ITS DISTANCE TO C?

A currently has direct:

$$4$$

Could A use B?

B initially says C is:

$$\infty$$

so no.

But after B learns a route:

TEXT

B !' D !' C

cost:

$$2 + 1 = 3$$

B advertises:

$$D_B(C) = 3$$

Then A considers:

Direct

$$4$$

Through B

$$\sqrt{1+3=4}$$

Equal.

So cost remains:

$$\sqrt{4}$$

There are potentially equal-cost alternatives.

23. FINAL SHORTEST DISTANCES FROM A

To B:

$$\sqrt{1}$$

To C:

$$\sqrt{4}$$

To D:

$$\sqrt{3}$$

A's final vector:

$$\sqrt{\boxed{\{0,1,4,3\}}}$$

24. DISTANCE TABLE

Some GATE questions use a **distance table** rather than only a vector.

Suppose A has neighbours B and C.

A may calculate destination D:

Destination D via	Calculation	Cost
B	$\sqrt{c(A,B)+D_B(D)}$	$\sqrt{1+2=3}$
C	$\sqrt{c(A,C)+D_C(D)}$	$\sqrt{4+1=5}$

Minimum:

$$\sqrt{\boxed{3}}$$

via:

$$\sqrt{\boxed{B}}$$

25. DISTANCE VECTOR vs DISTANCE TABLE

Distance Vector

Stores best distance to each destination.

Example:

Destination	Best Distance
B	1
C	4
D	3

Distance Table

Can show candidate distances via each neighbour.

Example:

Destination	Via B	Via C
D	3	5

The minimum becomes the vector entry.

26. FULL NUMERICAL

Router X has three neighbours:

- A
- B
- C

Direct link costs:

$$c(X,A)=2$$

$$c(X,B)=5$$

$$c(X,C)=1$$

Their advertised distances to destination Y:

$$D_A(Y)=7$$

$$D_B(Y)=2$$

$$D_C(Y)=8$$

Find X's route to Y.

Through A

$$2+7=9$$

Through B

$$5+2=7$$

Through C

$$1+8=9$$

Minimum:

$$\boxed{7}$$

Next hop:

$\boxed{B}$

So routing entry:

TEXT

Destination Y

Distance = 7

Next hop = B

27. GATE SHORTCUT

When you see a Distance Vector question, immediately draw:

Neighbour	Link cost to neighbour	Neighbour's distance	Total
-----------	------------------------	----------------------	-------

Then calculate:

$\text{Link} + \text{Advertised}$

Choose minimum.

This prevents most mistakes.

28. EXAMPLE USING TABLE

Suppose:

Neighbour	$c(X,v)$	$D_v(Z)$	Total
A	3	5	8
B	6	1	7
C	2	8	10

Therefore:

$D_X(Z)=7$

Next hop:

B

Done.

29. ROUTER ONLY TRUSTS ADVERTISED COST

A subtle but important idea:

Suppose B advertises:

$D_B(D)=4$

A doesn't necessarily know B's complete path to D.

A simply reasons:

If B says it costs 4 from B, and I can reach B for 2, then my candidate cost through B is 6.

This limited knowledge is both:

- an advantage — simplicity/distributed operation
- a disadvantage — can cause routing-loop/count-to-infinity problems.

We'll study those next.

30. LOCAL INFORMATION

Distance Vector is often described as:

Distributed and iterative.

Distributed

No single central router computes all routes.

Iterative

Routers repeatedly update based on neighbour information.

Asynchronous

Different routers may exchange/update at different times.

These are useful conceptual terms.

31. NEIGHBOUR UPDATE EXAMPLE

Router A currently has:

Destination	Distance	Next Hop
D	10	C

Then B advertises:

$[D_B(D)=4]$

and:

$[c(A,B)=2]$

Candidate:

$[2+4=6]$

Since:

$[6 < 10]$

A updates:

Destination	Distance	Next Hop
D	6	B

32. WHAT IF NEW ROUTE IS WORSE?

Suppose current route:

$[6]$

New candidate through C:

$[9]$

A keeps:

$[6]$

because Bellman-Ford selects:

$[\min]$

33. IMPORTANT NUANCE — WHAT IF CURRENT NEXT HOP'S COST CHANGES?

Suppose A reaches D through B.

Current:

$[D_A(D)=6]$

where:

$[c(A,B)=2, \quad D_B(D)=4]$

Now B says:

$[D_B(D)=10]$

A must not simply say:

"10 is worse, so keep old 6."

Why?

Because old 6 depended on B's old advertisement.

A must recompute candidates through all neighbours.

Perhaps:

Through B:

$[2+10=12]$

Through C:

$[5+3=8]$

Then A updates:

$\lceil 6 \rceil \rightarrow 8$

via C.

∞ important.

34. DISTANCE CAN INCREASE

Students sometimes think Distance Vector distances only decrease.

During initial shortest-path discovery in a stable graph, estimates often decrease from infinity.

But when:

- link cost increases
- link fails
- topology changes

a route's distance may increase.

This leads to some of Distance Vector's famous convergence problems.

35. LINK FAILURE PREVIEW

Suppose:

TEXT

A ---- B ---- D

A reaches D through B.

If:

TEXT

B-D

fails, B marks D unreachable.

But A may still mistakenly advertise an old route through B.

This can create:

bad news travelling slowly.

Eventually values can increase:

$\lceil 2 \rceil \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow \dots$

This is:

Count-to-Infinity

We will cover it next.

36. GOOD NEWS vs BAD NEWS

A classic Distance Vector observation:

Good news

A new shorter path often propagates quickly.

Bad news

A failed route may propagate slowly and cause loops/count-to-infinity.

Memory:

Good news travels fast; bad news travels slow.

This is a famous DV concept.

37. GATE-STYLE ITERATIVE EXAMPLE

Consider:

TEXT

A --1-- B --1-- C --1-- D

Initial vectors only know direct neighbours.

Round 0 — A

$\mathbb{V} [0, 1, \infty, \infty]$

After learning from B

B knows C at cost 1.

So A learns:

$\mathbb{V} [D_A(C)=1+1=2]$

A becomes:

$\mathbb{V} [0, 1, 2, \infty]$

Later B learns D through C

$\mathbb{V} [D_B(D)=1+1=2]$

Then A learns:

$\mathbb{V} [D_A(D)=1+2=3]$

Final:

$\mathbb{V} [\boxed{0, 1, 2, 3}]$

38. HOW MANY ITERATIONS?

In a simple synchronous conceptual model, information can propagate approximately:

one hop farther per exchange round.

Thus routes several hops away may take several rounds to become known.

However, actual protocol timing is asynchronous and implementation-dependent.

Don't blindly equate hop count with exact seconds or exact update count unless the question defines synchronous rounds.

39. ROUTING TABLE CONSTRUCTION

Suppose router A computes:

$\backslash D_A(B)=2 \backslash$

via B.

$\backslash D_A(C)=5 \backslash$

via D.

$\backslash D_A(E)=7 \backslash$

via B.

Then table:

Destination	Cost	Next Hop
B	2	B
C	5	D
E	7	B

Forwarding only needs the relevant next-hop/outgoing information.

40. EQUAL COST ROUTES

Suppose:

Through B:

$\backslash 3+4=7 \backslash$

Through C:

$\backslash 2+5=7 \backslash$

Then both are equally optimal.

If no tie-breaking rule is specified:

Both are valid minimum-cost next hops.

Do not invent a winner.

41. WHAT IF DIRECT ROUTE IS BEST?

Suppose A connects directly to D:

$$[c(A,D)=5]$$

but through B:

$$[c(A,B)+D_B(D) = 2+6 = 8]$$

Then direct route:

$$[\boxed{5}]$$

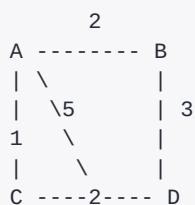
wins.

A directly connected destination can be treated as another candidate.

42. A MORE COMPLEX GATE EXAMPLE

Network:

TEXT



Edges:

$$[A-B=2]$$

$$[A-C=1]$$

$$[A-D=5]$$

$$[B-D=3]$$

$$[C-D=2]$$

Suppose A receives:

From B:

$$[D_B(D)=3]$$

From C:

$$[D_C(D)=2]$$

A also has direct D cost 5.

Candidates:

Direct

$$[5]$$

Via B

$$[2+3=5]$$

Via C

$\boxed{1+2=3}$

Therefore:

$\boxed{D_A(D)=3}$

Next hop:

$\boxed{C}$

43. A'S ROUTE TO B MAY ALSO IMPROVE INDIRECTLY

Direct:

$\boxed{A-B=2}$

Could A go:

TEXT

A !' C !' D !' B

Cost:

$\boxed{1+2+3=6}$

No.

So direct remains:

$\boxed{2}$

Bellman-Ford naturally evaluates alternatives.

44. WHY LOOPS DON'T APPEAR IN THE EQUATION ITSELF

The Bellman-Ford formula is mathematically sound when applied to correct stable shortest-path information.

The problem occurs because in a real distributed Distance Vector protocol:

routers use neighbour advertisements that may be stale or mutually dependent during topology changes.

That creates transient loops.

So don't blame the shortest-path equation itself.

The issue is:

distributed delayed information.

45. DISTANCE VECTOR CHARACTERISTICS

Remember:

- distributed

- neighbour-to-neighbour exchange
- iterative
- asynchronous
- uses Bellman-Ford principle
- does not require full topology map
- can suffer routing loops
- can suffer count-to-infinity

46. DISTANCE VECTOR vs LINK STATE— PREVIEW

Distance Vector	Link State
Exchanges distance vectors	Exchanges link-state info
With neighbours	Floods topology information
Bellman-Ford	Dijkstra
No complete topology required	Builds topology database
Simpler distributed view	More global view
Count-to-infinity possible	Not same DV problem

We'll make a full comparison after Link State.

47. HOW GATE CAN ASK DISTANCE VECTOR

Pattern 1 — Bellman-Ford calculation

Given neighbour costs and neighbour advertisements.

Find:

- best distance
- next hop

Pattern 2 — Initial vector

Direct neighbours finite.

Others:

∞

Pattern 3 — Iterative updates

Show successive distance vectors after exchanges.

Pattern 4 — Routing-table entry

Determine destination, cost and next hop.

Pattern 5 — Topology change

Ask what happens after link failure/cost increase.

This leads to count-to-infinity.

Pattern 6 — Algorithm identification

Neighbour distances + Bellman-Ford:

Distance Vector.

48. & pGATE TRAPS

Trap 1

Do not choose neighbour with smallest advertised distance alone.

You must add:

$\boxed{c(x,v)}$

first.

Trap 2

Suppose:

B says distance 2.

C says distance 3.

But costs from A are:

$A-B=10$

$A-C=1$

Then:

Via B:

12

Via C:

4

So choose C.

Trap 3

Distance Vector does not know complete topology.

Trap 4

Next hop is the neighbour you choose, not the destination necessarily.

Trap 5

A router can have a route cost increase after topology changes.

Trap 6

Infinity means:

currently unreachable according to known routing information.

It doesn't necessarily mean no physical path exists forever.

Trap 7

Equal-cost routes must be treated as ties unless a tie-breaking rule is supplied.

49. MASTER MEMORY TRICK

Distance Vector =

A-M-M

A — Ask neighbours

M — Me-to-neighbour + neighbour-to-destination

M — Minimum wins

So:

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

50. ANOTHER MEMORY TRICK

Think of travelling to college.

You ask three friends:

"How far are you from college?"

Friend B says:

5km

Friend C says:

$\backslash[3\text{km} \backslash]$

But you are:

$\backslash[1\text{km} \backslash]$

from B and:

$\backslash[5\text{km} \backslash]$

from C.

Total:

Through B:

$\backslash[1+5=6 \backslash]$

Through C:

$\backslash[5+3=8 \backslash]$

Even though C is closer to college, **B is your better next hop.**

That's exactly Distance Vector.

'p NOTEBOOK VERSION

Distance Vector Routing

Each router maintains:

minimum estimated cost to each destination.

Uses neighbour advertisements.

Does **not** require complete topology.

Bellman-Ford Equation

$\backslash[\boxed{D_x(y) = \min_{v \in N(x)} [c(x,v) + D_v(y)] } \backslash]$

Where:

- $\backslash(D_x(y)) = x\text{'s distance to } y$
- $\backslash(c(x,v)) = \text{direct cost } x\text{'s neighbour } v$
- $\backslash(D_v(y)) = v\text{'s advertised distance to } y$

Memory:

Minimum wins.

Initialization

$\backslash[D_x(x)=0 \backslash]$

Direct neighbour:

$$D_x(v) = c(x, v)$$

Unknown destination:

$$D_x(y) = \infty$$

Operation

1. Maintain distance vector.
2. Exchange vector with neighbours.
3. Recalculate candidate costs.
4. Select minimum.
5. Update cost + next hop.
6. Repeat until stable.

Characteristics

- distributed
- iterative
- neighbour-based
- Bellman-Ford
- asynchronous
- no full topology map
- can suffer routing loops/count-to-infinity

51. 30-SECOND REVISION

If you remember only this:

$$\boxed{\text{Cost through neighbour} = \text{Cost to neighbour} + \text{Neighbour's advertised cost}}$$

Calculate this for every neighbour.

Then:

$$\boxed{\text{minimum wins}}$$

That's Distance Vector.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Write the Bellman-Ford equation used in Distance Vector routing.

Q2

Router X has neighbours A and B.

$\forall c(X,A)=3, \text{ } \sqcap c(X,B)=6 \text{ }$

A advertises:

$\forall D_A(Y)=5 \text{ }$

B advertises:

$\forall D_B(Y)=1 \text{ }$

Find:

1. X's best distance to Y
 2. next hop
-

Q3

At initialization, what is a router's distance to:

1. itself?
 2. directly connected neighbour?
 3. unknown destination?
-

Q4 — GATE Trap

Router X hears:

$\forall D_A(Z)=2 \text{ }$

$\forall D_B(Z)=4 \text{ }$

Can X immediately conclude A is the better next hop?

What additional information is needed?

Q5

A currently reaches D through B with cost 7.

After an update:

- via B becomes 12
- via C becomes 9

What should A do?

Q6

Why doesn't Distance Vector require each router to know the complete network topology?

Q7 — Numerical

Router P has neighbours Q, R and S.

$\forall c(P,Q)=2 \text{ }$

$\forall c(P,R)=4 \text{ }$

$\forall c(P,S)=3 \text{ }$

They advertise distance to destination X:

$\{ D_Q(X)=7 \}$

$\{ D_R(X)=2 \}$

$\{ D_S(X)=4 \}$

Find P's:

1. candidate cost through each neighbour
2. final distance to X
3. next hop

Q8

True or False:

Distance Vector route costs can only decrease, never increase.

Explain.

Q9

What does routing **convergence** mean in a Distance Vector network?

Q10 — Preview

Why can a router mistakenly believe that its neighbour still has a path to a failed destination?

Think about:

stale neighbour advertisements.

NEXT !' Distance Vector Problems: Routing Loops, Count-to-Infinity, Split Horizon, Poison Reverse & Failure-Convergence GATE Questions

USER

next

ASSISTANT

CHAPTER 9 — DISTANCE VECTOR ROUTING

TOPIC 2 — Routing Loops, Count-to-Infinity, Split Horizon, Poison Reverse & Failure Convergence

VERY IMPORTANT GATE CONCEPTUAL AREA

Distance Vector works well when routes improve.

Its famous weakness appears when a route becomes worse or fails.

The core problem is:

Routers know what their neighbours claim, but they may not know where the neighbour's route actually goes.

That limited information can create:

- routing loops
- count-to-infinity
- slow convergence

The memory line is:

Good news travels fast; bad news travels slowly.

1. START WITH A VERY SIMPLE NETWORK

Consider:

TEXT

A ----1---- B ----1---- C

Suppose C represents a destination network.

Initially:

B's distance to C

$[D_B(C)=1]$

A's distance to C

A reaches C through B:

$[D_A(C)=1+1=2]$

So:

TEXT

A !' B !' C

is the route.

2. NOW SUPPOSE LINK B–C FAILS

The network becomes:

TEXT

A ----1---- B C

B immediately realizes:

My direct route to C is gone.

So ideally:

$D_B(C) = \infty$

But there is a problem.

A may still have its old information:

$D_A(C) = 2$

because its routing table has not yet been updated.

3. THE MISTAKE

A tells B:

"My distance to C is 2."

B thinks:

"Great! Maybe I can reach C through A."

So B computes:

$c(B,A) + D_A(C)$

$= 1 + 2$

$= 3$

Therefore B updates:

$D_B(C) = 3$

via A.

But where does A's supposed route to C go?

Through B!

So now:

TEXT

B thinks: C is through A

A thinks: C is through B

That creates a routing loop.

4. ROUTING LOOP

Packets for C may now travel:

TEXT

A !' B !' A !' B !' A !' B . . .

Neither router actually has a working path to C.

Yet both temporarily believe the other router does.

This is a:

Routing loop

5. WHY DID THIS HAPPEN?

Because Distance Vector advertisements normally tell a neighbour:

"My distance to C is 2."

They don't necessarily tell:

"And by the way, my route to C goes through you."

So B may accidentally use A's route even though A's route itself depends on B.

~~0-1%~~ This is the fundamental cause.

6. COUNT-TO-INFINITY

The situation gets worse.

Let's continue.

Initially after failure:

TEXT

A thinks C = 2 via B
B learns C = 3 via A

Then A receives B's new advertisement:

$[D_B(C)=3]$

A calculates:

$[1+3=4]$

So:

$[D_A(C)=4]$

Then B receives A's 4:

$[1+4=5]$

Then A:

$\backslash 1+5=6 \backslash$

Then B:

$\backslash 1+6=7 \backslash$

and so on:

TEXT

2 !' 3 !' 4 !' 5 !' 6 !' 7 !' . . .

The routers slowly increase the metric toward:

$\backslash \infty \backslash$

This is called:

Count-to-Infinity

7. SIMPLE TIMELINE

TEXT

Before failure:

A !' C = 2 v i a B
B !' C = 1 d i r e c t

B-C fails

B sees A advertising 2
B says: "Then I can reach C in 3"

A sees B advertising 3
A says: "Then I can reach C in 4"

B sees 4
B says 5

A sees 5
A says 6

...

That is exactly count-to-infinity.

8. WHY IS THIS CALLED “BAD NEWS TRAVELS SLOW”?

Suppose a new shorter path appears.

Routers quickly advertise a smaller metric.

Example:

$\backslash 8 \rightarrow 3 \backslash$

Neighbours immediately prefer it.

But when a route disappears:

∞

routers may mistakenly keep inventing alternative routes through one another.

Therefore:

Improvements propagate quickly.

Failures can propagate slowly.

Memory:

Good news ! "quickly. Bad news !" slowly.

9. WHAT DOES INFINITY ACTUALLY MEAN?

In routing protocols, infinity is often represented by some finite maximum metric.

For example, RIP uses hop count and treats:

∞

as unreachable.

Thus:

15

is the largest usable hop count in classic RIP.

You do not need RIP details for the syllabus unless relevant, but this illustrates why "infinity" is often a chosen protocol value.

10. WHY USE A FINITE INFINITY?

Imagine allowing the metric to increase:

$2, 3, 4, 5, \dots, 10^9$

Convergence would take absurdly long.

Instead, define:

Any metric "some threshold = unreachable.

This limits count-to-infinity duration.

11. GATE CONCEPT

If a DV routing protocol defines infinity as:

∞

and routers are counting:

TEXT

1 2 !' 1 3 !' 1 4 !' 1 5 !' 1 6

once they reach:

\[16 \]

the destination is considered unreachable.

12. CAN WE PREVENT THE SIMPLE LOOP?

Yes.

One important mechanism is:

Split Horizon

13. SPLIT HORIZON — BASIC IDEA

Suppose A learned its route to C from B.

Without split horizon, A advertises back to B:

"I can reach C with cost 2."

That is dangerous because A's route already depends on B.

Split horizon says:

Do not advertise a route back on the interface from which that route was learned.

So if:

TEXT

A learned C through B

then A does **not** tell B:

TEXT

A !' C = 2

14. MEMORY TRICK

Don't tell your teacher what your teacher taught you.

If B taught A the route to C:

A does not advertise that same route back to B.

That's split horizon.

15. HOW SPLIT HORIZON HELPS

Again:

TEXT

A ---- B ---- C

A's route to C is through B.

So A suppresses that route when advertising to B.

Then if B-C fails, B cannot incorrectly say:

"A claims a route to C, so I'll use A."

A never advertised that route back to B.

Thus the simple two-node loop is prevented.

16. SPLIT HORIZON RULE

If destination D is reached through neighbour N:

Do not advertise route D back to N.

Conceptually:

TEXT

Route learned from N

!"

Do not send same route back to N

17. POISON REVERSE

Another closely related technique is:

Poison Reverse

Instead of saying nothing, A explicitly tells B:

"My route to C through your direction is infinity."

So A advertises:

$\backslash D_A(C)=\infty \backslash$

to B.

But A may still advertise its actual finite route to other neighbours.

18. SPLIT HORIZON vs POISON REVERSE

Split Horizon

Don't advertise the route back to the neighbour it came from.

Poison Reverse

Advertise it back, but with metric infinity.

19. MEMORY

Split Horizon

Stay silent.

Poison Reverse

Say unreachable.

Very easy.

20. EXAMPLE

Suppose:

TEXT

A ---- B ---- C

A reaches C through B.

A's actual table:

$[D_A(C)=2]$

Advertisement from A to another neighbour D

A may say:

$[C=2]$

Advertisement from A back to B with Poison Reverse

A says:

∞

because A's route to C uses B.

21. WHY POISON REVERSE HELPS

B now knows:

A definitely cannot provide me an independent route to C.

So if B's own link to C fails, it won't choose A as an alternative.

This prevents the simple:

TEXT

A → B

loop.

22. IMPORTANT LIMITATION

Split horizon and poison reverse can prevent many simple **two-router loops**.

Split horizon and poison reverse can prevent many simple **two-router loops**.

But they do **not guarantee prevention of all routing loops** in larger topologies.

A loop involving three or more routers may still occur.

23. THREE-NODE LOOP INTUITION

Consider:

TEXT

A
/ \
B --- C

Destination D was reachable somewhere beyond this group.

After a topology change, outdated information can circulate:

TEXT

A thinks via B
B thinks via C
C thinks via A

This is a three-router dependency cycle.

Simple poison reverse based only on one immediate next hop may not eliminate every such scenario.

So don't memorize:

"Poison reverse solves count-to-infinity completely."

That is false.

24. & pGATE TRAP

Correct:

Poison reverse helps prevent two-node routing loops.

Incorrect:

Poison reverse prevents every possible Distance Vector loop.

25. ROUTE POISONING — RELATED IDEA

A related technique is:

When a route fails, explicitly advertise it with metric infinity.

For example:

Before failure:

$[D_B(C)=1]$

After failure:

$[D_B(C)=\infty]$

B quickly tells neighbours:

C is unreachable.

This is often called:

Route poisoning

Do not confuse:

Route poisoning

Advertise a failed route as infinity.

Poison reverse

Advertise infinity specifically back toward the neighbour used as next hop.

They are related but distinct ideas.

26. TRIGGERED UPDATES — SUPPORTING IDEA

Instead of waiting for the next periodic update after a route changes:

Send an update immediately.

This is called a:

Triggered update

This helps convergence happen faster.

27. HOLD-DOWN — SUPPORTING IDEA

Some Distance Vector protocols may temporarily distrust suspiciously "better" routes after a failure.

The idea is:

Don't immediately accept potentially stale information for a recently failed destination.

This can help prevent unstable oscillations.

For your GATE syllabus, know the intuition rather than protocol-specific timers.

28. COMPLETE FAILURE FLOW

Suppose link fails.

A robust DV-style protocol may use:

```
TEXT
Failure detected
    !"
Route marked infinity
    !"
Triggered advertisement
    !"
Neighbours update
    !"
Split horizon / poison reverse prevents simple loops
    !"
Network converges
```

29. COUNT-TO-INFINITY NUMERICAL EXAMPLE

Consider:

```
TEXT
A --1-- B --1-- C
```

Destination is C.

Before failure:

$\backslash D_B(C)=1 \backslash$

$\backslash D_A(C)=2 \backslash$

B-C fails.

Suppose no split horizon.

If B hears A's old value:

$\backslash 2 \backslash$

then:

$\backslash D_B(C)=1+2=3 \backslash$

A then hears:

$\backslash 3 \backslash$

and calculates:

$\backslash D_A(C)=1+3=4 \backslash$

Next:

$\backslash B=5 \backslash$

Then:

$\backslash A=6 \backslash$

and so on.

30. IF INFINITY = 8

Suppose protocol defines:

$\backslash 8=\infty \backslash$

Sequence may become:

TEXT

B: 3

A: 4

B: 5

A: 6

B: 7

A: 8

At:

$\backslash 8 \backslash$

route is considered unreachable.

Therefore finite infinity limits how long counting can continue.

31. GATE-STYLE QUESTION

Network:

TEXT

X --2-- Y --3-- D

Before failure:

$\{ D_Y(D)=3 \}$

$\{ D_X(D)=5 \}$

Link Y-D fails.

X still advertises:

$\{ D_X(D)=5 \}$

Y may calculate:

$\{ 2+5=7 \}$

and incorrectly select X.

This is the beginning of:

$\{ \boxed{\text{count-to-infinity / routing loop}} \}$

32. WHAT IF LINK COST DECREASES?

Suppose:

TEXT

A --10-- B

becomes:

TEXT

A --2-- B

This is good news.

A and B immediately know a better direct cost.

They advertise lower distances.

Nearby routers quickly update:

$\{ 12 \rightarrow 4 \}$

No mutual false path is generally needed.

Hence:

Good news usually propagates faster.

33. WHAT IF LINK COST INCREASES?

Suppose:

$\lceil A-B:1 \rightarrow 20 \rceil$

A route previously using this link becomes expensive.

Routers may temporarily still believe one another has an old cheaper path.

This can create the same kind of bad-news problem as complete failure.

So count-to-infinity can arise after:

- link failure
- link cost increase

not only when a destination disappears completely.

34. FAILURE CONVERGENCE

Convergence after a failure means routers eventually agree on:

- destination unreachable

or

- a valid alternate route

During convergence there may be:

- temporary loops
- outdated routes
- packet loss
- changing metrics

Fast convergence is desirable.

35. DISTANCE VECTOR LOOP EXAMPLE WITH ALTERNATE ROUTE

Suppose:

TEXT

```
A --1-- B --1-- D
 \         /
  \-----10-----/
```

A has:

Via B

$\lceil 1+1=2 \rceil$

Direct

$\lceil 10 \rceil$

So A chooses B.

Now B-D fails.

Correct final route for A should become:

$\lceil 10 \rceil$

directly.

But if B sees A's old advertisement of 2, B could falsely calculate:

$\backslash[1+2=3\backslash]$

via A.

Then A might see B advertising 3 and calculate:

$\backslash[1+3=4\backslash]$

which still looks better than direct 10.

Metrics may count upward:

$\backslash[2,3,4,5,\dots\backslash]$

until eventually the direct route of 10 becomes preferable or infinity is reached.

This illustrates that count-to-infinity can occur even when an alternate path exists.

36. IMPORTANT GATE INSIGHT

When a router recomputes its route after failure:

It considers all candidate neighbours and direct links.

So the count may stop before protocol infinity if a real alternate route becomes cheaper.

Example:

If direct alternate route cost:

$\backslash[10\backslash]$

and false loop metric reaches:

$\backslash[11\backslash]$

the valid 10-cost route will win.

37. SPLIT HORIZON IN THE SAME EXAMPLE

A's route to D uses B.

Therefore A does not advertise D back to B.

When B-D fails:

B cannot mistake A's old route as an independent route.

Then A can eventually switch to its direct 10-cost path.

Convergence becomes much cleaner.

38. POISON REVERSE IN SAME EXAMPLE

A tells B:

$D_A(D) = \infty$

because A's route to D uses B.

When B-D fails, B knows:

A is not a valid alternate path.

Again, simple loop prevented.

39. FULL COMPARISON

Mechanism	Main rule	Purpose
Split Horizon	Don't advertise route back where learned	Prevent simple loops
Poison Reverse	Advertise that route back as infinity	Explicitly prevent simple loops
Route Poisoning	Failed route advertised as infinity	Spread bad news
Triggered Update	Send change immediately	Faster convergence
Finite Infinity	Cap metric	Limit count-to-infinity

40. MEMORY TRICK

Use:

S-P-R-T

S — Split Horizon

Stay silent backward.

P — Poison Reverse

Put infinity backward.

R — Route Poisoning

Route failure = infinity.

T — Triggered Update

Tell neighbours immediately.

41. DISTANCE VECTOR FAILURE THINKING METHOD

Whenever GATE gives a link failure:

Step 1

Identify which routers used the failed path.

Step 2

Check what stale advertisements still exist.

Step 3

Ask:

Does router X think neighbour Y has a valid route that actually comes back through X?

If yes:

routing loop possible.

Step 4

Update metric using:

$$\min [c(x,v) + D_v(d)]$$

Step 5

Repeat until:

- valid alternate route wins

or

- infinity is reached.

42. GATE-STYLE EXAMPLE

Consider:

TEXT

A --1-- B --1-- C

Infinity:

$$\infty$$

Before B-C failure:

$$A(C)=2$$

$$B(C)=1$$

After failure, assume B uses A's stale advertisements.

Sequence:

B

$$1+2=3$$

A

$$1+3=4$$

B

$\backslash[1+4=5 \backslash]$

A

$\backslash[1+5=6 \backslash]$

At 6:

$\backslash[\boxed{\text{Unreachable}} \backslash]$

This is a classic numerical form.

43. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Identify phenomenon

Metrics gradually increase after failure.

Answer:

Count-to-infinity.

Pattern 2 — Trace updates

Given link costs and infinity threshold.

Find successive distance values.

Pattern 3 — Split Horizon

Ask whether a route learned from B should be advertised back to B.

Answer:

No.

Pattern 4 — Poison Reverse

Ask what metric is advertised back toward current next hop.

Answer:

$\backslash[\infty \backslash]$

Pattern 5 — Limitations

Does poison reverse prevent every loop?

No.

Pattern 6 — Good news vs bad news

Ask which changes converge more slowly.

Usually:

cost increase/failure.

Pattern 7 — Finite infinity

Given maximum routing metric.

Determine when destination becomes unreachable.

44. & pGATE TRAPS

Trap 1

Count-to-infinity does not mean the numerical value literally has to reach mathematical infinity.

Protocols generally define a finite "infinity."

Trap 2

Split horizon:

suppresses advertisement back toward the neighbour.

Poison reverse:

explicitly advertises infinity.

They are not exactly identical.

Trap 3

Poison reverse does not solve all possible multi-router loops.

Trap 4

A router can falsely believe a neighbour has a route because that neighbour's route actually depends on the router itself.

Trap 5

Failure may cause route costs to increase.

Distance Vector isn't only about finding progressively smaller values.

Trap 6

A valid alternate route can stop count-to-infinity before the protocol's infinity value is reached.

Trap 7

Triggered update means:

update immediately after meaningful change,

not wait for periodic timer.

45. WHY LINK STATE DOESN'T HAVE THE SAME PROBLEM

Link State works differently.

Routers distribute:

The actual state/cost of their directly connected links.

Then each router independently recomputes the network map.

If link B-C fails:

B advertises:

"B-C is down."

Routers update their topology database and rerun shortest-path computation.

They aren't simply asking:

"Neighbour, how far are you from C?"

Therefore Link State does not suffer from the same classic Distance Vector count-to-infinity mechanism.

This is one major reason we study the two approaches separately.

46. DISTANCE VECTOR — FINAL MENTAL MODEL

Normal operation:

TEXT

```
Ask neighbour
    !
Neighbour gives distance
    !
Add local link cost
    !
Choose minimum
```

Failure problem:

TEXT

Old information remains
!"
Neighbours may depend on each other
!"
Loop
!"
Metric increases
!"
Count-to-infinity

Solutions/mitigations:

TEXT

Split horizon
Poison reverse
Route poisoning
Triggered updates
Finite infinity

' p NOTEBOOK VERSION

Distance Vector — Failure Problems

Routing Loop

Two or more routers incorrectly use each other as the next hop to a destination.

Example:

TEXT

A !' B !' A !' B . . .

Cause:

stale/distributed routing information.

Count-to-Infinity

After route failure/cost increase, routers may repeatedly increase destination metric:

TEXT

2 !' 3 !' 4 !' 5 !' . . .

until the protocol-defined infinity or a valid alternate route is selected.

Memory:

Good news travels fast; bad news travels slow.

Split Horizon

If route to D is learned through neighbour B:

Do not advertise route D back to B.

Memory:

Don't tell back what you learned there.

Poison Reverse

If route to D uses B:

advertise to B:

$\boxed{D(D)=\infty}$

Memory:

Tell B: unreachable through me.

Route Poisoning

When route fails:

$\boxed{\text{Metric}=\infty}$

and advertise failure.

Triggered Update

Send routing update immediately after significant topology change.

Important Limitation

Split horizon / poison reverse:

prevent many two-router loops,

but:

do not guarantee removal of all larger routing loops.

47. 5-MINUTE REVISION — DISTANCE VECTOR COMPLETE

Core formula:

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

Operation:

- ask neighbours
- add local cost
- choose minimum

Problems:

- stale information
- routing loops
- count-to-infinity
- slow bad-news convergence

Mitigations:

- split horizon
- poison reverse
- route poisoning
- triggered updates
- finite infinity

48. 30-SECOND REVISION

Remember:

DV trusts neighbours.

That creates the danger:

"I think you know the way." "I think YOU know the way."

Then:

TEXT

A ! " B

loop.

Metric grows:

$2, 3, 4, 5, \dots$

= **count-to-infinity**.

Fix simple loops using:

Split Horizon = silence backward

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is count-to-infinity in Distance Vector routing?

Q2

Why can A and B create a routing loop after a destination becomes unreachable?

Q3

Network:

TEXT

A --1-- B --1-- C

Before B-C fails:

$[D_A(C)=2]$

After failure, A still advertises 2.

What new distance might B incorrectly calculate to C?

Q4

Continuing Q3, if B advertises its new value to A, what value does A calculate next?

Q5

What is the rule of **split horizon**?

Q6

What is the rule of **poison reverse**?

Q7 — GATE Trap

True or False:

Poison reverse guarantees that no routing loop of any size can ever occur.

Explain.

Q8

Why do routing protocols define a finite value for "infinity"?

Q9

Suppose infinity is 10 and routers are counting:

$[6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 10]$

What does metric 10 represent?

Q10 — Conceptual

What is meant by:

Good news travels fast, bad news travels slow

in Distance Vector routing?

Q11

Differentiate:

- route poisoning
- poison reverse

in one line each.

Q12 — Future Connection

Which routing approach avoids the classic count-to-infinity mechanism by distributing link-state/topology information and running a shortest-path algorithm locally?

NEXT !' Link State Routing: Link-State Advertisements, Flooding, Topology Database & Dijkstra's Algorithm Step-by-Step

USER

next

ASSISTANT

CHAPTER 10 — LINK STATE ROUTING

TOPIC 1 — Link-State Advertisements, Flooding, Topology Database & Dijkstra's Algorithm

~~OVER HIGH PRIORITY FOR GATE~~

Distance Vector works by asking:

"Neighbour, how far are you from the destination?"

Link State works differently:

“Tell everyone which links you have and what their costs are. Then everyone can build the network map and calculate shortest paths themselves.”

The core memory is:

Distance Vector = Ask neighbours Link State = Build complete map

1. WHAT IS LINK STATE ROUTING?

Link State is a routing approach in which every router learns the network topology by receiving information about the links connected to routers.

Then each router independently calculates shortest paths using:

`\[ \boxed{\text{Dijkstra's Algorithm}} \]`

So the overall process is:

TEXT

```
Discover neighbours
!
Measure link costs
!
Create link-state information
!
Flood it through network
!
Build topology database
!
Run Dijkstra
!
Construct routing table
```

2. WHY IS IT CALLED “LINK STATE”?

Each router advertises the **state of its links**.

For example, router A may say:

TEXT

```
I am A.

My links are:

A-B cost 2
A-C cost 5
A-D cost 1
```

That information describes:

A's local link state.

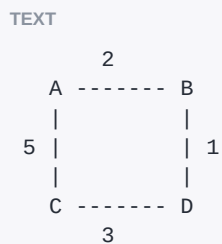
It does not simply say:

“My distance to destination X is 7.”

That would be Distance Vector thinking.

3. SIMPLE EXAMPLE

Network:



Router A knows its directly attached links:

\[A-B=2 \]

\[A-C=5 \]

So A can create an advertisement saying:

TEXT

Router A:

B	!	c o s t	2
C	!	c o s t	5

Router B might advertise:

TEXT

Router B:

A	!	c o s t	2
D	!	c o s t	1

And so on.

4. LINK-STATEADVERTISEMENT — LSA

A router creates a message describing its local links.

This is often called a:

Link-State Advertisement (LSA)

or broadly a link-state packet/message depending on protocol terminology.

An LSA may contain things such as:

- router identity

- directly connected neighbours
- link costs
- sequence number
- age/lifetime information

For GATE, the key concept is:

Router advertises its own directly connected links and their costs.

5. IMPORTANT DISTINCTION

A router normally advertises:

its local links

not:

its complete shortest-path routing table.

That is a crucial Link State vs Distance Vector difference.

6. WHAT HAPPENS TO THE LSA?

The advertisement is distributed throughout the routing domain using:

FLOODING

Flooding means:

A router sends received link-state information onward so that all relevant routers eventually receive it.

7. FLOODING INTUITION

Suppose:

TEXT

```
A ---- B ---- C
      |
      D
```

A creates an LSA.

A sends it to B.

B then forwards it toward:

- C
- D

So eventually:

TEXT

A's link-state information

```
      !"  
A  !'  B  !'  C  
      !"  
D
```

All routers learn A's link information.

8. DOES FLOODING MEAN INFINITE DUPLICATES?

It would, unless routers identify duplicate LSAs.

Therefore real link-state mechanisms use information such as:

- sequence numbers
- age
- router ID

to determine:

Is this new information or an old duplicate?

9. SEQUENCE NUMBER

Suppose A sends:

TEXT

A LSA sequence = 15

Later A sends:

TEXT

A LSA sequence = 16

Router receiving both knows:

16 is newer than 15.

If it later receives duplicate sequence 15:

Ignore the older information.

10. WHY AGE / LIFETIME?

Suppose a router disappears forever.

Its old LSA should not remain in databases permanently.

So link-state information typically has some:

age / expiration mechanism.

For GATE, remember the purpose:

Remove stale topology information.

11. LINK-STATE DATABASE

After receiving LSAs from all routers, each router builds a:

Link-State Database (LSDB)

Conceptually, it contains the network graph.

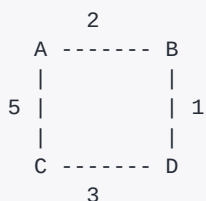
Example:

TEXT

A-B = 2
A-C = 5
B-D = 1
C-D = 3

Then every router can reconstruct:

TEXT



12. IMPORTANT

Under stable convergence:

Routers in the same link-state routing domain should have a consistent topology database.

But they calculate different shortest-path trees because each router uses:

itself as the root.

13. EXAMPLE

A runs Dijkstra with:

$\{A\}$

as root.

B runs Dijkstra with:

$\{B\}$

as root.

The underlying graph is the same.

But the shortest-path trees differ.

14. DISTANCE VECTOR vs LINK STATE— KNOWLEDGE

Distance Vector

Router A knows approximately:

TEXT

```
D e s t i n a t i o n   B   ! '   c o s t   2
D e s t i n a t i o n   C   ! '   c o s t   4
D e s t i n a t i o n   D   ! '   c o s t   5
```

It does not necessarily know why.

Link State

Router A knows topology like:

TEXT

```
A-B = 2
A-C = 5
B-D = 1
C-D = 3
```

Then calculates shortest routes itself.

15. MEMORY TRICK

DV knows answers. LS knows the map.

Nice way to remember it.

16. DIJKSTRA'S ALGORITHM

Priority queue is the main numerical part.

Dijkstra finds:

Shortest paths from one source to every other node in a graph with non-negative edge costs.

In Link State routing:

Each router runs Dijkstra with itself as source.

17. CORE IDEA OF DIJKSTRA

Suppose source is A.

We maintain:

N'

Set of nodes whose shortest distance from A has been permanently determined.

$D(v)$

Current best-known distance from A to node v .

$p(v)$

Predecessor of v on the current best path.

18. DON'T PANIC ABOUT NOTATION

Think:

TEXT

N' = finalized nodes

$D(v)$ = best distance currently known

$p(v)$ = which previous node gives that best route

That's all.

19. ALGORITHM IN SIMPLE WORDS

Start from source.

Then repeatedly:

1. Choose the unvisited node with smallest tentative distance.
2. Mark it final.
3. Check whether going through it improves distances to its neighbours.
4. Repeat.

This update process is called:

Relaxation

20. MEMORY TRICK

Dijkstra:

Pick smallest ! Fix it ! Improve neighbours ! Repeat

Use:

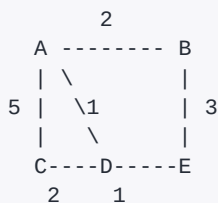
P-F-I-R

Pick minimum Finalize Improve neighbours Repeat

21. FULL GATE EXAMPLE

Consider:

TEXT



Let's define edges clearly:

$\{ A-B=2 \}$

$\{ A-C=5 \}$

$\{ A-D=1 \}$

$\{ B-E=3 \}$

$\{ C-D=2 \}$

$\{ D-E=1 \}$

We want shortest paths from:

$\{ \boxed{A} \}$

22. STEP 0 — INITIALIZATION

Source:

$\{ A \}$

So:

$\{ N'=\{A\} \}$

Distance to A:

$\{ 0 \}$

Direct neighbours:

$\{ D(B)=2 \}$

$\{ D(C)=5 \}$

$\{ D(D)=1 \}$

E isn't directly connected to A:

$\{ D(E)=\infty \}$

Table:

Node	Distance from A	Predecessor
B	2	A
C	5	A
D	1	A
E	"	—

23. STEP 1 — CHOOSE SMALLEST TENTATIVE NODE

Unfinalized distances:

$\{ B=2 \}$

$\{ C=5 \}$

$\{ D=1 \}$

$\{ E=\infty \}$

Smallest:

$\boxed{D}$

So:

$N'=\{A,D\}$

D's distance is permanently finalized:

$\{ D(D)=1 \}$

24. RELAX D'S NEIGHBOURS

D connects to:

- A
- C
- E

Check C through D

Current:

$\{ D(C)=5 \}$

Through D:

$$\boxed{D(D)+c(D,C)}$$

$$\boxed{=1+2=3}$$

Since:

$$\boxed{3<5}$$

update:

$$\boxed{D(C)=3}$$

Predecessor:

$$\boxed{p(C)=D}$$

Check E through D

Current:

$$\boxed{\infty}$$

Through D:

$$\boxed{1+1=2}$$

Update:

$$\boxed{D(E)=2}$$

and:

$$\boxed{p(E)=D}$$

Now table:

Node	Distance	Predecessor
B	2	A
C	3	D
D	1	A
E	2	D

25. STEP 2 — CHOOSE NEXT SMALLEST

Unfinalized:

$$\boxed{B=2}$$

$$\boxed{C=3}$$

$$\boxed{E=2}$$

There is a tie:

$$\boxed{B=2,\quad E=2}$$

Either may be selected depending on tie-breaking.

Let's choose B.

$N' = \{A, D, B\}$

26. RELAX B'S NEIGHBOURS

B connects to:

- A
- E

Check E through B:

$D(B) + c(B, E)$

$= 2 + 3 = 5$

Current E:

2

No improvement.

So E stays:

$\boxed{2}$

27. STEP 3 — NEXT MINIMUM

Remaining:

$E = 2$

$C = 3$

Choose:

E

Now:

$N' = \{A, D, B, E\}$

Check E's neighbours.

No shorter route appears.

28. STEP 4

Only C remains:

$D(C) = 3$

Finalize C.

Final distances from A:

Destination	Shortest Cost
A	0

Destination	Shortest Cost
B	2
C	3
D	1
E	2

29. BUILD SHORTEST-PATH TREE

Predecessors:

TEXT

B !• A
D !• A
C !• D
E !• D

So tree:

TEXT

graph TD; A --> B; A --> D; D --> C; D --> E

30. CONSTRUCT ROUTING TABLE

Now determine the **first hop** from A.

Destination B

Path:

TEXT

A !' B

Next hop:

\[B \]

Destination D

TEXT

A !' D

Next hop:

\[D \]

Destination C

Path:

TEXT

A !' D !' C

Next hop:

\[D \]

Destination E

Path:

TEXT

A !' D !' E

Next hop:

\[D \]

Routing table:

Destination	Cost	Next Hop
B	2	B
C	3	D
D	1	D
E	2	D

0-1 is exactly how shortest-path results become forwarding information.

31. VERY IMPORTANT — FINALIZED DISTANCE

Once Dijkstra selects the smallest tentative node and finalizes it:

Its shortest distance is permanent.

This relies on:

\[\boxed{\text{non-negative link costs}} \]

That is why Dijkstra does not correctly handle arbitrary negative edge weights.

32. & pGATE TRAP — DON'T UPDATE FINALIZED NODE

In ordinary Dijkstra:

Once a node is finalized, its shortest cost should not later improve.

If your calculation produces that situation with non-negative edges:

You probably made a mistake.

33. RELAXATION FORMULA

Suppose current finalized node is u .

For neighbour v :

$$D(v) = \min \{ D(v), D(u) + c(u, v) \}$$

This is the mathematical form of:

Check whether going through u gives a shorter path to v .

34. EXAMPLE

Current:

$$D(C) = 9$$

Finalized node B has:

$$D(B) = 4$$

Link:

$$B - C = 2$$

Candidate:

$$4 + 2 = 6$$

So:

$$D(C) = \min(9, 6)$$

$$= 6$$

35. DIJKSTRA TABLE METHOD

For GATE, the cleanest approach is to create a table.

Suppose nodes:

$$A, B, C, D, E$$

Write columns:

Step	N'	$D(B)$	$D(C)$	$D(D)$	$D(E)$
------	------	--------	--------	--------	--------

Then update each round.

This avoids mental mistakes.

36. SAME EXAMPLE IN TABLE FORM

Step	$\backslash(N\backslash)$	B	C	D	E
0	{A}	2	5	1	"
1	{A,D}	2	3	—	2
2	{A,D,B}	—	3	—	2
3	{A,D,B,E}	—	3	—	—
4	{A,D,B,E,C}	—	—	—	—

Final costs:

$\backslash[B=2, \backslash C=3, \backslash D=1, \backslash E=2 \backslash]$

37. GATE SHORTCUT FOR DIJKSTRA

Always do:

Step 1

Circle/finalize source with 0.

Step 2

Write direct neighbour costs.

Step 3

Choose smallest number.

Step 4

Update only that node's neighbours.

Step 5

Repeat.

Do not recompute all possible routes from scratch every time.

38. ANOTHER GATE NUMERICAL

Consider:

TEXT

```

A --4-- B --2-- D
|       |
1       1

```

```

|      |
C --5-- E --1-- D

```

Edges:

$\{ A-B=4 \}$

$\{ A-C=1 \}$

$\{ B-D=2 \}$

$\{ B-E=1 \}$

$\{ C-E=5 \}$

$\{ E-D=1 \}$

Find shortest path $A \rightarrow D$.

39. INITIALIZATION

From A:

$\{ B=4 \}$

$\{ C=1 \}$

$\{ D=\infty \}$

$\{ E=\infty \}$

Pick:

$\{ C=1 \}$

40. VIA C

$C \rightarrow E$ costs 5.

Candidate:

$\{ 1+5=6 \}$

So:

$\{ E=6 \}$

Current:

TEXT

$B=4$

$D = \infty$

$E=6$

Next pick:

$\{ B=4 \}$

41. VIA B

To D:

$$\sqrt{4+2=6}$$

So:

$$\sqrt{D=6}$$

To E:

$$\sqrt{4+1=5}$$

Current E was:

$$\sqrt{6}$$

update:

$$\sqrt{E=5}$$

Now:

TEXT

E=5

D=6

Pick E.

42. VIA E

E! D :

$$\sqrt{5+1=6}$$

Current D:

$$\sqrt{6}$$

Equal.

Therefore shortest cost:

$$\sqrt{\boxed{6}}$$

There are two equal-cost paths:

TEXT

A !' B !' D

cost:

$$\sqrt{4+2=6}$$

and:

TEXT

A !' B !' E !' D

cost:

\[4+1+1=6 \]

Depending on predecessor/tie handling, either may appear.

43. & pGATE TRAP — TIES

If two tentative nodes have equal minimum distance:

Either can usually be finalized first.

The final shortest distances remain valid.

Do not panic because your node order differs from an answer key unless the question specifies a tie-break rule.

44. LINK STATE AFTER A FAILURE

Suppose:

TEXT

A ---- B ---- C

link B-C fails.

B detects:

Its local link to C is down.

B creates a newer LSA.

It floods it.

Routers update topology database:

TEXT

Remove / invalidate B-C

Then each router reruns shortest-path computation.

45. DIFFERENCE FROM DISTANCE VECTOR FAILURE

Distance Vector may temporarily say:

“Maybe you still know a way.”

Link State says:

“The actual link B-C has failed.”

All routers eventually update their topology map.

Therefore it avoids the classic DV:

mechanism.

46. DOES LINK STATE HAVE NO PROBLEMS?

No protocol is perfect.

Link State still has costs:

- LSAs must be flooded
- routers store topology database
- each router runs shortest-path algorithm
- CPU/memory overhead
- transient inconsistencies can exist while updates propagate

But it avoids the specific classic DV count-to-infinity behavior.

47. LINK STATE OVERHEAD

Suppose network has many routers.

Every significant topology change can generate LSAs that must propagate throughout the routing domain.

So Link State generally uses more:

- topology information
- memory
- processing

than a very simple Distance Vector approach.

The tradeoff is usually:

better topology awareness and convergence properties.

48. WHAT INFORMATION IS FLOODED?

Important distinction.

Link State floods:

local link information.

Distance Vector exchanges:

destination distance estimates.

Do not swap these.

49. FLOODING vs FORWARDING

Another possible confusion:

Flooding LSAs

Send routing-control information broadly throughout network.

Forwarding normal data packets

Send based on destination forwarding table.

Link-state protocol doesn't mean:

every ordinary IP data packet is flooded everywhere.

Only the routing information uses flooding mechanisms.

& Important.

50. LINK STATE STEPS — MASTER SEQUENCE

Remember:

D-M-A-F-B-R

D — Discover neighbours

M — Measure costs

A — Advertise links

F — Flood advertisements

B — Build topology database

R — Run Dijkstra

A bit long, so a simpler memory is:

Know links !' Flood !' Map !' Dijkstra

51. LINK STATE vs DISTANCE VECTOR — MASTER COMPARISON

Feature	Distance Vector	Link State
Main knowledge	Distance estimates	Network topology
Shares with	Neighbours	Flooded throughout domain
Shares what	Distance vector	Link-state information
Main algorithm	Bellman-Ford	Dijkstra

Feature	Distance Vector	Link State
Complete topology needed?	No	Yes, locally reconstructed
Count-to-infinity	Possible	Not classic LS issue
Memory/CPU	Generally lower	Generally higher
Failure handling	Can converge slowly	Typically topology update + recomputation

Definitely notebook-worthy.

52. MEMORY TRICK

Distance Vector

Neighbour tells me the answer.

Link State

Neighbour/router tells me the roads; I calculate the answer.

53. DIJKSTRA vs BELLMAN-FORD

Bellman-Ford / DV idea

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

Uses neighbour distance estimates.

Dijkstra / LS idea

Repeatedly choose:

smallest tentative distance

and relax links.

$$D(v) = \min [D(v), D(u) + c(u,v)]$$

54. GATE QUESTION PATTERN— IDENTIFY ALGORITHM

Question says:

Every router has complete topology and calculates shortest-path tree.

Answer:

$\boxed{\text{Link State / Dijkstra}}$

Question says:

Routers exchange distance vectors with neighbours.

Answer:

$\boxed{\text{Distance Vector / Bellman-Ford}}$

55. GATE QUESTION PATTERN— NEXT HOP

Suppose Dijkstra gives shortest path:

TEXT

A !' C !' E !' F

For destination F, A's routing-table next hop is:

$\boxed{C}$

not:

- E
- F

56. GATE QUESTION PATTERN— LSA CHANGE

Suppose link cost changes:

$A-B: 3 \rightarrow 8$

A and/or B generate updated link-state information.

This is flooded.

Routers update LSDB.

Then:

rerun/recalculate shortest paths.

57. GATE QUESTION PATTERN— SEQUENCE NUMBER

Suppose router receives two LSAs from X:

TEXT

Seq 25
Seq 27

Generally:

$\boxed{27}$

is newer.

This helps suppress stale duplicates.

58. COMPLEXITY — USEFUL GATE FOUNDATION

A basic array implementation of Dijkstra can take approximately:

$$O(V^2)$$

Using appropriate priority-queue structures, it can be improved, for example:

$$O((V+E)\log V)$$

depending on implementation.

For Computer Networks GATE, numerical execution is usually more important than implementation complexity, but know:

Dijkstra is efficient for non-negative link costs.

59. & pGATE TRAPS

Trap 1

Link State does not exchange entire shortest-path routing tables with neighbours as its fundamental mechanism.

Trap 2

LSA flooding is for routing information, not normal user data.

Trap 3

Dijkstra needs non-negative link costs.

Trap 4

Shortest path means minimum total cost, not minimum number of hops.

Trap 5

Once a node is finalized in Dijkstra with non-negative edges, its distance should remain final.

Trap 6

Link State does not suffer the classic count-to-infinity mechanism.

Trap 7

Every router builds a topology database, but shortest-path tree is rooted at itself.

Trap 8

Tie in tentative distances does not necessarily mean calculation is wrong.

60. MASTER DIJKSTRA MEMORY

Remember:

TEXT

```
Source = 0
Direct neighbours = link costs
O t h e r s = "
```

Then:

Pick smallest !' Finalize !' Relax neighbours !' Repeat

Formula:

$$D(v) = \min[D(v), D(u) + c(u, v)]$$

' p NOTEBOOK VERSION

Link State Routing

Each router:

1. Discovers neighbours.
2. Determines local link costs.
3. Creates Link-State Advertisement.
4. Floods it through network.
5. Builds Link-State Database.
6. Runs Dijkstra.
7. Builds routing table.

LSA

Contains local topology information such as:

- router ID
- neighbours
- link costs
- sequence/lifetime information

Flooding

Link-state information is distributed throughout routing domain.

Duplicate/old advertisements are controlled using:

- sequence numbers
 - age/lifetime
-

LSDB

Each router reconstructs the topology graph.

Stable routers should have consistent topology knowledge.

Dijkstra

Initialize:

$\forall D(\text{source})=0$

Direct neighbours:

$\forall D(v)=c(\text{source},v)$

Others:

∞

Repeat:

1. choose smallest tentative node
2. finalize it
3. relax neighbours

Relaxation:

$\forall D(v) = \min[D(v), D(u) + c(u,v)]$

Link State vs Distance Vector

TEXT

```
DV !' Neighbour distances !' Bellman - Ford
LS !' Topology map !' Dijkstra
```

Memory:

DV asks for directions. LS gets the map.

61. 5-MINUTE REVISION — ROUTING COMPLETE SO FAR

Routing

Determine path.

Forwarding

Send packet to next hop.

Distance Vector

$$D_x(y) = \min_v [c(x,v) + D_v(y)]$$

- neighbour-based
- Bellman-Ford
- count-to-infinity possible

Mitigations:

- split horizon
- poison reverse

Link State

TEXT

```
Local links
!' L S A
!' F l o o d
!' L S D B
!' D i j k s t r a
!' R o u t i n g   t a b l e
```

No classic count-to-infinity.

62. 30-SECOND REVISION

Remember:

DV = Ask neighbour how far.

LS = Learn complete map.

Dijkstra:

$\boxed{\text{Pick smallest !' Finalize !' Relax !' Repeat}}$

And:

LSA is flooded; ordinary user packets are not automatically flooded.

ACTIVE RECALL — DON'T LOOK BACK

Q1

What information does a Link State router advertise?

A. Only final destination costs B. Its local links and their costs C. Application data D. Only MAC addresses

Q2

What is the purpose of flooding in Link State routing?

Q3

Why are sequence numbers useful in link-state advertisements?

Q4

What does every router build after receiving link-state information?

Q5

Which shortest-path algorithm is associated with Link State routing?

Q6 — Dijkstra Numerical

Router A currently has:

$[D(B)=3]$

$[D(C)=8]$

B is finalized.

Link:

$[B-C=2]$

What should the new tentative distance to C be?

Q7

Suppose shortest path from A to X is:

TEXT

A !' D !' E !' X

What is A's next hop for destination X?

Q8 — GATE Trap

True or False:

Link State routers flood every user data packet to all routers.

Explain.

Q9

Why does classic count-to-infinity occur in Distance Vector but not in the same form in Link State?

Q10

What condition on link costs is required for standard Dijkstra correctness?

Q11 — Full Small Graph

Given:

$\{ A-B=2 \}$

$\{ A-C=6 \}$

$\{ B-C=1 \}$

$\{ B-D=5 \}$

$\{ C-D=2 \}$

Run Dijkstra mentally from A and find:

1. shortest distance A to B
2. shortest distance A to C
3. shortest distance A to D
4. next hop from A to D

NEXT ! IPv4 Addressing Foundation: 32-bit Addresses, Network/Host Portion, Subnet Masks & Binary Conversion Before CIDR

USER

next

ASSISTANT

CHAPTER 11 — IPv4 ADDRESSING FOUNDATION

TOPIC 1 — 32-bit IPv4 Addresses, Network/Host Portion, Subnet Masks & Binary Conversion

OVERVIEW: HIGH PRIORITY FOUNDATION FOR CIDR, FRAGMENTATION & NAT

Before CIDR, you must be comfortable with:

1. IPv4 = **32 bits**
2. Dotted-decimal notation
3. Binary to decimal conversion
4. Network part vs host part
5. Subnet mask
6. Network address
7. Broadcast address
8. Number of addresses/hosts

Once these become automatic, CIDR numerals become much easier.

1. WHAT IS AN IPv4 ADDRESS?

An IPv4 address is a:

$\boxed{32\text{-bit logical address}}$

used at the:

Network Layer

Example:

TEXT

192 . 168 . 10 . 25

This is called:

Dotted-decimal notation

2. WHY "DOTTED DECIMAL"?

The 32 bits are divided into four groups of 8 bits.

Each 8-bit group is called an:

Octet

So:

TEXT

192 . 168 . 10 . 25

! ' ! ' ! ' ! '
8b 8b 8b 8b

Total:

$8+8+8+8=32\text{ bits}$

Therefore:

$\boxed{\text{IPv4} = 4 \text{ octets} = 32 \text{ bits}}$

3. MEMORY TRICK

IPv4:

$$4 \times 8 = 32$$

IPv4 is four octets.

Each:

8 bits

4. RANGE OF ONE OCTET

An octet has 8 bits.

Minimum:

TEXT

00000000

which is:

0

Maximum:

TEXT

11111111

which is:

255

Therefore every normal dotted-decimal octet lies between:

$\boxed{0 \text{ and } 255}$

5. WHY MAXIMUM 255?

For 8 bits:

11111111

represents:

$128+64+32+16+8+4+2+1$

$=255$

So:

$\boxed{2^8-1=255}$

6. BINARY PLACE VALUES

You need this for GATE subnetting/CIDR.

For one octet:

Bit position	Value
1st	128
2nd	64
3rd	32
4th	16
5th	8
6th	4
7th	2
8th	1

Memory:

TEXT

128 64 32 16 8 4 2 1

Memorize this sequence.

7. BINARY !' DECIMAL

Convert:

TEXT

11001010

Place values:

TEXT

1	1	0	0	1	0	1	0
128	64	32	16	8	4	2	1

Add positions containing 1 :

$\lfloor 128+64+8+2 \rfloor$

$\lfloor =\boxed{202} \rfloor$

Therefore:

TEXT

1 1 0 0 1 0 1 0 , = 2 0 2 • €

8. DECIMAL !' BINARY

Convert:

\[192 \]

Use powers:

\[192=128+64 \]

So:

TEXT

128 64 32 16 8 4 2 1
1 1 0 0 0 0 0 0

Therefore:

\[192=\boxed{11000000} \]

9. IMPORTANT COMMON VALUES

These appear constantly in subnet masks.

Binary	Decimal
00000000	0
10000000	128
11000000	192
11100000	224
11110000	240
11111000	248
11111100	252
11111110	254
11111111	255

Memorize this table.

It makes CIDR much faster.

10. MEMORY PATTERNFOR MASK VALUES

Watch the sequence:

TEXT

128
192
224
240
248
252
254
255

Each time, another 1 is added from the left.

TEXT

10000000 = 128
11000000 = 192
11100000 = 224
11110000 = 240
...

11. WHAT DOES AN IP ADDRESS IDENTIFY?

An IPv4 address conceptually contains two portions:

TEXT

NETWORK PORTION | HOST PORTION

The network part identifies:

Which network/subnet?

The host part identifies:

Which interface within that subnet?

12. SIMPLE EXAMPLE

Suppose:

TEXT

192.168.1.25/24

The /24 means:

First 24 bits are the network prefix.

So:

TEXT

```
192 . 168 . 1 . 25
|-----| |---|
 24 bits   8 bits
Network    Host
```

Network portion:

TEXT

192.168.1

Host portion:

TEXT

25

This is simplified notation; technically we work in bits.

13. WHAT IS A SUBNET MASK?

A subnet mask tells us:

Which bits belong to the network prefix and which bits belong to the host portion.

A mask uses:

- 1 !' network bit
- 0 !' host bit

Example:

TEXT

11111111.11111111.11111111.00000000

In decimal:

TEXT

255.255.255.0

Therefore:

$\boxed{255.255.255.0=24}$

14. WHY /24?

Count the 1 bits:

TEXT

```
11111111 = 8
11111111 = 8
```

```
11111111 = 8
00000000 = 0
```

Total:

$\lfloor 8+8+8=24 \rfloor$

Hence:

$\lfloor \boxed{/24} \rfloor$

15. CIDR PREFIX — PREVIEW

Notation:

```
TEXT
192.168.1.25/24
```

means:

IPv4 address = 192.168.1.25

Prefix length = 24 bits

Therefore:

$\lfloor 32-24=8 \rfloor$

host bits.

We will go much deeper into CIDR next.

16. NETWORK ADDRESS

Very important.

The **network address** is obtained by setting all host bits to:

$\lfloor 0 \rfloor$

Example:

```
TEXT
192.168.1.25/24
```

Host bits are final 8 bits.

Set them all to zero:

```
TEXT
192.168.1.0
```

Therefore:

$\boxed{\text{Network address}=192.168.1.0}$

17. BROADCAST ADDRESS

For an IPv4 subnet supporting broadcast, the directed broadcast address is obtained by setting all host bits to:

1

For:

TEXT

192.168.1.25/24

final 8 bits become:

TEXT

11111111 = 255

So:

$\boxed{192.168.1.255}$

18. USABLE HOST RANGE

For the common subnet case:

Network:

TEXT

192.168.1.0

Broadcast:

TEXT

192.168.1.255

Usable host addresses:

TEXT

192.168.1.1

through:

TEXT

192.168.1.254

19. WHY CAN'T NETWORK ADDRESS BE ASSIGNED TO A NORMAL HOST?

Because it identifies:

the subnet itself.

Example:

TEXT

192.168.1.0/24

represents the network.

20. WHY CAN'T THE BROADCAST ADDRESS BE A NORMAL HOST?

Because it represents:

all hosts in that IPv4 subnet.

Example:

TEXT

192.168.1.255

for /24.

Therefore in traditional IPv4 subnet calculations, network and broadcast addresses are reserved.

21. NUMBER OF ADDRESSES IN A SUBNET

If prefix length is:

$\backslash /n \backslash$

then host bits:

$\backslash \boxed{h=32-n} \backslash$

Number of total addresses:

$\backslash \boxed{2^h} \backslash$

or:

$\backslash \boxed{2^{32-n}} \backslash$

~~0=70%~~ Fundamental CIDR formula.

22. USABLE HOSTS

For ordinary IPv4 subnets where network and broadcast addresses are reserved:

$\backslash \boxed{\text{Usable}\backslash \text{hosts}=2^h-2} \backslash$

or:

$\backslash \boxed{2^{32-n}-2} \backslash$

23. EXAMPLE — /24

$n=24$

Host bits:

$32-24=8$

Total addresses:

$2^8=256$

Usable traditional hosts:

$256-2$

$\boxed{254}$

24. EXAMPLE — /26

Prefix:

$/26$

Host bits:

$32-26=6$

Total addresses:

$2^6=64$

Traditional usable hosts:

$64-2$

$\boxed{62}$

25. & pGATE TRAP — /31 AND /32

Do not blindly use:

2^h-2

for every possible prefix.

Special cases exist.

For example:

- $/31$ can be used on point-to-point links under modern standards.
- $/32$ identifies a single host/route.

But for ordinary exam subnetting questions unless special context is stated:

Use the standard network/broadcast reservation logic.

We'll flag exceptions when relevant.

26. BITWISE AND — HOW NETWORK ADDRESS IS ACTUALLY FOUND

Network address can be computed as:

$\boxed{\text{IP address AND subnet mask}}$

Bitwise AND rules:

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Memory:

AND gives 1 only when both are 1.

27. NETWORK ADDRESS EXAMPLE

IP:

TEXT

192.168.1.130

Mask:

TEXT

255.255.255.192

Last octet:

$\boxed{130}$

Binary:

TEXT

10000010

Mask 192:

TEXT

11000000

AND:

TEXT

10000010

11000000

10000000

which is:

$\backslash[128 \backslash]$

Therefore network address:

$\backslash \boxed{192.168.1.128} \backslash$

28. WHAT PREFIX IS 255.255.255.192?

First three octets:

$\backslash[24 \text{ones} \backslash]$

Last octet:

TEXT
11000000

contains:

$\backslash[2 \backslash]$

ones.

Total:

$\backslash[24+2=26 \backslash]$

Therefore:

$\backslash \boxed{255.255.255.192=26} \backslash$

29. /26 SUBNET RANGE

We found network:

TEXT
192.168.1.128/26

Host bits:

$\backslash[6 \backslash]$

Block contains:

$\backslash[2^6=64 \backslash]$

addresses.

Range therefore:

TEXT
192.168.1.128
through

192.168.1.191

Why 191?

$\lfloor 128+64-1=191 \rfloor$

So:

Network

$\lfloor \boxed{192.168.1.128} \rfloor$

Broadcast

$\lfloor \boxed{192.168.1.191} \rfloor$

Usable

TEXT

192.168.1.129

to:

TEXT

192.168.1.190

30. BLOCK SIZE SHORTCUT

$O = 256 - M$ is useful.

If the interesting subnet-mask octet is:

$\lfloor M \rfloor$

then block size is:

$\lfloor \boxed{256-M} \rfloor$

Example mask:

TEXT

255.255.255.192

Interesting octet:

$\lfloor 192 \rfloor$

Block size:

$\lfloor 256-192=64 \rfloor$

Subnet boundaries:

TEXT

0
64
128
192

Each block contains 64 addresses.

31. ANOTHER EXAMPLE — /27

Mask:

$\backslash$ /27 $\backslash$

First 24 bits plus 3 network bits:

```
TEXT
11100000
```

which is:

$\backslash$ 224 $\backslash$

So mask:

```
TEXT
255.255.255.224
```

Block size:

$\backslash$ $256 - 224 = \boxed{32}$ $\backslash$

Subnet boundaries:

```
TEXT
0
32
64
96
128
160
192
224
```

32. GIVEN IP — FIND SUBNET

IP:

```
TEXT
192.168.1.77/27
```

Block size:

$\backslash$ 32 $\backslash$

Blocks:

```
TEXT
0-31
32-63
64-95
```

96-127

...

77 lies in:

$\backslash [64-95]$

Therefore:

Network

$\backslash \boxed{192.168.1.64}$

Broadcast

$\backslash \boxed{192.168.1.95}$

Usable range

TEXT

192.168.1.65 - 192.168.1.94

33. GATE SHORTCUT — DON'T ALWAYS CONVERT EVERYTHING TO BINARY

For prefixes within the last octet, use:

Block size = 256 - mask value

Then find which block contains the IP.

Binary is still essential for understanding and unusual prefixes, but block-size method is much faster in exams.

34. IMPORTANT PREFIX/MASK TABLE

Worth memorizing.

Prefix	Last mask octet	Block size	Addresses
/24	0	256	256
/25	128	128	128
/26	192	64	64
/27	224	32	32
/28	240	16	16
/29	248	8	8
/30	252	4	4
/31	254	2	2
/32	255	1	1

This table alone saves a lot of time.

35. MEMORY PATTERN

Prefix increases by 1:

TEXT

/24 /25 /26 /27 /28 /29 /30

Addresses halve each time:

TEXT

256 128 64 32 16 8 4

Very important.

36. CLASSFUL ADDRESSING — HOW MUCH DO YOU NEED?

Historically IPv4 used classes:

- Class A
- Class B
- Class C
- Class D
- Class E

But your syllabus specifically says:

CIDR notation

Modern routing does not rely on old class boundaries in the same way.

Still, a little classful knowledge may appear as prerequisite/context.

37. CLASS A/B/C — BASIC HISTORICAL TABLE

Class	First bits	Traditional prefix	First-octet range
A	0	/8	1–126
B	10	/16	128–191
C	110	/24	192–223

Class D:

[224-239]

for multicast.

Class E:

\[240-255 \]

historically reserved/experimental.

38. & DO NOT USE CLASSFUL MASK IF CIDR PREFIX IS GIVEN

Suppose:

TEXT

10.20.30.40/27

Someone may think:

10.x.x.x is Class A, so mask is /8.

'L Wrong.

The explicit CIDR prefix:

\[/27 \]

wins.

Modern CIDR permits prefix boundaries independent of old classes.

Ø=important.

39. PRIVATE IPv4 RANGES — IMPORTANT FOR NAT

Because NAT is in your syllabus, remember these private ranges:

\[\boxed{10.0.0.0/8} \]

\[\boxed{172.16.0.0/12} \]

\[\boxed{192.168.0.0/16} \]

These are private address spaces commonly used inside local networks.

They are not globally routed on the public Internet in the normal way.

40. Ø> PRIVATE RANGE MEMORY

Think:

10 — 172.16 — 192.168

These three are worth memorizing because we'll use them when studying NAT.

41. LOOPBACK — SMALL IMPORTANT FACT

IPv4 loopback range:

`\[ 127.0.0.0/8 \]`

Most famous:

TEXT

`127.0.0.1`

means:

this host / loopback interface.

Do not use it as a normal remote-host destination.

42. LIMITED BROADCAST

IPv4:

TEXT

`255.255.255.255`

is the limited broadcast address in appropriate local-network context.

Do not confuse this with a subnet's directed broadcast such as:

TEXT

`192.168.1.255`

for `192.168.1.0/24`.

43. SAME SUBNET TEST

Suppose:

TEXT

Host A = `192.168.1.10/24`

Host B = `192.168.1.200/24`

A's network:

TEXT

`192.168.1.0`

B's network:

TEXT

`192.168.1.0`

Same network address:

`\[ \boxed{\text{Same subnet}} \]`

44. DIFFERENT SUBNET EXAMPLE

A:

TEXT

192.168.1.10/24

Network:

TEXT

192.168.1.0

B:

TEXT

192.168.2.20/24

Network:

TEXT

192.168.2.0

Different:

$\boxed{\text{Different subnets}}$

Communication normally requires a router/default gateway between them.

45. GATE-STYLE SAME-SUBNET TRAP

Consider:

TEXT

A = 192.168.1.70/26

B = 192.168.1.120/26

Don't say same subnet just because first three octets match.

For /26, block size:

$\boxed{64}$

Subnets:

TEXT

0-63

64-127

128-191

192-255

70 and 120 both lie:

$\boxed{64-127}$

So yes, same subnet.

Now consider:

TEXT

A = 192.168.1.70/26
B = 192.168.1.130/26

70 belongs to:

TEXT

64–127

130 belongs:

TEXT

128–191

Therefore:

$\boxed{\text{Different subnets}}$

even though first three octets match.

46. HOW TO FIND NETWORK ADDRESS QUICKLY

For an IP/prefix question:

Step 1

Find mask.

Step 2

Find block size.

Step 3

Locate address inside correct block.

Step 4

First address = network.

Step 5

Last address = broadcast.

Step 6

Middle addresses = hosts.

47. EXAMPLE — COMPLETE

Find subnet details for:

TEXT

172.16.10.141/28

Step 1 — Prefix

$\backslash$ /28 $\backslash$

Mask:

TEXT

255.255.255.240

Step 2 — Block size

$\backslash$ 256-240=16 $\backslash$

Step 3 — Boundaries

TEXT

0
16
32
48
64
80
96
112
128
144
160
...

141 lies in:

TEXT

128-143

Network

$\backslash$ $\boxed{172.16.10.128}$ $\backslash$

Broadcast

$\backslash$ $\boxed{172.16.10.143}$ $\backslash$

Usable hosts

TEXT

172.16.10.129

to:

TEXT

172.16.10.142

Number of addresses

Host bits:

$$\lfloor 32-28=4 \rfloor$$

$$\lfloor 2^4=16 \rfloor$$

Traditional usable:

$$\lfloor 16-2=14 \rfloor$$

48. PREFIX !' HOST BITS

Always:

$$\lfloor \boxed{\text{HostBits}=32-\text{Prefix}} \rfloor$$

Examples:

/20

$$\lfloor 32-20=12 \rfloor$$

host bits.

/29

$$\lfloor 32-29=3 \rfloor$$

host bits.

49. HOST REQUIREMENT !' HOW MANY BITS?

Suppose a subnet needs at least:

$$\lfloor 50 \rfloor$$

usable hosts.

Need:

$$\lfloor 2^h-2 \geq 50 \rfloor$$

Try:

$$\lfloor h=5 \rfloor$$

$$\lfloor 2^5-2=30 \rfloor$$

not enough.

Try:

$$\lfloor h=6 \rfloor$$

$$\lfloor 2^6-2=62 \rfloor$$

enough.

Therefore host bits:

$$\lfloor 6 \rfloor$$

Prefix:

$$\lfloor 32-6 = \boxed{/26} \rfloor$$

~~0=Very~~ Very common CIDR/subnetting pattern.

50. GATE SHORTCUT FOR HOST REQUIREMENT

Find smallest power of 2 satisfying:

$$\lfloor 2^h \geq \text{Hosts}+2 \rfloor$$

Then:

$$\lfloor \text{Prefix}=32-h \rfloor$$

Example 100 hosts:

$$\lfloor 100+2=102 \rfloor$$

next power of 2:

$$\lfloor 128=2^7 \rfloor$$

So:

$$\lfloor h=7 \rfloor$$

prefix:

$$\lfloor \boxed{/25} \rfloor$$

51. NUMBER OF SUBNETS — FOUNDATION

Suppose you begin conceptually with a $/24$ block and divide it into $/27$ subnets.

Borrowed bits:

$$\lfloor 27-24=3 \rfloor$$

Number of equal subnets:

$$\lfloor \boxed{2^3=8} \rfloor$$

Each contains:

$$\lfloor 2^{32-27}=32 \rfloor$$

addresses.

52. EXAMPLE

Block:

TEXT

192.168.1.0/24

Split into /26.

Additional prefix bits:

$\lfloor 26-24=2 \rfloor$

Number of subnets:

$\lfloor 2^2=4 \rfloor$

They are:

TEXT

192.168.1.0/26
192.168.1.64/26
192.168.1.128/26
192.168.1.192/26

Each:

$\lfloor 64 \rfloor$

addresses.

53. CONNECTION TO CIDR

Next topic.

CIDR will build directly on this foundation and add:

- arbitrary prefix lengths
- address blocks
- aggregation
- longest-prefix matching
- determining valid CIDR blocks
- subnetting calculations

Everything depends on:

$\lfloor \boxed{\text{Prefix bits} + \text{Host bits} = 32} \rfloor$

54. CONNECTION TO ROUTING

Routers don't usually need one entry per individual host.

Instead they can route based on prefixes such as:

TEXT

10.1.0.0/16

This represents many addresses.

When multiple prefixes match:

Longest Prefix Matching

selects the most specific route.

We'll cover that in CIDR.

55. CONNECTION TO IPv4 FRAGMENTATION

An IPv4 packet contains:

- source IP
- destination IP
- header fields

If too large for outgoing MTU, it may be fragmented.

Addressing tells us:

where the fragments are ultimately going.

Fragmentation will introduce completely different fields:

- Identification
- MF
- DF
- Fragment Offset

We'll handle those after CIDR.

56. CONNECTION TO NAT

Private address:

TEXT

192.168.1.10

may be translated by NAT to a public IP.

Therefore this addressing foundation will be reused when learning:

private vs public addresses + translation tables.

57. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Binary conversion

Convert decimal IP octet to binary or vice versa.

Pattern 2 — Prefix to mask

Example:

$\lfloor /27 \rfloor$

!

TEXT

255 . 255 . 255 . 224

Pattern 3 — Mask to prefix

Example:

TEXT

255 . 255 . 255 . 240

!

$\lfloor /28 \rfloor$

Pattern 4 — Number of addresses

$\lfloor 2^{\{32-n\}} \rfloor$

Pattern 5 — Network/broadcast

Given IP/prefix.

Find subnet range.

Pattern 6 — Same subnet

Calculate network address for both hosts.

Pattern 7 — Required host count

Find minimum prefix.

Pattern 8 — Divide a block into subnets

Calculate new prefixes and ranges.

58. & pGATE TRAPS

Trap 1

IPv4 is:

$\lfloor 32 \text{ bits} \rfloor$

not 32 bytes.

Trap 2

Subnet mask 1-bits indicate:

prefix/network portion.

0-bits indicate:

host portion.

Trap 3

Don't assume /24 just because address starts 192.x.x.x.

If CIDR prefix is given, use it.

Trap 4

Network address:

all host bits = 0.

Broadcast:

all host bits = 1.

Trap 5

Total addresses:

2^h

Traditional usable host addresses:

2^{h-2}

These are not the same.

Trap 6

Don't determine same subnet merely from visible decimal octets.

Prefix may divide inside an octet.

Trap 7

/26 means:

26

network bits, not 26 host bits.

Trap 8

Block size:

$\lfloor 256 - \text{mask} \rfloor$

works on the subnetted octet, but apply it carefully if prefix falls in an earlier octet.

59. MASTER MEMORY SHEET

IPv4

$\lfloor \boxed{32 \text{ bits} = 4 \times 8} \rfloor$

Prefix

$\lfloor \boxed{\text{Network Bits} = n} \rfloor$

for $/n$.

Host bits

$\lfloor \boxed{32 - n} \rfloor$

Total addresses

$\lfloor \boxed{2^{\{32 - n\}}} \rfloor$

Traditional hosts

$\lfloor \boxed{2^{\{32 - n\}} - 2} \rfloor$

Network

Host bits !' all 0.

Broadcast

Host bits !' all 1.

Fast block size

$\lfloor \boxed{256 - \text{MaskOctet}} \rfloor$

60. 'p NOTEBOOK VERSION

IPv4 Addressing

IPv4 address:

$\boxed{32\text{bits}}$

Written as four 8-bit octets:

TEXT

A . B . C . D

Each octet range:

$\boxed{0-255}$

Binary values

TEXT

128 64 32 16 8 4 2 1

Common mask values:

TEXT

128 192 224 240 248 252 254 255

Prefix

For:

$\boxed{/n}$

Network bits:

$\boxed{n}$

Host bits:

$\boxed{32-n}$

Subnet Mask

1 !' Network bit 0 !' Host bit

Example:

$\boxed{/24=255.255.255.0}$

Network Address

$\boxed{IP \text{ AND } Mask}$

Equivalent:

Set all host bits = 0.

Broadcast Address

Set all host bits = 1.

Address count

$$\boxed{\text{Total} = 2^{32-n}}$$

Traditional usable:

$$\boxed{2^{32-n} - 2}$$

Fast Block Method

$$\boxed{\text{BlockSize} = 256 - \text{MaskValue}}$$

Example:

TEXT

/27

Mask = 255.255.255.224

Block = 32

Private Ranges

$$\boxed{10.0.0.0/8}$$

$$\boxed{172.16.0.0/12}$$

$$\boxed{192.168.0.0/16}$$

61. 30-SECOND REVISION

Remember:

$$\boxed{\text{IPv4} = 32\text{bits}}$$

$$\boxed{\text{HostBits} = 32 - \text{Prefix}}$$

$$\boxed{\text{Addresses} = 2^{\text{HostBits}}}$$

Network:

host bits 0

Broadcast:

host bits 1

And memorize:

TEXT

/25 128
/26 192
/27 224
/28 240
/29 248
/30 252

ACTIVE RECALL — DON'T LOOK BACK

Q1

How many:

1. bits
2. octets

are present in IPv4?

Q2

Convert:

TEXT

11010010

to decimal.

Q3

Convert:

\[224 \]

to 8-bit binary.

Q4

What subnet mask corresponds to:

\[/27 \]

?

Q5

What CIDR prefix corresponds to:

TEXT

255.255.255.240

?

Q6

For:

TEXT

192.168.5.130/26

find:

1. network address
2. broadcast address
3. usable host range
4. total number of addresses

Q7

For:

\[/28 \]

how many:

1. host bits?
2. total addresses?
3. traditional usable host addresses?

Q8 — Same Subnet

Are these hosts in the same /26 subnet?

TEXT

192.168.1.90
192.168.1.125

Q9

Are these hosts in the same /26 subnet?

TEXT

192.168.1.90
192.168.1.150

Q10 — Host Requirement

You need at least:

\[100 \]

traditional usable IPv4 host addresses.

What is the smallest suitable prefix length?

Q11 — Subnet Division

How many /27 subnets can be formed from one /24 block?

Q12 — Future GATE Connection

What routing rule is used if destination IP matches multiple CIDR prefixes in a router's forwarding table?

NEXT !' CIDR: Prefix Calculations, Subnet Ranges, Route Aggregation & Longest Prefix Matching

USER

next

ASSISTANT

CHAPTER 11 — IPv4 ADDRESSING

TOPIC 2 — CIDR: Prefix Calculations, Valid Blocks, Subnet Ranges, Route Aggregation & Longest Prefix Matching

OVER HIGH PRIORITY FOR GATE

CIDR questions are among the most scoring IPv4 questions once you master four ideas:

1. Prefix length
2. Block size
3. Valid CIDR block boundaries
4. Longest Prefix Matching

The central formula is:

$$\boxed{\text{Number of addresses} = 2^{(32-n)}}$$

for a CIDR block with prefix:

$$/n$$

1. WHAT IS CIDR?

CIDR stands for:

Classless Inter-Domain Routing

The key word is:

Classless

Old IPv4 addressing depended heavily on Class A, B and C boundaries.

CIDR allows arbitrary prefix lengths such as:

TEXT

/13
/19
/22
/27
/30

So a network is represented as:

TEXT

IP-address/prefix-length

Example:

TEXT

192.168.10.0/24

2. WHAT DOES /n MEAN?

For:

$\lfloor /n \rfloor$

the first:

$\lfloor n \rfloor$

bits represent the network prefix.

Remaining:

$\lfloor 32-n \rfloor$

bits represent addresses inside the block.

Therefore:

$\lfloor \boxed{\text{PrefixBits} + \text{HostBits} = 32} \rfloor$

3. EXAMPLE — /20

For:

$\lfloor /20 \rfloor$

Prefix bits:

$\lfloor 20 \rfloor$

Host bits:

$$\lfloor 32-20=12 \rfloor$$

Number of addresses:

$$\lfloor 2^{12} \rfloor$$

$$\lfloor \boxed{4096} \rfloor$$

4. EXAMPLE — /27

Host bits:

$$\lfloor 32-27=5 \rfloor$$

Addresses:

$$\lfloor 2^5=32 \rfloor$$

So every /27 block contains:

$$\lfloor \boxed{32 \text{ addresses}} \rfloor$$

5. FAST ADDRESS TABLE

Memorize the pattern:

Prefix	Addresses
/24	256
/25	128
/26	64
/27	32
/28	16
/29	8
/30	4
/31	2
/32	1

Every time prefix increases by 1:

Address count halves.

6. CIDR MASK

Example:

$$\lfloor /20 \rfloor$$

First 16 bits:

TEXT
255.255

Need four more 1s:

TEXT
 $11110000 = 240$

Remaining octet:

TEXT
0

Therefore:

$\boxed{20=255.255.240.0}$

7. EXAMPLE — /22

Need:

$\boxed{22}$

ones.

First two octets:

$\boxed{16}$

ones.

Need:

$\boxed{6}$

more.

TEXT
 $11111100 = 252$

Therefore:

$\boxed{22=255.255.252.0}$

8. PREFIX-TO-MASK SHORTCUT

Remember mask-octet sequence:

TEXT
0
128
192
224

240
248
252
254
255

These correspond to:

TEXT

0, 1, 2, 3, 4, 5, 6, 7, 8

leading network bits in that octet.

Example:

/19

First 16 bits complete.

Need:

$\lfloor 3 \rfloor$

more.

Three leading 1s:

TEXT

11100000 = 224

So:

$\lfloor \boxed{/19=255.255.224.0} \rfloor$

9. FINDING THE BLOCK SIZE

If prefix ends inside an octet:

$\lfloor \boxed{\text{Block size}=256-\text{mask value}} \rfloor$

Example:

$\lfloor /27 \rfloor$

Mask last octet:

$\lfloor 224 \rfloor$

Block size:

$\lfloor 256-224=32 \rfloor$

Subnet boundaries:

TEXT

0, 32, 64, 96, 128, 160, 192, 224

10. EXAMPLE — FIND CIDR BLOCK

Given:

TEXT

192.168.10.77/27

Mask:

TEXT

255.255.255.224

Block size:

[32]

Ranges:

TEXT

0–31
32–63
64–95
96–127
...

77 lies inside:

TEXT

64–95

Therefore:

Network address

[192.168.10.64]

Broadcast address

[192.168.10.95]

Usable range

TEXT

192.168.10.65 – 192.168.10.94

11. CIDR BLOCK MUST START AT A VALID BOUNDARY

Very important GATE trap.

Not every address followed by /n is a valid **network address** for that CIDR block.

For example:

TEXT

192.168.1.70/26

is a valid host address with /26.

But:

TEXT

192.168.1.70/26

is **not** the network address of the block.

Why?

/26 block size:

[64]

Valid boundaries:

TEXT

0

64

128

192

70 lies in:

TEXT

64-127

So the actual block is:

$\boxed{192.168.1.64/26}$

12. VALID NETWORK ADDRESSES FOR /26

Within:

TEXT

192.168.1.x

valid /26 network addresses are:

TEXT

192.168.1.0/26

192.168.1.64/26

192.168.1.128/26

192.168.1.192/26

But:

TEXT

192.168.1.50/26

cannot be the base/network address of a /26 block.

13. GATE RULE FOR VALID CIDR BLOCK

A CIDR block of size:

$[2^h]$

must begin at an address aligned to that block size.

For a block size of 32:

valid starts include:

TEXT

0, 32, 64, 96, ...

For block size 64:

TEXT

0, 64, 128, 192

Memory:

Block must start on its own multiple.

14. CIDR RANGE WHEN PREFIX IS IN THIRD OCTET

Example:

TEXT

172.16.37.100/20

Mask:

TEXT

255.255.240.0

Interesting octet is third:

$[240]$

Block size:

$[256-240=16]$

Third-octet boundaries:

TEXT

0
16
32
48
64
...

37 lies in:

TEXT

32–47

Therefore:

Network

$\boxed{172.16.32.0}$

Broadcast

$\boxed{172.16.47.255}$

Notice:

The entire fourth octet is part of the host portion.

15. WHY BROADCAST IS .47.255

For $/20$:

TEXT

255.255.240.0

Network third-octet block:

TEXT

32–47

Last address in the block is:

TEXT

172.16.47.255

because:

- third octet reaches final value 47
- fourth octet becomes all ones = 255

16. EXAMPLE — $/22$

Find block containing:

TEXT

10.20.14.99/22

Mask:

TEXT

255.255.252.0

Third-octet block size:

$\lfloor 256 - 252 = 4 \rfloor$

Boundaries:

TEXT

0, 4, 8, 12, 16, 20, . . .

14 lies in:

TEXT

12-15

Therefore:

Network

$\lfloor \boxed{10.20.12.0} \rfloor$

Broadcast

$\lfloor \boxed{10.20.15.255} \rfloor$

Total addresses:

$\lfloor 2^{32-22} = 2^{10} = \boxed{1024} \rfloor$

17. FAST METHOD FOR ANY PREFIX

Suppose:

TEXT

A.B.C.D/n

Step 1

Convert $/n$ to subnet mask.

Step 2

Find the first mask octet that is not 255.

That is the **interesting octet**.

Step 3

Calculate:

\[256-M \]

Step 4

Find the block containing the corresponding IP octet.

Step 5

All octets to the right:

- Network address != 0
- Broadcast address != 255

This is extremely useful.

18. EXAMPLE WITH /19

IP:

TEXT

172.30.93.200/19

Mask:

TEXT

255.255.224.0

Interesting octet:

\[224 \]

Block:

\[256-224=32 \]

Third-octet ranges:

TEXT

0-31
32-63
64-95
96-127
...

93 lies in:

TEXT

64-95

So:

Network

\[\boxed{172.30.64.0} \]

Broadcast

\[\boxed{172.30.95.255} \]

19. FIRST AND LAST ADDRESS FROM BLOCK SIZE

If network is:

TEXT

192.168.1.64/27

Number of addresses:

$[32]$

Last address can be calculated as:

$[64+32-1]$

$[=95]$

Therefore broadcast:

TEXT

192.168.1.95

Very fast.

20. HOST REQUIREMENT !' PREFIX

~~0=0%~~ Common GATE numerical.

Suppose you need at least:

$[500]$

usable host addresses.

We need:

$[2^h-2 \geq 500]$

Check powers:

$[2^8-2=254]$

not enough.

$[2^9-2=510]$

enough.

So:

$[h=9]$

Prefix:

$[32-9 = \boxed{23}]$

21. ANOTHER HOST EXAMPLE

Need:

$$\lfloor 1000 \rfloor$$

hosts.

$$\lfloor 1000+2=1002 \rfloor$$

Next power of 2:

$$\lfloor 1024=2^{\{10\}} \rfloor$$

So:

$$\lfloor h=10 \rfloor$$

Prefix:

$$\lfloor 32-10 = \boxed{/22} \rfloor$$

22. GATE SHORTCUT

For ordinary IPv4 host-subnet questions:

$$\lfloor \boxed{2^h \geq H+2} \rfloor$$

Choose smallest $\lfloor h \rfloor$.

Then:

$$\lfloor \boxed{\text{Prefix}=32-h} \rfloor$$

23. SPLITTING ONE CIDR BLOCK INTO SMALLER BLOCKS

Suppose:

TEXT

192.168.10.0/24

is divided into /26 blocks.

Difference:

$$\lfloor 26-24=2 \rfloor$$

Number of equal subnets:

$$\lfloor 2^2=4 \rfloor$$

The four networks:

TEXT

192.168.10.0/26
192.168.10.64/26
192.168.10.128/26
192.168.10.192/26

24. GENERAL SUBNET COUNT

Original prefix:

$\backslash [/p \backslash]$

New prefix:

$\backslash [/q \backslash]$

where:

$\backslash [q > p \backslash]$

Number of equal smaller subnets:

$\backslash [\boxed{2^{q-p}} \backslash]$

25. EXAMPLE

How many $/28$ blocks fit inside $/22$?

$\backslash [28 - 22 = 6 \backslash]$

$\backslash [2^6 = \boxed{64} \backslash]$

26. SIZE CHECK

A $/22$ contains:

$\backslash [2^{10} = 1024 \backslash]$

addresses.

Each $/28$ contains:

$\backslash [2^4 = 16 \backslash]$

Number:

$\backslash [1024 / 16 = 64 \backslash]$

same answer.

27. CIDR AGGREGATION / ROUTE SUMMARIZATION

Very important concept.

CIDR can combine several contiguous networks into one larger prefix.

This is called:

Route aggregation

or:

Route summarization

Instead of routing table containing many entries:

TEXT

```
200.1.0.0/24
200.1.1.0/24
200.1.2.0/24
200.1.3.0/24
```

we may potentially summarize them as:

$\boxed{200.1.0.0/22}$

28. WHY /22 ?

Four /24 networks contain:

$4 \times 256 = 1024$

addresses.

$1024 = 2^{10}$

So host/address bits required:

10

Prefix:

$32 - 10 = 22$

Thus:

/22

29. CHECK CONTIGUITY

The four networks are:

TEXT

```
200.1.0.0
200.1.1.0
200.1.2.0
200.1.3.0
```

These are contiguous.

A /22 beginning at:

TEXT

```
200.1.0.0
```

covers:

TEXT

```
200.1.0.0
through
200.1.3.255
```

Exactly these four networks.

So aggregation is valid.

30. & pAGGREGATION REQUIRES ALIGNMENT

Suppose networks are:

TEXT

```
200.1.1.0/24
200.1.2.0/24
200.1.3.0/24
200.1.4.0/24
```

There are still four networks.

Can we summarize them exactly as one /22 ?

No.

Why?

A /22 must begin on a third-octet boundary that is a multiple of:

[4]

Valid /22 starts:

TEXT

```
0
4
8
12
...
```

Third octet 1 is not aligned.

Very important.

31. AGGREGATION CONDITIONS

To combine blocks exactly:

1. They should be contiguous.
2. Number of equal-sized blocks should normally be a power of 2.
3. Starting address must be correctly aligned.
4. Combined block must not unintentionally include unrelated addresses if exact aggregation is required.

32. > MEMORY

For aggregation:

Power of 2 + Continuous + Correct boundary

Think:

P-C-B

- Power of two
- Contiguous
- Boundary aligned

33. BINARY METHOD FOR AGGREGATION

The most general method:

Find the longest common binary prefix of all addresses/ranges.

Example network addresses:

TEXT

```
192.168.8.0
192.168.9.0
192.168.10.0
192.168.11.0
```

Look at third octet:

TEXT

```
8  = 00001000
9  = 00001001
10 = 00001010
11 = 00001011
```

Common bits:

TEXT

```
000010
```

First six bits of third octet are common.

First two full octets:

$\backslash$ 16 $\backslash$

bits.

Plus:

$\backslash$ 6 $\backslash$

common bits.

Total:

$\backslash$ 22 $\backslash$

So aggregate:

$\backslash$ $\boxed{192.168.8.0/22}$ $\backslash$

34. WHY START ADDRESS IS 8?

A `/22` has third-octet block size:

$\lfloor 4 \rfloor$

Valid starts:

TEXT

0, 4, 8, 12, 16, ...

8 is valid.

It covers:

TEXT

8-11

Exactly the four `/24` s.

35. ANOTHER AGGREGATION EXAMPLE

Combine:

TEXT

10.0.4.0/24

10.0.5.0/24

Two blocks:

$\lfloor 2=2^1 \rfloor$

Combining removes one prefix bit:

$\lfloor /24 \rightarrow /23 \rfloor$

Is 4 aligned for `/23` ?

A `/23` has third-octet block size 2.

Valid starts:

TEXT

0, 2, 4, 6, ...

Yes.

So:

$\lfloor \boxed{10.0.4.0/23} \rfloor$

covers:

TEXT

10.0.4.0 - 10.0.5.255

36. INVALID TWO-BLOCK AGGREGATION

Can these be exactly aggregated into one /23 ?

TEXT

```
10.0.5.0/24
10.0.6.0/24
```

A /23 start must have an even third octet:

TEXT

```
0, 2, 4, 6, ...
```

Starting at 5 is invalid.

The /23 containing 5 actually begins at 4:

TEXT

```
10.0.4.0/23
```

which includes 4 and 5.

The /23 containing 6 begins at:

TEXT

```
10.0.6.0/23
```

which includes 6 and 7.

Therefore 5 and 6 cannot be represented **exactly** by one /23 .

37. WHY ROUTE AGGREGATION IS USEFUL

Without aggregation:

TEXT

```
100 individual routing entries
```

With aggregation:

TEXT

```
one larger prefix
```

Benefits:

- smaller routing tables
- fewer routing advertisements
- simpler routing
- better scalability

This was one major motivation behind CIDR.

38. LONGEST PREFIX MATCHING

EXTREMELY IMPORTANT FOR GATE

Suppose router has:

Prefix	Interface
10.0.0.0/8	A
10.10.0.0/16	B
10.10.5.0/24	C

Destination:

TEXT

10.10.5.20

This destination matches:

- /8
- /16
- /24

Which route is chosen?

$\boxed{/24}$

because it is the:

Longest matching prefix

39. WHAT DOES "LONGEST" MEAN?

Not largest numerical network.

Not physically longest path.

It means:

Greatest number of matching prefix bits.

So:

$\boxed{/24}$

is longer/more specific than:

$\boxed{/16}$

which is more specific than:

$\boxed{/8}$

40. LONGEST PREFIX MATCH

Most specific route wins.

Or:

More prefix bits = more specific.

41. LONGEST PREFIX EXAMPLE

Routing table:

Prefix	Output
0.0.0.0/0	R1
172.16.0.0/12	R2
172.16.32.0/20	R3
172.16.35.0/24	R4

Destination:

TEXT

172.16.35.100

Matches:

- 0.0.0.0/0
- 172.16.0.0/12
- 172.16.32.0/20
- 172.16.35.0/24

Choose:

172.16.35.0/24 → R4

42. DEFAULT ROUTE

Prefix:

0.0.0.0/0

has:

0

prefix bits.

Therefore it matches:

every IPv4 destination.

But because its prefix length is 0, it is the **least specific route**.

It is used only when no longer matching route wins.

This is the:

Default route

43. MEMORY

Default route:

Matches everything, wins only when nothing more specific matches.

44. LONGEST PREFIX GATE EXAMPLE

Routing table:

Prefix	Interface
192.168.0.0/16	1
192.168.32.0/19	2
192.168.40.0/21	3
0.0.0.0/0	4

Destination:

TEXT
192.168.42.5

Let's check.

/16

Definitely matches 192.168.x.x.

/19

192.168.32.0/19

Third-octet block size:

\[32 \]

covers:

TEXT

32-63

42 matches.

/21

Mask:

TEXT

255.255.248.0

Block size:

$\backslash [8 \backslash]$

192.168.40.0/21 covers:

TEXT

40-47

42 matches.

Longest:

$\backslash [/21 \backslash]$

Therefore:

$\backslash [\boxed{\text{Interface 3}} \backslash]$

45. & pGATE TRAP — DON'T STOP AT FIRST MATCH

Routing table may not be ordered from most specific to least specific.

You must evaluate all relevant matches and choose:

$\backslash [\boxed{\text{longest prefix}} \backslash]$

46. BINARY LONGEST PREFIX METHOD

Suppose entry:

TEXT

200.20.16.0/20

Destination:

TEXT

200.20.31.8

/20 mask:

TEXT

255.255.240.0

Third-octet block beginning at 16 covers:

TEXT

16-31

31 is included.

So destination matches.

No need to convert all 32 bits if the block method is easier.

47. FORWARDING TABLE MAY HAVE OVERLAPPING PREFIXES

This is normal.

Example:

TEXT

10.0.0.0/8
10.1.0.0/16
10.1.2.0/24

These blocks overlap hierarchically.

Longest prefix matching makes this work predictably.

A packet for:

TEXT

10.1.2.50

goes via /24.

A packet for:

TEXT

10.1.8.50

doesn't match /24, but matches /16.

A packet for:

TEXT

10.50.8.50

matches only /8.

48. GATE NUMERICAL — FIND WHICH PREFIX MATCHES

Destination:

TEXT

10.4.6.10

Routes:

TEXT

10.0.0.0/8
10.4.0.0/16
10.4.4.0/22
10.4.8.0/21

Check /22:

10.4.4.0/22

Third-octet block size:

\[4 \]

covers:

TEXT

4-7

Destination third octet:

\[6 \]

matches.

10.4.8.0/21 covers third octets:

TEXT

8-15

does not match.

So longest matching prefix:

\[\boxed{10.4.4.0/22} \]

49. TWO CIDR BLOCKS — DO THEY OVERLAP?

Example:

TEXT

192.168.1.0/25

covers:

TEXT

0-127

and:

TEXT

192.168.1.64/26

covers:

TEXT

64-127

Therefore the /26 lies entirely inside the /25.

They overlap.

This is okay in routing tables because longest prefix handles specificity.

50. FIND SMALLEST CIDR BLOCK CONTAINING A RANGE

Suppose addresses from:

TEXT

192.168.8.0

through:

TEXT

192.168.11.255

must be represented.

Total:

$\lfloor 4 \times 256 = 1024 \rfloor$

addresses.

$\lfloor 1024 = 2^{\{10\}} \rfloor$

Prefix:

$\lfloor 32 - 10 = 22 \rfloor$

Start 8 is aligned to block size 4 in third octet.

Therefore:

$\lfloor \boxed{192.168.8.0/22} \rfloor$

51. RANGE SIZE MUST OFTEN BE POWER OF TWO FOR ONE EXACT CIDR BLOCK

One CIDR block always contains:

$\lfloor 2^h \rfloor$

addresses.

So exact block sizes are:

TEXT

1, 2, 4, 8, 16, 32, 64, . . .

If a requested arbitrary range contains, say:

$\backslash [1000 \backslash]$

addresses, one exact CIDR block of exactly 1000 addresses is impossible.

You may need:

- a larger block that includes extra addresses

or

- multiple CIDR blocks

depending on the question.

52. ADDRESS-COUNT QUESTION

How many addresses are represented by:

TEXT

172.20.0.0/18

Host bits:

$\backslash [32-18=14 \backslash]$

Addresses:

$\backslash [2^{\{14\}} \backslash]$

$\backslash [\boxed{16384} \backslash]$

53. FIND LAST ADDRESS OF /18

Mask:

TEXT

255.255.192.0

Third-octet block size:

$\backslash [64 \backslash]$

If network:

TEXT

172.20.0.0/18

covers third octets:

TEXT

0-63

So final address:

$\boxed{172.20.63.255}$

54. ANOTHER /18

Network:

TEXT

172.20.128.0/18

Block size 64.

Starts:

TEXT

0, 64, 128, 192

So range:

TEXT

172.20.128.0
to
172.20.191.255

55. CIDR AND CLASSFUL THINKING

Suppose:

TEXT

172.20.10.0/24

Historically 172.x is in Class B territory.

But explicit CIDR says:

$\boxed{/24}$

So use:

$\boxed{/24}$

not /16.

~~0=CIDR~~ CIDR prefix overrides historical class assumptions.

56. ROUTE AGGREGATION GATE METHOD

Given several prefixes to aggregate:

Method 1 — Binary

Find longest common prefix.

Method 2 — Power/block method

If equal-sized contiguous networks:

1. Count blocks.
2. Check if count = power of 2.
3. Reduce prefix by $\lfloor \log_2 \rfloor$ of block count.
4. Check starting alignment.

57. EXAMPLE

Aggregate eight contiguous /24 s:

TEXT

```
192.168.16.0/24
through
192.168.23.0/24
```

Number:

$\lfloor 8=2^3 \rfloor$

So:

$\lfloor 24-3=21 \rfloor$

Candidate:

$\lfloor 192.168.16.0/21 \rfloor$

Check /21 block size in third octet:

$\lfloor 8 \rfloor$

Valid starts:

TEXT

```
0, 8, 16, 24, . . .
```

16 is aligned.

Therefore:

$\lfloor \boxed{192.168.16.0/21} \rfloor$

58. INVALID AGGREGATION EXAMPLE

Eight /24 s from:

TEXT

```
192.168.18.0
```

through:

TEXT

192.168.25.0

Eight is a power of two.

But candidate /21 needs a multiple-of-8 third-octet boundary.

18 is not.

Therefore these eight networks cannot be exactly represented by a single /21.

Power-of-two count alone is not enough.

59. SUPER IMPORTANT CIDR FORMULAS

$$\boxed{\text{HostBits}=32-n}$$

$$\boxed{\text{Addresses}=2^{32-n}}$$

For ordinary host subnetting:

$$\boxed{\text{Usable hosts}=2^{32-n}-2}$$

If splitting /p into /q:

$$\boxed{\text{\#subnets}=2^{q-p}}$$

For equal block aggregation:

$$\boxed{\text{NewPrefix}=\text{OldPrefix}-\log_2(\text{\#blocks})}$$

provided alignment/contiguity are valid.

60. HOW GATE CAN ASK CIDR

Pattern 1

Find network and broadcast from IP/prefix.

Pattern 2

Convert prefix !” subnet mask.

Pattern 3

Find number of addresses/hosts.

Pattern 4

Find required prefix for a host count.

Pattern 5

Split a larger block into subnets.

Pattern 6

Check whether a proposed network address is valid/aligned.

Pattern 7

Aggregate multiple prefixes.

Pattern 8

Longest prefix matching.

Pattern 9

Find smallest CIDR block covering an address range.

Pattern 10

Determine whether two addresses are in the same subnet.

61. & pGATE TRAPS

Trap 1

/n means:

\[n \]

prefix bits, not host bits.

Trap 2

CIDR block size is always a power of 2.

Trap 3

A network base address must be correctly aligned.

Trap 4

Contiguous networks are not automatically aggregatable into one exact block.

Alignment matters.

Trap 5

Longest Prefix Matching chooses:

most specific matching route

not lowest numerical address.

Trap 6

Default route:

$\backslash [0.0.0.0/0 \backslash]$

matches everything but loses to every more-specific matching route.

Trap 7

Do not blindly subtract 2 when question asks:

total addresses.

Subtract 2 only for ordinary usable-host count where that model applies.

Trap 8

For prefixes such as $/20$, the subnet boundary occurs in the third octet, not necessarily the fourth.

Trap 9

Historical Class A/B/C does not override an explicit CIDR prefix.

62. 0> MASTER MEMORY SYSTEM

For a CIDR question, use:

P-H-A-B-R

P — Prefix

Given $/n$.

H — Host bits

$\backslash [32-n \backslash]$

A — Addresses

$\backslash [2^h \backslash]$

B — Block boundary

Use mask/block size.

R — Range

Find:

- network
- broadcast
- usable addresses

63. LONGEST PREFIX MEMORY

More bits = More specific = More preferred

Example:

TEXT

/8
/16
/24

If all match:

$\boxed{/24 \text{ wins}}$

'p NOTEBOOK VERSION

CIDR

CIDR = Classless Inter-Domain Routing

Notation:

TEXT

IP/prefix

Example:

TEXT

192.168.1.0/24

Core Formulas

For $/n$:

$\text{HostBits} = 32 - n$

$\boxed{\text{TotalAddresses} = 2^{\{32-n\}}}$

Traditional usable hosts:

$\boxed{2^{\{32-n\}} - 2}$

Prefix !' Mask

Example:

$/20$

$\boxed{255.255.240.0}$

Common mask values:

TEXT

128 192 224 240 248 252 254 255

Block Size

$\boxed{\text{Block=256-MaskOctet}}$

Example:

TEXT

/ 2 7 ! ' m a s k 2 2 4 ! ' b l o c k 3 2

Valid Block

Network address must be aligned to block size.

For /26 :

TEXT

0, 64, 128, 192

are valid last-octet boundaries.

Aggregation

Requirements:

- power-of-two number of equal blocks
- contiguous
- correct alignment

Memory:

Power + Continuous + Boundary

Longest Prefix Matching

If several routes match destination:

$\boxed{\text{choose the longest prefix}}$

Example:

TEXT

10.0.0.0/8
10.1.0.0/16
10.1.2.0/24

Destination:

TEXT

10.1.2.5

uses:

$\boxed{/24}$

Default Route

$\boxed{0.0.0.0/0}$

Matches every IPv4 address.

Used when no more-specific route matches.

64. 30-SECOND REVISION

Memorize:

$\boxed{\text{Addresses}=2^{\{32-\text{prefix}\}}}$

$\boxed{\text{BlockSize}=256-\text{MaskOctet}}$

And:

CIDR network must start on a valid block boundary.

Aggregation:

Power of 2 + contiguous + aligned.

Forwarding:

Longest matching prefix wins.

ACTIVE RECALL — DON'T LOOK BACK

Q1

How many total addresses are in:

$/21$

?

Q2

Convert:

$/19$

to dotted-decimal subnet mask.

Q3

Find the network and broadcast addresses for:

TEXT

172.16.46.170/20

Q4 — Alignment Trap

Is:

TEXT

192.168.1.96/26

a valid /26 network address?

If not, what /26 block contains it?

Q5

How many /28 blocks fit inside one /23 block?

Q6 — Host Requirement

You need at least:

[700]

traditional usable host addresses.

What is the smallest suitable CIDR prefix?

Q7 — Aggregation

Can these four networks be aggregated exactly into one prefix?

TEXT

10.10.8.0/24
10.10.9.0/24
10.10.10.0/24
10.10.11.0/24

If yes, give the aggregate.

Q8 — Aggregation Trap

Can these four /24 s be represented exactly by one /22 ?

TEXT

10.10.9.0/24
10.10.10.0/24
10.10.11.0/24
10.10.12.0/24

Why or why not?

Q9 — Longest Prefix

Routing table:

TEXT

0 . 0 . 0 . 0 / 0	!'	A
1 0 . 0 . 0 . 0 / 8	!'	B
1 0 . 2 0 . 0 . 0 / 1 6	!'	C
1 0 . 2 0 . 1 6 . 0 / 2 0	!'	D

Destination:

TEXT

10.20.23.100

Which output is selected?

Q10

What is the role of:

\[0.0.0.0/0]

in an IPv4 forwarding table?

Q11 — Range

Find the complete address range represented by:

TEXT

192.168.64.0/18

Q12 — Future Connection

If an IPv4 datagram is larger than the MTU of the outgoing link, which IPv4 mechanism may divide it into smaller pieces?

NEXT !' IPv4 Fragmentation: MTU, Identification, DF/MF Flags, Fragment Offset, 8-Byte Rule & Full GATE Numericals

USER

next

ASSISTANT

CHAPTER 11 — IPv4

TOPIC 3 — IPv4 Fragmentation: MTU, DF/MF, Identification, Fragment Offset & GATE Numericals

EXTREMELY HIGH PRIORITY FOR GATE NUMERICALS

IPv4 fragmentation looks difficult mainly because students confuse:

- MTU
- IP header
- data size
- fragment total length
- fragment offset
- MF flag

Once you remember one rule, most questions become mechanical:

Fragment Offset is measured in units of 8 bytes

And for every non-final fragment:

Data size should be a multiple of 8 bytes

1. WHY IS IPv4 FRAGMENTATION NEEDED?

Different networks can support different maximum packet sizes.

Suppose an IPv4 datagram has:

4000 bytes

but the outgoing link supports an MTU of only:

1500 bytes

The entire datagram cannot fit.

IPv4 may divide it into smaller pieces called:

fragments

Each fragment becomes its own IPv4 packet.

2. WHAT IS MTU?

MTU stands for:

Maximum Transmission Unit

It is the maximum Network-layer packet size that can be carried in one frame over that link, in this context.

For ordinary Ethernet:

MTU=1500B

So an IPv4 packet placed inside the Ethernet data field normally cannot exceed:

1500B

3. & pVERY IMPORTANT — MTU INCLUDES IP HEADER

Suppose:

$[\text{MTU}=1500\text{B}]$

and IPv4 header:

$[20\text{B}]$

Then maximum IP **data** that can fit in one fragment is not 1500.

It is initially:

$[1500-20 = 1480\text{B}]$

So:

$[\boxed{\text{Fragment} \setminus \text{Total} \setminus \text{Length} = \text{Header} + \text{Fragment} \setminus \text{Data}}]$

4. IPv4 HEADER SIZE

Without options, basic IPv4 header size is:

$[\boxed{20\text{B}}]$

IPv4 header can be larger if options exist, up to 60 bytes, but GATE fragmentation questions commonly specify or assume:

$[20\text{B}]$

unless told otherwise.

& pNever blindly assume 20 if the question explicitly gives another header length.

5. WHAT HAPPENS DURING FRAGMENTATION?

Suppose original packet:

``text id="frag1" +-----+-----+

IPv4 Header	Original Data
-------------	---------------

+-----+-----+

It may become:

``text id="frag2"

Fragment 1:

Header | Data part 1

Fragment 2:

Header | Data part 2

Fragment 3:

Header | Data part 3

Every fragment gets its **own IPv4 header**.

So header overhead increases.

6. IMPORTANT IPv4 FRAGMENTATION FIELDS

You need four key fields:

1. Identification
2. DF — Don't Fragment
3. MF — More Fragments
4. Fragment Offset

These let the destination recognize and reconstruct the original datagram.

7. IDENTIFICATION FIELD

All fragments belonging to the same original IPv4 datagram carry the same:

Identification value

Example:

```
```text id="frag3" Original packet ID = 500
```

Fragment 1 !' ID = 500 Fragment 2 !' ID = 500 Fragment 3 !' ID = 500

This helps the destination know:

> These fragments belong together.

---

```
8 . 0>Yà M E M O R Y
```

```
> * * S a m e o r i g i n a l p a c k e t ! ' S a m e I d e n t i f i c a t i o n * *
```

Do not assign a new Identification number to every fragment in a standard fragmentation question.

---

```
9. DF FLAG
```

DF means:

> **\*\*Don't Fragment\*\***

If:

```
\[
DF=1
\]
```

the packet must **\*\*not\*\*** be fragmented.

If it is too large for the outgoing link's MTU, the router cannot simply fragment it.

In IPv4, it typically drops the datagram and can send an ICMP error indicating fragmentation is needed.

For GATE:

```
\[
\boxed{DF=1\rightarrow fragmentation\ prohibited}
\]
```

---

# 10. DF = 0

If:

```
\[
DF=0
\]
```

fragmentation is permitted.

Provided fragmentation is otherwise necessary and supported.

---

# 11. MF FLAG

MF means:

> \*\*More Fragments\*\*

It tells the receiver whether more pieces follow after this fragment.

### For every fragment except the final one:

```
\[
\boxed{MF=1}
\]
```

### For the final fragment:

```
\[
\boxed{MF=0}
\]
```

---

# 12. EXAMPLE

Three fragments:

Fragment	MF
1	1
2	1
3	0

Memory:

> \*\*More coming? MF = 1.\*\*

> \*\*Last fragment? MF = 0.\*\*

---

# 13. FRAGMENT OFFSET



0=Ý%0=Ý%0=Ý% This is where most mistakes happen .

Fragment Offset tells:

> Where this fragment's **\*\*data\*\*** belongs inside the original datagram's payload.

But IPv4 does **\*\*not\*\*** store the offset directly in bytes.

It stores it in units of:

\[  
\\boxed{8\\text{ bytes}}  
\\]

Therefore:

\[  
\\boxed{  
FragmentOffset  
=  
\\frac{\\text{Starting byte position of fragment data}}{8}  
}  
\\]

---

#### # 14. FIRST FRAGMENT OFFSET

The first fragment contains the beginning of original data.

Starting data byte:

\[  
0  
\\]

Therefore:

\[  
\\boxed{Offset=0}  
\\]

Always.

---

#### # 15. SIMPLE OFFSET EXAMPLE

Suppose first fragment carries:

\[  
1480B  
\\]

of original data.

Then second fragment begins after 1480 bytes.

Its offset:

\[  
\\frac{1480}{8}  
=  
\\boxed{185}  
\\]

If second fragment also carries 1480 bytes, third begins after:

```
\[
1480+1480=2960B
\]
```

So third offset:

```
\[
\frac{2960}{8}
=
\boxed{370}
\]
```

0=Ý%    T h i s   e x a c t   p a t t e r n   a p p e a r s   c o n s t a n t l y .

---

# 16. WHY MUST DATA BE A MULTIPLE OF 8?

Because Fragment Offset can represent only positions in units of:

```
\[
8B
\]
```

Suppose a non-final fragment carried:

```
\[
1001B
\]
```

Then the next fragment would start at byte 1001.

Offset:

```
\[
1001/8
=
125.125
\]
```

But the Fragment Offset field stores an integer.

Impossible.

Therefore, all fragments that are followed by more fragments must normally have data length divisible by 8.

```
\[
\boxed{\text{Non-last fragment data}=multiple\ of\ 8}
\]
```

---

# 17. DOES THE LAST FRAGMENT NEED DATA MULTIPLE OF 8?

'L    N o .

The final fragment can contain whatever data remains.

Example:

```
\[
1480,\ 1480,\ 1020
\]
```

\]

The last:

\[  
1020  
\]

doesn't need to be a multiple of 8 because no later fragment needs to reference a starting position after it.

---

#### # 18. MASTER FORMULA – MAXIMUM FRAGMENT DATA

Given:

- MTU =  $\backslash(M\backslash)$
- IP header =  $\backslash(H\backslash)$

Maximum raw data space:

\[  
M-H  
\]

But for a non-final fragment we need a multiple of 8:

\[  
\boxed{  
Data\_{\max}  
=  
8  
\left\lfloor\frac{M-H}{8}\right\rfloor  
}  
\]

Ø=Ý%    T h i s   i s   t h e   s a f e s t   g e n e r a l   f o r m u l a .

---

#### # 19. COMMON ETHERNET CASE

MTU:

\[  
1500B  
\]

Header:

\[  
20B  
\]

Available data:

\[  
1500-20=1480B  
\]

And:

\[

1480/8=185

\]

exact integer.

Therefore maximum non-final fragment payload:

\[

\boxed{1480B}

\]

---

# 20. FULL GATE NUMERICAL 1

Original IPv4 datagram total length:

\[

4000B

\]

Header:

\[

20B

\]

Outgoing MTU:

\[

1500B

\]

Find all fragments.

---

# 21. STEP 1 — ORIGINAL DATA SIZE

Original total length includes header.

So original data:

\[

4000-20

\]

\[

=\boxed{3980B}

\]

& p    F i r s t   m a j o r   t r a p :

Do not fragment all 4000 bytes as payload.

---

# 22. STEP 2 — MAX DATA PER FULL FRAGMENT

MTU:

\[

1500

\]

Header:

\[  
20  
\]

So:

\[  
 $1500 - 20 = 1480B$   
\]

1480 is divisible by 8.

Thus first fragments can carry:

\[  
\boxed{1480B}  
\]

each.

---

#### # 23. STEP 3 – SPLIT DATA

Original data:

\[  
3980  
\]

First fragment data:

\[  
1480  
\]

Remaining:

\[  
 $3980 - 1480 = 2500$   
\]

Second:

\[  
1480  
\]

Remaining:

\[  
 $2500 - 1480$   
 $=$   
1020  
\]

So:

```
```text id="frag4"  
F1 data = 1480 B  
F2 data = 1480 B  
F3 data = 1020 B
```

24. STEP 4 — TOTAL LENGTH OF EACH FRAGMENT

Remember each fragment gets a 20-byte header.

Fragment 1

$$\lfloor 1480+20 = \boxed{1500\text{B}} \rfloor$$

Fragment 2

$$\lfloor 1480+20 = \boxed{1500\text{B}} \rfloor$$

Fragment 3

$$\lfloor 1020+20 = \boxed{1040\text{B}} \rfloor$$

25. STEP 5 — OFFSETS

Fragment 1

Starts at byte:

$$\lfloor 0 \rfloor$$

$$\lfloor \text{Offset}=0 \rfloor$$

Fragment 2

Starts after:

$$\lfloor 1480\text{B} \rfloor$$

$$\lfloor \text{Offset}=\frac{1480}{8} = \boxed{185} \rfloor$$

Fragment 3

Starts after:

$$\lfloor 1480+1480=2960\text{B} \rfloor$$

$$\lfloor \text{Offset}=\frac{2960}{8} = \boxed{370} \rfloor$$

26. STEP 6 — MF FLAGS

Fragment 1:

$$\lfloor \text{MF}=1 \rfloor$$

Fragment 2:

$$\lfloor \text{MF}=1 \rfloor$$

Fragment 3 is last:

$$\lfloor \text{MF}=0 \rfloor$$

27. FINAL ANSWER TABLE

Fragment	Data	Total Length	Offset	MF
1	1480 B	1500 B	0	1
2	1480 B	1500 B	185	1
3	1020 B	1040 B	370	0

0=1 is the standard GATE fragmentation pattern.

28. GATE SOLUTION TEMPLATE

For every fragmentation numerical write:

GIVEN

- Original total length
- Header length
- MTU

FIND

- fragment sizes
- offsets
- MF

SOLUTION

1. Original data = Total " Header
2. Max fragment data = floor-to-multiple-of-8(MTU " Header)
3. Divide original data
4. Add header to each fragment
5. Offset = previous data bytes / 8
6. MF = 1 except last

This prevents almost every mistake.

29. FULL NUMERICAL 2 — MTU NOT FRIENDLY TO 8

Original datagram:

[2400B]

IPv4 header:

[20B]

MTU:

[620B]

Find fragmentation.

30. ORIGINAL DATA

$\lfloor 2400 - 20 = \boxed{2380} \rfloor$

31. RAW AVAILABLE DATA

$\lfloor 620 - 20 = 600 \rfloor$

Is 600 divisible by 8?

$\lfloor 600 / 8 = 75 \rfloor$

Yes.

So maximum fragment data:

$\lfloor 600 \rfloor$

32. SPLIT

$\lfloor 2380 = 600 + 600 + 600 + 580 \rfloor$

Thus 4 fragments.

33. TOTAL LENGTHS

First three:

$\lfloor 600 + 20 = 620 \rfloor$

Last:

$\lfloor 580 + 20 = 600 \rfloor$

34. OFFSETS

F1

$\lfloor 0 \rfloor$

F2

$\lfloor 600 / 8 = 75 \rfloor$

F3

Starts at 1200:

$\lfloor 1200 / 8 = 150 \rfloor$

F4

Starts at 1800:

$\lfloor 1800 / 8 = 225 \rfloor$

35. FINAL TABLE

Fragment	Data	Total	Offset	MF
F1	600	620	0	1
F2	600	620	75	1
F3	600	620	150	1
F4	580	600	225	0

36. NOW A MORE IMPORTANT CASE

Suppose:

$\backslash$ MTU=500B $\backslash$

Header:

$\backslash$ 20B $\backslash$

Raw available:

$\backslash$ 480B $\backslash$

480 is divisible by 8:

$\backslash$ $480/8=60$ $\backslash$

Easy.

But what if:

$\backslash$ MTU=500B $\backslash$

and header:

$\backslash$ 24B $\backslash$

Then raw space:

$\backslash$ $500-24=476$ $\backslash$

476 is not divisible by 8.

37. FLOOR TO MULTIPLE OF 8

Largest multiple of 8 not exceeding 476:

$\backslash$ 472 $\backslash$

because:

$\backslash$ $472/8=59$ $\backslash$

Therefore maximum non-final fragment data:

$\backslash$ $\boxed{472\text{B}}$ $\backslash$

Not:

$\lfloor 476/8 \rfloor$

Important GATE trap.

38. SHORTCUT FOR MULTIPLE OF 8

Calculate:

$\lfloor \text{MTU} - H \rfloor$

Then divide by 8 and take integer floor.

Example:

$\lfloor 476/8 = 59.5 \rfloor$

Take:

$\lfloor 59 \rfloor$

Then:

$\lfloor 59 \times 8 = 472 \rfloor$

Done.

39. FRAGMENT OFFSET IS ABOUT DATA, NOT HEADER

Suppose first fragment:

- Header = 20 B
- Data = 1480 B
- Total = 1500 B

Second offset is:

$\lfloor 1480/8 = 185 \rfloor$

not:

$\lfloor 1500/8 \rfloor$

Very common mistake.

Headers of fragments are **not part of the original payload position**.

40. GATE TRAP

Wrong:

$\lfloor \text{Offset}_2 = \frac{\text{Fragment1TotalLength}}{8} \rfloor$

Correct:

$\lfloor \boxed{\text{Offset}_2 = \frac{\text{Fragment1DataLength}}{8}} \rfloor$

41. OFFSET OF FRAGMENT (i)

General rule:

$$\boxed{\text{Offset}_i = \frac{\text{Total original data preceding fragment } i}{8}}$$

Not:

- number of previous fragments
- total bytes including headers
- current fragment size

42. EXAMPLE

Fragment payloads:

```text id="frag5" F1 = 800 F2 = 800 F3 = 800 F4 = 300

Offsets:

### F1

$$\boxed{0}$$

### F2

$$\boxed{800/8=100}$$

### F3

$$\boxed{1600/8=200}$$

### F4

$$\boxed{2400/8=300}$$

So:

```text id="frag6"  
0, 100, 200, 300

43. FIND DATARANGE FROM OFFSET

GATE can reverse the question.

Suppose a fragment has:

$$\boxed{\text{Offset}=200}$$

That means its data begins at original payload byte:

$$\boxed{200 \times 8}$$

$$\boxed{1600}$$

If fragment data length is 800 B, it covers payload bytes conceptually from offset 1600 through 2399 if zero-indexed.

44. OFFSET IS 13 BITS

IPv4 Fragment Offset field has:

$\boxed{13 \text{ bits}}$

Maximum stored value:

$2^{13} - 1 = 8191$

Since each unit = 8 bytes:

$8191 \times 8 = 65528$

This aligns with IPv4's maximum datagram-size range.

You usually don't need deep field-width calculations, but it's useful to know why offset uses 8-byte units.

45. IPv4 TOTAL LENGTH FIELD

IPv4 Total Length field is:

16 bits

So maximum total IPv4 datagram size is:

$2^{16} - 1$

$\boxed{65535 \text{ B}}$

including:

- IPv4 header
- IPv4 data

Useful fact.

46. WHO PERFORMS FRAGMENTATION IN IPv4?

In IPv4, fragmentation may be performed by:

- the sending host
- an intermediate router

if required and if:

$DF=0$

This differs from IPv6, where routers do not fragment packets.

For your syllabus:

Focus on IPv4.

47. WHO REASSEMBLES FRAGMENTS?

It's important.

IPv4 fragments are normally reassembled:

Only at the final destination

Intermediate routers do not normally reassemble them just to fragment again later.

Why?

Reassembling at every router would:

- require buffering
- increase delay
- waste processing
- complicate forwarding

So:

[Reassembly occurs at destination]

48. WHAT IF A FRAGMENT ENCOUNTERS AN EVEN SMALLER MTU?

An IPv4 fragment can itself be fragmented again if:

- DF permits it
- later link MTU is smaller

The resulting pieces still relate to the same original datagram.

Their offsets continue to refer to positions in the:

original IPv4 payload.

This is important conceptually.

49. SECOND-LEVEL FRAGMENTATIONEXAMPLE

Suppose original fragment contains data starting at original offset:

[Offset=100]

Since offsets use 8-byte units, starting byte is:

[$100 \times 8 = 800$]

If this fragment is split into:

- 400 B data
- remaining data

then first new piece starts at:

[800]

offset:

\[100 \]

Second starts:

\[800+400=1200 \]

offset:

\[1200/8 = 150 \]

Offsets remain relative to the original datagram payload, not relative to the fragment being split.

50. IDENTIFICATION DURING REFRAGMENTATION

Fragments originating from the same original datagram retain the same Identification value for reassembly purposes in the conventional model.

So destination can associate them.

51. MF DURING REFRAGMENTATION— CONCEPT

Suppose a non-final fragment with:

\[MF=1 \]

is fragmented again.

Its resulting subfragments are not the end of the original datagram, so their MF information must reflect that more original data still exists.

For ordinary GATE questions, focus on the simpler first-level fragmentation case unless re-fragmentation is explicitly asked.

52. WHAT IF ONE FRAGMENT IS LOST?

Suppose original packet becomes:

```text id="frag7" F1 F2 F3

and F2 is lost.

The destination cannot reconstruct the complete original IPv4 datagram.

IPv4 itself does not provide end-to-end retransmission of the missing fragment.

If TCP is being used above IP:

> TCP recovery mechanisms may eventually cause data to be retransmitted.

---

# 53. FRAGMENTATION IS EXPENSIVE

Fragmentation creates disadvantages:

- extra headers
- more packets
- processing overhead

- if one fragment is lost, original packet reassembly fails
- more work for destination
- possible performance reduction

Therefore modern networking often tries to avoid unnecessary fragmentation.

---

#### # 54. PATH MTU IDEA – SMALL PREVIEW

Rather than repeatedly fragmenting packets, systems can try to determine the smallest MTU along a path and send packets accordingly.

This idea is related to:

> Path MTU Discovery.

Not a central syllabus requirement here, but it explains why DF is useful.

---

#### # 55. FULL GATE NUMERICAL 3

Original IPv4 datagram:

```
\[
5000B
\]
```

Header:

```
\[
20B
\]
```

MTU:

```
\[
1500B
\]
```

Find:

- number of fragments
- total length
- offset
- MF

---

#### # 56. ORIGINAL DATA

```
\[
5000-20
=
4980B
\]
```

---

#### # 57. MAX DATA PER FULL FRAGMENT

```
\[
1500-20
=
1480B
```

```

\]

58. SPLIT DATA

\[
4980
\]

First:

\[
1480
\]

remaining:

\[
3500
\]

Second:

\[
1480
\]

remaining:

\[
2020
\]

Third:

\[
1480
\]

remaining:

\[
540
\]

So:

```text id="frag8"
1480
1480
1480
540

```

59. TOTAL LENGTHS

First three:

\[1500B \]

Last:

\[540+20 = 560B \]

60. OFFSETS

F1:

$\lfloor 0 \rfloor$

F2:

$\lfloor 1480/8=185 \rfloor$

F3:

$\lfloor 2960/8=370 \rfloor$

F4:

$\lfloor 4440/8=555 \rfloor$

61. MF

```text id="frag9" F1 !' 1 F2 !' 1 F3 !' 1 F4 !' 0

---

# 62. FINAL TABLE

| Fragment | Data | Total Length | Offset | MF  |
|----------|------|--------------|--------|-----|
| ---      | ---  | ---          | ---    | --- |
| F1       | 1480 | 1500         | 0      | 1   |
| F2       | 1480 | 1500         | 185    | 1   |
| F3       | 1480 | 1500         | 370    | 1   |
| F4       | 540  | 560          | 555    | 0   |

---

# 63. NUMBER OF FRAGMENTS FORMULA

Let original data size be:

$\lfloor D \rfloor$

Maximum full-fragment data:

$\lfloor F \rfloor$

Then number of fragments:

$\lfloor \boxed{N = \left\lceil \frac{D}{F} \right\rceil} \rfloor$

provided  $\lfloor F \rfloor$  is the properly 8-byte-aligned non-final payload capacity.

Example:

```
\[
D=4980
\]
```

```
\[
F=1480
\]
```

```
\[
N=\lceil 4980/1480 \rceil
\]
```

```
\[
= \lceil 3.364... \rceil
\]
```

```
\[
= \boxed{4}
\]
```

---

# 64. GATE QUESTION – GIVEN PAYLOAD, NOT TOTAL LENGTH

& p   c a r e f u l l y   r e a d   w o r d i n g .

If question says:

> "IP datagram has \*\*4000 bytes of data\*\* and 20-byte header"

then total original IP size is:

```
\[
4020B
\]
```

Do not subtract 20 from 4000.

But if question says:

> "IP datagram total length is 4000 bytes"

then payload is:

```
\[
3980B
\]
```

Ø=Ý%   w o r d i n g   m a t t e r s   e n o r m o u s l y .

---

# 65. EXAMPLE

Question A:

> Datagram size = 4000 B, including 20 B header.

Payload:

```
\[
3980B
```

\]

Question B:

> Datagram carries 4000 B of data, header = 20 B.

Payload:

\[  
4000B  
\]

Total:

\[  
4020B  
\]

These produce different fragment sizes/counts.

---

# 66. GATE TRAP – EACH FRAGMENT GETS HEADER

Suppose original packet:

\[  
4000B  
\]

becomes three fragments.

Do not simply split:

\[  
1500+1500+1000  
\]

because each fragment must include its own header.

You need to split the **\*\*payload\*\*** while reserving header space in every MTU-sized fragment.

---

# 67. HEADER OVERHEAD AFTER FRAGMENTATION

Our 4000 B example produced:

- 3 fragments
- each with 20 B header

Total transmitted IPv4 bytes:

\[  
3980\text{ original data}  
+  
3(20)\text{ headers}  
\]

\[  
=4040B  
\]

Original packet was:

\[

4000B

\]

So fragmentation introduced:

\[

4040-4000

=

\boxed{40B}

\]

extra IP-header overhead.

Why 40 rather than 60?

Original packet already had one 20-byte header.

Fragmentation added two additional headers beyond that original one.

---

#### # 68. GENERAL EXTRA HEADER OVERHEAD

If one original packet with header size  $H$  becomes  $N$  fragments:

Total fragment header bytes:

\[

NH

\]

Original header bytes:

\[

H

\]

Additional overhead due to fragmentation:

\[

\boxed{(N-1)H}

\]

Nice GATE shortcut.

---

#### # 69. EXAMPLE

Header:

\[

20B

\]

Fragments:

\[

5

\]

Additional header overhead:

\[

$(5-1)20$

\]

```
\[
=\boxed{80B}
\]
```

---

#### # 70. WHICH FIELDS STAY SAME?

Across fragments of one original datagram, important shared information includes:

- source IP
- destination IP
- Identification
- protocol indication

But fields such as:

- Total Length
- Fragment Offset
- MF
- header checksum

can differ between fragments.

---

#### # 71. WHY HEADER CHECKSUM DIFFERS

IPv4 header checksum covers the:

> IPv4 header.

Since fields such as:

- fragment offset
- flags
- total length

change across fragments, each fragment has its own header checksum.

This also connects back to our checksum topic.

---

#### # 72. ETHERNET CONNECTION

Recall:

```
\[
Ethernet\ MTU=1500B
\]
```

Suppose IPv4 total datagram:

```
\[
1500B
\]
```

It fits exactly.

Suppose:

```
\[
1501B
\]
```

It no longer fits in that ordinary MTU.

If:

```
\[
DF=0
\]
```

fragmentation may be needed.

If:

```
\[
DF=1
\]
```

router cannot fragment it.

---

# 73. EXACTLY EQUAL TO MTU

If:

```
\[
DatagramTotalLength=MTU
\]
```

No fragmentation.

For example:

```
\[
1500B
\]
```

packet over MTU:

```
\[
1500B
\]
```

fits.

Do not fragment unnecessarily.

---

# 74. SMALL DATAGRAM

If:

```
\[
DatagramTotalLength<MTU
\]
```

No fragmentation.

Even if the payload itself is not a multiple of 8.

The 8-byte condition matters for non-last fragments when fragmentation actually occurs.

---

# 75. ORIGINAL UNFRAGMENTED PACKET FLAGS/OFFSET

For an ordinary unfragmented datagram:

```
\[
Offset=0
\]
```

and:

```
\[
MF=0
\]
```

because there are no more fragments.

DF may be either 0 or 1 depending on sender choice.

---

#### # 76. GATE QUESTION – IDENTIFY LAST FRAGMENT

Given fragments:

| Offset | MF |
|--------|----|
| 0      | 1  |
| 185    | 1  |
| 370    | 0  |

Last fragment:

```
\[
\boxed{Offset=370}
\]
```

because:

```
\[
MF=0
\]
```

---

#### # 77. CAN OFFSET ALONE IDENTIFY THE LAST FRAGMENT?

No.

A large offset means fragment belongs later in the data.

But:

> MF tells whether more fragments follow.

So don't assume highest offset automatically based on an incomplete set is definitely last unless context establishes all fragments.

---

#### # 78. REASSEMBLY ORDER

Destination uses offsets to place fragment data correctly.

For example:

```
```text id="frag10"
Offset 0      !'  bytes  starting  at  0
```

```
Offset 185 !' bytes starting at 1480
Offset 370 !' bytes starting at 2960
```

Thus fragments can even arrive out of order.

The offset allows correct placement.

79. WHY IDENTIFICATION + OFFSET BOTH?

Identification answers:

Which original datagram?

Offset answers:

Where inside that datagram?

MF answers:

Is this the end?

Excellent memory trio:

```text id="frag11" ID !' Which packet? Offset !' Where? MF !' More?

---

```
80 . 0>Yà MASTER MEMORY TRICK
```

For fragmentation remember:

```
I-D-M-O-8
```

```
I – Identification
Same for all fragments.
```

```
D – DF
1 means don't fragment.
```

```
M – MF
1 except last.
```

```
O – Offset
Position in original data.
```

```
8 – Offset unit
8 bytes.
```

---

```
81. HOW GATE CAN ASK THIS TOPIC
```

```
Pattern 1 – Number of fragments
```

Given:



```

- packet size
- MTU
- header.

Pattern 2 – Fragment total lengths

Remember each gets header.

Pattern 3 – Fragment offsets

\[
\boxed{\text{preceding original payload bytes}/8}
\]

Pattern 4 – MF values

All except last:

\[
1
\]

Last:

\[
0
\]

Pattern 5 – DF behavior

If DF=1 and MTU too small:

> packet cannot be fragmented.

Pattern 6 – Reassembly location

Answer:

> Destination host.

Pattern 7 – 8-byte alignment

Determine valid fragment payload.

Pattern 8 – Refragmentation

Offsets remain relative to original datagram data.

Pattern 9 – Additional overhead

```

```

\[
(N-1)H
\]

8 2 . & p G A T E T R A P S - M A S T E R L I S T

Trap 1

MTU includes IP header.

Trap 2

Original total length includes original header.

Trap 3

Every fragment gets its own IP header.

Trap 4

Offset is measured in:

\[
\boxed{8B\ units}
\]

not bytes.

Trap 5

Calculate offset using preceding **payload data**, not fragment total length.

Trap 6

Non-final fragment data must be divisible by 8.

Trap 7

Last fragment's data need not be divisible by 8.

Trap 8

All fragments have same Identification value for the original datagram.

Trap 9

MF:

```

```
```text id="frag12"
1 !'   m o r e   f r a g m e n t s
0 !'   l a s t / n o   m o r e
```

Trap 10

DF:

```
```text id="frag13" 1 !' do not fragment 0 !' fragmentation allowed
```

```

Trap 11

Reassembly happens at destination, not every router.

Trap 12

If the question says "4000 B payload," don't subtract the header from 4000.

' p N O T E B O O K V E R S I O N

IPv4 Fragmentation

Occurs when IPv4 datagram is larger than outgoing link:

\[
\boxed{MTU}
\]

and fragmentation is allowed.

Core Fields

Identification

Same value for all fragments of one original datagram.

DF

\[
DF=1\rightarrow \text{Don't\ Fragment}
\]

\[
DF=0\rightarrow \text{Fragmentation\ allowed}
\]

MF

\[
MF=1\rightarrow \text{More\ fragments}
\]

\[
```

```

MF=0\rightarrow Last\ fragment
\]

Fragment Offset

\[
\boxed{
Offset=
\frac{\text{Original data bytes before fragment}}{8}
}
\]

Offset unit:

\[
\boxed{8B}
\]

Maximum Fragment Data

Given MTU (M) and header (H) :

\[
\boxed{
F=
8\left\lfloor\frac{M-H}{8}\right\rfloor
}
\]

for non-final fragments.

Fragment Total Length

\[
\boxed{\text{Header}+\text{FragmentData}}
\]

Number of Fragments

If original payload (D) :

\[
\boxed{
N=\left\lceil\frac{D}{F}\right\rceil
}
\]

8-Byte Rule

All non-final fragment payload sizes:

\[
\boxed{\text{multiple of 8}}
\]

Last fragment:

> may be any remaining size.

```

---

## ## Reassembly

Occurs at:

\[  
\boxed{\text{Final destination}}  
\]

---

## ## Additional Header Overhead

For  $(N)$  fragments, header  $(H)$ :

\[  
\boxed{(N-1)H}  
\]

extra bytes compared with the original datagram.

---

## # 83. MASTER SOLUTION METHOD

Whenever you see an IPv4 fragmentation numerical:

### ### Step 1

Determine whether given size is:

- total datagram size
- or
- payload size.

### ### Step 2

Find original data:

\[  
Total-Header  
\]

if needed.

### ### Step 3

Calculate:

\[  
MTU-Header  
\]

### ### Step 4

Round **down** to nearest multiple of 8 for non-final fragment data.

### ### Step 5

Split payload.

### ### Step 6

For each fragment:

```
\[
TotalLength=Data+Header
\]
```

### Step 7

Offsets:

```
\[
0,\quad
\frac{Data_1}{8},\quad
\frac{Data_1+Data_2}{8},\dots
\]
```

### Step 8

MF:

> 1, 1, ..., 0.

---

# 84. 30-SECOND REVISION

Remember just this:

```
\[
\boxed{FragmentData\le MTU-Header}
\]
```

and non-last data must be:

```
\[
\boxed{\text{multiple of 8}}
\]
```

Then:

```
\[
\boxed{
Offset=
\frac{\text{previous payload bytes}}{8}
}
\]
```

Finally:

```
```text id="frag14"
M F = 1 !' more coming
M F = 0 !' last
D F = 1 !' don't fragment
```

ACTIVE RECALL — DON'T LOOK BACK

Q1

An IPv4 datagram has total length:

\[3000B \]

and a 20-byte header.

How many bytes of original payload does it contain?

Q2

Outgoing MTU:

\[1500B \]

IPv4 header:

\[20B \]

What is the maximum data payload of a non-final fragment?

Q3

Why must non-final fragment data normally be a multiple of 8 bytes?

Q4

A fragment has:

\[Offset=250 \]

At what byte position in the original IP payload does its data begin?

Q5

What are the MF values for a datagram split into four fragments?

Q6

What does:

\[DF=1 \]

mean?

Q7 — GATE Numerical

Original IPv4 datagram total length:

\[4000B \]

Header:

\[20B \]

MTU:

\[1500B \]

Find for every fragment:

1. payload size
 2. total length
 3. offset
 4. MF
-

Q8 — GATE Trap

First fragment has:

- total length = 1500 B
- header = 20 B

What is the second fragment's offset?

A. $\lfloor (1500/8) \rfloor$ B. $\lfloor (1480/8) \rfloor$ C. $\lfloor (1500/20) \rfloor$ D. $\lfloor (1480/20) \rfloor$

Q9

If:

$\lfloor \text{MTU} = 500\text{B} \rfloor$

and IPv4 header:

$\lfloor 24\text{B} \rfloor$

what is the largest valid payload size for a **non-final** fragment?

Q10

Where is an IPv4 fragmented datagram normally reassembled?

A. Every router B. First router C. Destination host D. Ethernet switch

Q11

All fragments of one original IPv4 datagram normally share which important field used for reassembly association?

Q12 — Overhead

A 20-byte-header IPv4 datagram becomes 6 fragments.

How many **additional** IP-header bytes are introduced by fragmentation compared with the original single datagram?

Q13 — Concept

Why can fragmentation increase the impact of packet loss?

NEXT !' Network Address Translation (NAT): Private/Public IPs, NAT Table, Port Address Translation (PAT), Packet Header Changes & GATE Questions

USER

next

CHAPTER 11 — IPv4

TOPIC 4 — Network Address Translation (NAT): Private/Public IPs, NAT Table, PAT, Header Changes & GATE Questions

Ø=Ý% HIGH PRIORITY CONCEPTUAL TOPIC

NAT becomes easy if you remember one idea:

NAT changes IP addressing information as packets move between a private network and the public Internet.

A very common form, **PAT/NAPT**, also changes:

Transport-layer port numbers

This lets many private hosts share one public IPv4 address.

1. WHY DO WE NEED NAT?

IPv4 has only:

$[2^{32}]$

possible addresses.

That is roughly:

$[4.29 \times 10^9]$

addresses.

But the world has:

- phones
- laptops
- servers
- TVs
- IoT devices
- routers
- tablets

So globally unique IPv4 addresses are limited.

NAT helps conserve public IPv4 addresses.

2. PRIVATE IPv4 ADDRESSES

We already learned the three important private ranges:

$[\boxed{10.0.0.0/8}]$

$\boxed{172.16.0.0/12}$

$\boxed{192.168.0.0/16}$

These addresses are commonly used inside private networks.

Example:

TEXT

192.168.1.10
192.168.1.11
192.168.1.12

Different homes can reuse these same private addresses.

3. PRIVATE ADDRESSES ARE NOT GLOBALLY UNIQUE

Your laptop may be:

TEXT

192.168.1.10

Another person's laptop somewhere else may also be:

TEXT

192.168.1.10

That's okay because these addresses exist in separate private networks.

But they cannot normally be routed directly across the public Internet as globally unique destinations.

So when communicating with the Internet, translation is needed.

4. SIMPLE NAT TOPOLOGY

Suppose:

TEXT

```
Private LAN                                Internet
PC A                                     Web Server
192.168.1.10 ----\
                        \
PC B                                     NAT Router ----- 8.8.8.8
192.168.1.20 ----/      Public IP:
                        203.0.113.5
```

Inside:

TEXT

192.168.1.10
192.168.1.20

Outside, the NAT router may use:

TEXT

203.0.113.5

as a public-facing address.

5. WHAT DOES BASIC NAT DO?

Suppose PC A sends:

TEXT

Source IP = 192.168.1.10
Destination IP = 8.8.8.8

When packet crosses NAT router, source may become:

TEXT

Source IP = 203.0.113.5
Destination IP = 8.8.8.8

So outgoing translation is conceptually:

$\boxed{\text{Private Source IP}} \rightarrow \text{Public Source IP}$

6. RETURN PACKET

Server replies to:

TEXT

Destination IP = 203.0.113.5

The NAT router consults its translation state and determines:

This traffic belongs to the internal host 192.168.1.10.

It changes destination:

TEXT

203.0.113.5

to:

TEXT

192.168.1.10

and forwards it inside.

7. BASIC MENTAL MODEL

Outgoing packet:

Private != Public

Incoming reply:

Public != Correct private host

8. PROBLEM WITH ONLY ONE PUBLIC ADDRESS

Suppose two hosts communicate simultaneously:

TEXT

192.168.1.10
192.168.1.20

Both use the same NAT public IP:

TEXT

203.0.113.5

How does the router know which reply belongs to which internal machine?

This is where:

Port Address Translation

becomes important.

9. PAT/ NAPT

PAT means:

Port Address Translation

You may also see:

NAPT — Network Address and Port Translation

Many consumer routers use this idea.

It maps:

TEXT

Private IP : Private Port

to:

TEXT

Public IP : Translated Port

10. EXAMPLE

Host A creates a TCP connection:

TEXT

192.168.1.10:5000

Host B creates another:

TEXT

192.168.1.20:5000

Notice both may even use the same source port internally.

NAT can translate:

TEXT

192.168.1.10:5000
! "
203.0.113.5:40001

and:

TEXT

192.168.1.20:5000
! "
203.0.113.5:40002

Now the outside world sees two different transport endpoints.

11. NAT TRANSLATION TABLE

The router maintains mappings such as:

Inside Private Endpoint	Outside/Public Endpoint
192.168.1.10:5000	203.0.113.5:40001
192.168.1.20:5000	203.0.113.5:40002

When a reply arrives for:

TEXT

203.0.113.5:40001

router knows:

TEXT

```
!' 192.168.1.10:5000
```

When reply arrives for:

TEXT

```
203.0.113.5:40002
```

router knows:

TEXT

```
!' 192.168.1.20:5000
```

0-10 is the essential NAT/PAT mechanism.

12. MEMORY TRICK

PAT:

One public IP, many ports, many private hosts.

Or:

Same house address, different room numbers.

Public IP = building address. Port = room number.

13. WHY PORTS HELP

A transport endpoint is typically identified using:

`[ \boxed{IP\ Address + Port\ Number} \]`

For TCP, a connection is more fully identified by values involving:

- source IP
- source port
- destination IP
- destination port

NAT can use these fields to distinguish simultaneous flows.

14. FULL OUTGOING EXAMPLE

Internal host:

TEXT

```
IP   = 192.168.1.10
Port = 51000
```

Web server:

TEXT

IP = 93.184.216.34
Port = 80

Before NAT:

TEXT

Source = 192.168.1.10:51000
Destination = 93.184.216.34:80

NAT translates source endpoint:

TEXT

Source = 203.0.113.5:62000
Destination = 93.184.216.34:80

15. RETURN TRAFFIC

Server replies:

TEXT

Source = 93.184.216.34:80
Destination = 203.0.113.5:62000

NAT table says:

TEXT

203.0.113.5:62000
!"
192.168.1.10:51000

So inside packet becomes:

TEXT

Source = 93.184.216.34:80
Destination = 192.168.1.10:51000

16. WHAT FIELDS CHANGE?

For ordinary outbound PAT:

IP layer

Source IP changes:

TEXT

P r i v a t e ! ' P u b l i c

Transport layer

Source port may change.

So:

$\boxed{\text{IP address and port may both be translated}}$

17. RETURN DIRECTION

For the returning packet:

Destination IP

Public NAT address becomes private host IP.

Destination port

Translated external port becomes original internal port.

18. CHECKSUMS ALSO NEED ATTENTION

Useful GATE-level concept.

IPv4 header contains a header checksum.

If NAT modifies the IPv4 header, for example source or destination IP:

IPv4 header checksum must be updated/recomputed appropriately.

TCP and UDP checksums also include IP addressing information via a pseudo-header.

So if NAT changes:

- IP addresses
- ports

then TCP/UDP checksum values may also need adjustment.

19. DO NOT SAY NAT CHANGES ONLY IP ADDRESS

With simple one-to-one NAT, perhaps mainly IP translation occurs.

With PAT/NAPT:

Both IP address and transport port can change.

This distinction matters.

20. STATIC NAT

One conceptual NAT form is:

Static NAT

A private address is permanently mapped to a particular public address.

Example:

```
TEXT
10.0.0.10
!
203.0.113.10
```

This is a fixed one-to-one mapping.

21. DYNAMIC NAT

Another form:

Dynamic NAT

A private host can be mapped to an available public address from a pool.

Example public pool:

```
TEXT
203.0.113.10
203.0.113.11
203.0.113.12
```

A host obtains one temporarily as needed.

22. PAT / NAT OVERLOAD

Most home/office NAT systems commonly use:

Many-to-one translation

Many private hosts share one public IP, distinguished by ports.

This is often called:

- PAT
- NAPT
- NAT overload

For GATE, understand the mechanism rather than memorizing vendor terminology.

23. COMPARISON

Type	Mapping idea
Static NAT	One private IP !" one fixed public IP
Dynamic NAT	Private IP !" available public IP from pool
PAT/NAPT	Many private endpoints !" one/few public IPs using ports

24. WHY PAT SAVES MORE ADDRESSES

Suppose 100 devices are inside a network.

Without sharing, you might need many public addresses.

With PAT:

TEXT

```
100 private devices
      !"
1 public IPv4 address
  +
many translated ports
```

So public-address consumption is drastically reduced.

25. HOW MANY PORT VALUES?

A TCP/UDP port field is:

$[16\text{ bits}]$

So theoretical port-number range:

$[0\text{ to }65535]$

In practice, not every port is available for arbitrary NAT mapping, but conceptually PAT gains a large translation space from port numbers.

26. & pGATE TRAP — PORTS ARE TRANSPORT LAYER

IP address belongs to:

Network Layer

Port number belongs to:

Transport Layer

PAT deliberately reaches across this boundary and modifies both kinds of information.

27. PRIVATE vs PUBLIC ADDRESS

Private address

Used internally.

Examples:

TEXT

10.1.2.3
172.20.5.7
192.168.1.100

Public address

Globally routable address assigned for public Internet connectivity.

Do not decide public/private merely by whether an address "looks familiar."

Use the defined private ranges.

28. GATE PRIVATE-ADDRESS QUESTION

Which are private?

A

TEXT

10.20.30.40

Yes.

Because:

\[10.0.0.0/8 \]

B

TEXT

172.20.10.5

Yes.

Because 20 lies between:

\[16\text{ and }31 \]

inside:

\[172.16.0.0/12 \]

C

TEXT

172.40.1.1

No.

It is outside private 172.16-31.x.x .

D

TEXT

192.168.50.10

Yes.

29. PRIVATE RANGE MEMORY

TEXT

10.x.x.x

1 7 2 . 1 6 . x . x !' 1 7 2 . 3 1 . x . x

192.168.x.x

30. NAT AND ROUTING

NAT and routing are related but different.

Routing

Decides:

Where packet should go.

NAT

Modifies:

Addressing information.

A NAT router may do both:

1. translate packet fields
2. route/forward packet

But don't treat NAT itself as the routing algorithm.

31. NAT AND FIREWALL — IMPORTANT DISTINCTION

& pNAT is **not the same thing as a firewall**.

NAT changes address/port mappings.

A firewall enforces security policy such as:

- allow

- deny
- filter

A home router often performs both NAT and firewalling, which causes people to confuse them.

Correct:

NAT may incidentally make unsolicited inbound connections harder without mappings.

Incorrect:

NAT is automatically a complete security firewall.

32. INBOUND CONNECTION PROBLEM

Suppose Internet host wants to initiate a new connection to:

TEXT

203.0.113.5:8080

But NAT has no mapping telling it which internal host should receive the packet.

It cannot automatically know whether it should go to:

TEXT

192.168.1.10

or:

TEXT

192.168.1.20

or another host.

This is one limitation of NAT.

33. PORT FORWARDING

One solution is an explicit static mapping.

Example:

TEXT

203.0.113.5:8080

! "

192.168.1.50:80

This is often called:

Port forwarding

It allows selected inbound traffic to reach a chosen internal service.

34. NAT BREAKS PURE END-TO-END ADDRESSING

Original Internet design ideal:

Any host can address another host directly using globally unique addresses.

NAT introduces an intermediate device that rewrites packet addressing.

So NAT breaks or complicates the pure:

end-to-end addressing model.

Some protocols that embed IP/port information inside application payloads may need special handling.

35. WHY NAT CAN COMPLICATE APPLICATIONS

Suppose an application sends inside its payload:

TEXT

"My IP is 192.168.1.10"

NAT may translate the IP header to:

TEXT

203.0.113.5

but the application payload might still contain:

TEXT

192.168.1.10

That can create issues for protocols that embed network addressing information inside data.

You don't need protocol-specific depth here, just understand:

NAT can complicate end-to-end protocols.

36. NAT STATE

NAT is generally stateful in the sense that the NAT device maintains translation mappings.

Example table:

Protocol	Internal	External mapping
TCP	192.168.1.10:51000	203.0.113.5:62000

Protocol	Internal	External mapping
TCP	192.168.1.20:53000	203.0.113.5:62001

Without this state, replies could not be translated correctly.

37. SAME INTERNAL PORT IS POSSIBLE

Suppose:

TEXT

```
Host A = 192.168.1.10:5000
Host B = 192.168.1.20:5000
```

Both use port:

`\[ 5000 \]`

No problem.

NAT can translate to:

TEXT

```
203.0.113.5:40000
203.0.113.5:40001
```

The translated external ports differentiate them.

38. SAME PUBLIC IP, SAME DESTINATION SERVER

Even if both hosts connect to the same web server:

TEXT

```
93.184.216.34:443
```

NAT can distinguish them by different translated source ports.

Example:

TEXT

```
203.0.113.5:40000 !' server:443
203.0.113.5:40001 !' server:443
```

39. CONNECTION IDENTIFICATION

A TCP flow is commonly distinguished by a 4-tuple:

`\boxed{ (SourceIP, \SourcePort, \DestinationIP, \DestinationPort) } \]`

Example:

TEXT

192.168.1.10:51000
!
93.184.216.34:443

NAT rewrites part of this tuple.

40. GATE NUMERICAL-STYLE TRANSLATION

Suppose NAT table is:

Private endpoint	Public endpoint
10.0.0.2:2000	150.10.1.1:5001
10.0.0.3:3000	150.10.1.1:5002

A reply arrives:

TEXT

Destination IP = 150.10.1.1
Destination Port = 5002

Where does NAT forward it?

Look up:

TEXT

150.10.1.1:5002

Mapping:

TEXT

10.0.0.3:3000

Therefore:

$\boxed{10.0.0.3:3000}$

41. ANOTHER GATE QUESTION

Outgoing packet:

TEXT

Source = 10.0.0.7:4000
Destination = 20.1.1.5:80

NAT mapping:

TEXT

```
10.0.0.7:4000
!"
100.5.5.5:6200
```

What packet does the server see?

`\boxed{ Source=100.5.5.5:6200 } \`

Destination remains:

`\ 20.1.1.5:80 \`

assuming ordinary outbound NAT with no destination translation.

42. RETURN PACKET

Server sends:

TEXT

```
Source      = 20.1.1.5:80
Destination = 100.5.5.5:6200
```

NAT translates destination:

TEXT

```
Destination = 10.0.0.7:4000
```

and forwards internally.

43. SOURCE NAT vs DESTINATION NAT — CONCEPTUAL

On outbound traffic, NAT often changes:

Source address/port.

This is sometimes described as:

Source NAT.

For selected incoming traffic/port forwarding, NAT may change:

Destination address/port.

Conceptually:

TEXT

```
Outbound:
p r i v a t e   s o u r c e   ! '   p u b l i c   s o u r c e

Inbound forwarding:
```

public destination !' private destination

44. DOES NAT CHANGE DESTINATION IP ON NORMAL OUTBOUND PACKETS?

Usually not in the simple Internet-access case.

Example:

Before:

TEXT

1 0 . 0 . 0 . 2 !' 8 . 8 . 8 . 8

After source NAT:

TEXT

2 0 3 . 0 . 1 1 3 . 5 !' 8 . 8 . 8 . 8

Destination remains:

TEXT

8.8.8.8

The source changes.

45. DOES NAT CHANGE SOURCE ON THE REPLY?

In the simple return case:

Server source remains server.

Example:

TEXT

8 . 8 . 8 . 8 !' 2 0 3 . 0 . 1 1 3 . 5

NAT changes destination to:

TEXT

8 . 8 . 8 . 8 !' 1 0 . 0 . 0 . 2

So direction matters.

46. NAT vs PROXY

Do not confuse these.

NAT

Rewrites network/transport addressing while forwarding packets.

Proxy

Acts as an application/intermediary endpoint and can create a separate connection on behalf of a client.

For this syllabus:

Focus on NAT, not proxy architecture.

47. ADVANTAGES OF NAT

Important benefits:

- conserves public IPv4 addresses
- allows reuse of private address ranges
- hides internal addressing structure from ordinary external routing
- allows internal renumbering with reduced public-address impact
- PAT lets many devices share one public IP

48. DISADVANTAGES/ LIMITATIONS

Important drawbacks:

- breaks pure end-to-end addressing
- inbound connections need mappings/port forwarding
- NAT maintains translation state
- can complicate some protocols
- packet fields/checksums must be modified
- creates dependence on the NAT device

49. & DON'T OVERCLAIM PRIVACY

NAT hides private IP addresses from direct public routing, but that does not mean:

complete anonymity

or:

complete privacy

or:

complete security.

Those are different concepts.

50. NAT AND TCP/UDP

PAT commonly works with TCP and UDP because both have:

port numbers

This makes multiplexing many private flows onto one public IP convenient.

Other protocols may require different handling.

51. GATE CONNECTION TO TCP

Later, when studying TCP, remember:

A TCP connection uses endpoint addresses and ports.

NAT may transform:

TEXT

Private IP:Port

into:

TEXT

Public IP:TranslatedPort

But end hosts still run TCP.

NAT is an intermediate translation function.

52. GATE CONNECTION TO DNS

DNS may return a public IP for a server.

Your private host sends traffic toward that public address.

NAT translates your private source endpoint before traffic enters the public Internet.

So:

TEXT

```
D N S ! '   l e a r n s   d e s t i n a t i o n   I P
N A T ! '   t r a n s l a t e s   l o c a l   s o u r c e   i d e n t i t y
```

Different jobs.

53. HOW GATE CAN ASK NAT

Pattern 1 — Private address identification

Choose which IP belongs to private ranges.

Pattern 2 — NAT table lookup

Given public port, determine private host.

Pattern 3 — Outgoing translation

Find changed source IP/port.

Pattern 4 — Incoming reply

Find changed destination IP/port.

Pattern 5 — Why ports are used

Answer:

distinguish many simultaneous private connections sharing one public IP.

Pattern 6 — Address conservation

Explain why NAT/PAT helps with IPv4 exhaustion.

Pattern 7 — Header modification

Ask which fields/checksums may change.

Pattern 8 — End-to-end property

Ask whether NAT preserves pure end-to-end addressing.

Answer:

No.

54. & pGATE TRAPS

Trap 1

Private range is:

\[172.16.0.0/12 \]

not all 172.x.x.x.

Only:

TEXT

172.16.x.x
through
172.31.x.x

are in that private block.

Trap 2

NAT and PAT are related but not identical.

NAT may translate IP addresses.

PAT additionally uses transport ports to multiplex many connections.

Trap 3

On outgoing Internet traffic, the destination server's IP does not normally change under basic source NAT.

Trap 4

On return traffic, NAT typically translates the destination back to the internal host.

Trap 5

NAT is not the same as routing.

Trap 6

NAT is not automatically equivalent to a firewall.

Trap 7

Port numbers belong to the Transport layer.

Trap 8

NAT may require checksum updates because packet headers changed.

Trap 9

Static NAT does not necessarily conserve addresses as efficiently as PAT.

55. 0> MASTER MEMORY SYSTEM

Remember:

P-P-T

P — Private inside

P — Public outside

T — Table connects them

For PAT:

Private IP:Port → Public IP:Port

56. SECOND MEMORY TRICK

Outgoing:

Change SOURCE

Incoming reply:

Change DESTINATION

This is the standard simple NAT flow.

57. THIRD MEMORY TRICK — PRIVATE RANGES

TEXT

10

172.16-31

192.168

That's enough for GATE.

'p NOTEBOOK VERSION

NAT — Network Address Translation

NAT translates addressing information between private and public networks.

Main purpose:

Conserve public IPv4 addresses and allow reuse of private address space.

Private IPv4 Ranges

$\boxed{10.0.0.0/8}$

$\boxed{172.16.0.0/12}$

$\boxed{192.168.0.0/16}$

Basic NAT

Outgoing:

TEXT

```
Private Source IP
! "
Public Source IP
```

Incoming reply:

TEXT

```
Public Destination IP
! "
Private Destination IP
```

PAT/ NATP

Many private hosts share one public IP using different port mappings.

Example:

TEXT

```
192.168.1.10:5000
! ' 2 0 3 . 0 . 1 1 3 . 5 : 4 0 0 0 1

192.168.1.20:5000
! ' 2 0 3 . 0 . 1 1 3 . 5 : 4 0 0 0 2
```

NAT table stores mappings.

Types

Static NAT

One fixed private !" public mapping.

Dynamic NAT

Uses an available public address from a pool.

PAT

Many private endpoints share a public IP using ports.

Fields Changed

PAT may modify:

- IP address
- TCP/UDP port
- related checksums

Advantages

- public IPv4 conservation
- private-address reuse
- many hosts share public IP

Limitations

- breaks pure end-to-end addressing
 - inbound connections need explicit mapping
 - maintains state
 - can complicate protocols
-

58. 30-SECOND REVISION

Remember:

TEXT

Private:
10/8
172.16/12
192.168/16

PAT:

$\boxed{\text{PrivateIP:Port} \rightarrow \text{PublicIP:TranslatedPort}}$

Outgoing:

source translated.

Incoming reply:

destination translated back.

And:

NAT "firewall. NAT "routing.

ACTIVE RECALL — DON'T LOOK BACK

Q1

List the three private IPv4 address blocks.

Q2

Is this address private?

TEXT

172.25.10.20

Explain briefly.

Q3

Is this address private?

TEXT

172.40.10.20

Q4

What is the main difference between basic address translation and PAT?

Q5 — NAT Table

Given:

Private	Public
10.0.0.5:3000	200.1.1.1:5001
10.0.0.6:3000	200.1.1.1:5002

A reply arrives for:

TEXT

200.1.1.1:5002

To which private endpoint should it be delivered?

Q6

Why can two private hosts both use source port 5000 and still share one public IPv4 address through PAT?

Q7

For ordinary outbound source NAT, which IP field usually changes?

A. Destination IP B. Source IP C. Both always D. Neither

Q8

For the returning reply, which endpoint is translated back to the internal host?

Q9

Why might TCP/UDP checksums need adjustment when PAT modifies packet fields?

Q10 — GATE Trap

True or False:

Every address beginning with 172 is private.

Explain.

Q11

Does NAT completely preserve the original Internet end-to-end addressing model?

Q12

Why is PAT more effective at conserving public IPv4 addresses than simple one-to-one static NAT?

NEXT !' TCP Foundation: Connection-Oriented Service, Segments, Ports, Sequence Numbers, ACKs, Three-Way Handshake & Reliable Byte Stream

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 1 — TCP Foundation: Connection-Oriented Service, Ports, Segments, Sequence Numbers, ACKs & Three-Way Handshake

VERY HIGH PRIORITY FOUNDATION

Your syllabus specifically includes:

- TCP **flow control**
- TCP **congestion control**
- socket API

But none of those will make sense unless these TCP fundamentals are clear first.

The core mental model is:

TCP provides a reliable, connection-oriented, full-duplex byte stream between two processes.

And the most important GATE idea today is:

`\boxed{\text{TCP sequence numbers count BYTES, not segments}}`

1. WHERE DOES TCP OPERATE?

TCP operates at the:

Transport Layer

Recall:

TEXT

Application

T r a n s p o r t ! • T C P

N e t w o r k ! • I P

Data Link

Physical

TCP provides communication between:

Processes

while IP provides delivery between:

Hosts

2. DELIVERY TYPES — RECALL

We already learned:

Transport Layer

Process-to-process delivery

Network Layer

Host-to-host delivery

Data Link

Node-to-node delivery

So TCP must know:

Which application/process should receive the data?

For that, TCP uses:

Port Numbers

3. WHAT IS A PORT NUMBER?

A port number identifies a transport-layer application endpoint on a host.

Examples conceptually:

TEXT

```
W e b   s e r v e r   ! '   p o r t
D N S   s e r v e r   ! '   p o r t
A p p l i c a t i o n   p r o c e s s   ! '   p o r t
```

A TCP/UDP port field is:

$\boxed{16 \text{ bits}}$

Therefore possible numerical values:

$0 \text{ to } 65535$

because:

$2^{16} = 65536$

4. IP ADDRESS vs PORT NUMBER

Important distinction.

IP address

Identifies:

Host/interface in the network-layer context.

Port number

Identifies:

Process/service on that host.

Memory:

IP finds the machine; port finds the application.

5. EXAMPLE

Suppose:

TEXT

```
Client IP   = 10.1.1.5
Client port = 51000

Server IP   = 20.1.1.10
Server port = 443
```

TCP communication is between:

```
TEXT
10.1.1.5:51000
! •
20.1.1.10:443
```

This notation:

```
TEXT
IP:Port
```

represents a transport endpoint.

6. TCP CONNECTION IDENTIFICATION

A TCP connection is commonly identified by the 4-tuple:

$\boxed{(\text{SourceIP}, \text{SourcePort}, \text{DestinationIP}, \text{DestinationPort})}$

Example:

```
TEXT
10.1.1.5 : 51000
! '
20.1.1.10 : 443
```

Another client can simultaneously connect to the same server because its source IP or source port differs.

7. CONNECTION-ORIENTED TCP

TCP is:

Connection-oriented

Before normal application data transfer, endpoints establish TCP connection state.

This does **not** mean:

a dedicated physical circuit is reserved.

Important connection to our switching chapter.

TCP connection-oriented transport runs over:

connectionless IP.

There is no contradiction.

8. CIRCUIT SWITCHING vs TCP CONNECTION

Do not confuse:

Circuit switching

May reserve network resources/path.

TCP connection

Maintains logical transport-layer state between endpoints.

TCP does not reserve every router/link along the Internet path.

Memory:

TCP connection = logical state, not physical circuit.

9. WHAT SERVICE DOES TCP PROVIDE?

Important TCP characteristics:

- connection-oriented
- reliable
- byte-stream service
- full duplex
- ordered delivery
- error detection
- flow control
- congestion control

We will study the last two deeply later.

10. WHAT DOES RELIABLE MEAN?

TCP tries to ensure the receiving application gets the byte stream:

- without missing data
- without duplicates
- in order

It uses mechanisms such as:

- sequence numbers
- acknowledgements
- checksum
- retransmissions
- timers

Later:

- flow control
- congestion control

11. TCP IS A BYTE STREAM

Order is important.

Applications give TCP a sequence of bytes.

Conceptually:

```
TEXT
A B C D E F G H I J ...
```

TCP sees:

a continuous ordered byte stream.

It does **not preserve application message boundaries** in the way a message-oriented protocol might.

12. EXAMPLE

Application performs two writes:

```
TEXT
W r i t e   1   !'   " A B C "
W r i t e   2   !'   " D E F "
```

TCP conceptually carries byte stream:

```
TEXT
ABCDEF
```

Receiver might read:

```
TEXT
AB
```

then:

```
TEXT
CDEF
```

depending on buffering/API behavior.

TCP guarantees byte ordering, not that one application write equals one received segment/read.

13. BYTE-STREAM ORIENTED CONCEPT

TCP is:

`\boxed{\text{Byte-stream oriented}}`

not:

message-boundary preserving.

14. TCP SEGMENT

TCP takes portions of the byte stream and places them inside:

TCP segments

A TCP segment contains:

TEXT

```
+-----+
| TCP Header |
+-----+
| TCP Data   |
+-----+
```

This TCP segment becomes payload of an IP packet.

15. ENCAPSULATION RECALL

TEXT

```
Application Data
! "
TCP Segment
! "
IP Packet
! "
Ethernet Frame
! "
Bits
```

Mnemonic from earlier:

Data !' Segment !' Packet !' Frame !' Bits

16. TCP HEADER SIZE

Basic TCP header without options:

$\boxed{20B}$

TCP header can be larger when options are present.

Maximum header size:

$\boxed{60B}$

because the TCP data-offset field measures header length in 32-bit words.

For most basic GATE calculations:

20-byte TCP header is common unless another value is specified.

17. IMPORTANT TCP HEADER FIELDS

You don't need every bit immediately.

The most important ones for our syllabus are:

- Source Port
- Destination Port
- Sequence Number
- Acknowledgement Number
- Flags such as SYN, ACK, FIN
- Window Size
- Checksum

Later:

Window Size != flow control.

18. SEQUENCE NUMBER — THE HEART OF TCP

TCP numbers the bytes in its stream.

Suppose a sender's first data byte has sequence number:

[1000]

and it sends 500 bytes.

Then those bytes correspond conceptually to sequence numbers:

[1000]

through:

[1499]

The next byte has sequence number:

[1500]

This is fundamental.

19. OVERFLOW MEMORY

Sequence number = number of the FIRST byte in this segment's data.

If segment contains:

[L]

data bytes and starts at sequence number:

S

then next sequence number is:

$S+L$

20. GATE NUMERICAL

Segment:

$\text{Seq}=3000$

Data:

800B

Which byte numbers are carried?

First:

3000

Last:

$3000+800-1$

$\boxed{3799}$

Next byte:

$\boxed{3800}$

21. & pSEQUENCE NUMBER COUNTS BYTES

Suppose TCP sends:

Segment 1

500 data bytes

Segment 2

500 data bytes

If Segment 1 starts at:

$\text{Seq}=1000$

Segment 2 starts at:

$\boxed{1500}$

not:

1001

Sequence numbers don't simply increase by one per segment.

They advance by:

Number of bytes carried.

22. ACKNOWLEDGEMENT NUMBER

TCP acknowledgements are generally:

Cumulative

The ACK number tells the sender:

The next byte I expect from you.

When I receive exactly this.

23. EXAMPLE

Receiver successfully receives bytes:

[1000\text{ through }1499]

What ACK does it send?

It expects next:

[1500]

Therefore:

[\boxed{ACK=1500}]

Meaning:

I have received everything through byte 1499; send me byte 1500 next.

24. MEMORY

ACK = Next Expected Byte

Not:

last byte received.

So if last correctly received byte is:

[5000]

then ACK is:

[\boxed{5001}]

assuming contiguous delivery from the expected starting point.

25. GATE TRAP

Segment:

\[Seq=7000 \]

Payload:

\[1000B \]

Receiver ACK:

\[\boxed{8000} \]

not:

- 7999
- 7001
- 1000

26. CUMULATIVE ACK

Suppose receiver has correctly received:

TEXT

Bytes 1000–1499
Bytes 1500–1999
Bytes 2000–2499

It can acknowledge all of them using:

\[\boxed{ACK=2500} \]

This means:

Everything before 2500 has been received contiguously.

27. WHAT IF A SEGMENT IS MISSING?

Suppose receiver gets:

TEXT

```
1 0 0 0 - 1 4 9 9   '
1 5 0 0 - 1 9 9 9   'L   m i s s i n g
2 0 0 0 - 2 4 9 9   '   a r r i v e s
```

With ordinary cumulative ACK behavior, receiver is still waiting for:

\[1500 \]

So it may send:

\[\boxed{ACK=1500} \]

again.

This is called a:

Duplicate ACK

because the acknowledgement number does not advance.

We'll connect this to fast retransmit during congestion control.

28. WHY SEQUENCE NUMBERS ARE NEEDED

Without sequence numbers, receiver couldn't reliably determine:

- correct order
- missing data
- duplicate data

Example packets might arrive:

TEXT

Segment 3
Segment 1
Segment 2

TCP can use sequence numbers to reconstruct:

TEXT

1 !' 2 !' 3

in byte order.

29. WHY ACKs ARE NEEDED

The sender needs to know:

What successfully reached the receiver?

ACKs provide that feedback.

If expected ACK does not arrive within appropriate conditions/timing:

sender may retransmit.

30. CHECKSUM

TCP also includes a:

checksum

to detect corrupted segments.

Recall from error detection:

TCP uses a checksum mechanism involving:

one's-complement arithmetic.

So:

TEXT

```
T C P ! ' C h e c k s u m
E t h e r n e t ! ' C R C / F C S
```

Do not swap them.

31. TCP IS FULL DUPLEX

TCP supports communication in both directions simultaneously.

If A and B are connected:

TEXT

```
A ! Ä B
```

A can send data to B while B sends data to A.

Each direction has its own:

- sequence-number space
- acknowledgement behavior

This is important.

32. TWO INDEPENDENT BYTE STREAMS

Conceptually:

TEXT

```
A ! ' B : S e q u e n c e   n u m b e r s   s t a r t i n g   a r o u n d   X
B ! ' A : S e q u e n c e   n u m b e r s   s t a r t i n g   a r o u n d   Y
```

They do **not** share one universal sequence counter.

Each direction chooses its own initial sequence number.

33. WHY INITIAL SEQUENCE NUMBERS?

When TCP creates a connection, each endpoint chooses an:

Initial Sequence Number (ISN)

Suppose:

Client chooses:

[x]

Server chooses:

[y]

These establish initial sequence spaces for the two directions.

34. TCP THREE-WAY HANDSHAKE

VERY IMPORTANT

TCP establishes a connection using:

Three-Way Handshake

Basic form:

TEXT

Client

Server

SYN ----->

<----- SYN + ACK

ACK ----->

After this:

connection is established.

35. WHY THREE STEPS?

Both sides need to:

- synchronize sequence-number state
- confirm ability to communicate in both directions

The three messages establish that both endpoints have received the necessary information.

36. HANDSHAKE STEP 1

Client chooses initial sequence:

[x]

Client sends:

TEXT

SYN = 1

Seq = x

Conceptually:

TEXT

Client ---- SYN, Seq=x ----> Server

37. IMPORTANT — SYN CONSUMES ONE SEQUENCE NUMBER

Common GATE trap.

Although SYN normally carries no application data, it consumes:

$\boxed{\text{sequence number}}$

Therefore after sending SYN with:

$\text{Seq} = x$

client's next sequence number is:

$\boxed{x+1}$

38. HANDSHAKE STEP 2

Server receives client's SYN.

Server chooses its own initial sequence:

y

It sends:

TEXT

SYN = 1
ACK = 1
Seq = y
Ack = x + 1

So:

TEXT

Client <---- SYN+ACK ---- Server
Seq=y
Ack=x+1

Why:

$x+1$

?

Because client's SYN consumed one sequence number.

39. HANDSHAKE STEP 3

Client receives server SYN.

Server's SYN also consumes one sequence number.

So client replies:

TEXT

ACK = 1
Seq = x + 1
Ack = y + 1

Now both sides have synchronized their initial sequence spaces.

40. COMPLETE THREE-WAY HANDSHAKE

Suppose:

\[Client\ ISN=100 \]

\[Server\ ISN=500 \]

Then:

1

Client !' Server

TEXT

SYN
Seq = 100

2

Server !' Client

TEXT

SYN + ACK
Seq = 500
Ack = 101

3

Client !' Server

TEXT

ACK
Seq = 101
Ack = 501

Ø=Yes pattern should become automatic.

41. HANDSHAKE MEMORY

Think:

TEXT

Client: "My number is x."

```
Server: "Got x, expecting x+1.  
        My number is y."  
  
Client: "Got y, expecting y+1."
```

That's the handshake.

42. WHY IS ACK = x+1?

Because ACK means:

Next sequence number expected.

Client sent SYN at:

$\backslash [x]$

and SYN occupies one sequence position.

So receiver expects:

$\backslash [x+1]$

43. DOES A PURE ACK CONSUME A SEQUENCE NUMBER?

Normally:

No

A TCP segment carrying only an ACK with no data and no SYN/FIN does not consume sequence-number space.

This is an important contrast.

44. WHAT CONSUMES SEQUENCE SPACE?

Application data

Consumes:

one sequence number per byte.

SYN

Consumes:

$\backslash [1]$

FIN

Consumes:

$\backslash [1]$

Pure ACK

Consumes:

$\lfloor 0 \rfloor$

$\emptyset = 0\%$ useful GATE memory.

45. MEMORY TABLE

TCP content	Sequence space consumed
100 data bytes	100
SYN	1
FIN	1
ACK only	0

Memory:

Data counts bytes; SYN/FIN count one; ACK alone counts none.

46. DATA AFTER HANDSHAKE

Using:

$\lfloor \text{ClientISN}=100 \rfloor$

client's first data byte normally starts at:

$\lfloor \boxed{101} \rfloor$

because the SYN used sequence 100.

Suppose client sends 500 data bytes.

Segment:

TEXT

Seq = 101

Data = 500B

These bytes occupy:

$\lfloor 101 \text{ through } 600 \rfloor$

because:

$\lfloor 101+500-1=600 \rfloor$

Next expected:

$\lfloor \boxed{601} \rfloor$

So server ACK:

$\boxed{601}$

47. FULL NUMERICAL

Client ISN:

1000

Server ISN:

5000

Handshake:

Client SYN

Seq=1000

Server SYN+ACK

Seq=5000

Ack=1001

Client ACK

Seq=1001

Ack=5001

Now client sends:

1200B

starting at:

Seq=1001

Last byte:

$1001 + 1200 - 1 = 2200$

Server next expects:

2201

Therefore:

$\boxed{ACK=2201}$

48. ACK CAN BE PIGGYBACKED

Because TCP is full duplex, suppose B has:

- an ACK to send to A
- its own application data to send A

Rather than sending a separate ACK segment and then data, B can include the acknowledgement in its outgoing data segment.

This is called:

Piggybacking

Conceptually:

TEXT

B ! ' A :

Data + ACK

49. ACK FLAG vs ACKNOWLEDGEMENT NUMBER

TCP header has an ACK flag indicating:

acknowledgement field is meaningful.

Don't confuse:

- ACK flag
- acknowledgement number itself.

During most established TCP communication, ACK flag is set.

50. SYN FLAG

SYN means:

Synchronize sequence numbers.

It is used during connection establishment.

Memory:

SYN = Synchronize

51. FIN FLAG — PREVIEW

FIN means:

Sender has finished sending data in that direction.

TCP connection termination is normally more involved than connection establishment because the two directions close independently.

FIN also consumes:

\[1 \]

sequence number.

We'll briefly understand it today but not over-expand.

52. CONNECTION TERMINATION — BASIC IDEA

A typical graceful TCP close can involve:

TEXT

```
A !' B : F I N
B !' A : A C K
B !' A : F I N
A !' B : A C K
```

often described as a:

four-segment termination

because full-duplex directions may close separately.

53. FIN NUMERICAL

Suppose A has sent through byte:

`\[ 999 \]`

So next sequence:

`\[ 1000 \]`

A sends:

TEXT

```
FIN
Seq = 1000
```

FIN consumes one sequence number.

B acknowledges:

`\[ \boxed{ACK=1001} \]`

Same basic idea as SYN.

54. THREE-WAY HANDSHAKE vs FOUR-WAY CLOSE

General memory:

Establish

SYN !' SYN+ACK !' ACK

"B steps.

Close

Often:

FIN !' ACK !' FIN !' ACK

"H segments.

Why?

Because TCP is full duplex, and each direction may terminate separately.

55. TCP vs UDP — SMALL FOUNDATION

UDP isn't explicitly named in your syllabus, but knowing the contrast helps.

TCP

- connection-oriented
- reliable
- ordered byte stream
- flow control
- congestion control

UDP

- connectionless
- no TCP-style reliability
- preserves datagram/message boundaries
- lower protocol overhead

Do not expand further unless needed.

56. SEGMENTATION vs IP FRAGMENTATION

Ø = important confusing pair.

TCP segmentation

TCP divides application byte stream into:

TCP segments.

Occurs at:

Transport layer.

IPv4 fragmentation

IP divides an oversized IP datagram because of:

MTU.

Occurs at:

Network layer.

These are not the same.

57. EXAMPLE

Application provides:

\[5000B \]

TCP may divide this into several TCP segments.

Each TCP segment becomes an IP datagram.

If one IP datagram is then too large for an outgoing MTU:

IPv4 fragmentation could further split that IP datagram.

So:

```
TEXT
Application data
      !"
TCP segmentation
      !"
IP packets
      !"
Possible IP fragmentation
```

58. MSS — USEFUL PREVIEW

TCP often tries to limit how much TCP data it places in a segment using:

MSS — Maximum Segment Size

MSS refers to maximum TCP **payload**, not total IP packet size.

Common relationship in a simple IPv4/Ethernet case:

Ethernet MTU:

\[1500B \]

IPv4 header:

\[20B \]

TCP header:

\[20B \]

Therefore a common TCP MSS:

\[$1500 - 20 - 20 = \boxed{1460B}$ \]

under those assumptions.

$\text{MTU} - \text{IP} - \text{TCP}$ connects Ethernet MTU to TCP.

59. MSS vs MTU

Do not confuse:

MTU

Maximum IP packet carried by link.

Example:

1500B

MSS

Maximum TCP payload.

Example with 20B IP + 20B TCP:

1460B

So:

$\boxed{\text{MSS} \neq \text{MTU}}$

60. SEQUENCE NUMBER FIELD SIZE

TCP Sequence Number field is:

32 bits

Therefore sequence numbers range modulo:

2^{32}

They eventually wrap around.

For most GATE questions, simply remember:

32-bit byte sequence space.

61. ACKNOWLEDGEMENT FIELD SIZE

TCP Acknowledgement Number is also:

32 bits

It represents:

next sequence number expected from the other direction.

62. SEQUENCE WRAPAROUND — SMALL FOUNDATION

Suppose sequence reaches near:

$$2^{32}-1$$

Then numbering wraps modulo:

$$2^{32}$$

This can appear in theoretical GATE numericals.

Don't treat sequence numbers as infinitely increasing integers.

63. GATE QUESTION — NUMBER OF BYTES

Segment:

$$\text{Seq}=400$$

Next segment:

$$\text{Seq}=900$$

Assume first carried only normal application data and there are no SYN/FIN complications.

How much data was in the first segment?

$$900-400 = \boxed{500\text{B}}$$

64. GATE QUESTION — ACK

Sender sends:

TEXT

Seq = 5000

Data = 700B

Receiver gets everything correctly.

Last byte:

$$5000+700-1=5699$$

Next expected:

$$5700$$

Therefore:

$$\boxed{\text{ACK}=5700}$$

65. GATE QUESTION — MULTIPLE SEGMENTS

Sender sends:

Segment 1

$$\text{Seq}=1000, \text{Length}=500$$

Segment 2

$$\text{Seq}=1500, \text{Length}=300$$

Both arrive correctly and contiguously.

Receiver ACK:

$\lfloor 1000+500+300 = \boxed{1800} \rfloor$

66. OUT-OF-ORDER EXAMPLE

Expected:

$\lfloor 1000 \rfloor$

Receiver gets:

TEXT

```
1 0 0 0 - 1 4 9 9 '
2 0 0 0 - 2 4 9 9 '
```

but:

TEXT

```
1 5 0 0 - 1 9 9 9 'L
```

The cumulative ACK remains:

$\lfloor \boxed{1500} \rfloor$

because receiver's next missing byte is 1500.

67. WHY DUPLICATE ACKS MATTER

If later segments arrive while one earlier segment is missing, receiver may repeatedly send:

$\lfloor \text{ACK}=1500 \rfloor$

These repeated ACKs can inform sender:

Something around sequence 1500 was likely lost.

Later we will see:

Three duplicate ACKs != Fast Retransmit in classic TCP behavior.

~~Save~~ Save this connection for congestion control.

68. RETRANSMISSION — BASIC IDEA

TCP can retransmit data when it believes it has been lost.

Two major signals we will later encounter:

1. Retransmission timeout
2. Duplicate acknowledgements

Don't go deep yet; today just understand why sequence/ACK numbers enable retransmission.

69. RELIABILITY MENTAL MAP

TCP reliability:

```
TEXT
Checksum
    !"
Detect corruption

Sequence numbers
    !"
Know byte positions

ACKs
    !"
Know what arrived

Timer / duplicate ACK
    !"
Detect likely loss

Retransmission
    !"
Recover data
```

70. FLOW CONTROL — PREVIEW

Suppose sender is extremely fast but receiver's buffer is small.

The sender could overwhelm the receiver.

TCP solves this with:

Flow control

Receiver advertises how much buffer space it currently has.

This value is commonly represented as:

$\boxed{rwnd}$

Receiver window.

Next topic.

71. CONGESTION CONTROL — PREVIEW

Even if receiver can handle lots of data, the network itself may be congested.

TCP also limits sending based on network conditions using:

$\boxed{cwnd}$

Congestion window.

Later the usable sending window becomes conceptually constrained by:

$\lfloor \min(\text{rwnd}, \text{cwnd}) \rfloor$

subject to TCP details.

Save this distinction.

72. FLOW vs CONGESTION PREVIEW

Flow Control

Protect:

Receiver

Congestion Control

Protect:

Network

Memory:

Flow !' Receiver Congestion !' Network

This will become one of the biggest TCP GATE topics.

73. SOCKET API — PREVIEW

Applications do not manually construct every TCP sequence number.

They normally interact with the operating system using:

Sockets

For example, server conceptually does:

```
TEXT
socket()
bind()
listen()
accept()
```

Client:

```
TEXT
socket()
connect()
```

Then both use operations like:

TEXT

```
send()/write()  
recv()/read()
```

We'll study this after TCP flow/congestion control.

74. HOW GATE CAN ASK TCP FOUNDATION

Pattern 1

Identify TCP service:

reliable connection-oriented byte stream.

Pattern 2

Sequence number numerical.

Given starting sequence + data length.

Find next sequence.

Pattern 3

ACK numerical.

Find next expected byte.

Pattern 4

Three-way handshake.

Given ISNs, fill missing:

- Seq
 - ACK
-

Pattern 5

SYN/FIN sequence consumption.

Both consume one.

Pattern 6

Pure ACK.

Consumes no sequence number.

Pattern 7

Out-of-order segments.

Determine cumulative ACK.

Pattern 8

MSS from MTU.

$\lfloor \text{MSS} = \text{MTU} - \text{IPHeader} - \text{TCPHeader} \rfloor$

under specified assumptions.

75. & pGATE TRAPS

Trap 1

TCP sequence numbers count:

$\lfloor \boxed{\text{text{bytes}}} \rfloor$

not segments.

Trap 2

ACK number means:

next byte expected.

Not:

last byte received.

Trap 3

If segment starts at $\lfloor (S) \rfloor$ and carries $\lfloor (L) \rfloor$ bytes:

$\lfloor \boxed{\text{NextSeq} = S + L} \rfloor$

Trap 4

SYN consumes:

$\lfloor 1 \rfloor$

sequence number.

Trap 5

FIN also consumes:

$\lfloor 1 \rfloor$

sequence number.

Trap 6

Pure ACK consumes:

$\lfloor 0 \rfloor$

sequence numbers.

Trap 7

Client and server use separate sequence-number spaces.

Trap 8

TCP connection-oriented does not mean circuit switching.

Trap 9

TCP segmentation "" IPv4 fragmentation.

Trap 10

MSS refers to TCP payload, while MTU concerns the link's supported IP-packet size.

76. MASTER MEMORY SYSTEM

Remember:

S-A-N

S — Sequence

First byte number.

A — ACK

Next expected byte.

N — Number of bytes

$\lfloor \text{NextSeq} = \text{Seq} + \text{NumberOfBytes} \rfloor$

77. HANDSHAKE MEMORY

If:

\[ClientISN=x \]

\[ServerISN=y \]

then:

TEXT

C l i e n t ! ' S e r v e r :

SYN

Seq = x

S e r v e r ! ' C l i e n t :

SYN + ACK

Seq = y

Ack = x+1

C l i e n t ! ' S e r v e r :

ACK

Seq = x+1

Ack = y+1

Ø=Memorize this pattern.

' p NOTEBOOK VERSION

TCP Foundation

TCP = **Transmission Control Protocol**

Transport-layer protocol providing:

- connection-oriented service
- reliable delivery
- ordered byte stream
- full duplex
- flow control
- congestion control

Ports

Port number:

\[\boxed{16\text{bits}} \]

Endpoint:

TEXT

IP:Port

TCP connection commonly identified by:

\[\boxed{ SrcIP, SrcPort, DstIP, DstPort } \]

TCP Segment

TEXT

TCP Header | TCP Data

Minimum TCP header:

$\boxed{20\text{B}}$

Sequence Number

Sequence numbers count:

$\boxed{\text{BYTES}}$

Sequence field identifies:

first data byte in segment.

If:

$\boxed{\text{Seq} = S}$

and data:

$\boxed{L}$

then:

$\boxed{\text{NextSeq} = S + L}$

ACK Number

$\boxed{\text{ACK} = \text{NextExpectedByte}}$

Example:

Received bytes 1000–1499:

$\boxed{\text{ACK} = 1500}$

TCP ACKs are generally cumulative.

Three-Way Handshake

If client ISN $\boxed{x}$, server ISN $\boxed{y}$:

TEXT

1. SYN
Seq = x
2. SYN+ACK
Seq = y
Ack = $x+1$
3. ACK

Seq = x+1
Ack = y+1

Sequence Space Consumption

Item	Consumes
(L) data bytes	(L)
SYN	1
FIN	1
ACK only	0

MSS

Under ordinary IPv4:

$\boxed{MSS = MTU - IPHeader - TCPHeader}$

Example:

$\boxed{1500 - 20 - 20 = 1460B}$

Reliability

TCP uses:

- checksum
- sequence numbers
- ACKs
- retransmission
- timers

78. 30-SECOND REVISION

Remember only:

TCP = reliable byte stream.

$\boxed{Seq = FirstByte}$

$\boxed{ACK = NextExpectedByte}$

$\boxed{NextSeq = Seq + DataBytes}$

Handshake:

TEXT

S Y N ! ' S Y N + A C K ! ' A C K

If ISNs are (x, y) :

$[x, x+1]$

and:

$[y, y+1]$

because:

SYN consumes one sequence number.

ACTIVE RECALL — DON'T LOOK BACK

Q1

TCP provides communication between:

A. Routers B. Processes C. Ethernet switches D. Physical links

Q2

What is the difference between an IP address and a port number?

Q3

A TCP segment has:

$[Seq=2000]$

and carries:

$[750B]$

Find:

1. last byte sequence number
2. next sequence number
3. ACK expected if everything arrives correctly

Q4 — GATE Trap

True or False:

TCP sequence number identifies the segment number.

Explain.

Q5

Receiver has correctly received all contiguous bytes through:

\[8999 \]

What ACK number should it send?

Q6 — Three-Way Handshake

Client ISN:

\[500 \]

Server ISN:

\[900 \]

Fill:

1. Client SYN sequence
 2. Server SYN+ACK sequence and acknowledgement
 3. Client final ACK sequence and acknowledgement
-

Q7

How many sequence numbers are consumed by:

1. 1000 data bytes?
 2. SYN?
 3. FIN?
 4. pure ACK?
-

Q8 — Cumulative ACK

Receiver gets:

TEXT

```
1 0 0 0 - 1 4 9 9 '
1 5 0 0 - 1 9 9 9 'L
2 0 0 0 - 2 4 9 9 '
```

What cumulative ACK should it send?

Q9

Why can TCP deliver data in order even if IP packets arrive out of order?

Q10

Ethernet MTU is:

\[1500B \]

IPv4 header:

\[20B \]

TCP header:

\[20B \]

What is the TCP MSS under these assumptions?

Q11

Why is saying:

“TCP is connection-oriented, therefore it reserves a circuit through the network”

incorrect?

Q12 — Future Connection

What is the main purpose of TCP flow control?

A. Protect receiver from being overwhelmed B. Choose IP routing path C. Detect Ethernet collision D. Allocate MAC addresses

NEXT !' TCP Flow Control: Receiver Buffer, Sliding Window, `rwnd`, Cumulative ACKs, Zero Window & GATE Window Numericals

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 2 — TCP Flow Control: Receiver Buffer, Sliding Window, `rwnd`, Cumulative ACKs, Zero Window & GATE Numericals

~~Ø=Ý%Ø~~ ~~VÉRNOH~~ HIGH PRIORITY FOR GATE

TCP flow control answers one basic question:

What if the sender can transmit data faster than the receiving application can consume it?

TCP protects the receiver using the:

$$\boxed{\text{Receive Window } (rwnd)}$$

The central memory line is:

Flow control protects the RECEIVER.

Later, congestion control will protect the:

NETWORK.

Do not mix them.

1. WHY IS FLOW CONTROL NEEDED?

Suppose:

TEXT

Sender	Receiver
Fast CPU	Slow application
Fast network	Small receive buffer

Sender generates data at:

100 Mbps

but receiving application consumes only:

10 Mbps

If sender continues without limitation:

Receiver's buffer eventually fills.

New incoming data may then have nowhere to go.

TCP prevents this using:

Flow Control

2. BASIC IDEA

The receiver tells the sender:

"This is how much additional data I currently have room for."

This advertised amount is called:

$\boxed{rwnd}$

or:

Receiver Window / Advertised Window

3. RECEIVER BUFFER

Suppose TCP receiver has buffer size:

$RcvBuffer = 10000B$

Currently:

$6000B$

are occupied by received data not yet consumed by the application.

Free space:

$$\boxed{10000 - 6000 = 4000B}$$

So receiver can advertise approximately:

$$\boxed{rwnd = 4000B}$$

4. BASIC FORMULA

Conceptually:

$$\boxed{rwnd = RcvBuffer - BufferedData}$$

Exact TCP implementations have additional details, but for GATE numericals this is the essential model.

5. EXAMPLE

Receive buffer:

$$\boxed{12000B}$$

Currently occupied:

$$\boxed{7500B}$$

Then:

$$\boxed{rwnd = 12000 - 7500}$$

$$\boxed{= 4500B}$$

Meaning:

Receiver can currently accept about 4500 additional bytes.

6. WINDOW MEMORY

Think:

Window = Empty space in receiver's room.

If room becomes full:

$$\boxed{rwnd = 0}$$

If application consumes data:

Free space increases !' rwnd increases.

7. TCP SLIDING WINDOW

TCP does not normally require:

```
TEXT
Send one segment
!"
Wait for ACK
!"
Send next segment
```

That would waste bandwidth.

Instead, TCP can have multiple bytes/segments outstanding before receiving individual acknowledgements.

This is called:

Sliding Window

8. STOP-AND-WAIT vs SLIDING WINDOW

Stop-and-Wait

```
TEXT
Send 1
Wait
ACK
Send 2
Wait
ACK
```

Only one packet/segment at a time.

Sliding Window

```
TEXT
Send 1
Send 2
Send 3
Send 4
! "
ACKS
```

Multiple bytes can be "in flight."

This provides much better utilization.

9. WINDOW IS MEASURED IN BYTES

Ø=16K important.

TCP is a:

byte-stream protocol.

Therefore TCP receive window is fundamentally expressed in terms of:

$\boxed{\text{bytes}}$

not number of segments.

If:

$\text{rwnd}=5000$

it means roughly:

receiver permits a window of 5000 bytes from the acknowledgement point.

10. HOW ACK AND WINDOW WORK TOGETHER

Suppose receiver sends:

$\text{ACK}=5000$

and:

$\text{rwnd}=3000$

What does this mean?

ACK = 5000

I have received contiguous bytes before 5000. My next expected byte is 5000.

rwnd = 3000

I have room for bytes in the receive window beginning at 5000.

So allowed sequence range is conceptually:

5000

through:

$5000+3000-1$

$\boxed{7999}$

11. MASTER FORMULA

If receiver advertises:

$\text{ACK}=A$

and:

$\text{rwnd}=W$

then receive window covers:

$\boxed{\text{A through A+W-1}}$

The right edge is:

$\boxed{\text{A+W}}$

as the first sequence number beyond the advertised window.

12. GATE NUMERICAL

Receiver advertises:

$\boxed{\text{ACK=10000}}$

$\boxed{\text{rwnd=4000}}$

Permitted sequence numbers:

$\boxed{10000}$

through:

$\boxed{13999}$

because:

$\boxed{10000+4000-1 = 13999}$

So sender may have bytes within that receive window outstanding, subject also to congestion control.

13. WHY IS IT CALLED "SLIDING"?

Suppose initially:

$\boxed{\text{ACK=1000, rwnd=4000}}$

Window:

TEXT
1000 ----- 4999

Receiver obtains some data and application consumes data.

Later receiver advertises:

$\boxed{\text{ACK=2000, rwnd=4000}}$

Window becomes:

TEXT
2000 ----- 5999

The acceptable sequence range moved forward.

Hence:

Sliding window

14. VISUAL

Initially:

TEXT

```
Already ACKed | Receiver Window
-----|-----
              1000              4999
```

Later:

TEXT

```
Already ACKed | Receiver Window
-----|-----
              2000              5999
```

The window "slides" as ACKs advance.

15. SENDER SIDE — OUTSTANDING DATA

Suppose receiver advertises:

`\[ rwnd=5000B \]`

Sender has already transmitted:

`\[ 3000B \]`

that have not yet been acknowledged.

How much more can sender send due to flow control?

`\[ 5000-3000 = \boxed{2000B} \]`

16. FLOW-CONTROL USABLE WINDOW

Conceptually:

`\[ \boxed{ UsableWindow = rwnd-OutstandingBytes } \]`

where outstanding bytes are sent but not yet cumulatively acknowledged, relative to the current advertised receive window.

For basic GATE problems this is extremely useful.

17. EXAMPLE

Advertised receive window:

`\[ 8000B \]`

Unacknowledged data:

`\[ 2500B \]`

Additional data allowed by flow control:

$\lfloor 8000-2500 \rfloor$

$\lfloor \boxed{5500} \rfloor$

18. BUT CONGESTION CONTROL ALSO EXISTS

Later we will learn another sender limit:

$\lfloor \text{cwnd} \rfloor$

= congestion window.

Therefore the sender cannot necessarily use the entire:

$\lfloor \text{rwnd} \rfloor$

The actual amount of unacknowledged data permitted is broadly bounded by:

$\lfloor \boxed{\min(\text{rwnd}, \text{cwnd})} \rfloor$

subject to implementation details.

Remember the distinction now:

rwnd ! receiver limitation

cwnd ! network/congestion limitation

19. EFFECTIVE SENDING WINDOW

For GATE-level understanding:

$\lfloor \boxed{\text{SendWindow} = \min(\text{rwnd}, \text{cwnd})} \rfloor$

Then if some bytes are already outstanding:

$\lfloor \boxed{\text{AdditionalSend} \approx \text{SendWindow} - \text{OutstandingBytes}} \rfloor$

when positive.

We will properly use this in congestion-control numericals.

20. FLOW CONTROL DOES NOT MEAN ACK EVERY BYTE

TCP can send many bytes before ACKs arrive.

Suppose:

$\lfloor \text{rwnd} = 10000\text{B} \rfloor$

Sender may transmit several segments whose total outstanding data is up to the permitted window.

Example with 1000-byte segments:

TEXT

```
S e q    1 0 0 0  !'    1 0 0 0  B
S e q    2 0 0 0  !'    1 0 0 0  B
S e q    3 0 0 0  !'    1 0 0 0  B
...
```

Multiple segments can be outstanding.

21. CUMULATIVE ACK RECALL

Suppose receiver correctly gets:

TEXT

```
1000-1999
2000-2999
3000-3999
```

It can simply send:

$\boxed{\text{ACK}=4000}$

This acknowledges all contiguous data before:

4000

No separate ACK number is required for every byte.

22. FLOW CONTROL NUMERICAL

Receiver sends:

$\text{ACK}=4000$

$\text{rwnd}=6000$

Sender's acceptable receive-window range:

4000

through:

9999

because:

$4000+6000-1=9999$

Suppose sender has already transmitted through byte:

7499

Then outstanding range from ACK point:

TEXT

```
4000-7499
```

Number outstanding:

$7500-4000 = 3500\text{B}$

Additional permitted by flow control:

$$\lfloor 6000 - 3500 = \boxed{2500\text{B}} \rfloor$$

23. GATE SHORTCUT

If:

- $\text{ACK} = \lfloor A \rfloor$
- $\text{advertised window} = \lfloor W \rfloor$
- $\text{next sequence sender wants to send} = \lfloor N \rfloor$

then right edge is:

$$\lfloor A + W \rfloor$$

Additional bytes possible before reaching right edge:

$$\lfloor \boxed{A + W - N} \rfloor$$

provided $\lfloor N \rfloor$ is the next unsent sequence number within the current window.

24. EXAMPLE

Receiver advertises:

$$\lfloor \text{ACK} = 10000 \rfloor$$

$$\lfloor \text{rwnd} = 5000 \rfloor$$

Sender's next new byte:

$$\lfloor 13000 \rfloor$$

Right edge:

$$\lfloor 10000 + 5000 = 15000 \rfloor$$

Additional new bytes:

$$\lfloor 15000 - 13000 = \boxed{2000\text{B}} \rfloor$$

25. RECEIVER APPLICATION MATTERS

Suppose buffer:

$$\lfloor 10000\text{B} \rfloor$$

Receiver has:

$$\lfloor 8000\text{B} \rfloor$$

buffered.

So:

$$\lfloor \text{rwnd} = 2000 \rfloor$$

Now application reads:

$$\lfloor 5000\text{B} \rfloor$$

from the buffer.

Buffered data remaining:

$\lfloor 3000B \rfloor$

Free:

$\lfloor 10000 - 3000 = 7000B \rfloor$

Receiver can advertise a larger window:

$\lfloor \boxed{rwnd=7000B} \rfloor$

26. WINDOW CAN CHANGE WITHOUT ACK ADVANCING

Interesting point.

Suppose receiver already acknowledged up to byte:

$\lfloor 5000 \rfloor$

but its application has not consumed much data.

It advertises:

$\lfloor rwnd=1000 \rfloor$

Later application drains buffer.

Even if no new network data arrives, receiver may send an update indicating a larger window.

So:

ACK number and advertised window are related but represent different things.

27. ACK vs rwnd

ACK

Answers:

What contiguous byte do I expect next?

rwnd

Answers:

How much receive-buffer capacity am I currently advertising?

Memory:

TEXT

A C K ! ' W h e r e ?
r w n d ! ' H o w m u c h ?

28. ZERO WINDOW

0 = 0% important.

What if receive buffer becomes completely full?

Then:

$\boxed{rwnd=0}$

Receiver advertises:

Zero Window

Meaning:

Stop sending ordinary additional data because I currently have no buffer space.

29. ZERO-WINDOW EXAMPLE

Receive buffer:

$\boxed{5000B}$

Currently occupied:

$\boxed{5000B}$

Thus:

$\boxed{rwnd=5000-5000}$

$\boxed{0}$

Receiver sends something like:

TEXT

ACK = X

Window = 0

Sender must stop sending new bulk data based on that advertised receive window.

30. WHAT HAPPENS NEXT?

Eventually receiving application reads some data.

Suppose it frees:

$\boxed{2000B}$

Receiver can advertise:

$\boxed{rwnd=2000}$

Then sender may resume.

But there's a subtle problem.

31. LOST WINDOW UPDATE PROBLEM

Imagine:

1. Receiver says:

\[rwnd=0 \]

2. Later receiver frees space.

3. Receiver sends:

\[rwnd=3000 \]

but that window-update segment is lost.

Now:

Sender thinks

Window still zero. I must wait.

Receiver thinks

I already told sender window is open.

Both could wait indefinitely.

That would be bad.

32. TCP PERSIST TIMER

To avoid this deadlock, TCP uses a mechanism commonly called the:

Persist Timer

When sender sees a zero window, it periodically probes the receiver.

The goal:

Find out whether the receiver's window has opened.

33. ZERO-WINDOW PROBE

Conceptually:

TEXT

Sender

Receiver

Window = 0 <-----

(wait)

Probe ----->

<----- Window update
rwnd > 0

Now sender knows it may resume.

34. ZERO WINDOW

Zero window !' Persist, don't disappear forever.

Persist timer protects against:

lost window-opening updates.

35. ZERO WINDOW IS FLOW CONTROL, NOT CONGESTION

& Important.

If:

$\lceil \text{rwnd}=0 \rceil$

the problem is:

Receiver buffer full.

Not necessarily:

Network congested.

So zero receive window belongs to:

$\lceil \boxed{\text{Flow control}} \rceil$

36. RECEIVER WINDOW FIELD

TCP header contains a:

Window Size field.

Its basic field width is:

$\lceil \boxed{16 \text{ bits}} \rceil$

Without scaling, the largest raw advertised value is:

$\lceil 2^{16}-1 = \boxed{65535} \rceil$

bytes.

37. WINDOW SCALING — SMALL USEFUL FOUNDATION

Modern high-bandwidth TCP connections may need receive windows much larger than:

$\lfloor 65535B \rfloor$

TCP can negotiate a:

Window Scale option

which effectively allows much larger windows.

For GATE basics:

Know that the header's raw window field is 16 bits, but window scaling can extend effective window size.

Don't over-focus on it unless explicitly asked.

38. WHY WOULD 65 KB BE TOO SMALL?

Suppose link:

$\lfloor 1Gbps \rfloor$

RTT:

$\lfloor 100ms \rfloor$

Bandwidth-delay product:

$\lfloor R \times RTT \rfloor$

$\lfloor = 10^9 \times 0.1 \rfloor$

$\lfloor = 100,000,000bits \rfloor$

Convert:

$\lfloor = 12.5MB \rfloor$

To fully utilize such a path, sender may need many megabytes outstanding.

A 65 KB window would be far too small.

This connects directly to:

bandwidth-delay product.

39. WINDOW AND THROUGHPUT

If sender can have at most:

$\lfloor W \rfloor$

bytes outstanding and must wait roughly one RTT for acknowledgements, a simplified throughput upper bound is:

$$\boxed{\text{Throughput} \approx \frac{W}{\text{RTT}}}$$

when window size is the limiting factor.

Useful GATE numerical.

40. UNITS

If W is in bytes:

$$\frac{W}{\text{RTT}}$$

gives:

bytes/s.

Multiply by 8 for:

bits/s.

41. GATE NUMERICAL — WINDOW-LIMITED THROUGHPUT

Window:

$$64\text{KB}$$

Assume:

$$64\text{KB} = 64 \times 1024 = 65536\text{B}$$

RTT:

$$100\text{ms} = 0.1\text{s}$$

Maximum approximate throughput:

$$\frac{65536}{0.1}$$

$$= 655360\text{B/s}$$

Convert to bits:

$$655360 \times 8$$

$$= 5,242,880\text{bps}$$

Approximately:

$$\boxed{5.24\text{Mbps}}$$

Even if physical link is:

$$100\text{Mbps}$$

the window could limit achievable throughput substantially.

42. REQUIRED WINDOW FOR A TARGET RATE

Rearrange:

$$\text{Throughput} \approx \frac{W}{RTT}$$

So:

$$W \approx \text{Throughput} \times RTT$$

This is essentially the:

bandwidth-delay product.

43. EXAMPLE

Want:

$$100\text{Mbps}$$

over:

$$RTT=50\text{ms}$$

Required bits in flight:

$$100 \times 10^6 \times 0.05$$

$$= 5,000,000 \text{ bits}$$

Convert to bytes:

$$625,000 \text{ B}$$

So window should be at least around:

$$625\text{KB}$$

to sustain that rate under the simplified model.

44. CONNECTION TO BANDWIDTH-DELAY PRODUCT

Earlier we learned:

$$BDP = \text{Rate} \times \text{Delay}$$

For TCP end-to-end utilization, commonly:

$$BDP = \text{Bandwidth} \times RTT$$

This gives roughly:

amount of unacknowledged data needed "in flight" to fill the path.

If TCP window is much smaller than BDP:

link may be underutilized.

45. WINDOW NUMERICAL WITH SEGMENTS

Suppose:

$\text{rwnd} = 6000\text{B}$

MSS:

1000B

Ignoring congestion control, how many full MSS-sized segments could be outstanding?

$6000/1000 = 6$

If:

$\text{rwnd} = 6500\text{B}$

then:

- six full 1000-B segments
- plus up to 500 B additional data

subject to sender's segmentation decisions.

46. WINDOW IS NOT "NUMBER OF SEGMENTS"

Even if we calculate:

$6000/1000 = 6$

that doesn't change the underlying rule.

TCP receive window is:

6000 bytes

not:

six packets.

If segment sizes differ, number of outstanding segments can differ.

47. FULL NUMERICAL

Receiver sends:

$\text{ACK} = 5001$

$\text{rwnd} = 4000$

Sender has already transmitted data covering:

TEXT

5001–7000

How much more can sender transmit due to flow control?

Outstanding:

$\backslash [7000-5001+1 \backslash]$

$\backslash [=2000B \backslash]$

Available:

$\backslash [4000-2000 \backslash]$

$\backslash [=\boxed{2000B} \backslash]$

The advertised receive window covers:

$\backslash [5001 \backslash]$

through:

$\backslash [9000 \backslash]$

because:

$\backslash [5001+4000-1=9000 \backslash]$

48. GATE SHORTCUT — RANGE LENGTH

Inclusive sequence range:

$\backslash [a\text{text}\{ \text{through } \} b \backslash]$

contains:

$\backslash [\boxed{b-a+1} \backslash]$

bytes.

Don't forget **+1** .

Example:

TEXT

5001–7000

contains:

$\backslash [7000-5001+1 = 2000 \backslash]$

bytes.

49. WINDOW UPDATE EXAMPLE

Receiver initially:

$\backslash [ACK=1000 \backslash]$

$\backslash [rwnd=3000 \backslash]$

Allowed range:

$\backslash [1000-3999 \backslash]$

Receiver then gets first 1000 bytes but its application consumes none.

Those bytes occupy buffer.

If buffer was exactly 3000 bytes, remaining free space:

$\boxed{2000}$

ACK becomes:

$\boxed{2000}$

and receiver may advertise:

$\boxed{rwnd=2000}$

New window:

$\boxed{2000-3999}$

Notice the right edge did **not** move.

Why?

ACK advanced by 1000, but free space shrank by 1000.

50. WINDOW OPENS WHEN APP READS DATA

Now suppose application consumes those 1000 bytes.

Free space returns.

Receiver may advertise:

$\boxed{ACK=2000}$

$\boxed{rwnd=3000}$

New range:

$\boxed{2000-4999}$

Right edge advances.

This is a deeper understanding of how sliding windows work.

51. LEFT EDGE AND RIGHT EDGE

For advertised receive window:

Left edge

$\boxed{\text{ACK}}$

= first currently expected byte.

Right edge

Exclusive boundary:

$\boxed{ACK+rwnd}$

So valid byte sequence numbers are:

$\boxed{ACK}$

through:

$\backslash [\text{ACK} + \text{rwnd} - 1]$

52. WINDOW MOVEMENT

The left edge generally advances when:

contiguous data is received.

The right edge depends on both:

- ACK progress
- available receive-buffer space

So window size can:

- increase
- decrease
- become zero

depending on receiver state.

53. SILLY WINDOW SYNDROME — OPTIONAL FOUNDATION

There is a performance problem where very small amounts of buffer space/data are advertised or transmitted repeatedly.

This is called:

Silly Window Syndrome

Solutions include mechanisms such as:

- Clark's solution at receiver
- Nagle's algorithm at sender

But this is outside the central flow-control syllabus focus.

Do not recognize the term if encountered; don't prioritize it over `rwnd` numericals.

54. FLOW CONTROL vs ERROR CONTROL

Do not confuse them.

Flow control

Prevents:

Receiver buffer overflow.

Error/reliability mechanisms

Handle:

- loss
- corruption
- ordering
- retransmission.

A receiver window of zero does not mean a packet was corrupted.

55. FLOW CONTROL vs CONGESTION CONTROL

One of the most important TCP distinctions.

Flow Control	Congestion Control
Protects receiver	Protects network
Based on receiver buffer	Based on network congestion
Main variable: rwnd	Main variable: cwnd
Receiver advertises	Sender maintains/adjusts
Avoid receiver overflow	Avoid network overload

Memory:

R in rwnd = Receiver

C in cwnd = Congestion

56. EXAMPLE — WHICH LIMIT WINS?

Suppose:

$\backslash$ rwnd=8000B $\backslash$

$\backslash$ cwnd=3000B $\backslash$

Then sender is restricted approximately to:

$\backslash$ $\min(8000, 3000)$ $\backslash$

$\backslash$ $\boxed{3000\text{B}}$ $\backslash$

Network congestion limit wins.

57. ANOTHER EXAMPLE

$\backslash$ rwnd=2000B $\backslash$

$\backslash$ cwnd=10000B $\backslash$

Effective limit:

\[\boxed{2000B} \]

Receiver is the bottleneck.

58. FLOW-CONTROL ACK QUESTION

Receiver sends:

```
TEXT
ACK = 6000
Window = 0
```

This means:

ACK 6000

All contiguous bytes before 6000 received.

Window 0

Do not send additional normal data now; receive buffer currently has no advertised space.

These two fields are not contradictory.

59. IF WINDOW BECOMES ZERO, IS CONNECTION CLOSED?

'L No.

A zero receive window does **not** mean:

TCP connection terminated.

It means:

Temporarily pause new data because receiver has no space.

Connection remains established.

60. DOES ZERO WINDOW MEAN RECEIVER IS DEAD?

No.

The receiver may simply have:

- slow application
- temporarily full buffer

Eventually application can free buffer and window reopens.

61. HOW GATE CAN ASK FLOW CONTROL

Pattern 1 — Compute `rwnd`

$\lfloor \text{RcvBuffer-Occupied} \rfloor$

Pattern 2 — Allowed sequence range

Given:

$\lfloor \text{ACK}, \lfloor \text{rwnd} \rfloor$

find:

$\lfloor \text{ACK} \rfloor$ to $\text{ACK} + \text{rwnd} - 1$

Pattern 3 — Additional bytes sender can send

$\lfloor \text{rwnd} - \text{Outstanding} \rfloor$

Pattern 4 — Window-limited throughput

$\lfloor \frac{\text{Window}}{\text{RTT}} \rfloor$

Pattern 5 — Required window

$\lfloor \text{Bandwidth} \times \text{RTT} \rfloor$

Pattern 6 — Zero window

Ask what sender does.

Pause and use persist mechanism/probes as needed.

Pattern 7 — Flow vs congestion

`rwnd` vs `cwnd`.

62. & pGATE TRAPS

Trap 1

`rwnd` is measured in:

$\lfloor \boxed{\text{bytes}} \rfloor$

not packets.

Trap 2

ACK identifies:

next expected byte.

Trap 3

If:

$\backslash [\text{ACK}=\text{A}, \text{rwnd}=\text{W} \backslash]$

valid range ends at:

$\backslash [\text{A}+\text{W}-1 \backslash]$

not:

$\backslash [\text{A}+\text{W} \backslash]$

Trap 4

Do not send **rwnd** more **new** bytes if some bytes are already outstanding.

Outstanding data already occupies part of the sender's usable window.

Trap 5

Zero window does not close TCP connection.

Trap 6

Zero window is receiver-side flow control, not proof of network congestion.

Trap 7

Flow control protects receiver.

Congestion control protects network.

Trap 8

Actual sender limit is not always simply **rwnd** ; congestion window can be smaller.

Trap 9

Window Size field is 16 bits, but Window Scaling can produce larger effective windows.

63. MASTER MEMORY SYSTEM

Remember:

A-W-R

A — ACK

Where the window starts.

W — Window

How much room.

R — Range

$\boxed{\text{ACK} \rightarrow \text{ACK} + \text{rwnd} - 1}$

64. FLOW-CONTROL MEMORY

Receiver says: "I expect byte X, and I have room for W bytes."

That's exactly:

TEXT

ACK = X

rwnd = W

'p NOTEBOOK VERSION

TCP Flow Control

Purpose:

Prevent sender from overwhelming receiver buffer.

Main variable:

$\boxed{\text{rwnd}}$

Receiver advertises available buffer space.

Receive Window

Conceptually:

$\boxed{\text{rwnd} = \text{RcvBuffer} - \text{BufferedData}}$

ACK + Window

If:

$\lfloor \text{ACK} = A \rfloor$

and:

$\lfloor \text{rwnd} = W \rfloor$

acceptable range:

$\lfloor \boxed{A \text{ through } A+W-1} \rfloor$

ACK:

Next expected byte.

Window:

How many bytes receiver can currently accept.

Outstanding Data

Flow-control additional send capacity:

$\lfloor \boxed{\text{rwnd} - \text{OutstandingBytes}} \rfloor$

subject also to congestion control.

Effective TCP Send Limit

Conceptually:

$\lfloor \boxed{\min(\text{rwnd}, \text{cwnd})} \rfloor$

where:

- rwnd !' receiver capacity
- cwnd !' congestion/network capacity

Zero Window

$\lfloor \boxed{\text{rwnd} = 0} \rfloor$

means:

Receiver buffer has no advertised free space.

Sender pauses normal new-data transmission.

Persist timer / window probes prevent deadlock if a window update is lost.

Window Throughput

Simplified:

$\lfloor \boxed{\text{Throughput} \approx \frac{\text{Window}}{\text{RTT}}} \rfloor$

Required window to fill path:

$\boxed{\text{Window} \approx \text{Bandwidth} \times \text{RTT}}$

65. 30-SECOND REVISION

Remember:

Flow control protects receiver.

$\boxed{rwnd = \text{receiver's advertised free space}}$

If:

$\boxed{\text{ACK}=5000, rwnd=3000}$

window is:

$\boxed{5000-7999}$

If buffer becomes full:

$\boxed{rwnd=0}$

And later:

$\boxed{\text{ActualLimit} \approx \min(rwnd, cwnd)}$

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the main purpose of TCP flow control?

Q2

A receiver has:

$\boxed{10000\text{B}}$

buffer capacity, of which:

$\boxed{6500\text{B}}$

is occupied.

What approximate `rwnd` can it advertise?

Q3

Receiver sends:

$\boxed{\text{ACK}=8000}$

$\boxed{rwnd=5000}$

What sequence-number range does the advertised receive window cover?

Q4 — GATE Numerical

Receiver advertises:

\[rwnd=7000B \]

Sender currently has:

\[4500B \]

unacknowledged.

Ignoring congestion control, how many additional bytes can sender transmit?

Q5

What does:

TEXT

```
ACK = 12000  
window = 0
```

mean?

Q6

Why is a TCP persist timer useful after a receiver advertises zero window?

Q7 — Throughput

TCP window:

\[50KB \]

RTT:

\[100ms \]

Using:

\[1KB=1000B \]

for this question, what approximate maximum window-limited throughput is possible in bits/s?

Q8

A link has bandwidth:

\[20Mbps \]

and RTT:

\[50ms \]

Approximately how many bits should be allowed in flight to fully utilize it under the simple BDP model?

Q9 — Confusing Pair

What is the difference between:

- rwnd
-

- cwnd

?

Q10

If:

\[rwnd=10000B \]

and:

\[cwnd=4000B \]

what approximate sender window limit applies?

Q11 — GATE Trap

True or False:

If TCP's receive window becomes zero, the TCP connection is closed.

Explain.

Q12 — Sequence Range

Receiver sends:

\[ACK=20001,\quad rwnd=4096 \]

What is the largest sequence number currently inside the advertised receive window?

NEXT !' TCP Congestion Control Part 1: cwnd, ssthresh, Slow Start, Congestion Avoidance & Congestion-Window Growth Numericals

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 3 — TCP Congestion Control Part 1: cwnd, ssthresh, Slow Start & Congestion Avoidance

EXTREMELY HIGH PRIORITY FOR GATE

TCP flow control protects:

Receiver

TCP congestion control protects:

Network

The two most important variables are:

$\boxed{cwnd}$

and

$\boxed{ssthresh}$

where:

- $cwnd$ = **Congestion Window**
- $ssthresh$ = **Slow-Start Threshold**

The core idea is:

TCP starts cautiously, increases its sending rate, watches for congestion, and reduces the rate when congestion is inferred.

1. WHAT IS NETWORK CONGESTION?

Suppose many senders inject too much traffic into a network:

TEXT

```
S e n d e r   A   % %   \
S e n d e r   B   % % % ! '   R o u t e r   % % ! '   O u t p u t   L i n k
S e n d e r   C   % %   /
```

The router's outgoing link has limited capacity.

Packets begin accumulating in its queue.

If traffic continues increasing:

- queueing delay increases
- buffers fill
- packets are dropped

This condition is:

Network congestion

2. FLOW CONTROL vs CONGESTION CONTROL

Memorize this first.

Flow Control	Congestion Control
Protects receiver	Protects network
Receiver may be slow	Routers/links may be overloaded

Flow Control	Congestion Control
Uses rwnd	Uses cwnd
Receiver advertises limit	Sender adjusts limit

Memory:

R !' rwnd !' Receiver

C !' cwnd !' Congestion

3. WHAT IS cwnd ?

cwnd means:

Congestion Window

It is maintained by the TCP sender.

It limits how much unacknowledged data the sender can have in flight based on its estimate of network capacity/congestion.

Conceptually:

$\boxed{\text{Outstanding data} \leq \text{cwnd}}$

But flow control also exists.

So more accurately:

$\boxed{\text{Outstanding} \leq \min(\text{cwnd}, \text{rwnd})}$

4. EXAMPLE

Suppose:

$\text{cwnd} = 8000\text{B}$

and:

$\text{rwnd} = 12000\text{B}$

Then network congestion control is more restrictive.

Effective window:

$\min(8000, 12000)$

$= \boxed{8000\text{B}}$

5. ANOTHER EXAMPLE

$\text{cwnd} = 20000\text{B}$

$\lceil \text{rwnd}=5000\text{B} \rceil$

Then:

$\lceil \boxed{\text{EffectiveWindow}=5000\text{B}} \rceil$

Receiver limitation wins.

6. WHY DOES TCP NEED TO DISCOVER NETWORK CAPACITY?

The sender normally doesn't know in advance:

- bottleneck bandwidth
- number of competing flows
- router buffer occupancy
- current congestion

So it probes the network.

Basic strategy:

Start relatively small !' increase !' observe feedback.

Loss or other congestion signals indicate:

You may be sending too aggressively.

7. THE TWO MAIN GROWTH PHASES

Classical TCP congestion control has two important growth modes:

1. Slow Start
2. Congestion Avoidance

The boundary is controlled using:

$\lceil \boxed{\text{ssthresh}} \rceil$

8. WHAT IS **ssthresh**?

ssthresh means:

Slow-Start Threshold

It determines when TCP should transition from aggressive exponential growth toward slower linear growth.

Broad rule:

If:

$\lceil \text{cwnd} < \text{ssthresh} \rceil$

use:

Slow Start

Once the threshold is reached/crossed:

use:

Congestion Avoidance

For textbook/GATE questions, follow the exact convention stated in the problem if it specifies whether equality belongs to one phase or the other.

9. 0>MEMORY

TEXT

```
B e l o w   t h r e s h o l d   ! '   G r o w   F A S T
A f t e r   t h r e s h o l d   ! '   G r o w   S L O W
```

Ironically:

Slow Start is actually the faster-growing phase.

That's a famous naming trap.

10. WHY IS IT CALLED SLOW START THEN?

Because historically TCP starts with a relatively small congestion window instead of immediately transmitting a huge amount of data.

But once it starts receiving ACKs, the congestion window grows exponentially per RTT.

So:

Slow initial start, fast subsequent growth.

PART A — SLOW START

11. SLOW START GROWTH

During Slow Start, `cwnd` approximately doubles every RTT under ideal ACK behavior.

For example, if initial:

`\[ cwnd=1\ MSS \]`

then approximately:

TEXT

R T T	0	!	'	1	M S S
R T T	1	!	'	2	M S S
R T T	2	!	'	4	M S S
R T T	3	!	'	8	M S S
R T T	4	!	'	1 6	M S S

So:

$\lfloor \boxed{1,2,4,8,16,\dots} \rfloor$

$\Theta = \sqrt{cwnd}$ is very important.

12. WHY DOES IT DOUBLE?

In Slow Start, a simplified rule is:

For each ACK acknowledging new data, increase `cwnd` by about 1 MSS.

Suppose initial window is:

$\lfloor 1 \text{ MSS} \rfloor$

Send one segment.

One ACK arrives.

Window increases to about:

$\lfloor 2 \text{ MSS} \rfloor$

Next RTT, send two segments.

Each ACK increases `cwnd` by 1 MSS.

After two ACKs:

$\lfloor 2+2=4 \text{ MSS} \rfloor$

Hence roughly doubling per RTT.

13. MSS RECALL

MSS means:

Maximum Segment Size

It refers to TCP payload.

Example under common Ethernet/IPv4 assumptions:

$\lfloor \text{MTU}=1500\text{B} \rfloor$

$\lfloor \text{IPHeader}=20\text{B} \rfloor$

$\lfloor \text{TCPHeader}=20\text{B} \rfloor$

then:

$[\text{MSS}=1460 \text{B}]$

So if:

$[\text{cwnd}=4\text{MSS}]$

then in bytes:

$[4 \times 1460 = \boxed{5840\text{B}}]$

14. GATE NUMERICAL — SLOW START

Initial:

$[\text{cwnd}=1\text{MSS}]$

No loss.

What is `cwnd` after 5 RTT growth rounds?

Sequence:

TEXT

S	t	a	r	t	!	'	1
1	R	T	T	!	'	2	
2	R	T	T	!	'	4	
3	R	T	T	!	'	8	
4	R	T	T	!	'	16	
5	R	T	T	!	'	32	

Therefore:

$[\boxed{32\text{MSS}}]$

assuming the connection remains in Slow Start throughout.

15. & pGATE TRAP — COUNTING RTTs

If question says:

Initial `cwnd` = 1 MSS. What is `cwnd` after 3 RTTs?

Answer:

$[1 \rightarrow 2 \rightarrow 4 \rightarrow 8]$

So:

$[\boxed{8\text{MSS}}]$

Do not answer 4 by counting the initial state as RTT 1 unless the problem defines rounds differently.

16. GENERAL SLOW-START FORMULA

If:

$[\text{cwnd}_0 = W_0]$

and ideal doubling continues for (k) RTTs:

$$cwnd_k = W_0 2^k$$

Example:

$$W_0 = 2MSS$$

after 4 RTTs:

$$2 \times 2^4 = 32MSS$$

17. PACKETS SENT PER ROUND

Suppose:

$$cwnd = 1MSS$$

at start.

Segments allowed per ideal RTT round:

TEXT

R o u n d	1	!	1
R o u n d	2	!	2
R o u n d	3	!	4
R o u n d	4	!	8

Total segments sent over first (k) complete rounds forms a geometric sum.

For $(1, 2, 4, \dots, 2^{k-1})$:

$$\boxed{\text{Total} = 2^k - 1}$$

This can be useful in GATE.

18. EXAMPLE

How many full-MSS segments are transmitted during the first 4 Slow-Start rounds starting at 1 MSS?

$$1 + 2 + 4 + 8$$

$$= \boxed{15}$$

Using:

$$2^4 - 1 = 15$$

PART B — ssthresh

19. EXAMPLE WITH THRESHOLD

Suppose:

$$cwnd_{\text{initial}} = 1MSS$$

$$ssthresh = 8MSS$$

Growth:

TEXT

1
2
4
8

At approximately the threshold:

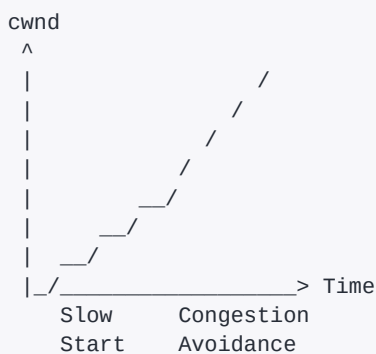
TCP transitions toward Congestion Avoidance.

So later growth becomes linear rather than exponential.

20. GRAPH SHAPE

Conceptually:

TEXT



More accurately:

- Slow Start !' exponential increase
- Congestion Avoidance !' approximately linear increase

PART C — CONGESTION AVOIDANCE

21. CONGESTION AVOIDANCE

In Congestion Avoidance, TCP grows more cautiously.

Approximate textbook rule:

$\boxed{\text{cwnd increases by } 1 \text{ MSS per RTT}}$

So if:

$\boxed{\text{cwnd} = 8\text{MSS}}$

then ideal progression:

TEXT

8 !' 9 !' 1 0 !' 1 1 !' 1 2 . . .

instead of:

TEXT

8 !' 1 6 !' 3 2 . . .

22. WHY SLOWER GROWTH?

Once TCP has reached a substantial sending rate, continuing exponential growth would quickly overload the network.

So it probes more carefully:

Additive increase.

This is part of the classic:

AIMD

idea.

23. AIMD

AIMD means:

Additive Increase, Multiplicative Decrease

Additive Increase

During Congestion Avoidance:

$\lfloor \text{cwnd} \rfloor$

grows roughly:

$\lfloor +1\text{MSS} \rfloor$

per RTT.

Multiplicative Decrease

When congestion is detected:

reduce sending window significantly, commonly related to halving.

We'll study the exact reactions to timeout vs duplicate ACKs next.

24. ~~O~~ MEMORY

No congestion !' cautiously add.

Congestion !' cut significantly.

That's AIMD.

25. CONGESTION AVOIDANCE PER-ACK RULE

A more detailed textbook approximation is:

For each ACK:

$$\boxed{cwnd \leftarrow cwnd + \frac{MSS^2}{cwnd}}$$

Why?

If roughly:

$$cwnd/MSS$$

segments are acknowledged per RTT, total increase over one RTT is approximately:

$$1MSS$$

26. INTUITION

Suppose:

$$cwnd = 10MSS$$

There are about 10 full-sized segments in a window.

Each ACK increases `cwnd` by about:

$$\frac{MSS}{10}$$

Across 10 ACKs:

$$10 \times \frac{MSS}{10} = 1MSS$$

So after one RTT:

$$10MSS \rightarrow 11MSS$$

approximately.

27. FOR GATE, WHICH RULE TO USE?

If the question gives `cwnd` in MSS units and asks RTT-by-RTT growth:

Use the usual simplified rules:

Slow Start

$$\boxed{\times 2 \text{ per RTT}}$$

Congestion Avoidance

$\boxed{+1\text{MSS per RTT}}$

unless the question explicitly asks for per-ACK calculation.

28. FULL GATE NUMERICAL

Given:

$\boxed{\text{cwnd}_0 = 1\text{MSS}}$

$\boxed{\text{ssthresh} = 8\text{MSS}}$

No packet loss.

Find $\boxed{\text{cwnd}}$ after 7 RTTs.

Start:

$\boxed{1}$

RTT 1

$\boxed{2}$

RTT 2

$\boxed{4}$

RTT 3

$\boxed{8}$

Threshold reached.

Now Congestion Avoidance.

RTT 4

$\boxed{9}$

RTT 5

$\boxed{10}$

RTT 6

$\boxed{11}$

RTT 7

$\boxed{12}$

Therefore:

$\boxed{\text{cwnd} = 12\text{MSS}}$

under this standard simplified convention.

29. & ssthresh CONVENTION

Some textbook formulations say Slow Start continues while:

$\boxed{\text{cwnd} < \text{ssthresh}}$

and switches when:

$\lfloor \text{cwnd} \geq \text{ssthresh} \rfloor$

Others may phrase the exact transition slightly differently.

For GATE:

Follow the stated algorithm/graph in the question if equality handling matters.

Usually the expected qualitative rule is clear.

30. EXAMPLE WITH DIFFERENT INITIAL WINDOW

Suppose:

$\lfloor \text{cwnd}_0 = 2\text{MSS} \rfloor$

$\lfloor \text{ssthresh} = 16\text{MSS} \rfloor$

Growth:

TEXT

```
Start = 2
RTT1 = 4
RTT2 = 8
RTT3 = 16
RTT4 = 17
RTT5 = 18
```

Thus after five RTTs:

$\lfloor \boxed{18\text{MSS}} \rfloor$

under the simplified transition rule.

31. NUMBER OF RTTs TO REACH THRESHOLD

Suppose:

$\lfloor \text{cwnd}_0 = 1\text{MSS} \rfloor$

and:

$\lfloor \text{ssthresh} = 32\text{MSS} \rfloor$

Slow Start:

$\lfloor 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rfloor$

Number of doublings:

$\lfloor 5 \rfloor$

Because:

$\lfloor 2^5 = 32 \rfloor$

So:

$\lfloor \boxed{5 \text{ RTTs}} \rfloor$

to grow from 1 MSS to 32 MSS.

32. GENERAL FORM

If ideal powers line up exactly:

$$cwnd_0 \cdot 2^k = ssthresh$$

then:

$$k = \log_2 \left(\frac{ssthresh}{cwnd_0} \right)$$

33. EXAMPLE

$$cwnd_0 = 2$$

$$ssthresh = 64$$

$$k = \log_2(64/2)$$

$$= \log_2(32)$$

$$= 5$$

RTTs.

34. HOW MANY SEGMENTS CAN BE OUTSTANDING?

If:

$$cwnd = 12MSS$$

and:

$$MSS = 1000B$$

then congestion-control window:

$$12000B$$

If:

$$rwnd = 20000B$$

then effective window:

$$\min(12000, 20000) = 12000B$$

So up to approximately:

$$12$$

full MSS-sized segments may be outstanding.

35. IF RECEIVER WINDOW IS SMALLER

Same:

$$cwnd = 12MSS$$

$\lfloor \text{MSS}=1000\text{B} \rfloor$

so:

$\lfloor \text{cwnd}=12000\text{B} \rfloor$

But receiver advertises:

$\lfloor \text{rwnd}=5000\text{B} \rfloor$

Then effective:

$\lfloor \boxed{5000\text{B}} \rfloor$

So congestion window can be large but flow control still restricts sender.

36. CONGESTION WINDOW IS NOT ROUTER BUFFER

& pGATE trap.

cwnd is:

Sender-maintained TCP state.

It is **not**:

- router queue size
- receiver buffer size
- physical link capacity

It is an estimate/control variable used to regulate network load.

37. CONGESTION SIGNALS

Classic TCP primarily interprets packet loss as congestion evidence.

Loss can be inferred from things such as:

1. Retransmission timeout
2. Duplicate ACKs

These do not trigger identical reactions in all TCP variants.

That's exactly what we'll cover next.

38. WHY DIFFERENT LOSS SIGNALS?

A timeout often suggests:

serious congestion / very little progress.

Multiple duplicate ACKs suggest:

packets are still flowing, but one segment was probably lost.

So TCP can react less drastically to duplicate ACKs than to a timeout in variants such as TCP Reno.

39. ssthresh AFTER CONGESTION — PREVIEW

When congestion is detected, a typical classical rule sets:

$\boxed{\text{ssthresh} \approx \frac{\text{cwnd}}{2}}$

using the flight size/current window according to the exact algorithm.

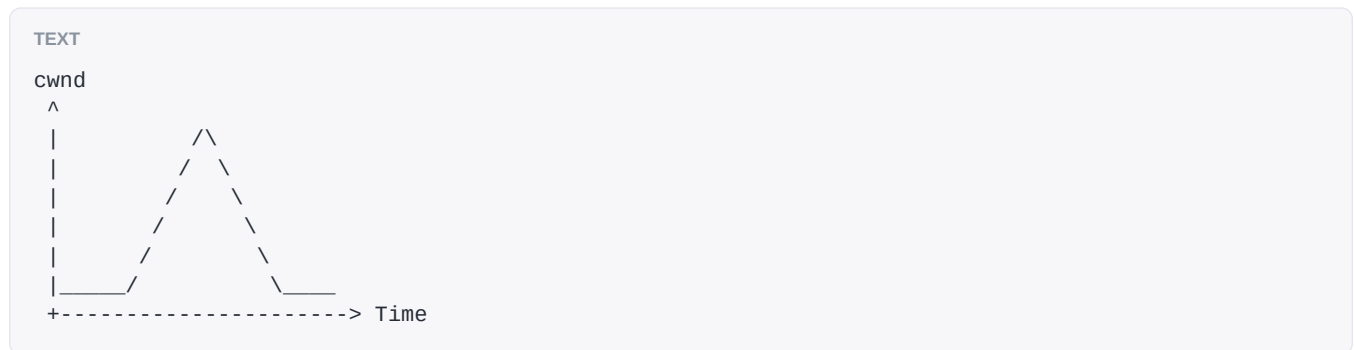
Then what happens to `cwnd` depends on whether loss was detected by:

- timeout
- triple duplicate ACK

We'll do this precisely next.

40. SAWTOOTH BEHAVIOR

TCP Congestion Avoidance often creates a characteristic sawtooth:



TCP:

1. increases gradually
2. detects congestion
3. reduces window
4. increases again

This is AIMD behavior.

41. WHY AIMD HELPS FAIRNESS

Suppose multiple TCP connections share a bottleneck.

If each:

- increases additively
- decreases multiplicatively

their rates can tend toward a fairer sharing of the bottleneck under ideal assumptions.

You do not need formal fairness proofs for this syllabus, but know:

AIMD is used because it provides stable distributed congestion adaptation and useful fairness properties.

42. SLOW START GRAPH

If initial:

$\lfloor 1\text{MSS} \rfloor$

then:

TEXT

```
RTT : 0  1  2  3  4
cwnd: 1  2  4  8 16
```

Plot shape:

exponential.

43. CONGESTION AVOIDANCE GRAPH

Starting:

$\lfloor 16\text{MSS} \rfloor$

TEXT

```
RTT : 0  1  2  3  4
cwnd:16 17 18 19 20
```

Plot:

approximately linear.

44. & p"SLOW START" DOES NOT MEAN LINEAR

This is a classic MCQ trap.

Slow Start

$\lfloor \boxed{\text{Exponential growth}} \rfloor$

Congestion Avoidance

$\lfloor \boxed{\text{Linear growth}} \rfloor$

Names can mislead you.

45. GATE NUMERICAL — DATA SENT DURING SLOW START

Initial:

$\lfloor \text{cwnd}=1\text{MSS} \rfloor$

For first four transmission rounds, sender can send:

$$[1+2+4+8]$$

$$[=15\text{MSS}]$$

If:

$$[\text{MSS}=1000\text{B}]$$

then total:

$$[15 \times 1000]$$

$$[=\boxed{15000\text{B}}]$$

assuming each round fully uses the window and ignoring handshake/details.

46. MIXED PHASE NUMERICAL

Suppose:

$$[\text{cwnd}_0=1\text{MSS}]$$

$$[\text{ssthresh}=4\text{MSS}]$$

Window at starts of successive transmission rounds:

TEXT

Round 1: 1
Round 2: 2
Round 3: 4
Round 4: 5
Round 5: 6

Total full MSS segments sent over these 5 rounds:

$$[1+2+4+5+6]$$

$$[=\boxed{18}]$$

This kind of GATE problem is easier if you draw a table.

47. GATE TABLE METHOD

Always create:

RTT/Round	Phase	cwnd
0	Slow Start	1
1	Slow Start	2
2	Slow Start	4
3	Congestion Avoidance	8
4	Congestion Avoidance	9

Then continue.

This prevents mixing exponential and linear growth.

48. PHASE IDENTIFICATION FROM GRAPH

If graph shows:

TEXT

1, 2, 4, 8, 16

that's:

$\boxed{\text{Slow Start}}$

If it then shows:

TEXT

16, 17, 18, 19, 20

that's:

$\boxed{\text{Congestion Avoidance}}$

If a sudden drop occurs:

congestion/loss was detected.

The magnitude of the drop can indicate what event/variant occurred.

We'll learn that next.

49. CONNECTION TO ACK CLOCKING

TCP window growth depends on acknowledgements returning.

ACKs act as feedback that data has left the network path and reached the receiver.

This creates an approximate:

ACK clock.

You don't need deep control-theory detail.

Just know:

ACK arrival drives TCP's sending/growth behavior.

50. WHAT IF ACKs ARE DELAYED?

Real TCP receivers may use delayed acknowledgements.

Therefore actual growth may not be exactly textbook doubling per RTT in every real implementation.

For GATE, unless the question specifies otherwise:

use the standard simplified congestion-control model.

51. INITIAL `cwnd`

Historically TCP used very small initial congestion windows such as 1 MSS.

Modern TCP standards commonly permit larger initial windows.

But many exam questions explicitly state an initial window or use a textbook model.

Therefore:

Never assume a modern real-world initial `cwnd` if the GATE question gives a value. Follow the question.

If no value is provided, use the model implied by the problem/context rather than mixing standards.

52. WHY THIS MATTERS FOR GATE

A question may say:

"Initially `cwnd` = 1 MSS."

Then use:

TEXT

1 !' 2 !' 4 !' 8 . . .

Do not replace it with a modern implementation default you happen to know.

53. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Identify variable

`cwnd` means:

Congestion Window.

Pattern 2 — Slow Start growth

Find window after $\backslash(n)$ RTTs.

Pattern 3 — Threshold transition

Given:

- initial `cwnd`
- `ssthresh`

find when Congestion Avoidance begins.

Pattern 4 — Congestion Avoidance

Add:

$\lfloor 1\text{MSS}/\text{RTT} \rfloor$

in simplified model.

Pattern 5 — Segments transmitted

Sum window sizes over rounds.

Pattern 6 — Effective sending window

$\lfloor \min(\text{cwnd}, \text{rwnd}) \rfloor$

Pattern 7 — Identify graph phase

Exponential vs linear.

54. & pGATE TRAPS

Trap 1

Slow Start is:

$\lfloor \boxed{\text{exponential}} \rfloor$

not linear.

Trap 2

Congestion Avoidance is approximately:

$\lfloor \boxed{+1\text{MSS}/\text{RTT}} \rfloor$

not doubling.

Trap 3

`ssthresh` is not the maximum congestion window.

It marks the transition between growth modes.

Trap 4

`cwnd` protects the:

network,

not receiver.

Trap 5

`rwnd` and `cwnd` are different.

Actual limit roughly uses:

$\lfloor \min(rwnd, cwnd) \rfloor$

Trap 6

When counting RTTs, don't count the initial value as if one RTT already passed unless the question labels rounds that way.

Trap 7

Slow Start's doubling is a per-RTT approximation produced by per-ACK increases.

Trap 8

Do not apply congestion-loss reduction rules yet unless the problem tells you what event occurred.

Timeout and duplicate ACK loss signals can cause different `cwnd` behavior.

55. MASTER MEMORY SYSTEM

Remember:

S-T-C

S — Slow Start

$\lfloor \boxed{\times 2} \text{ per RTT} \rfloor$

T — Threshold

$\lfloor \boxed{ssthresh} \rfloor$

C — Congestion Avoidance

$\lfloor \boxed{+1MSS} \text{ per RTT} \rfloor$

56. SECOND MEMORY TRICK

Think:

Below threshold !' multiply

After threshold !' add

That's enough for most basic GATE numericals.

' p NOTEBOOK VERSION

TCP Congestion Control

Purpose:

Prevent sender from overloading the network.

Main sender variable:

$\boxed{cwnd}$

Receiver flow-control variable:

$\boxed{rwnd}$

Effective outstanding-data limit:

$\boxed{\min(cwnd, rwnd)}$

Slow Start

Used when congestion window is below slow-start threshold in the standard model.

Growth:

$\boxed{\text{approximately doubles every RTT}}$

Example:

TEXT

1 !' 2 !' 4 !' 8 !' 16

General:

$\boxed{cwnd_k = cwnd_0 \cdot 2^k}$

while Slow Start continues.

ssthresh

Slow-Start Threshold.

Marks transition toward Congestion Avoidance.

Congestion Avoidance

Growth approximately:

$\lceil \boxed{+1\text{MSS per RTT}} \rceil$

Example:

TEXT

8 !' 9 !' 10 !' 11 !' 12

Detailed per-ACK approximation:

$\lceil \boxed{cwnd \leftarrow cwnd + \frac{\text{MSS}^2}{cwnd}} \rceil$

AIMD

$\lceil \boxed{\text{Additive Increase, Multiplicative Decrease}} \rceil$

No congestion:

increase.

Congestion:

reduce window substantially.

Exact loss reactions come next.

57. 30-SECOND REVISION

Remember:

Flow control !' `rwnd` !' receiver

Congestion control !' `cwnd` !' network

Then:

TEXT

Slow Start:

1, 2, 4, 8, 16...

$\lceil \boxed{\text{exponential}} \rceil$

After `ssthresh`:

TEXT

16, 17, 18, 19...

$\lceil \boxed{\text{linear}} \rceil$

And actual sender limit is roughly:

$\lceil \boxed{\min(cwnd, rwnd)} \rceil$

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the purpose of TCP congestion control?

Q2

What is the difference between:

- `cwnd`
- `rwnd`

?

Q3

Initial:

`\[ cwnd=1MSS \]`

TCP remains in Slow Start for 4 RTTs.

What is `cwnd` after those 4 RTTs?

Q4

Initial:

`\[ cwnd=2MSS \]`

After 3 Slow-Start RTTs, what is `cwnd` ?

Q5 — Threshold

Given:

`\[ cwnd_0=1MSS \]`

`\[ ssthresh=8MSS \]`

Write the `cwnd` values from the start through the next 5 RTTs under the simplified model.

Q6

During Congestion Avoidance, by approximately how much does `cwnd` increase per RTT?

Q7 — Effective Window

`\[ cwnd=12KB \]`

`\[ rwnd=20KB \]`

What approximate effective sender window applies?

Q8

\[cwnd=32KB \]

\[rwnd=8KB \]

Which mechanism is currently limiting the sender?

Q9 — Segments Sent

Initial `cwnd = 1 MSS`.

How many total full-MSS segments can be sent during the first 5 ideal Slow-Start transmission rounds?

Q10 — GATE Trap

True or False:

Slow Start increases `cwnd` linearly because its name contains the word “slow.”

Explain.

Q11

What does `ssthresh` represent?

Q12 — Preview

If a TCP sender detects congestion through:

1. a retransmission timeout
2. three duplicate ACKs

should it necessarily react identically in classic TCP Reno-style congestion control?

NEXT !' TCP Congestion Control Part 2: Timeout vs Triple Duplicate ACK, Fast Retransmit, Fast Recovery, `ssthresh` Updates & Reno/Tahoe GATE Numericals

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 4 — TCP Congestion Control Part 2: Timeout vs Triple Duplicate ACK, Fast Retransmit, Fast Recovery, Tahoe vs Reno

EXTREMELY HIGH PRIORITY FOR GATE

This topic is where many TCP congestion-control numericals are asked.

You already know:

Slow Start

$\boxed{\text{Exponential growth}}$

Congestion Avoidance

$\boxed{\text{Linear growth}}$

Now we answer:

What happens when TCP detects packet loss?

The answer depends on **how the loss was detected** and **which TCP variant the question uses**.

The most important distinction:

Timeout = severe congestion signal

Triple Duplicate ACK = packets are still reaching receiver, so congestion may be less severe

1. TWO COMMON LOSS SIGNALS

TCP can infer packet loss mainly through:

1. Retransmission Timeout (RTO)
2. Three Duplicate ACKs

These signals are treated differently in classic TCP Reno.

2. WHAT IS A TIMEOUT?

Suppose sender transmits data but does not receive the expected acknowledgement within its retransmission timeout.

Conceptually:

```
TEXT
Sender                      Receiver
Segment ----->
      ... no useful ACK ...
Timer expires
      !
Retransmit
```

A timeout suggests:

The network may be heavily congested or progress has stopped.

Therefore TCP reacts strongly.

3. TIMEOUT — CLASSIC TEXTBOOK RESPONSE

In the traditional GATE-style model:

If loss is detected by timeout:

1. Set:

$\boxed{ssthresh = \frac{\text{current congestion window}}{2}}$

2. Reset:

$\boxed{cwnd = 1MSS}$

3. Enter:

Slow Start

$\frac{cwnd}{MSS} = 1$

4. EXAMPLE — TIMEOUT

Before loss:

$\boxed{cwnd = 16MSS}$

Timeout occurs.

Then:

$ssthresh = \frac{16}{2} = \boxed{8MSS}$

and:

$\boxed{cwnd = 1MSS}$

Growth restarts:

TEXT

1 !' 2 !' 4 !' 8

then Congestion Avoidance:

TEXT

9 !' 10 !' 11 . . .

5. ~~0~~ > MEMORY

Timeout:

Cut threshold in half, restart window from 1.

Use:

T !' 1

Timeout !' cwnd = 1 MSS

for classic textbook Tahoe/Reno loss handling.

6. WHY IS TIMEOUT TREATED HARSHLY?

If ACKs stop arriving, TCP has weak evidence that the network is successfully delivering later packets.

So sender assumes:

Congestion may be serious.

It drastically reduces load.

PART A — DUPLICATE ACKS

7. WHAT IS A DUPLICATE ACK?

Recall cumulative ACKs.

Suppose receiver expects:

\[Seq=2000 \]

but gets later data while byte 2000 is missing.

Example:

TEXT

```
1 0 0 0 - 1 4 9 9 '
1 5 0 0 - 1 9 9 9 '
2 0 0 0 - 2 4 9 9 'L
2 5 0 0 - 2 9 9 9 '
3 0 0 0 - 3 4 9 9 '
3 5 0 0 - 3 9 9 9 '
```

Receiver is still waiting for:

\[2000 \]

So it repeatedly sends:

\[\boxed{ACK=2000} \]

These repeated identical ACK numbers are:

Duplicate ACKs

8. WHY DO DUPLICATE ACKS TELL US SOMETHING?

If later segments arrive:

TEXT

2500–2999
3000–3499
3500–3999

then packets are still moving through the network.

So:

Network is not completely stalled.

Likely one earlier segment was lost.

This is why TCP can retransmit before waiting for a timeout.

9. FAST RETRANSMIT

When the sender receives:

`\boxed{3\text{ duplicate ACKs}}\]`

classic TCP infers a segment was likely lost and retransmits it immediately.

This mechanism is:

Fast Retransmit

Very important.

10. WHY “FAST”?

Without duplicate ACK logic:

Sender may wait for retransmission timer to expire.

With three duplicate ACKs:

Retransmit immediately.

So recovery happens sooner.

11. EXAMPLE

Sender transmitted:

TEXT

Seq 1000
Seq 2000
Seq 3000

Seq 4000
Seq 5000

Suppose segment beginning at:

$\lfloor 2000 \rfloor$

is lost.

Receiver gets later segments and repeatedly sends:

$\lfloor \text{ACK}=2000 \rfloor$

After sender sees three duplicate ACK=2000 messages:

Retransmit segment beginning at sequence 2000.

12. & pGATE TRAP

Three duplicate ACKs means:

Three additional ACKs with the same acknowledgement number after the original ACK.

Be careful if a question explicitly counts ACK messages.

Normally textbook terminology:

$\lfloor \boxed{\text{3 duplicate ACKs}} \rfloor$

triggers Fast Retransmit.

PART B — TCP TAHOE

13. TCP TAHOE

Classic TCP Tahoe includes:

- Slow Start
- Congestion Avoidance
- Fast Retransmit

But after detecting loss—whether by:

- timeout
- triple duplicate ACK

Tahoe behaves aggressively.

It sets:

$\lfloor ssthresh \approx \frac{cwnd}{2} \rfloor$

and then:

$\lfloor \boxed{cwnd=1MSS} \rfloor$

Then enters:

Slow Start.

14. TAHOE RULE

Timeout

$\lfloor \text{ssthresh} = \text{cwnd} / 2 \rfloor$

$\lfloor \text{cwnd} = 1 \rfloor$

Triple duplicate ACK

$\lfloor \text{ssthresh} = \text{cwnd} / 2 \rfloor$

$\lfloor \text{cwnd} = 1 \rfloor$

So Tahoe's key property is:

Any detected loss sends cwnd back to 1 MSS.

15. Ø>TAHOE MEMORY

Tahoe falls to the bottom.

Whenever it sees loss:

$\lfloor \boxed{\text{cwnd} \rightarrow 1} \rfloor$

PART C — TCP RENO

16. TCP RENO IMPROVEMENT

TCP Reno asks:

If I received three duplicate ACKs, later packets are clearly still reaching the receiver. Why restart completely from 1 MSS?

So Reno handles triple duplicate ACKs less drastically.

It introduces:

Fast Recovery

17. RENO — TIMEOUT

For a timeout, Reno still reacts strongly.

Textbook rule:

$$\boxed{ssthresh = \frac{cwnd}{2}}$$

$$\boxed{cwnd = 1MSS}$$

Then:

Slow Start.

So Tahoe and Reno react similarly to:

$$\boxed{\text{Timeout}}$$

18. RENO — TRIPLE DUPLICATE ACK

This is where Reno differs.

On 3 duplicate ACKs:

1. Perform Fast Retransmit.
2. Set:

$$\boxed{ssthresh \approx \frac{\text{current flight size or cwnd}}{2}}$$

3. Enter Fast Recovery instead of restarting from 1.

In simplified GATE questions, after recovery the congestion window is often treated as approximately:

$$\boxed{ssthresh}$$

and Congestion Avoidance continues.

19. IMPORTANT TEXTBOOK SIMPLIFICATION

For many GATE numericals, use this simplified Reno rule:

Timeout

$$\boxed{ssthresh = cwnd/2}$$

$$\boxed{cwnd = 1}$$

Triple duplicate ACK

$$\boxed{ssthresh = cwnd/2}$$

$$\boxed{cwnd = cwnd/2}$$

then continue with:

Congestion Avoidance.

$\frac{cwnd}{2}$ is often the expected exam-level model.

20. REAL FAST RECOVERY DETAIL

More precisely in classic Reno Fast Recovery:

After three duplicate ACKs:

$$\lfloor ssthresh = \frac{\text{FlightSize}}{2} \rfloor$$

and `cwnd` is temporarily inflated, commonly to approximately:

$$\lfloor ssthresh + 3MSS \rfloor$$

because the three duplicate ACKs indicate three segments have left the network.

For each additional duplicate ACK, `cwnd` may increase further.

When an ACK acknowledging the retransmitted lost data arrives:

$$\lfloor cwnd \rightarrow ssthresh \rfloor$$

and Reno enters:

Congestion Avoidance.

For GATE:

Use the detailed rule only if the question explicitly models Fast Recovery ACK-by-ACK.

Otherwise, the simplified "halve on triple duplicate ACK" model is usually what is intended.

21. RENO EXAMPLE — SIMPLIFIED

Suppose before loss:

$$\lfloor cwnd = 20MSS \rfloor$$

Three duplicate ACKs arrive.

Then:

$$\lfloor ssthresh = \frac{20}{2} = 10MSS \rfloor$$

Fast retransmit lost segment.

After recovery, treat:

$$\lfloor \boxed{cwnd = 10MSS} \rfloor$$

Then Congestion Avoidance:

TEXT

1 0 !' 1 1 !' 1 2 !' 1 3 . . .

22. SAME EXAMPLE WITH TIMEOUT

Before timeout:

$$\lfloor cwnd = 20MSS \rfloor$$

Then:

$\lfloor \text{ssthresh} = 10\text{MSS} \rfloor$

but:

$\lfloor \boxed{\text{cwnd} = 1\text{MSS}} \rfloor$

Growth:

TEXT

1 !' 2 !' 4 !' 8

Then near threshold:

TEXT

1 0 !' 1 1 !' 1 2 . . .

depending on the exact textbook threshold transition convention.

23. TAHOE vs RENO — MASTER TABLE

Event	Tahoe	Reno
Timeout	ssthresh = half, cwnd = 1	ssthresh = half, cwnd = 1
3 duplicate ACKs	ssthresh = half, cwnd = 1	Fast Retransmit + Fast Recovery
After triple dup ACK	Slow Start	Congestion Avoidance after recovery
Fast Recovery	No	Yes

Definitely notebook-worthy.

24. O > MEMORY

Tahoe

Any loss !' Start over

Reno

Timeout !' Start over

3 duplicate ACKs !' Half and recover

This one memory line solves many MCQs.

25. WHY IS RENO BETTER FOR SINGLE PACKET LOSS?

Three duplicate ACKs mean:

Data continues flowing.

So instead of reducing:

$\lfloor 16 \rfloor$

Reno may effectively reduce around:

$\lfloor 16 \rfloor$

after recovery.

This avoids unnecessarily destroying throughput.

26. CONGESTION WINDOW GRAPH — TIMEOUT

Suppose:

TEXT

1 !' 2 !' 4 !' 8 !' 16

Then timeout.

Window drops:

TEXT

16 !' 1

New threshold:

$\lfloor 8 \rfloor$

Then:

TEXT

1 !' 2 !' 4 !' 8 !' 9 !' 10 . . .

Graph shows a:

sharp drop to bottom.

27. CONGESTION WINDOW GRAPH — RENO TRIPLE DUP ACK

Suppose:

$\lfloor cwnd=16 \rfloor$

Three duplicate ACKs.

Then simplified graph:

TEXT

1 6 !' 8 !' 9 !' 1 0 !' 1 1 . . .

This suggests:

Triple duplicate ACK / Fast Recovery.

28. IDENTIFY LOSS TYPE FROM GRAPH

If a GATE graph shows:

$\backslash \text{ cwnd: } 20 \rightarrow 1 \backslash$

likely:

Timeout

or Tahoe-style reaction to loss.

If Reno graph shows:

$\backslash 20 \rightarrow 10 \backslash$

likely:

Three duplicate ACKs.

~~Very~~ common interpretation question.

29. FULL GATE NUMERICAL 1

TCP Reno:

Initial:

$\backslash \text{ cwnd} = 1\text{MSS} \backslash$

$\backslash \text{ ssthresh} = 8\text{MSS} \backslash$

Assume no loss for five RTTs.

Then triple duplicate ACK occurs.

Find progression.

30. BEFORE LOSS

Start:

$\backslash 1 \backslash$

RTT 1:

$\backslash 2 \backslash$

RTT 2:

$\backslash 4 \backslash$

RTT 3:

$\backslash 8 \backslash$

Enter Congestion Avoidance.

RTT 4:

$\backslash 9 \backslash$

RTT 5:

$\backslash 10 \backslash$

Now triple duplicate ACK.

31. AFTER TRIPLE DUP ACK

Current:

$\backslash \text{cwnd}=10 \backslash$

Set:

$\backslash \text{ssthresh}=10/2 \backslash$

$\backslash \boxed{5} \backslash$

Simplified Reno behavior:

$\backslash \boxed{\text{cwnd} \approx 5\text{MSS}} \backslash$

after recovery.

Then Congestion Avoidance:

TEXT

5 !' 6 !' 7 !' 8 . . .

32. SAME PROBLEM IF TIMEOUT OCCURS

Current:

$\backslash \text{cwnd}=10 \backslash$

Set:

$\backslash \text{ssthresh}=5 \backslash$

Then:

$\backslash \boxed{\text{cwnd}=1} \backslash$

Slow Start:

Then transition toward Congestion Avoidance around threshold.

33. ODD `cwnd` VALUES

Suppose:

$\lfloor \text{cwnd} = 15\text{MSS} \rfloor$

Loss occurs.

What is half?

$\lfloor 7.5\text{MSS} \rfloor$

Real implementations operate in bytes and use specified rules.

In simplified exam problems, the question may specify:

- floor
- exact bytes
- approximate half

If not, preserve:

$\lfloor 7.5\text{MSS} \rfloor$

or follow the convention implied by options.

& pDon't invent rounding without need.

34. `ssthresh` IS UPDATED ON CONGESTION

Suppose initially:

$\lfloor \text{ssthresh} = 32\text{MSS} \rfloor$

but congestion occurs at:

$\lfloor \text{cwnd} = 20\text{MSS} \rfloor$

New threshold is not necessarily still 32.

Instead:

$\lfloor \text{ssthresh} \approx 10\text{MSS} \rfloor$

So `ssthresh` dynamically changes.

35. WHY HALF?

The multiplicative decrease principle attempts to substantially reduce offered load.

Conceptually:

$\lfloor \boxed{\text{Congestion !' cut window roughly in half}} \rfloor$

This lets network queues recover while maintaining some throughput.

36. TIMEOUT NUMERICAL

Given:

$\lfloor \text{cwnd}=24\text{KB} \rfloor$

$\lfloor \text{MSS}=1\text{KB} \rfloor$

Timeout occurs.

Assume textbook TCP.

Then:

$\lfloor \text{ssthresh}=12\text{KB} \rfloor$

and:

$\lfloor \text{cwnd}=1\text{MSS}=1\text{KB} \rfloor$

So:

$\lfloor \boxed{\text{ssthresh}=12\text{KB}, \text{cwnd}=1\text{KB}} \rfloor$

37. TRIPLE DUP ACK NUMERICAL — RENO

Same:

$\lfloor \text{cwnd}=24\text{KB} \rfloor$

$\lfloor \text{MSS}=1\text{KB} \rfloor$

Three duplicate ACKs.

Simplified Reno:

$\lfloor \text{ssthresh}=12\text{KB} \rfloor$

and after Fast Recovery:

$\lfloor \boxed{\text{cwnd}=12\text{KB}} \rfloor$

Then additive growth resumes.

38. TRIPLE DUP ACK NUMERICAL — TAHOE

Same starting window:

$\lfloor 24\text{KB} \rfloor$

Triple duplicate ACKs with Tahoe:

$\lfloor \text{ssthresh}=12\text{KB} \rfloor$

but:

$\lfloor \boxed{\text{cwnd}=1\text{KB}} \rfloor$

Then Slow Start.

This is the critical Tahoe/Reno difference.

39. FAST RETRANSMIT vs FAST RECOVERY

Do not confuse them.

Fast Retransmit

Retransmit lost segment after 3 duplicate ACKs without waiting for timeout.

Fast Recovery

Reno's congestion-window behavior after Fast Retransmit, allowing it to avoid restarting from 1 MSS.

So:

Fast Retransmit detects/recovers lost data early.

Fast Recovery controls congestion window afterward.

40. OWM MEMORY

Retransmit = resend packet

Recovery = manage window

41. DUPLICATE ACK COUNT EXAMPLE

Receiver sends:

TEXT

ACK 5000
ACK 5000
ACK 5000
ACK 5000

If the first `ACK 5000` is the original acknowledgement and the next three are duplicates:

Sender has received three duplicate ACKs.

Fast Retransmit condition is satisfied.

42. WHY WAIT FOR THREE DUPLICATES?

A small amount of packet reordering can produce duplicate ACKs even without loss.

Waiting for three duplicate ACKs provides stronger evidence that:

a segment was actually lost.

So one duplicate ACK alone usually does not trigger Fast Retransmit.

43. MULTIPLE LOSSES — RENO LIMITATION

Classic Reno works well for:

one loss in a window.

But multiple packet losses within the same window can cause poorer recovery and may eventually lead to timeout.

Later TCP variants improved this.

For this syllabus, you mainly need:

- Tahoe
- Reno logic
- timeout
- triple duplicate ACK
- fast retransmit
- fast recovery

44. TCP NEW RENO — SMALL NOTE

New Reno improves recovery when multiple segments are lost from one window.

But unless specifically asked:

Do not spend study time on detailed New Reno state transitions.

Your GATE syllabus says TCP congestion control, so Tahoe/Reno principles are the high-value foundation.

45. AIMD CONNECTION

Congestion Avoidance:

Additive increase.

Loss detection:

Multiplicative decrease.

For Reno triple duplicate ACK:

$\lfloor \text{cwnd} \rightarrow \text{cwnd}/2 \rfloor$

is the classic multiplicative decrease.

Then:

$\lfloor +1\text{MSS}/\text{RTT} \rfloor$

again.

This creates the sawtooth.

46. SAWTOOTH NUMERICAL

Suppose Reno Congestion Avoidance window evolves:

TEXT

8, 9, 10, 11, 12, 13, 14, 15, 16

Then triple duplicate ACK.

New approximately:

$\lfloor 8 \rfloor$

Then:

TEXT

8, 9, 10, 11, . . .

So each congestion event roughly halves the operating point.

47. TIMEOUT AFTER CONGESTION AVOIDANCE

If at:

$\lfloor \text{cwnd}=16 \rfloor$

a timeout occurs:

$\lfloor \text{ssthresh}=8 \rfloor$

$\lfloor \text{cwnd}=1 \rfloor$

Then:

TEXT

1 !' 2 !' 4 !' 8

and afterward:

TEXT

9 !' 10 !' 11 . . .

So TCP uses Slow Start again until near the new threshold.

48. FULL GATE TIMELINE

TCP Reno:

Initial:

\[cwnd=1 \]

\[ssthresh=16 \]

Progression:

TEXT

1
2
4
8
16
17
18
19
20

At cwnd = 20 , triple duplicate ACK occurs.

New:

\[ssthresh=10 \]

After recovery:

\[cwnd=10 \]

Then:

TEXT

11
12
13
14

At cwnd = 14 , timeout occurs.

New:

\[ssthresh=7 \]

\[cwnd=1 \]

Then Slow Start resumes:

TEXT

1 !' 2 !' 4

and transitions around 7 according to the question's convention.

49. GATE GRAPH READING

A graph might look like:

TEXT



Interpret:

Curved/steep exponential beginning

Slow Start.

Straight linear increase

Congestion Avoidance.

Drop by approximately half

Triple duplicate ACK / Reno Fast Recovery.

Drop to 1

Timeout.

50. & IMPORTANT VARIANT RULE

Do not automatically apply Reno rules if the question explicitly says:

TCP Tahoe.

Tahoe after triple duplicate ACK:

$\boxed{cwnd=1}$

Reno after triple duplicate ACK:

does Fast Recovery and does not fall all the way to 1 in the usual model.

51. GATE TABLE METHOD

For each event maintain:

Event	cwnd before	New ssthresh	New cwnd	Next phase
Timeout	$\frac{W}{2}$	$\frac{W}{2}$	1 MSS	Slow Start
3 dup ACK, Tahoe	$\frac{W}{2}$	$\frac{W}{2}$	1 MSS	Slow Start

Event	cwnd before	New ssthresh	New cwnd	Next phase
3 dup ACK, Reno	$\frac{W}{2}$	$\frac{W}{2}$	"Half-duplex" after recovery	Congestion Avoidance

This is the safest exam approach.

52. EXAMPLE WITH BYTES

MSS:

$\lfloor 1500B \rfloor$

Current:

$\lfloor cwnd=12MSS \rfloor$

So:

$\lfloor cwnd=18000B \rfloor$

Timeout:

$\lfloor ssthresh=9000B \rfloor$

and:

$\lfloor cwnd=1500B \rfloor$

53. RENO DUP ACK WITH SAME VALUES

Current:

$\lfloor 18000B \rfloor$

Three duplicate ACKs.

New:

$\lfloor ssthresh=9000B \rfloor$

After simplified Fast Recovery:

$\lfloor cwnd \approx 9000B \rfloor$

which is:

$\lfloor \boxed{6MSS} \rfloor$

54. **rwnd** STILL MATTERS

Suppose after Reno recovery:

$\lfloor cwnd=10MSS \rfloor$

MSS:

$\lfloor 1000B \rfloor$

So:

$\lfloor \text{cwnd}=10000\text{B} \rfloor$

But:

$\lfloor \text{rwnd}=4000\text{B} \rfloor$

Actual effective outstanding-data limit remains:

$\lfloor \min(10000, 4000) \rfloor$

$\lfloor \boxed{4000\text{B}} \rfloor$

Congestion control never replaces flow control.

Both operate simultaneously.

55. TIMEOUT vs TRIPLE DUP ACK — INTUITION TABLE

Signal	Interpretation
Timeout	Little/no useful ACK progress; stronger congestion indication
3 duplicate ACKs	Later segments are arriving; isolated loss likely

Hence Reno treats duplicate ACK loss less severely.

56. CONNECTION TO CUMULATIVE ACK

Why can three duplicate ACKs exist?

Because TCP ACK means:

next contiguous byte expected.

A missing byte prevents ACK from moving forward even if later segments arrive.

Thus reliability and congestion control are tightly connected.

57. FAST RETRANSMIT EXAMPLE WITH BYTES

Receiver expects:

$\lfloor 5000 \rfloor$

Sender sends:

TEXT

```
5 0 0 0 - 5 9 9 9   'L   l o s t
6 0 0 0 - 6 9 9 9   '
7 0 0 0 - 7 9 9 9   '
8 0 0 0 - 8 9 9 9   '
```

Receiver sends duplicate:

$\lfloor \text{ACK}=5000 \rfloor$

for each out-of-order arrival.

After 3 duplicate ACKs:

retransmit bytes 5000–5999 immediately.

58. WHAT HAPPENS WHEN MISSING DATA ARRIVES?

Once bytes:

$[5000-5999]$

arrive, and receiver already buffered:

$[6000-8999]$

it now has contiguous data through:

$[8999]$

So cumulative ACK can jump to:

$\boxed{9000}$

This is why cumulative ACKs can make large jumps.

59. ACK JUMP IS NOT DATA LOSS

If ACK jumps:

$5000 \rightarrow 9000$

that may simply mean:

The missing gap was filled and already-buffered later data became contiguous.

Do not assume the receiver suddenly received all 4000 bytes in one segment.

60. HOW GATE CAN ASK THIS TOPIC

Pattern 1

Given `cwnd`, timeout occurs.

Find:

- new `ssthresh`
- new `cwnd`
- next phase.

Pattern 2

Given Reno + triple duplicate ACK.

Find:

- new threshold
 - window after recovery.
-

Pattern 3

Compare Tahoe vs Reno.

Pattern 4

Identify Fast Retransmit.

Answer:

retransmission after 3 duplicate ACKs.

Pattern 5

Identify Fast Recovery.

Answer:

Reno avoids resetting to 1 after triple duplicate ACK loss.

Pattern 6

Read `cwnd` graph.

Drop to 1 vs half.

Pattern 7

Continue the window progression after congestion event.

61. & pGATE TRAPS

Trap 1

Three duplicate ACKs do not mean:

three packets were lost.

They are a signal suggesting an earlier packet/segment was lost.

Trap 2

Fast Retransmit is not the same as timeout retransmission.

Trap 3

Reno still resets to 1 on timeout in the classic textbook model.

Trap 4

Tahoe resets to 1 even after triple duplicate ACK loss.

Trap 5

Reno's triple-duplicate-ACK response is not identical to its timeout response.

Trap 6

`ssthresh` is updated after congestion.

Don't keep the old threshold.

Trap 7

When halving, use the congestion/flight value specified by the question.

If the question explicitly gives `FlightSize`, use its stated rule rather than blindly using `cwnd`.

Trap 8

Fast Recovery has temporary `cwnd` inflation in the detailed Reno algorithm.

Use the simplified half-window rule only when the question uses RTT-level textbook behavior.

62. 0> MASTER MEMORY SYSTEM

Remember:

T-D-R

T — Timeout

$\lfloor \boxed{cwnd} \rfloor$

D — Duplicate ACKs

3 duplicates:

Fast Retransmit

R — Reno

63. TAHOE/RENO MEMORY

TEXT

TAHOE:

L o s s ? ! ' g o t o 1 .

RENO:

T i m e o u t ? ! ' g o t o 1 .
3 d u p A C K ? ! ' h a l v e a n d r e c o v e r .

That's the exam shortcut.

'p NOTEBOOK VERSION

TCP Loss Detection

Two important loss signals:

1. Retransmission Timeout
2. Three Duplicate ACKs

Timeout

Strong congestion indication.

Classic rule:

$\boxed{\text{ssthresh} \approx \text{cwnd}/2}$

$\boxed{\text{cwnd} = 1\text{MSS}}$

Then:

Slow Start.

Three Duplicate ACKs

Indicate that later segments are still arriving.

Trigger:

$\boxed{\text{Fast Retransmit}}$

Lost segment retransmitted without waiting for timeout.

TCP Tahoe

For both timeout and triple duplicate ACK:

$\lfloor \text{ssthresh} = \text{cwnd} / 2 \rfloor$

$\lfloor \boxed{\text{cwnd} = 1\text{MSS}} \rfloor$

Then Slow Start.

TCP Reno

Timeout

$\lfloor \text{ssthresh} = \text{cwnd} / 2 \rfloor$

$\lfloor \text{cwnd} = 1\text{MSS} \rfloor$

! Slow Start.

Triple Duplicate ACK

- Fast Retransmit
- `s s t h r e s h "H c w n d / 2`
- Fast Recovery
- after recovery:

$\lfloor \boxed{\text{cwnd} \approx \text{ssthresh}} \rfloor$

! Congestion Avoidance.

Fast Retransmit

$\lfloor \boxed{3 \text{ duplicate ACKs} \rightarrow \text{retransmit}} \rfloor$

Fast Recovery

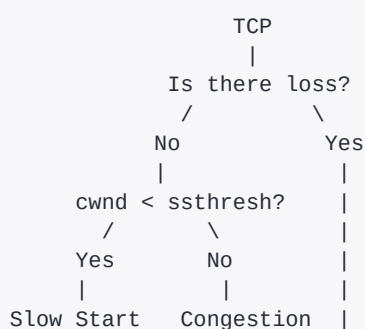
Reno avoids returning to:

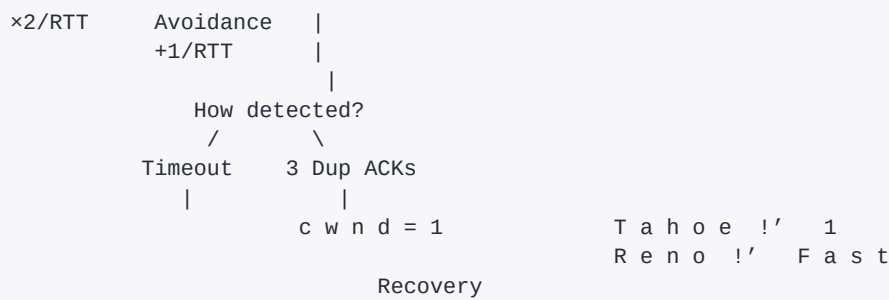
$\lfloor 1\text{MSS} \rfloor$

after triple duplicate ACK loss.

64. COMPLETE TCP CONGESTION CONTROL MEMORY MAP

TEXT





65. 5-MINUTE TCP CONGESTION REVISION

Slow Start

$\backslash$ 1,2,4,8,16 $\backslash$

Exponential.

Congestion Avoidance

$\backslash$ 16,17,18,19... $\backslash$

Linear.

Timeout

$\backslash$ ssthresh $\backslash$ approx cwnd/2 $\backslash$

$\backslash$ cwnd=1 $\backslash$

Three duplicate ACKs

Fast Retransmit.

Tahoe:

$\backslash$ cwnd=1 $\backslash$

Reno:

$\backslash$ cwnd $\backslash$ approx half $\backslash$

after recovery.

66. 30-SECOND REVISION

Memorize:

Timeout = bad !' go to 1

3 dup ACK = packet flow continues !' Fast Retransmit

Then:

TEXT

Tahoe !' always falls to 1 on loss .

Reno:

time out !' 1

3 dup ACK !' half + Fast Recovery

ACTIVE RECALL — DON'T LOOK BACK

Q1

What are the two major ways classical TCP infers packet loss?

Q2

A TCP sender has:

\[cwnd=20MSS \]

A timeout occurs.

Find:

1. new ssthresh
2. new cwnd
3. next congestion-control phase

Q3

Why are three duplicate ACKs considered a less severe congestion signal than a timeout?

Q4

What mechanism is triggered by three duplicate ACKs?

A. Slow Start B. Fast Retransmit C. Fragmentation D. Flow control

Q5 — Tahoe

TCP Tahoe has:

\[cwnd=16MSS \]

Three duplicate ACKs arrive.

What are the new approximate:

1. ssthresh
2. cwnd
3. next phase?

Q6 — Reno

TCP Reno has:

$\lfloor \text{cwnd} = 16\text{MSS} \rfloor$

Three duplicate ACKs arrive.

Under the simplified GATE model, what are:

1. new `ssthresh`
 2. `cwnd` after recovery
 3. next phase?
-

Q7 — Reno Timeout

Same Reno connection:

$\lfloor \text{cwnd} = 16\text{MSS} \rfloor$

but this time a timeout occurs.

What happens?

Q8 — Graph Interpretation

A Reno `cwnd` graph suddenly changes:

$\lfloor 24\text{MSS} \rightarrow 12\text{MSS} \rfloor$

and then resumes linear growth.

Which event most likely occurred?

Q9

A graph changes:

$\lfloor 24\text{MSS} \rightarrow 1\text{MSS} \rfloor$

What loss event is most likely in Reno?

Q10 — Full Progression

TCP Reno:

$\lfloor \text{cwnd}_0 = 1\text{MSS}, \text{ ssthresh} = 8\text{MSS} \rfloor$

No loss for 5 RTTs, then a timeout occurs.

Write:

1. `cwnd` values up to the timeout
 2. new `ssthresh`
 3. new `cwnd` immediately after timeout
-

Q11

Differentiate in one line:

- Fast Retransmit
 - Fast Recovery
-

Q12 — Confusing Pair

Why does Tahoe behave differently from Reno after three duplicate ACKs?

NEXT !' TCP Timing & Reliability: RTT Estimation, Retransmission Timeout, Duplicate ACKs, Fast Retransmit and End-to-End TCP Numerical Patterns

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 5 — TCP Timing & Reliability: RTT Estimation, Retransmission Timeout, Timers, Duplicate ACKs & End-to-End Numericals

Ø=Ý%
HIGH PRIORITY FOR GATE

You already know that TCP provides reliable delivery using:

- sequence numbers
- cumulative ACKs
- retransmissions
- checksum

Now comes an important question:

How long should TCP wait for an ACK before deciding that data is lost?

If TCP waits too little:

it retransmits packets unnecessarily.

If TCP waits too long:

recovery from an actual loss becomes very slow.

So TCP dynamically estimates:

$\boxed{RTT}$

and calculates a:

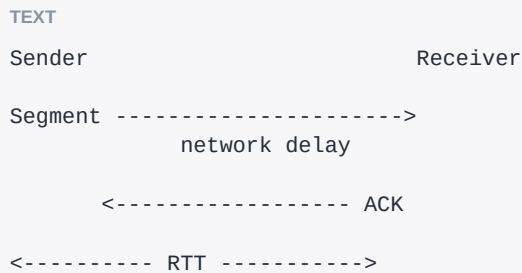
$\boxed{\text{Retransmission Timeout (RTO)}}$

1. WHAT IS RTT?

RTT means:

Round-Trip Time

Conceptually:



RTT is approximately the time from:

sending a TCP segment

until:

receiving the corresponding ACK.

2. WHY CAN'T TCP USE ONE FIXED TIMEOUT?

Internet delay is not constant.

RTT changes because of:

- queueing
- route conditions
- traffic load
- link speeds
- network congestion

Example RTT samples:

TEXT

80 ms
110 ms
90 ms
180 ms
95 ms

So a fixed timeout such as:

\[100ms \]

would sometimes be too small.

3. WHAT HAPPENS IF TIMEOUT IS TOO SMALL?

Suppose actual RTT:

[150ms]

but timeout:

[100ms]

At 100 ms TCP concludes:

ACK hasn't arrived !' retransmit.

But original packet may simply still be travelling normally.

This causes a:

Premature / spurious retransmission

and unnecessary network load.

4. WHAT IF TIMEOUT IS TOO LARGE?

Actual packet is genuinely lost after transmission.

Suppose TCP timeout:

[5s]

even though normal RTT:

[100ms]

Then TCP waits a long time before recovering.

So:

[$\boxed{RTO \text{ should be slightly larger than expected RTT}}$]

and should account for RTT variability.

5. SampleRTT

TCP measures individual RTT observations.

A measured RTT for one transmission is:

[$\boxed{\text{SampleRTT}}$]

Example:

[$\text{SampleRTT}=120\text{ms}$]

But individual samples can fluctuate considerably.

Therefore TCP maintains a smoothed estimate.

6. EstimatedRTT

Instead of simply using the latest sample, TCP commonly uses an exponentially weighted moving average:

$$\boxed{\text{EstimatedRTT} = (1-\alpha)\text{EstimatedRTT}_{\text{old}} + \alpha \text{SampleRTT}}$$

A commonly used value is:

$$\boxed{\alpha = \frac{1}{8} = 0.125}$$

So:

$$\text{EstimatedRTT} = 0.875(\text{Old}) + 0.125(\text{Sample})$$

7. WHAT DOES THIS FORMULA MEAN?

Most weight is placed on:

previous history

and a smaller weight on:

newest measurement.

This prevents one unusual RTT sample from completely changing the timeout.

Memory:

Estimated RTT = Old history + little new information.

8. NUMERICAL 1 — ESTIMATED RTT

Given:

$$\text{EstimatedRTT}_{\text{old}} = 100\text{ms}$$

$$\text{SampleRTT} = 140\text{ms}$$

$$\alpha = 0.125$$

Find new EstimatedRTT.

SOLUTION

$$\text{EstimatedRTT} = (1-0.125)(100) + 0.125(140)$$

$$= 0.875(100) + 0.125(140)$$

$$= 87.5 + 17.5$$

$$= \boxed{105\text{ms}}$$

Notice:

Latest sample was 140 ms, but estimate only moved from 100 to 105 ms.

That's smoothing.

9. GENERAL EWMA IDEA

Formula:

$$New = (1 - \alpha)Old + \alpha Sample$$

If α is:

Small

More smoothing / history matters more.

Large

Newest sample matters more.

& pGATE may test this concept.

10. IS AVERAGE RTT ENOUGH?

No.

Consider two networks.

Network A

RTT:

TEXT

99, 101, 100, 100

Average RTT 100 ms.

Network B

RTT:

TEXT

20, 180, 30, 170

Average RTT 100 ms.

Both have similar average RTT.

But Network B has much greater:

variability.

Therefore TCP should account for deviation too.

11. DevRTT

TCP maintains an estimate of RTT variation:

$$\boxed{DevRTT} = (1 - \beta)DevRTT_{old} + \beta$$

SampleRTT-EstimatedRTT

} \]

A common value:

$\beta = \frac{1}{4} = 0.25$

This tells us approximately:

How far RTT samples tend to deviate from the expected RTT.

12. WHY ABSOLUTE VALUE?

Suppose:

$\text{SampleRTT} = 80$

$\text{EstimatedRTT} = 100$

Difference:

$80 - 100 = -20$

But variability shouldn't be negative.

So use:

$|$

80-100

$= 20$

13. RETRANSMISSION TIMEOUT FORMULA

A common TCP timeout calculation is:

$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\text{DevRTT}$

This is extremely important.

Memory:

Expected delay + safety margin

where:

4DevRTT

is the safety margin.

14. MASTER FORMULAS

Estimated RTT

$$\boxed{\text{EstimatedRTT} = (1-\alpha)\text{OldEstimated} + \alpha \text{ SampleRTT}}$$

Typically:

$$\alpha=0.125$$

Deviation

$$\boxed{\text{DevRTT} = (1-\beta)\text{OldDev} + \beta|\text{SampleRTT}-\text{EstimatedRTT}|}$$

Typically:

$$\beta=0.25$$

Timeout

$$\boxed{\text{RTO}=\text{EstimatedRTT}+4\text{DevRTT}}$$

Put these in your formula sheet.

15. FULL GATE NUMERICAL

Given:

$$\text{EstimatedRTT}_{\text{old}}=100\text{ms}$$

$$\text{DevRTT}_{\text{old}}=10\text{ms}$$

$$\text{SampleRTT}=140\text{ms}$$

$$\alpha=0.125$$

$$\beta=0.25$$

Find new timeout.

16. STEP 1 — ESTIMATED RTT

$$\text{EstimatedRTT} = 0.875(100)+0.125(140)$$

$$=87.5+17.5$$

$$=\boxed{105\text{ms}}$$

17. STEP 2 — DEVIATION

Using the new estimate in this standard calculation:

$$|$$

SampleRTT-EstimatedRTT

$$|$$

$$=|140-105|$$

$$=35\text{ms}$$

Now:

$$\text{DevRTT} = 0.75(10) + 0.25(35)$$

$$= 7.5 + 8.75$$

$$= \boxed{16.25\text{ms}}$$

18. STEP 3 — TIMEOUT

$$\text{RTO} = 105 + 4(16.25)$$

$$= 105 + 65$$

$$= \boxed{170\text{ms}}$$

19. WHY $4 \times \text{DevRTT}$?

If network delay fluctuates a lot:

$$\text{DevRTT} \uparrow$$

then:

$$\text{RTO} \uparrow$$

So TCP gives the network more time before falsely declaring loss.

If RTT is very stable:

$$\text{DevRTT} \downarrow$$

the timeout can stay closer to:

$$\text{EstimatedRTT}$$

20. GATE SHORTCUT

When given:

- old EstimatedRTT
- SampleRTT
- old DevRTT
- $\pm$
- 2

do exactly:

1

$$E = (1 - \alpha)E_{\text{old}} + \alpha S$$

2

$$D = (1 - \beta)D_{\text{old}} + \beta |S - E|$$

3

$$\boxed{\text{RTO} = E + 4D}$$

Don't mix the order.

21. TCP RELIABILITY — BIG PICTURE

Suppose sender transmits data.

Three possibilities:

Case 1 — Everything works

TEXT

```
D a t a ! ' R e c e i v e r
A C K ! • R e c e i v e r
```

No retransmission.

Case 2 — Data is lost

TEXT

```
Data ----X
```

No appropriate ACK.

Eventually:

timeout !' retransmit.

Case 3 — Data arrives but ACK is lost

TEXT

```
D a t a ! ' R e c e i v e r
A C K ----X
```

Sender may eventually timeout and retransmit the data.

Receiver uses sequence numbers to recognize duplicate data.

22. DATA LOST EXAMPLE

Sender transmits:

TEXT

```
Seq = 1000
Length = 500 B
```

Segment is lost.

Receiver never gets bytes:

[1000-1499]

Sender eventually experiences:

[RTO]

and retransmits.

23. ACK LOST EXAMPLE

Sender sends:

\[1000-1499 \]

Receiver gets it and sends:

\[ACK=1500 \]

But ACK gets lost.

Sender doesn't know receiver already has the bytes.

After timeout, sender may retransmit:

\[1000-1499 \]

Receiver sees duplicate sequence numbers and can discard duplicate data while acknowledging appropriately.

Sequence numbers prevent duplicate delivery to the application.

24. ACK LOSS DOES NOT NECESSARILY REQUIRE RETRANSMISSION

Because ACKs are cumulative.

Suppose:

TEXT

```
Sender !' Segment 1
Receiver !' ACK 1500 'L l o s t

Sender !' Segment 2
Receiver !' ACK 2000 ' a r r i v e s
```

ACK:

\[2000 \]

cumulatively confirms all bytes before 2000.

Therefore lost:

\[ACK=1500 \]

doesn't necessarily matter.

25. CUMULATIVE ACK BENEFIT

If ACK \((A_1)\) is lost but later ACK \((A_2 > A_1)\) arrives:

\[A_2 \]

can cover the earlier data too.

This reduces the need to recover every individual lost ACK.

26. OUT-OF-ORDER DELIVERY RECALL

Suppose receiver expects:

\[2000 \]

but segment:

\[2500-2999 \]

arrives.

Receiver still lacks:

\[2000-2499 \]

So cumulative ACK remains:

\[\boxed{2000} \]

This duplicate ACK provides information to the sender.

27. THREE DUPLICATE ACKS

Suppose missing:

\[2000-2499 \]

but later segments arrive:

TEXT

```
2 5 0 0 - 2 9 9 9 '
3 0 0 0 - 3 4 9 9 '
3 5 0 0 - 3 9 9 9 '
```

Each can cause another:

\[ACK=2000 \]

After:

\[\boxed{3\ \text{duplicate\ ACKs}} \]

sender performs:

Fast Retransmit

without waiting for RTO.

28. TWO WAYS TO RETRANSMIT

Timeout-based retransmission

Wait until timer expires.

Fast Retransmit

Three duplicate ACKs indicate likely loss.

Then:

retransmit earlier.

29. WHICH IS FASTER?

Usually:

Fast Retransmit

because it can recover before the timer expires.

That's why duplicate ACKs are useful.

30. RETRANSMISSION TIMER

A sender doesn't necessarily run a separate independent timer for every byte in the simplified TCP model.

A common conceptual TCP model maintains a retransmission timer associated with:

the oldest unacknowledged data.

When that data is acknowledged:

timer can be restarted/stopped as appropriate.

For GATE, follow the timer behavior specified in the question.

31. OLDEST UNACKNOWLEDGED BYTE

Suppose sender transmitted:

TEXT

1000–1499

1500–1999

2000–2499

Receiver ACK:

\[1500 \]

Then bytes:

\[1000-1499 \]

are acknowledged.

Oldest unacknowledged byte becomes:

\[\boxed{1500} \]

The retransmission timer conceptually concerns the earliest outstanding data.

32. ACK ADVANCES

If sender receives:

[ACK=2500]

all data before 2500 is cumulatively acknowledged.

So all three ranges:

TEXT

1000–1499

1500–1999

2000–2499

are now acknowledged.

Outstanding data from those segments becomes:

[0]

33. RETRANSMISSION AMBIGUITY

Suppose sender sends a segment.

No ACK arrives.

Sender retransmits.

Then an ACK arrives.

Question:

Did this ACK correspond to the original transmission or the retransmission?

Often impossible to know.

This creates:

RTT measurement ambiguity

34. KARN'S ALGORITHM — IMPORTANT CONCEPT

A classic solution is:

Do not use RTT measurements from retransmitted segments to update the RTT estimator.

Why?

Because you cannot reliably tell which transmission caused the ACK.

This principle is associated with:

Karn's Algorithm

35. RETRANSMISSION MEMORY

Retransmitted? Don't trust its RTT sample.

Very easy.

36. EXAMPLE OF AMBIGUITY

Original transmitted at:

$[t=0]$

Retransmitted at:

$[t=200\text{ms}]$

ACK arrives:

$[t=250\text{ms}]$

Possible RTT:

If ACK is for original

$[250\text{ms}]$

If ACK is for retransmission

$[50\text{ms}]$

TCP cannot know for sure.

So avoid using that sample.

37. TIMEOUT BACKOFF

Suppose TCP times out and retransmits.

If it keeps using the exact same RTO during serious congestion, it may repeatedly inject traffic too quickly.

So TCP commonly uses:

Exponential backoff

After repeated timeout events, retransmission timeout is increased.

Conceptually:

TEXT

RTO

2RTO

4RTO

8RTO

...

subject to implementation limits.

38. WHY BACKOFF?

Timeout suggests the network may be congested.

Increasing waiting time:

reduces retransmission pressure on the network.

This complements congestion control.

39. DON'T CONFUSE BACKOFFS

We have seen two different backoff ideas:

Ethernet CSMA/CD

Binary exponential backoff after collisions.

TCP

Retransmission timeout backoff after timeout.

They solve different problems.

& pGATE trap.

40. TCP CHECKSUM

TCP detects corruption using:

checksum.

If segment fails checksum:

receiver treats it as corrupted rather than delivering corrupted bytes normally.

The sender eventually recovers through retransmission mechanisms when acknowledgement does not properly progress.

41. CHECKSUM DOES NOT CORRECT ERRORS

Recall from earlier:

Error detection "error correction.

TCP checksum detects corruption.

TCP reliability recovers by:

retransmitting data.

42. TCP vs ETHERNET RELIABILITY

Ethernet:

FCS/CRC detects corrupted frame and typically discards it.

TCP:

end-to-end reliability can retransmit missing/corrupted data.

This is a good example of functionality at different layers.

43. END-TO-END RELIABILITY

Suppose:

TEXT

S e n d e r ! ' R o u t e r 1 ! ' R o u t e r 2 ! ' R o u t e r 3 ! ' R e c e i v e r

Routers do not provide TCP's end-to-end ACK/retransmission mechanism.

TCP reliability is primarily between:

sender host !"receiver host.

This is:

End-to-end reliability

44. GATE NUMERICAL — ACK + TIMEOUT

Sender sends:

\[Seq=5000 \]

\[Length=1000B \]

Expected cumulative ACK:

\[\boxed{6000} \]

If no suitable ACK arrives before RTO:

retransmit data beginning at sequence 5000.

45. MULTIPLE SEGMENTS EXAMPLE

Sender sends:

Segment	Sequence	Length
S1	1000	500
S2	1500	500
S3	2000	500
S4	2500	500

S2 is lost.

Receiver gets S1:

\[ACK=1500 \]

Then S3:

\[ACK=1500 \]

duplicate.

Then S4:

\[ACK=1500 \]

another duplicate.

If enough later segments generate 3 duplicate ACKs:

Fast Retransmit S2.

46. HOW MANY LATER SEGMENTS ARE NEEDED?

In the simple model, to obtain:

\[3 \]

duplicate ACKs, the receiver generally needs to receive at least three later out-of-order segments after the missing segment.

Example missing S2:

TEXT

```
S 3 !'   d u p l i c a t e   A C K   # 1
S 4 !'   d u p l i c a t e   A C K   # 2
S 5 !'   d u p l i c a t e   A C K   # 3
```

Then Fast Retransmit.

47. WHAT IF THERE ARE NOT ENOUGH LATER SEGMENTS?

Suppose last segment in a small transfer is lost.

There may be no later segments to generate duplicate ACKs.

Then sender may need to rely on:

retransmission timeout.

This explains why RTO remains necessary even though Fast Retransmit exists.

48. FAST RETRANSMIT DOES NOT REPLACE RTO

Ø=important.

TCP still needs timeout because:

- first segment might be lost
- final segment might be lost
- too few later packets may exist
- ACK feedback may disappear

So:

$\boxed{\text{Fast Retransmit complements RTO}}$

rather than replacing it.

49. SEQUENCE NUMBER + ACK NUMERICAL

Sender sends three segments:

S1

$\boxed{\text{Seq}=4000, \text{Length}=600}$

S2

Next sequence:

$\boxed{4600}$

Length:

$\boxed{600}$

S3

Next:

$\boxed{5200}$

Length:

$\boxed{300}$

If all arrive contiguously, final ACK:

$\boxed{5200+300}$

$\boxed{=5500}$

50. LOSS NUMERICAL

Using same segments:

TEXT

```
S 1 :   4 0 0 0 - 4 5 9 9   '
S 2 :   4 6 0 0 - 5 1 9 9   'L
S 3 :   5 2 0 0 - 5 4 9 9   '
```

Receiver's cumulative ACK after S3 arrives remains:

$\boxed{4600}$

because byte 4600 is still missing.

51. WHEN S2 IS RETRANSMITTED

Receiver already has S3 buffered.

Retransmitted S2 arrives:

TEXT

```
4 6 0 0 - 5 1 9 9   '
```

Now receiver has contiguous bytes:

$\boxed{4000-5499}$

So ACK can jump to:

$\boxed{5500}$

52. GATE NUMERICAL — RTT EWMA WITH MULTIPLE SAMPLES

Suppose:

$\boxed{\text{EstimatedRTT}_0=100\text{ms}}$

$\boxed{\alpha=0.25}$

Samples:

$\boxed{120\text{ms}, 140\text{ms}}$

Find estimate after both.

First sample

$\boxed{E_1=0.75(100)+0.25(120)}$

$\boxed{=75+30}$

$\boxed{=105\text{ms}}$

Second sample

$\boxed{E_2=0.75(105)+0.25(140)}$

$\boxed{=78.75+35}$

$\boxed{113.75\text{ms}}$

53. ← DO NOT AVERAGE SAMPLES DIRECTLY

Incorrect:

$$\frac{100+120+140}{3}$$

unless question specifically asks arithmetic mean.

EWMA repeatedly applies:

$$(1-\alpha)\text{Old} + \alpha \text{New}$$

Order matters.

54. CLOSED-FORM WEIGHT INTUITION

After several samples, newer observations have relatively greater influence than very old observations, but historical influence decays gradually.

That's why it's called:

Exponentially Weighted Moving Average

You don't need a full derivation for standard GATE problems.

55. RTT vs ONE-WAY DELAY

Do not confuse:

One-way propagation/delay

Sender !' Receiver.

RTT

Sender !' Receiver !' Sender.

You cannot simply say:

$$RTT = 2d_{\text{prop}}$$

unless the question explicitly ignores:

- transmission delays
- queueing
- processing
- ACK transmission
- asymmetry

Recall this from performance metrics.

56. RTO vs RTT

RTT

Measured/estimated round-trip network time.

RTO

How long TCP waits before declaring timeout.

Generally:

$$\boxed{RTO > EstimatedRTT}$$

because variability margin is added.

57. RTT, RTO, cwnd — DON'T MIX THEM

RTT

Timing measurement.

RTO

Retransmission timer.

cwnd

Amount of data sender may have outstanding due to congestion control.

Different concepts.

58. RTT, rwnd, cwnd CONNECTION

Together they affect performance.

If:

$$\text{Window} = W$$

and RTT:

$$T$$

window-limited throughput is roughly:

$$\frac{W}{T}$$

Actual outstanding window is approximately constrained by:

$$\min(rwnd, cwnd)$$

So simplified:

$$\boxed{\text{Throughput} \lesssim \frac{\min(rwnd, cwnd)}{RTT}}$$

and also cannot exceed path capacity.

59. FULL CONNECTION NUMERICAL

Given:

$\text{rwnd} = 40\text{KB}$

$\text{cwnd} = 25\text{KB}$

$\text{RTT} = 50\text{ms}$

Effective window:

$\min(40, 25) = 25\text{KB}$

Assume:

$1\text{KB} = 1000\text{B}$

Then:

$25\text{KB} = 25,000\text{B}$

Window-limited throughput:

$\frac{25,000}{0.05} = 500,000\text{B/s}$

Convert:

$500,000 \times 8 = 4,000,000\text{bps}$

$\boxed{4\text{Mbps}}$

if no other bottleneck dominates.

60. ADD LINK CAPACITY

Suppose physical bottleneck:

3Mbps

Our window calculation allowed:

4Mbps

Actual ideal throughput cannot exceed bottleneck:

$\boxed{3\text{Mbps}}$

So a useful simplified bound is:

$\boxed{\text{Throughput} \leq \min(\text{PathRate}, \frac{\text{Window}}{\text{RTT}})}$

61. TCP RELIABILITY MENTAL MAP

TEXT

Data bytes

!"

Sequence numbers

!"

Receiver checks order/corruption

!"

Cumulative ACK

!"

ACK arrives?

/ \


```

Yes      No
|        |
Continue RTO
          |
          ! "
          Retransmit

```

If duplicate ACKs arrive:

```

TEXT
3 duplicate ACKs
          |
          ! "
Fast Retransmit

```

62. HOW GATE CAN ASK THIS TOPIC

Pattern 1 — Estimated RTT

Use EWMA.

Pattern 2 — DevRTT

Use weighted deviation.

Pattern 3 — Timeout Interval

$\lfloor \text{EstimatedRTT} + 4\text{DevRTT} \rfloor$

Pattern 4 — Lost segment

Determine ACK number and retransmission.

Pattern 5 — Lost ACK

Use cumulative ACK reasoning.

Pattern 6 — Duplicate ACK

Find missing sequence range.

Pattern 7 — Fast Retransmit

Three duplicate ACKs.

Pattern 8 — Karn's algorithm

Do not sample RTT for retransmitted segments.

Pattern 9 — Window/RTT throughput

$\lfloor \text{Window/RTT} \rfloor$

63. & pGATE TRAPS

Trap 1

RTO should not simply equal SampleRTT.

Trap 2

EstimatedRTT is not a plain arithmetic average in the standard EWMA model.

Trap 3

Higher RTT variability should generally cause a larger timeout margin.

Trap 4

ACK means:

next expected byte.

Still true here.

Trap 5

Lost ACK does not always require retransmission because a later cumulative ACK may cover it.

Trap 6

Three duplicate ACKs trigger Fast Retransmit; a timeout is not required first.

Trap 7

Fast Retransmit cannot replace retransmission timeout completely.

Trap 8

Do not use RTT sample from a retransmitted segment under Karn's classic rule.

Trap 9

TCP checksum detects corruption; it does not itself repair corrupted bits.

Trap 10

RTO backoff and Ethernet collision backoff are different mechanisms.

64. 0> MASTER MEMORY SYSTEM

Remember:

S-E-D-T

S — Sample RTT

Measure one RTT.

E — Estimated RTT

Smooth it.

$$E = (1 - \alpha)E + \alpha S$$

D — Deviation

Measure variability.

T — Timeout

$$T = E + 4D$$

65. RELIABILITY MEMORY

S-A-T-R

S — Sequence numbers

Where are the bytes?

A — ACKs

What arrived?

T — Timer

Did ACK take too long?

R — Retransmit

Recover missing data.

'p NOTEBOOK VERSION

TCP RTT & Reliability

RTT

Round-trip time between transmitting data and receiving corresponding acknowledgement.

Estimated RTT

$$\text{EstimatedRTT} = (1 - \alpha)\text{EstimatedRTT}_{\text{old}} + \alpha \text{SampleRTT}$$

Common:

$\alpha=0.125$

RTT Deviation

$$\text{DevRTT} = (1-\beta)\text{DevRTT}_{\text{old}} + \beta|\text{SampleRTT}-\text{EstimatedRTT}|$$

Common:

$\beta=0.25$

Retransmission Timeout

$$\text{RTO} = \text{EstimatedRTT} + 4\text{DevRTT}$$

Reliability

TCP uses:

- checksum
- sequence numbers
- cumulative ACKs
- retransmission timer
- retransmissions
- duplicate ACKs
- Fast Retransmit

Fast Retransmit

$$3 \text{ duplicate ACKs} \rightarrow \text{retransmit}$$

without waiting for timeout.

Karn's Algorithm

Do not use retransmitted segments for RTT measurement because ACK timing is ambiguous.

RTO Backoff

Repeated timeout:

increase retransmission timeout, commonly exponentially.

Cumulative ACK

If later ACK acknowledges more data, it also covers earlier contiguous bytes.

Therefore a lost intermediate ACK may not matter.

66. TCP MASTER CONNECTION SO FAR

Sequence number

$\boxed{\text{First byte in segment}}$

ACK

$\boxed{\text{Next byte expected}}$

Flow control

$\boxed{rwnd}$

Protects receiver.

Congestion control

$\boxed{cwnd}$

Protects network.

Effective sending limit

$\boxed{\min(rwnd, cwnd)}$

Slow Start

$\times 2 / RTT$

Congestion Avoidance

$+1MSS / RTT$

Timeout

Classic:

$cwnd \rightarrow 1MSS$

Three duplicate ACKs

Fast Retransmit.

Reno

Fast Recovery after triple duplicate ACK.

67. 30-SECOND REVISION

Remember:

$\boxed{\text{EstimatedRTT} = (1 - \alpha)\text{Old} + \alpha \text{ Sample}}$

$\boxed{\text{RTO} = \text{EstimatedRTT} + 4\text{DevRTT}}$

Then reliability:

No ACK !' timeout !' retransmit

3 duplicate ACKs !' Fast Retransmit

And:

Retransmitted segment? Don't trust its RTT sample.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Why doesn't TCP simply set:

$\boxed{\text{RTO} = \text{SampleRTT}}$

?

Q2

Given:

$\boxed{\text{EstimatedRTT}_{\text{old}} = 80\text{ms}}$

$\boxed{\text{SampleRTT} = 120\text{ms}}$

$\boxed{\alpha = 0.25}$

find new EstimatedRTT.

Q3

If:

$\boxed{\text{EstimatedRTT} = 100\text{ms}}$

and:

$\boxed{\text{DevRTT} = 20\text{ms}}$

find:

$\boxed{\text{RTO}}$

Q4 — Full Numerical

Given:

$\boxed{\text{EstimatedRTT}_{\text{old}} = 100\text{ms}}$

$\boxed{\text{DevRTT}_{\text{old}} = 20\text{ms}}$

$\backslash$ SampleRTT=140ms $\backslash$

$\backslash$ $\alpha=0.125$, $\beta=0.25$ $\backslash$

Calculate:

1. new EstimatedRTT
 2. new DevRTT
 3. new RTO
-

Q5

A sender transmits:

$\backslash$ Seq=5000, Length=800B $\backslash$

What cumulative ACK is expected after successful reception?

Q6 — Lost ACK

Receiver successfully gets bytes through:

$\backslash$ 1999 $\backslash$

and sends:

$\backslash$ ACK=2000 $\backslash$

but that ACK is lost.

Later receiver gets bytes 2000–2499.

What cumulative ACK can it now send?

Q7

Why can a later cumulative ACK make an earlier lost ACK irrelevant?

Q8

What event triggers Fast Retransmit in classic TCP?

Q9

Why can Fast Retransmit not completely replace the retransmission timer?

Q10

Why does Karn's algorithm avoid RTT measurements from retransmitted segments?

Q11 — Window + RTT

$\backslash$ rwnd=20KB, cwnd=12KB, RTT=40ms $\backslash$

Using $\backslash$ (1KB=1000B) $\backslash$, what is the approximate window-limited throughput in Mbps?

Q12 — Confusing Pair

Differentiate:

- RTT
- RTO

in one line each.

NEXT !' TCP Socket API: Client–Server Model, `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()` & GATE Call-Sequence Questions

USER

next

ASSISTANT

CHAPTER 12 — TCP

TOPIC 6 — Socket API: Client–Server Model, `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/recv()` & GATE Call Sequences

HIGH PRIORITY FOR GATE CONCEPTUAL QUESTIONS

The socket API is how an application asks the operating system to use TCP/IP networking.

The easiest way to remember TCP sockets is:

Server prepares and waits. Client connects. Then both communicate.

The most important sequence is:

`\boxed{\text{Server: socket !' bind !' listen !' accept}} \]`

`\boxed{\text{Client: socket !' connect}} \]`

Then both:

`\boxed{\text{send/recv !' close}} \]`

1. WHAT IS A SOCKET?

A socket is a software interface between:

Application process

and:

Transport layer/networking services provided by the OS

Conceptually:

```
TEXT
Application
  |
  | Socket API
  |
TCP / UDP
  |
IP
```

A process uses a socket to send and receive network data.

2. SOCKET "PORT"

& Important.

A **port number** is a numeric transport-layer identifier.

A **socket** is an OS-managed communication endpoint/interface.

A TCP endpoint is associated with information such as:

```
TEXT
IP address + Port
```

but:

socket and port are not the same thing.

3. CLIENT-SERVER MODEL

Most socket questions use:

Server

Waits for incoming connection requests.

Client

Actively initiates a connection to the server.

Example:

```
TEXT
Client                                Server
|                                     |
|----- connect ----->|
|<===== TCP connection =====>|
|----- data ----->|
|<----- data -----|
```

4. TCP SERVER — MASTER SEQUENCE

A TCP server typically performs:

TEXT

```
socket()  
    !"   
bind()  
    !"   
listen()  
    !"   
accept()  
    !"   
send()/recv()  
    !"   
close()
```

Memorize:

S-B-L-A

Socket !' Bind !' Listen !' Accept

5. TCP CLIENT — MASTER SEQUENCE

Client usually performs:

TEXT

```
socket()  
    !"   
connect()  
    !"   
send()/recv()  
    !"   
close()
```

Memory:

S-C

Socket !' Connect

6. COMPLETE PICTURE

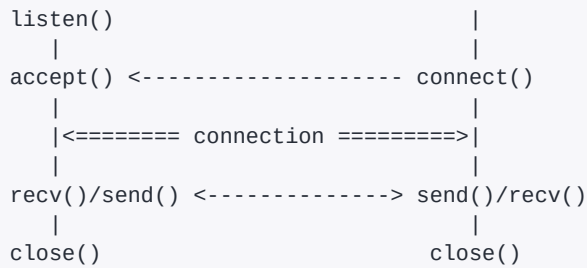
TEXT

SERVER

```
socket()  
|  
bind()  
|
```

CLIENT

```
socket()  
|  
|  
|
```



This diagram alone answers many GATE questions.

7. socket()

`socket()` creates a socket.

Conceptually:

"Operating system, give this process a network communication endpoint."

For TCP, the application requests:

stream-oriented transport.

After `socket()`:

- socket exists
- but server is not necessarily bound/listening yet
- client is not connected yet

8. bind()

`bind()` associates a local socket with a local address, typically:

`[ \boxed{\text{Local IP + Local Port}} \]`

Example server wants to receive connections on:

TEXT
192.168.1.10:8080

The server binds its socket to that address/port.

9. WHY DOES A SERVER NEED `bind()`?

Clients need to know:

Where should I connect?

Suppose a web service listens at:

TEXT

```
Server IP = 20.1.1.5
Port      = 443
```

The server needs a known local port.

So it binds its socket to that service endpoint.

10. CAN A CLIENT USE `bind()` ?

Yes, but usually it doesn't need to explicitly do so.

A normal TCP client often lets the OS automatically select:

- appropriate local IP
- ephemeral source port

when `connect()` is called.

So typical client sequence:

TEXT

```
socket()
connect()
```

not:

TEXT

```
socket()
bind()
listen()
...
```

& But `bind()` is **not forbidden** on clients.

It is simply often optional.

11. EPHEMERAL PORT

Suppose client connects to:

TEXT

```
Server = 93.184.216.34:443
```

Operating system might assign client:

TEXT

```
192.168.1.20:53142
```

Here:

`\[ 53142 \]`

is an automatically selected temporary source port.

This is commonly called:

Ephemeral port

12. `listen()`

After binding, TCP server calls:

`\boxed{listen()}\`

This changes the socket into a:

Listening socket

Meaning:

"I am ready to receive incoming TCP connection requests."

13. `listen()` DOES NOT ACCEPT A CLIENT

& Important GATE trap.

`listen()`:

prepares the socket to accept connections.

`accept()`:

obtains a particular established incoming connection.

Do not swap them.

14. `accept()`

The server calls:

`\boxed{accept()}\`

to wait for / obtain an incoming TCP connection.

When a client successfully connects, `accept()` returns:

a new connected socket

used for communication with that client.

~~Extremely~~ extremely important.

15. WHAT HAPPENS TO THE ORIGINAL LISTENING SOCKET?

It remains available for:

accepting additional clients.

So server has conceptually:

TEXT

```
Listening socket
|
+---- connected socket for Client A
|
+---- connected socket for Client B
|
+---- connected socket for Client C
```

This distinction is a favorite conceptual question.

16. OVERVIEW

Listening socket welcomes everyone.

Accepted socket talks to one client.

17. EXAMPLE

Server creates:

TEXT

```
Socket L
```

Then:

TEXT

```
bind(L)
listen(L)
```

Client A connects.

Server executes:

TEXT

```
A = accept(L)
```

Now:

- **L** remains listening.
- **A** is connected to Client A.

Then Client B connects:

TEXT

```
B = accept(L)
```

Now:

- L still listens.
- A communicates with Client A.
- B communicates with Client B.

18. GATE TRAP

`accept()` does **not** normally transform the listening socket itself into the connected client socket.

Instead:

`\boxed{\text{accept() returns a new connected socket}}`

while the listening socket remains.

19. `connect()`

The TCP client calls:

`\boxed{connect()}\`

with the server's:

- IP address
- port number

Conceptually:

TEXT

```
connect(serverIP, serverPort)
```

This initiates TCP connection establishment.

20. CONNECTION TO THREE-WAY HANDSHAKE

When client calls:

TEXT

```
connect()
```

the OS's TCP implementation performs the TCP handshake:

TEXT

```
SYN  
SYN + ACK  
ACK
```

The application does **not** manually create these TCP handshake packets under normal socket usage.

21. SERVER SIDE AND HANDSHAKE

The server's TCP stack receives the connection request while the socket is listening.

Once connection establishment succeeds, `accept()` can return the connected socket.

Thus conceptually:

```
TEXT
Client connect()
! "
TCP three-way handshake
! "
Server accept() returns connection
```

Exact blocking/timing details depend on OS implementation, but this is the right GATE mental model.

22. `send()` / `write()`

After TCP connection is established, an application can send data.

Common APIs include:

```
TEXT
send()
```

or:

```
TEXT
write()
```

They place application bytes into the socket/TCP send buffer.

TCP then handles:

- segmentation
- sequence numbers
- retransmission
- ACK processing
- flow control
- congestion control

The application normally doesn't manage those manually.

23. `recv()` / `read()`

The receiver retrieves available TCP byte-stream data using calls such as:


```
TEXT
recv()
```

or:

```
TEXT
read()
```

Again remember:

TCP provides a byte stream.

One `send()` is not guaranteed to correspond to exactly one `recv()`.

24. VERY IMPORTANT — TCP DOES NOT PRESERVE MESSAGE BOUNDARIES

Suppose sender executes:

```
TEXT
send("ABC")
send("DEF")
```

Receiver might obtain:

```
TEXT
recv() ! " A B C D E F "
```

or:

```
TEXT
recv() ! " A B "
recv() ! " C D E F "
```

or other divisions of the same ordered byte stream.

TCP guarantees:

byte order/reliability,

not application-message boundaries.

25. `close()`

`close()` releases the socket from the application's perspective.

For an established TCP socket, orderly closure may trigger TCP termination behavior such as FIN exchange.

Recall:

TEXT

F I N ! ' A C K ! ' F I N ! ' A C K

in a typical graceful close.

26. SERVER CLOSING CONNECTED SOCKET vs LISTENING SOCKET

Suppose server finished serving Client A.

It should close:

Client A's connected socket.

That does not necessarily mean:

stop the server.

The listening socket can remain open to serve other clients.

27. SERVER TERMINATION

Only when the server no longer wants to accept new connections would it also close its:

listening socket.

This distinction is important.

28. BLOCKING CALLS

Many basic socket APIs are blocking by default.

For example:

accept()

May wait until a connection becomes available.

recv()

May wait until data arrives.

Conceptually:

TEXT

```
accept()  
!"  
No client yet?  
!"  
wait
```

This is called:

29. BLOCKING DOES NOT MEAN FAILURE

If `accept()` is waiting:

The program isn't necessarily broken.

It's waiting for an incoming connection.

Similarly, `recv()` may wait because no data has arrived yet.

30. NON-BLOCKING / ASYNCHRONOUS SOCKETS

Operating systems also support more advanced:

- non-blocking I/O
- select/poll
- asynchronous/event-driven networking

But this is beyond your main GATE socket API requirement.

Focus on:

`[ \boxed{socket, \ bind, \ listen, \ accept, \ connect, \ send, \ recv, \ close} ]`

31. TCP SERVER FULL FLOW

Suppose a server wants to listen on port:

`[ 8080 ]`

Step 1 — `socket()`

Create TCP socket.

Step 2 — `bind()`

Associate with:

TEXT

LocalIP:8080

Step 3 — `listen()`

Mark as passive/listening socket.

Step 4 — `accept()`

Wait for client.

Step 5 — `recv()/send()`

Exchange application data using new connected socket.

Step 6 — `close()`

Close client connection when finished.

Then server can continue accepting other clients.

32. TCP CLIENT FULL FLOW

Client wants server:

```
TEXT
20.1.1.10:8080
```

Step 1

```
TEXT
socket()
```

Step 2

```
TEXT
connect(20.1.1.10, 8080)
```

Step 3

```
TEXT
send()/recv()
```

Step 4

```
TEXT
close()
```

33. MASTER CALL ORDER

Server

```
\boxed{ socket \rightarrow bind \rightarrow listen \rightarrow accept \rightarrow read/write \rightarrow close } \]
```

Client

```
\boxed{ socket \rightarrow connect \rightarrow read/write \rightarrow close } \]
```

34. MEMORY TRICK FOR SERVER

S-B-L-A

Some Boys Like Apples

- Socket
- Bind
- Listen
- Accept

Then:

communicate !' close.

35. MEMORY TRICK FOR CLIENT

Create !' Connect !' Communicate !' Close

Very simple.

36. WHAT DOES `bind()` ACTUALLY BIND?

The socket gets associated with local addressing information.

For Internet sockets, think:

`\boxed{LocalIP:LocalPort}`

Example:

TEXT

`0.0.0.0:8080`

can conceptually mean:

listen on port 8080 on applicable local interfaces,

depending on API/system semantics.

You don't need OS-specific detail for basic GATE.

37. SERVER PORT vs CLIENT PORT

Suppose:

TEXT

`Server = 20.1.1.10:80`

Three clients:

TEXT

```
Client A = 10.0.0.1:50001
Client B = 10.0.0.2:50002
Client C = 10.0.0.3:50003
```

All can connect to the same:

$\boxed{\text{ServerPort}=80}$

because TCP connections differ by their endpoint tuple.

38. CONNECTION IDENTIFICATION RECALL

Each TCP connection can be distinguished by:

$\boxed{(\text{SrcIP}, \text{SrcPort}, \text{DstIP}, \text{DstPort})}$

So:

TEXT

```
1 0 . 0 . 0 . 1 : 5 0 0 0 1 ! ' 2 0 . 1 . 1 . 1 0 : 8 0
```

and:

TEXT

```
1 0 . 0 . 0 . 2 : 5 0 0 0 2 ! ' 2 0 . 1 . 1 . 1 0 : 8 0
```

are different connections.

39. CAN TWO CLIENTS USE THE SAME SOURCE PORT?

Potentially yes if other tuple components differ.

For example:

TEXT

```
1 0 . 0 . 0 . 1 : 5 0 0 0 ! ' S e r v e r : 8 0
1 0 . 0 . 0 . 2 : 5 0 0 0 ! ' S e r v e r : 8 0
```

are distinct because source IPs differ.

Recall NAT can also translate identical private ports into different public ports.

40. LISTENING SOCKET IDENTIFICATION

The listening server mainly waits for connections directed toward its local service endpoint, such as:

TEXT

```
20.1.1.10:80
```

When accepted, a particular connection is associated with both local and remote endpoint details.

Example:

TEXT

```
Local  = 20.1.1.10:80
Remote = 10.0.0.1:50001
```

41. SOCKET DESCRIPTOR

On Unix-like systems, `socket()` and `accept()` return a:

file descriptor

which is essentially an integer handle used by the process.

Example conceptually:

TEXT

```
listenfd = socket(...)
connfd   = accept(listenfd, ...)
```

Then:

TEXT

```
recv(connfd, ...)
send(connfd, ...)
```

GATE may ask conceptually:

`accept()` returns another descriptor/socket for the established connection.

42. WHY DOES SERVER NEED TWO SOCKETS?

Suppose only one listening socket existed and it became permanently dedicated to first client.

Then server could no longer accept new clients conveniently.

Instead:

TEXT

```
Listening socket
      !"  a c c e p t
Connected socket #1

Listening socket
      !"  a c c e p t
Connected socket #2
```

This allows one server service to support many clients.

43. CONCURRENT SERVER — SMALL FOUNDATION

A real server can handle multiple accepted sockets using:

- processes
- threads
- event loops

Example:

TEXT

```
listen socket
|
+ - - C l i e n t   A   s o c k e t   ! '   W o r k e r   A
|
+ - - C l i e n t   B   s o c k e t   ! '   W o r k e r   B
```

For your syllabus:

understand why multiple connected sockets exist.

No need to study server architecture deeply.

44. LISTEN BACKLOG

A server's `listen()` call usually includes a backlog-related parameter.

Conceptually it concerns:

pending connection requests waiting to be accepted.

Do not interpret backlog simply as:

maximum total clients server can ever serve.

It relates to queued/pending connection state.

For GATE, this distinction is enough.

45. `listen()` vs `accept()` — MASTER COMPARISON

<code>listen()</code>	<code>accept()</code>
Makes socket passive/listening	Gets a particular incoming connection
Called after <code>bind()</code>	Called after <code>listen()</code>
Keeps listening socket	Returns new connected socket
Does not transfer app data	Returned socket can transfer data

46. `bind()` vs `connect()`

bind()	connect()
Assign local endpoint	Initiate peer connection
Commonly server	Commonly client
Local IP/port	Remote server IP/port
Does not itself establish TCP session	Initiates TCP handshake

47. accept() vs connect()

Client:

\[connect() \]

actively initiates.

Server:

\[accept() \]

receives/accepts a connection.

Memory:

Client connects. Server accepts.

48. GATE CALL-ORDER QUESTION

Which server order is correct?

A

TEXT

s o c k e t ! ' a c c e p t ! ' b i n d ! ' l i s t e n

'L

B

TEXT

s o c k e t ! ' b i n d ! ' l i s t e n ! ' a c c e p t

,

C

TEXT

b i n d ! ' s o c k e t ! ' l i s t e n ! ' c o n n e c t

'L

D

TEXT

```
s o c k e t ! '   c o n n e c t ! '   a c c e p t
```

'L for standard TCP server.

Answer:

\[\boxed{B} \]

49. ANOTHER GATE QUESTION

Client sequence:

A

TEXT

```
s o c k e t ! '   l i s t e n ! '   c o n n e c t
```

'L

B

TEXT

```
s o c k e t ! '   b i n d ! '   a c c e p t
```

'L

C

TEXT

```
s o c k e t ! '   c o n n e c t ! '   s e n d / r e c v
```

,

50. CONNECTION ESTABLISHMENT TIMELINE

Conceptually:

TEXT

SERVER

CLIENT

```
socket()
bind()
listen()
```

```
accept() <--- waiting
```

```
socket()
connect()
```

```
SYN <-----+
SYN+ACK ----->
ACK <-----
```

```
accept() returns
```

Then:

```
TEXT
send/recv <=====> send/recv
```

51. DOES `accept()` SEND THE SYN+ACK ITSELF?

At API level:

The kernel's TCP implementation handles protocol packets.

The application calls:

```
TEXT
accept()
```

It does not manually construct SYN+ACK.

Similarly, `connect()` triggers kernel TCP behavior.

52. SOCKET API vs TCP PROTOCOL

Keep the distinction:

Socket API

Functions applications call:

```
TEXT
socket()
connect()
send()
recv()
```

TCP protocol

Network behavior underneath:

- SYN
- ACK
- sequence numbers
- retransmissions
- flow control
- congestion control

The socket API is the application's interface to that protocol stack.

53. GATE CONNECTION — LAYERING

This is a beautiful application of our first chapter.

Application:

uses transport-layer service through an interface.

Socket API is essentially:

an interface through which application processes access networking/transport services.

Recall:

Service = what lower layer provides.

Interface = how upper layer accesses it.

The socket API is a concrete example of this idea.

54. CLIENT DOES NOT NEED TO KNOW ROUTE

The client application calls:

TEXT

```
connect(ServerIP, Port)
```

It doesn't calculate:

- Dijkstra
- routing table
- Ethernet next-hop MAC
- CRC

Lower layers handle those tasks.

This is layering in practice.

55. TCP SOCKET vs UDP SOCKET — SMALL PREREQUISITE

Although your syllabus says socket API in the TCP section, GATE may contrast TCP and UDP socket calls.

For TCP server:

TEXT

```
socket  
bind  
listen  
accept
```

For UDP server, typically:

TEXT

```
socket  
bind  
recvfrom/sendto
```

UDP does not use TCP's:

- `listen()`
- `accept()`
- connection handshake

in the standard connectionless server model.

56. TCP CLIENT vs UDP CLIENT

TCP client:

TEXT

```
socket()  
connect()  
send()/recv()
```

UDP commonly can use:

TEXT

```
socket()  
sendto()/recvfrom()
```

without a connection establishment handshake.

Some systems allow `connect()` on UDP sockets for convenience, but that does not create a TCP-like connection.

For GATE basics:

TCP has connection establishment; UDP does not.

57. MESSAGE BOUNDARIES — TCP vs UDP

TCP

Byte stream:

does not preserve message boundaries.

UDP

Datagram-oriented:

preserves datagram boundaries.

This can appear in socket API conceptual questions.

58. GATE QUESTION — SERVER HAS THREE CLIENTS

Suppose a TCP server has:

- one listening socket
- three active accepted client connections

How many sockets does server process conceptually need?

$\lfloor 1+3 = \boxed{4} \rfloor$

One:

listening socket

Three:

connected sockets.

$\emptyset = \text{None}$ numerical/conceptual pattern.

59. EXAMPLE

Server currently has:

$\lfloor N \rfloor$

simultaneous active client connections and one listening socket.

Number of server-side sockets:

$\lfloor \boxed{N+1} \rfloor$

assuming one socket per active connection in the simple model.

60. `accept()` LOOP

A basic iterative/concurrent server often does conceptually:

TEXT

```
socket()
bind()
listen()

while server_running:
    client_socket = accept()
    serve(client_socket)
    close(client_socket)
```

The listening socket remains open across iterations.

61. SERVER PORT STAYS SAME

Suppose server listens:

```
TEXT
20.1.1.1:80
```

Client A:

```
TEXT
10.0.0.1:5001
```

Client B:

```
TEXT
10.0.0.2:6001
```

Server can keep local port:

`\[ 80 \]`

for both connections.

Connections differ because remote endpoints differ.

62. WHAT HAPPENS IF `bind()` PORT ALREADY USED?

Generally, binding another incompatible socket to the same local address/port can fail unless special socket options/configurations allow sharing.

For GATE:

A port cannot normally be arbitrarily bound by multiple conflicting server sockets.

No need to dive into OS-specific reuse options.

63. WELL-KNOWN PORTS — USEFUL

Some common service ports:

Protocol/Service	Port
HTTP	80
HTTPS	443
DNS	53

DNS may use UDP and TCP depending on operation.

We'll cover DNS later.

These values can occasionally appear in networking questions.

64. PORT NUMBER RANGES — LOW PRIORITY

Port field:

\[16bits \]

Range:

\[0-65535 \]

Traditional broad categories include:

- well-known ports
- registered ports
- dynamic/private ports

But exact range memorization is lower priority than the socket call sequence.

65. SOCKET ERROR CASE

Suppose client calls:

TEXT

```
connect ( )
```

but:

- server is not listening
- address unreachable
- connection times out
- connection refused

Then `connect ( )` may fail.

A successful `socket ( )` call alone does not guarantee:

a working TCP connection.

66. `send ( )` SUCCESS DOESN'T MEAN RECEIVER APP ALREADY READ IT

One subtle conceptual point.

When `send ( )` returns successfully, it often means:

data has been accepted into local networking buffers according to API semantics.

It does not necessarily mean:

receiver application has already processed the data.

TCP acknowledgements and application reads happen separately.

For GATE, just avoid equating `send ( )` completion with remote application completion.

67. `recv()` RETURNING ZERO — USEFUL CONCEPT

In many stream socket APIs, if `recv()` returns zero bytes:

peer has performed an orderly shutdown of its sending side / connection reached EOF.

This is more programming-specific and lower priority, but useful if a question uses socket semantics.

68. HALF-CLOSE — LOW PRIORITY

TCP is full duplex.

One side can conceptually stop sending while still receiving.

This is why connection closure can be directional.

The socket API can support this using operations such as shutdown.

But not central for your syllabus.

69. SOCKET ADDRESS

A server endpoint needs something equivalent to:

`\[ (IP,\Port) \]`

This is often called:

socket address

Don't confuse "socket address" with the entire TCP connection identity.

A connection uses both endpoints.

70. TCP CONNECTION = TWO ENDPOINTS

Client endpoint:

`\[ (IP_c,Port_c) \]`

Server endpoint:

`\[ (IP_s,Port_s) \]`

Connection:

`\[ \boxed{ (IP_c,Port_c,IP_s,Port_s) } \]`

This is why one server port can support thousands of clients.

71. HOW GATE CAN ASK SOCKET API

Pattern 1 — Correct call sequence

Server:

`\[ socket\to bind\to listen\to accept \]`

Client:

`\[ socket\to connect \]`

Pattern 2 — Which call creates listening socket?

Answer:

`listen()` changes bound socket into listening/passive state.

Pattern 3 — Which call initiates TCP connection?

Answer:

`connect()`.

Pattern 4 — Which call returns connected socket?

Answer:

`accept()`.

Pattern 5 — Number of sockets

One listening socket + one accepted socket per active connection.

Pattern 6 — `bind()` purpose

Associate socket with local IP/port.

Pattern 7 — Connection identity

4-tuple.

Pattern 8 — TCP vs UDP API

TCP:

listen/accept/connect handshake behavior.

UDP:

connectionless sendto/recvfrom style.

72. & pGATE TRAPS

Trap 1

Correct TCP server sequence:

`\[ \boxed{socket\to bind\to listen\to accept} \]`

Trap 2

`listen()` does not accept a particular client.

Trap 3

`accept()` returns a **new connected socket**.

Trap 4

Original listening socket remains available for future clients.

Trap 5

Client usually doesn't call `listen()` or `accept()`.

Trap 6

Server normally doesn't use `connect()` to passively receive ordinary client connections.

Trap 7

`bind()` does not perform the three-way handshake.

Trap 8

`connect()` initiates TCP connection establishment.

Trap 9

Socket "" port.

Trap 10

TCP does not preserve application message boundaries.

Trap 11

One server port can support many simultaneous TCP connections because connection identity includes both endpoints.

73. MASTER MEMORY SYSTEM

Server

S-B-L-A

Socket Bind Listen Accept

Then:

Send/Receive ! Close

Client

S-C-S-R-C

Socket Connect Send Receive Close

Don't worry about exact send/receive order; application decides that.

74. CONNECTION MEMORY

Client connects. Server accepts.

Listen socket listens; accepted socket talks.

Those two lines are enough for most conceptual questions.

'p NOTEBOOK VERSION

TCP Socket API

A socket is an application interface to transport/network services.

TCP Server

$\boxed{\text{socket() } \rightarrow \text{bind() } \rightarrow \text{listen() } \rightarrow \text{accept() } }$

Then:

TEXT

```
send()/recv()  
close()
```

socket()

Create socket.

bind()

Associate local:

$\boxed{\text{IP:Port}}$

listen()

Make socket passive/listening.

accept()

Accept incoming TCP connection and return:

$\boxed{\text{new connected socket}}$

Original listening socket remains open for other clients.

TCP Client

$\boxed{\text{socket() } \rightarrow \text{connect() } }$

Then:

TEXT

```
send()/recv()  
close()
```

connect()

Initiates TCP connection/three-way handshake to server.

Connection Identity

$\boxed{\text{SrcIP, SrcPort, DstIP, DstPort}}$

Socket Count

For (N) active clients:

$\boxed{N+1}$

server-side sockets in simple model:

- 1 listening
- N connected

TCP vs UDP

TCP server:

TEXT

```
s o c k e t ! '   b i n d ! '   l i s t e n ! '   a c c e p t
```

UDP server typically:

TEXT

```
s o c k e t ! '   b i n d ! '   r e c v f r o m / s e n d t o
```

TCP:

byte stream + connection establishment.

UDP:

datagram-oriented + connectionless.

75. TCP CHAPTER — 5-MINUTE REVISION

TCP Service

- reliable
- connection-oriented
- ordered
- full-duplex byte stream

Sequence Numbers

$\boxed{\text{Seq=FirstByte}}$

If payload length (L) :

$\boxed{\text{NextSeq=Seq+L}}$

SYN and FIN consume one sequence number.

ACK

$\boxed{\text{ACK=NextExpectedByte}}$

Cumulative.

Flow Control

$\lfloor \boxed{\text{rwnd}} \rfloor$

Protects:

receiver.

Congestion Control

$\lfloor \boxed{\text{cwnd}} \rfloor$

Protects:

network.

Effective window:

$\lfloor \boxed{\min(\text{rwnd}, \text{cwnd})} \rfloor$

Slow Start

$\lfloor 1, 2, 4, 8, \dots \rfloor$

Exponential.

Congestion Avoidance

$\lfloor 8, 9, 10, 11, \dots \rfloor$

Linear.

Loss

Timeout:

$\lfloor \text{cwnd} \backslash \text{to} 1 \rfloor$

Three duplicate ACKs:

Fast Retransmit.

Reno:

Fast Recovery.

RTO

$\lfloor \text{EstimatedRTT} = (1 - \alpha) \text{Old} + \alpha \text{Sample} \rfloor$

$RTO = EstimatedRTT + 4DevRTT$

Socket API

Server:

$\boxed{S \rightarrow B \rightarrow L \rightarrow A}$

Client:

$\boxed{S \rightarrow C}$

76. 30-SECOND REVISION

Memorize:

TEXT

SERVER:

s o c k e t ! ' b i n d ! ' l i s t e n ! ' a c c e p t

CLIENT:

s o c k e t ! ' c o n n e c t

Then:

TEXT

s e n d ! " r e c v

And:

accept returns a new connected socket.

Listening socket remains listening.

ACTIVE RECALL — DON'T LOOK BACK

Q1

Write the correct TCP server socket-call sequence before data transfer.

Q2

Write the normal TCP client sequence before data transfer.

Q3

What is the purpose of:

$\boxed{\text{bind()}}$

?

Q4

What is the difference between:

- `listen()`
- `accept()`

?

Q5 — GATE Trap

A server executes:

TEXT

```
client_fd = accept(listen_fd)
```

After successful `accept()`, which statement is correct?

A. `listen_fd` is destroyed B. `client_fd` is a new connected socket C. `client_fd` becomes a router D. Server must call `bind()` again

Q6

Which socket API call on the client normally initiates the TCP three-way handshake?

Q7

A server has 8 simultaneous connected clients and one listening socket.

How many server-side sockets are present in the simple model?

Q8

Why can many clients connect simultaneously to the same server IP and server port?

Q9 — Layering Connection

What is the relationship between:

- socket API
- TCP protocol

?

Q10

True or False:

One TCP `send()` call is guaranteed to correspond to exactly one `recv()` call.

Explain.

Q11

Which calls are typically used by a UDP server instead of TCP's `listen()` and `accept()` ?

Q12 — Master Question

Arrange these calls for a TCP server:

TEXT

```
accept()  
socket()  
listen()  
bind()
```

in the correct order.

NEXT !' DNS: Name Resolution, DNS Hierarchy, Recursive vs Iterative Queries, Resource Records, Caching & GATE Resolution Numericals

USER

next

ASSISTANT

CHAPTER 13 — APPLICATION LAYER

TOPIC 1 — DNS: Hierarchy, Name Resolution, Recursive vs Iterative Queries, Resource Records, Caching & GATE Numericals

VERY HIGH PRIORITY FOR GATE CONCEPTUAL + RTT QUESTIONS

DNS answers a simple problem:

Humans prefer names like `www.example.com`, but IP networking ultimately needs an IP address.

So DNS performs:

$\boxed{\text{Domain Name}} \rightarrow \text{IP Address}$

The core mental model is:

Root !' TLD !' Authoritative Server !' Answer

1. WHAT IS DNS?

DNS stands for:

Domain Name System

It is a distributed, hierarchical naming system.

Example:

TEXT

www.example.com

!”

93.184.216.34

DNS allows applications to use human-readable names instead of memorizing numerical IP addresses.

2. WHY NOT KEEP ONE CENTRAL DNS SERVER?

Imagine one server storing the mapping for every domain on Earth.

Problems:

- single point of failure
- enormous database
- enormous traffic
- high latency for distant users
- difficult administration
- poor scalability

Therefore DNS is:

$\boxed{\text{Distributed + Hierarchical}}$

Ø=6% keyword.

3. DNS HIERARCHY

Consider:

TEXT

www.example.com

DNS hierarchy conceptually looks like:

TEXT

```
Root
|
.
|
com
|
example
|
www
```

The hierarchy contains different levels of DNS servers.

The three key server types are:

1. Root DNS servers
2. Top-Level Domain (TLD) DNS servers
3. Authoritative DNS servers

Plus the client normally talks to a:

4. Local/recursive DNS resolver

4. ROOT DNS SERVER

At the top is the DNS root:

TEXT

.

A root server usually does not directly tell you the final IP of arbitrary sites.

Instead, it can direct the resolver toward the appropriate:

TLD server

Example request:

TEXT

www.example.com

Root essentially indicates:

Ask the .com DNS infrastructure.

5. TLD SERVER

TLD means:

Top-Level Domain

Examples:

TEXT

.com
.org
.net
.edu
.in

For:

TEXT

www.example.com

the relevant TLD is:

TEXT

.com

The .com TLD server can direct the resolver to the authoritative DNS server responsible for:

TEXT

example.com

6. AUTHORITATIVE DNS SERVER

The authoritative DNS server contains DNS records for the domain it is authoritative for.

For example, it may know:

TEXT

www.example.com !' IP address

Thus the typical hierarchy is:

TEXT

Root

!"

.com TLD

!"

example.com authoritative DNS

!"

Final record

7. LOCAL DNS RESOLVER

Your computer usually does not manually contact root, then TLD, then authoritative servers itself.

Instead, it normally asks a DNS resolver.

This may be operated by:

- ISP
- enterprise
- public DNS service
- local network

Conceptually:

TEXT

Your Host

!"

Local / Recursive Resolver

!"

DNS hierarchy

The resolver performs the lookup on your behalf and usually caches the result.

8. COMPLETE DNS LOOKUP

Suppose browser needs:

```
TEXT
www.example.com
```

A simplified lookup is:

```
TEXT
Host
!"
Local Resolver
!"
Root DNS
!"
.com TLD DNS
!"
Authoritative DNS for example.com
!"
IP address
```

Then the answer returns to your host.

9. MEMORY TRICK

Remember:

R-T-A

R — Root

Which TLD?

T — TLD

Which authoritative server?

A — Authoritative

Here is the actual record.

So:

```
\[ \boxed{\text{Root !' TLD !' Authoritative}} \]
```

10. DOMAIN NAME STRUCTURE

Take:

TEXT

`www.cs.example.com`

Read hierarchy from right to left:

TEXT

`. ! ' c o m ! ' e x a m p l e ! ' c s ! ' w w w`

The right side is more general.

The left side becomes more specific.

11. FQDN

A fully qualified domain name conceptually specifies the complete name in the DNS hierarchy.

Example:

TEXT

`www.example.com.`

The final dot represents the root.

It is usually omitted in everyday writing:

TEXT

`www.example.com`

12. LABELS

A DNS name consists of labels separated by dots.

Example:

TEXT

`mail.example.com`

Labels:

TEXT

`mail
example
com`

Do not interpret the dots as part of individual labels.

13. DNS IS DISTRIBUTED

No one server normally stores all Internet DNS information.

Different organizations control their own portions of the namespace.

Example:

```
TEXT
.com
|
example.com
|
www.example.com
```

Administration can be delegated down the hierarchy.

This is one reason DNS scales globally.

14. DNS NAME RESOLUTION

Name resolution means:

Finding DNS information associated with a domain name.

Often this means finding an IP address, but DNS can store several kinds of records.

So DNS is not merely an:

"IP-address database."

It stores different resource records.

15. TWO QUERY STYLES

One important GATE concept.

DNS lookup discussions commonly distinguish:

1. Recursive query
2. Iterative query

16. RECURSIVE QUERY

In a recursive query, the queried DNS server is asked:

Give me the final answer, not merely a referral.

Conceptually:

```
TEXT
Client !' Resolver :
"What is the address of www.example.com?
Find the answer for me."
```

The recursive resolver takes responsibility for completing the lookup or returning an error.

17. ITERATIVE QUERY

In an iterative query, a DNS server can respond:

I don't have the final answer, but here is a server that is closer to it.

Example:

Resolver asks root:

TEXT

Where is `www.example.com`?

Root replies:

TEXT

I don't know the final address.
Ask the `.com` TLD server.

That's an iterative/referral-style interaction.

18. TYPICAL REAL-WORLD PATTERN

Common conceptual model:

Host !' local recursive resolver

Recursive request.

Resolver !' root/TLD/authoritative hierarchy

Iterative queries/referrals.

So:

TEXT

```
Host
  | recursive
  v
Resolver
  |
  | iterative
  v
Root !' TLD !' Authoritative
```

Ø=Yes is the standard textbook picture.

19. RECURSIVE vs ITERATIVE

Recursive	Iterative
Queried server obtains final answer	Queried server can return referral
More work for queried server	Resolver follows referrals

Recursive	Iterative
Client asks "solve completely"	Resolver asks successive servers
Common host!'resolver behavior	Common resolver!'DNS hierarchy behavior

20. MEMORY

Recursive

You solve it for me.

Iterative

Tell me who to ask next.

That's enough for GATE.

21. EXAMPLE OF ITERATIVE RESOLUTION

Resolver asks root:

TEXT
www.example.com?

Root says:

TEXT
Ask .com.

Resolver asks .com:

TEXT
www.example.com?

TLD says:

TEXT
Ask example.com's authoritative server.

Resolver asks authoritative:

TEXT
www.example.com?

Authoritative gives:

TEXT

IP = X

Resolver returns X to host.

22. RESOURCE RECORDS

DNS information is stored as:

Resource Records (RRs)

A simplified DNS resource record contains fields such as:

`\boxed{(Name, \ Value, \ Type, \ TTL)} \`

Different `Type` values mean different things.

23. A RECORD

An **A record** maps a domain name to an:

IPv4 address.

Example:

TEXT

`w w w . e x a m p l e . c o m ! ' 1 9 2 . 0 . 2 . 2 0`

Type:

`\boxed{A} \`

Memory:

A !' IPv4 Address

24. AAAA RECORD

An **AAAA record** maps a domain name to an:

IPv6 address.

Why four A's?

IPv6 addresses are much larger than IPv4.

For your GATE CN syllabus, know the distinction but IPv4 is your main addressing focus.

25. NS RECORD

NS means:

Name Server

An NS record specifies an authoritative name server for a domain/zone.

Example conceptually:

TEXT

```
example.com ! ns1.example.net
```

Type:

\boxed{NS}

26. CNAME RECORD

CNAME means:

Canonical Name

It creates an alias.

Example:

TEXT

```
shop.example.com  
! "  
store.example.net
```

The first name may be an alias for the canonical hostname.

So:

\boxed{CNAME = Alias \rightarrow Canonical\ Name}

27. MX RECORD

MX means:

Mail Exchange

It identifies mail servers responsible for receiving email for a domain.

Example:

TEXT

```
example.com  
! "  
mail.example.com
```

Type:

`\boxed{MX}`

28. IMPORTANT RECORD TABLE

Record	Main purpose
A	Name !' IPv4 address
AAAA	Name !' IPv6 address
NS	Domain !' authoritative name server
CNAME	Alias !' canonical name
MX	Domain !' mail server

Memorize this.

29. & pGATE TRAP — CNAME DOES NOT DIRECTLY MEAN IP

Example:

TEXT

`www.example.com CNAME server.example.net`

This tells you another hostname.

You may then need an A/AAAA record for:

TEXT

`server.example.net`

So CNAME is:

name-to-name aliasing,

not necessarily name-to-IP directly.

30. DNS CACHING

Very important.

DNS lookup results are cached.

Suppose your resolver just looked up:

TEXT

`www.example.com`

A second user asks for the same domain shortly afterward.

Instead of contacting:

- root
- TLD
- authoritative server

again, resolver may answer from cache.

Benefits:

- lower latency
- fewer DNS messages
- less load on upstream servers
- improved scalability

31. TTL

Cached DNS records don't remain forever.

They contain a:

TTL — Time To Live

TTL tells a caching resolver roughly:

how long this record may remain cached before it should be considered expired and refreshed.

32. EXAMPLE

Suppose:

TEXT

TTL = 3600 seconds

Then the record may be cached for roughly:

[3600s=1hour]

Once the TTL expires:

resolver may need a fresh lookup.

33. MEMORY

TTL = cache lifetime

Do not confuse DNS TTL with:

IPv4 packet TTL.

Same abbreviation, different roles.

Ø=important confusing pair.

34. DNS TTL vs IP TTL

DNS TTL	IPv4 TTL
Controls cache lifetime	Limits packet lifetime/hops
Application-layer DNS data	Network-layer IP header
Measured as time	Router decrements field as packet travels
Helps expire cached records	Prevents forwarding loops forever

Do not mix them.

35. CACHING CAN GIVE TEMPORARILY OLD INFORMATION

Suppose authoritative DNS changes:

TEXT

www.example.com

Old IP !' New IP

Some resolvers may still have the old record cached until its TTL expires.

So DNS updates are not necessarily seen everywhere instantly.

This is an expected consequence of caching.

36. NEGATIVE CACHING — SMALL NOTE

DNS can also cache information such as:

a name does not exist

for a controlled period.

This is called negative caching.

Low priority for your syllabus, but useful context.

37. DNS USES UDP OR TCP?

Ø=Ø%TE-worthy.

Traditional DNS queries commonly use:

`\boxed{UDP}`

on port:

`\boxed{53}`

But DNS can also use:

`\boxed{TCP}`

port 53 when needed.

So the correct statement is not:

DNS always uses UDP.

Correct:

DNS commonly uses UDP, but TCP is also supported/used.

38. WHY UDP COMMONLY?

Many DNS queries and responses are small.

UDP avoids:

- TCP connection establishment
- extra transport overhead

So simple request-response resolution can be efficient.

39. WHEN TCP MAY BE USED

Historically/common examples include:

- zone transfers
- responses requiring TCP fallback
- cases where transport/reliability requirements make TCP appropriate

Exact modern DNS behavior has evolved, but for GATE:

DNS supports both UDP and TCP; port 53.

40. DNS OVERHEAD CONNECTION

Suppose DNS uses UDP.

Application DNS message becomes:

TEXT

DNS message

! "

UDP datagram


```

! "
IP packet
! "
Frame

```

If DNS uses TCP:

```

TEXT
DNS
! "
TCP
! "
IP

```

Layering still applies.

41. DNS RESOLUTION TIME — BASIC RTT MODEL

Common GATE-style numerical.

Suppose:

- no DNS entry cached
- resolver contacts three DNS servers sequentially:
 - root
 - TLD
 - authoritative
- each query-response costs 1 RTT
- ignore processing/transmission delay

Then DNS lookup time:

$\boxed{3RTT}$

if all three RTTs are equal to the same value.

42. NUMERICAL

Each DNS RTT:

$\boxed{20ms}$

Three sequential lookups:

$\boxed{Root=20}$

$\boxed{TLD=20}$

$\boxed{Authoritative=20}$

Total:

$\boxed{20+20+20}$

$\boxed{=60ms}$

43. DIFFERENT RTTs

Suppose:

$[RTT_{\text{root}}=30\text{ms}]$

$[RTT_{\text{TLD}}=20\text{ms}]$

$[RTT_{\text{auth}}=40\text{ms}]$

Total iterative lookup delay:

$[30+20+40]$

$[=90\text{ms}]$

assuming sequential queries and no cache.

44. INCLUDE HOST !' LOCAL RESOLVER?

& ¶Read the question carefully.

Some problems say:

Local DNS resolver already received the host's query.

Then only count external DNS lookups.

Others explicitly provide:

$[RTT_{\text{host-localDNS}}]$

Then you may need to include it.

Never automatically add or omit it.

45. EXAMPLE

Host !' Local DNS:

$[5\text{ms}]$

Local DNS !' Root:

$[20\text{ms}]$

Local DNS !' TLD:

$[30\text{ms}]$

Local DNS !' Auth:

$[25\text{ms}]$

If each represents a full query-response RTT and all are sequential:

$[5+20+30+25]$

$[=80\text{ms}]$

for the complete lookup.

46. CACHE HIT NUMERICAL

Suppose resolver already has the final A record cached.

Then it may not need:

- root lookup
- TLD lookup
- authoritative lookup

So DNS latency may reduce to approximately:

host !"local resolver RTT,

under the simplified model.

Caching can dramatically reduce lookup time.

47. PARTIAL CACHE

Resolver might know:

authoritative name server information

but not the final A record.

Then it may skip root and TLD and contact the authoritative server directly.

So GATE may ask:

Which DNS queries are avoided because of caching?

Answer depends on what exactly is cached.

48. CACHE EXAMPLE

Without cache:

```
TEXT
Root
!"
TLD
!"
Authoritative
```

Three RTTs.

Suppose resolver caches the authoritative server address/delegation.

Then next lookup for another name in same domain may require only:

TEXT

R e s o l v e r ! ' A u t h o r i t a t i v e

approximately one external DNS RTT.

49. DNS MESSAGE-COUNT QUESTION

Suppose local resolver uses iterative resolution and has no cached information.

It queries:

1. root
2. TLD
3. authoritative

That's:

$\backslash 3 \backslash$

DNS queries and:

$\backslash 3 \backslash$

responses.

Total DNS messages between resolver and those servers:

$\backslash \boxed{6} \backslash$

If host-resolver query and response are also counted:

$\backslash 6+2 = \boxed{8} \backslash$

~~0=4~~ways check what the question counts.

50. RECURSIVE QUERY MESSAGE INTUITION

Suppose a root server truly performed recursion on behalf of requester.

Then requester might send one query to root and wait for final answer, while servers communicate down the hierarchy.

From the requester's perspective:

fewer direct interactions.

But total network DNS traffic can still involve several query-response exchanges.

This is why message count depends on architecture given.

51. ITERATIVE LOOKUP NUMERICAL

Local resolver wants:

TEXT

www.abc.com

No cache.

It performs:

TEXT

```
Resolver !' Root
Root !' Resolver referral

Resolver !' .com
.com !' Resolver referral

Resolver !' abc.com authoritative
Authoritative !' Resolver final A record
```

Number of resolver-originated queries:

$\boxed{3}$

Number of responses:

$\boxed{3}$

52. DNS + HTTP CONNECTION

Very important for next chapter.

When you enter:

TEXT

http://www.example.com

browser cannot immediately contact the web server unless it already knows its IP.

Typical high-level sequence:

TEXT

1. DNS lookup
2. Obtain server IP
3. Establish TCP connection
4. Send HTTP request
5. Receive HTTP response

So page-loading GATE questions often combine:

DNS RTT + TCP RTT + HTTP RTT.

We'll do these next.

53. EXAMPLE PREVIEW

Suppose:

- DNS requires 3 RTTs
- TCP handshake takes 1 RTT before request
- HTTP request/first response takes 1 RTT
- ignore transmission time

Total:

$3RTT + 1RTT + 1RTT$

$5RTT$

This is a classic type of question.

We'll derive HTTP timing carefully next.

54. DNS AND PORT NUMBERS

DNS server listens on well-known port:

53

A client resolver usually uses a temporary source port.

Example:

TEXT

Client: 53000

!"

DNS Server: 53

Don't confuse:

- server port 53
- arbitrary client source port.

55. DNS AND NAT

Your host may have private IP:

TEXT

192.168.1.20

It sends DNS query to a public resolver.

NAT may translate the source endpoint.

DNS itself still performs:

name resolution.

NAT and DNS are completely different jobs.

56. DNS vs ARP — IMPORTANT CONFUSING PAIR

ARP wasn't explicitly in your syllabus, but this distinction is useful.

DNS

$\boxed{\text{Name} \rightarrow \text{IP}}$

ARP conceptually

$\boxed{\text{Local IP} \rightarrow \text{MAC}}$

So:

TEXT

```
www.example.com
!" DNS
IP address
!" local link resolution
MAC/next-hop delivery
```

Do not say DNS finds MAC addresses.

57. DNS vs ROUTING

DNS answers:

What IP corresponds to this name?

Routing answers:

Which next hop/path should packets use toward this IP?

So:

TEXT

```
DNS !' Destination identity / address
Routing !' How to reach destination
```

58. AUTHORITATIVE vs CACHED ANSWER

An authoritative server answers based on the zone data for which it has authority.

A caching resolver may answer from:

previously cached information.

Both may give the same result, but they are not the same role.

59. AUTHORITATIVE DOES NOT MEAN ROOT

& pGATE trap.

A root server is authoritative for the root zone/delegation information.

But the authoritative server for:

```
TEXT
example.com
```

is normally not the root server.

The hierarchy delegates responsibility.

60. ROOT SERVER DOES NOT NORMALLY STORE EVERY WEBSITE IP

Another trap:

Root DNS server knows every website's IP.

'LWrong.

Root mostly knows how to direct queries toward appropriate TLD infrastructure.

61. TLD SERVER DOES NOT NECESSARILY RETURN FINAL HOST IP

For:

```
TEXT
www.example.com
```

the `.com` server typically points toward authoritative DNS for:

```
TEXT
example.com
```

The final host record is obtained from authoritative data.

62. CNAME CHAIN

Suppose:

```
TEXT
www.example.com
  CNAME
cdn.example.net
```

Then resolver may need to resolve:

```
TEXT
cdn.example.net
```

to an address.

So one logical hostname lookup can result in additional DNS work.

For basic GATE numericals, only count such extra queries if explicitly indicated.

63. MX RECORD EXAMPLE

Suppose DNS contains:

```
TEXT
example.com MX mail.example.com
```

This says:

Mail for example.com should be handled by mail.example.com.

Then another lookup may resolve:

```
TEXT
mail.example.com
```

to an IP address.

64. NS RECORD EXAMPLE

Suppose:

```
TEXT
example.com NS ns1.provider.net
```

Meaning:

ns1.provider.net is a name server associated with authority for example.com.

Again, NS gives a hostname of a name server, not necessarily the desired web server's IP directly.

65. GATE NUMERICAL — CACHE

Without cache:

- Root RTT = 10 ms
- TLD RTT = 20 ms
- Authoritative RTT = 30 ms

Total:

$\backslash$ $10+20+30 = 60\text{ms}$ $\backslash$

If authoritative answer is cached locally:

these external lookups can be skipped.

Suppose local resolver RTT is:

[5ms]

Then lookup may take approximately:

[\boxed{5ms}]

instead of:

[65ms]

if the first case also included local resolver RTT.

66. GATE NUMERICAL — TTL

Resolver caches an A record at:

```
TEXT
10:00:00
```

with:

[TTL=600s]

TTL duration:

[600s=10min]

Ignoring implementation details, it remains valid until around:

```
TEXT
10:10:00
```

A request at:

```
TEXT
10:05
```

can use cache.

A request after expiry may require fresh resolution.

67. DNS IS APPLICATION LAYER

~~0=144~~ Layer question.

DNS is an:

[\boxed{\text{Application-layer protocol}}]

It uses transport services underneath.

Commonly:

```
TEXT
DNS
! "
UDP/TCP
```

! "
IP

68. DNS DOES NOT ESTABLISH THE WEB TCP CONNECTION

DNS may tell browser:

TEXT

w w w . e x a m p l e . c o m ! ' 2 0 3 . 0 . 1 1 3 . 2 0

After that:

browser/web client establishes TCP connection to that web server separately.

The DNS lookup itself does not automatically create the HTTP connection.

69. GATE QUESTION PATTERNS

Pattern 1 — DNS hierarchy

Order:

\[Root\rightarrow TLD\rightarrow Authoritative \]

Pattern 2 — Recursive vs iterative

Determine which party follows referrals.

Pattern 3 — Resource record

Identify:

- A
- CNAME
- MX
- NS

Pattern 4 — Cache / TTL

Determine whether query needs upstream resolution.

Pattern 5 — DNS delay

Sum sequential DNS RTTs.

Pattern 6 — Message count

Count queries + responses.

Pattern 7 — Protocol/port

DNS:

UDP/TCP, port 53.

Pattern 8 — DNS + HTTP

Calculate total webpage retrieval time.

70. & pGATE TRAPS

Trap 1

DNS:

`\[ \boxed{\text{Name !' IP}} \]`

not IP !' MAC.

Trap 2

Root server does not normally return every final website IP.

Trap 3

Recursive:

final answer requested.

Iterative:

referral may be returned.

Trap 4

A record:

IPv4.

AAAA:

IPv6.

Trap 5

CNAME:

alias != canonical hostname,

not necessarily directly to IP.

Trap 6

MX:

mail server.

NS:

name server.

Trap 7

DNS commonly uses UDP, but saying "DNS always uses UDP" is false.

Trap 8

Port:

$\boxed{53}$

Trap 9

DNS caching reduces both:

- lookup latency
- upstream DNS traffic.

Trap 10

DNS TTL and IPv4 TTL are unrelated concepts despite the same abbreviation.

Trap 11

When calculating lookup delay, only add RTTs actually required by the stated cache/query model.

71. 0> MASTER MEMORY SYSTEM

Remember:

R-T-A-C

R — Root

Find TLD.

T — TLD

Find authoritative server.

A — Authoritative

Find requested record.

C — Cache

Remember it for later.

72. RESOURCE RECORD MEMORY

A-C-M-N

A — A

IPv4 Address.

C — CNAME

Canonical alias.

M — MX

Mail.

N — NS

Name server.

And:

AAAA !' IPv6.

' p NOTEBOOK VERSION

DNS — Domain Name System

Purpose:

$\boxed{\text{Domain name} \rightarrow \text{IP address}}$

DNS is:

- distributed
 - hierarchical
-

- application layer

DNS Hierarchy

[\boxed{ Root \rightarrow TLD \rightarrow Authoritative }]

Example:

```
TEXT
www.example.com
```

Resolution:

```
TEXT
Root
!"
.com
!"
example.com authoritative
!"
final record
```

Recursive Query

Queried server/resolver should obtain final answer.

Memory:

Solve it for me.

Iterative Query

Server may return a referral to another DNS server.

Memory:

Ask this server next.

Typical:

```
TEXT
Host !' Resolver = recursive
Resolver !' Root / TLD / Auth = iterative
```

Resource Records

Type	Meaning
A	Hostname !' IPv4
AAAA	Hostname !' IPv6
CNAME	Alias !' canonical hostname
MX	Mail server
NS	Name server

Caching

Resolver stores DNS results temporarily.

Benefits:

- less delay
- less DNS traffic
- better scalability

TTL:

$\boxed{\text{Time To Live}}$

determines cache validity duration.

DNS Transport

Commonly:

$\boxed{\text{UDP} \backslash \text{port} \backslash 53}$

but DNS also supports:

$\boxed{\text{TCP} \backslash \text{port} \backslash 53}$

DNS Delay

With sequential lookups:

$\boxed{T_{\text{DNS}} = \text{RTT}_{\text{root}} + \text{RTT}_{\text{TLD}} + \text{RTT}_{\text{auth}}}$

plus host-local-resolver RTT if explicitly relevant.

Caching can remove some/all of these terms.

73. 30-SECOND REVISION

Remember:

DNS = Name !' IP

Hierarchy:

\[\boxed{\text{Root} \rightarrow \text{TLD} \rightarrow \text{Authoritative}} \]

Queries:

Recursive = solve it

Iterative = tell me who to ask

Records:

TEXT

```
A           !'   I P v 4
A A A A     !'   I P v 6
C N A M E   !'   A l i a s
M X         !'   M a i l
N S         !'   N a m e   S e r v e r
```

And:

\[\boxed{\text{DNS} \text{ uses } \text{port } 53} \]

Caching:

TTL decides how long the answer stays reusable.

ACTIVE RECALL — DON'T LOOK BACK

Q1

What is the primary purpose of DNS?

Q2

Arrange these DNS server types in the usual lookup hierarchy:

- Authoritative server
- Root server
- TLD server

Q3

For:

TEXT

www.example.com

what does the root DNS server typically help the resolver locate first?

Q4

Differentiate in one line:

- recursive DNS query
 - iterative DNS query
-

Q5 — Resource Records

Match:

1. A
2. CNAME
3. MX
4. NS

with:

A. Mail server B. IPv4 address C. Authoritative/name server D. Alias/canonical name

Q6 — GATE Trap

True or False:

A CNAME record directly maps a hostname to an IPv4 address.

Explain.

Q7

What is the purpose of DNS caching?

Give two benefits.

Q8

What does DNS record TTL control?

Q9 — Confusing Pair

What is the difference between:

- DNS TTL
- IPv4 TTL

?

Q10 — RTT Numerical

A resolver has no cached information.

RTTs are:

\[RTT_{root}=20ms \]

\[RTT_{TLD}=30ms \]

\[RTT_{auth}=40ms \]

Assuming sequential iterative lookups and ignoring all other delay, how long does resolution take?

Q11 — Message Count

The local resolver performs three iterative DNS query-response exchanges: root, TLD and authoritative.

How many DNS messages are exchanged between the resolver and these servers in total?

Q12

Which transport protocols can DNS use, and what is the well-known DNS port?

Q13 — Cache Reasoning

If the final A record is already valid in the resolver's cache, which hierarchy lookups can normally be skipped?

Q14 — Future Connection

After DNS resolves a web server's IP address, which application-layer protocol from your syllabus is typically used to request the webpage?

NEXT !' HTTP: Request–Response Model, Persistent vs Non-Persistent Connections, HTTP Methods, Status Codes & GATE RTT/Page-Load Numericals

USER

next

ASSISTANT

CHAPTER 13 — APPLICATION LAYER

TOPIC 2 — HTTP: Request–Response Model, Persistent vs Non-Persistent Connections, Methods, Status Codes & GATE RTT Numericals

~~Q=Ý%Q=Ý%Q=Ý%Q=Ý%Q=Ý%~~ **OVER HIGH PRIORITY FOR GATE**

HTTP questions often combine:

- DNS delay
- TCP connection establishment
- HTTP request/response timing
- persistent vs non-persistent connections
- parallel connections
- object download time

The single most important timing result is:

Non-persistent HTTP without transmission delay usually needs 2 RTTs per object.

Why?

$\boxed{1 \text{ RTT for TCP connection} + 1 \text{ RTT for HTTP request/response}}$

1. WHAT IS HTTP?

HTTP stands for:

Hypertext Transfer Protocol

It is an:

$\boxed{\text{Application-layer protocol}}$

used for communication between:

- web clients
- web servers

Typical model:

TEXT

```
Browser / Client
  |
  | HTTP Request
  v
Web Server
  |
  | HTTP Response
  v
Browser / Client
```

2. CLIENT-SERVER MODEL

The browser generally acts as:

HTTP client

The website's server acts as:

HTTP server

Example:

TEXT

```
Client:
"Give me /index.html"

Server:
"Here is /index.html"
```

This is a:

Request-Response protocol

3. HTTP AND TCP

Traditional HTTP versions such as HTTP/1.0 and HTTP/1.1 commonly operate over:

`\boxed{TCP}`

So conceptually:

```
TEXT
HTTP
! "
TCP
! "
IP
! "
Data Link
```

This is why HTTP timing questions often include TCP RTT.

4. HTTP PORTS

Common traditional ports:

HTTP

`\boxed{80}`

HTTPS

`\boxed{443}`

HTTPS is essentially HTTP semantics protected by TLS.

For basic GATE HTTP questions, port:

`\boxed{80}`

is worth remembering.

5. WHAT HAPPENS WHEN YOU ENTER A URL?

Suppose:

```
TEXT
http://www.example.com/page.html
```

A simplified sequence is:

TEXT

1. DNS resolves `www.example.com`
2. Client obtains server IP
3. TCP connection established
4. HTTP request sent
5. Server sends HTTP response
6. Browser processes returned object

0-1% connects DNS + TCP + HTTP.

6. URL COMPONENTS

Example:

TEXT

`http://www.example.com/images/a.jpg`

Contains conceptually:

Scheme/protocol

TEXT

`http`

Hostname

TEXT

`www.example.com`

Resource/path

TEXT

`/images/a.jpg`

DNS resolves the hostname.

HTTP requests the resource/path.

7. HTTP REQUEST

A client sends an HTTP request containing things such as:

- method
- requested resource
- protocol version
- headers
- optional body

Conceptually:

TEXT

```
GET /index.html HTTP/1.1
Host: www.example.com
...
```

You do not need to memorize full header syntax for GATE.

Focus on:

- GET
- POST
- request-response behavior
- connection timing

8. HTTP RESPONSE

Server response contains:

- status line
- headers
- optional body/content

Conceptually:

TEXT

```
HTTP/1.1 200 OK
...
<html>...</html>
```

The body might contain:

- HTML
- image
- CSS
- JavaScript
- other content

9. HTTP METHODS

Important common methods:

GET

Retrieve a resource.

POST

Send data to server for processing/resource action.

HEAD

Like GET but response normally contains headers without the resource body.

PUT

Create/replace resource at a specified URI according to semantics.

DELETE

Request deletion of a resource.

For basic GATE:

Do not memorize:

`\[ \boxed{GET, \POST, \HEAD} \]`

10. GET vs POST

GET	POST
Primarily retrieves resource	Sends data for processing/action
Parameters may appear in URL/query	Data commonly placed in request body
Common for fetching pages/resources	Common for forms/API submissions

Do not memorize:

GET = secure, POST = secure.

'L That's false.

Security depends on protocols such as HTTPS/TLS, not simply GET vs POST.

11. HTTP STATUSCODES

HTTP responses include numerical status codes.

Main categories:

Range	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client-side request error
5xx	Server-side error

Do not memorize:

2 Good, 3 Move, 4 You, 5 Server

12. COMMON STATUSCODES

200 OK

Request succeeded.

\[\boxed{200} \]

301 Moved Permanently

Resource permanently redirected.

\[\boxed{301} \]

304 Not Modified

Useful with caching/conditional requests.

Means cached copy may still be used.

400 Bad Request

Malformed/invalid request.

404 Not Found

Requested resource not found.

\[\boxed{404} \]

500 Internal Server Error

Server encountered an internal error.

\[\boxed{500} \]

13. STATUS MEMORY

TEXT

```
2 0 0  !'  O K
3 0 1  !'  M o v e d
3 0 4  !'  N o t   M o d i f i e d
4 0 4  !'  N o t   F o u n d
5 0 0  !'  S e r v e r   E r r o r
```

These are enough for most basic CN questions.

14. HTTP IS STATELESS— BASIC IDEA

HTTP is traditionally described as:

Stateless

Meaning:

Each request can be understood independently at the HTTP protocol level; the server need not inherently remember previous application state between unrelated requests.

But websites often need sessions.

They use mechanisms such as:

- cookies
- session identifiers
- authentication tokens

So:

HTTP statelessness does not mean websites can never maintain user state.

15. COOKIES — SMALL PREREQUISITE

A server can send a cookie.

Browser stores it and sends it in later relevant requests.

Conceptually:

TEXT

First request

Client -----> Server

Set-Cookie

Client <----- Server

Later request:

Cookie: xyz

Client -----> Server

This helps maintain application-level state.

Low-to-moderate GATE priority.

16. MAIN HTTP GATE TOPIC — CONNECTION TYPES

Two very important terms:

1. **Non-Persistent HTTP**
2. **Persistent HTTP**

Remember to note their difference.

PART A — NON-PERSISTENT HTTP

17. WHAT IS NON-PERSISTENT HTTP?

In non-persistent HTTP:

A separate TCP connection is used for each requested object.

Suppose webpage consists of:

- 1 HTML file
- 3 images

Total objects:

4

Non-persistent HTTP may use:

4 separate TCP connections

assuming one connection per object.

18. OBJECT MEANS WHAT?

A web page is not necessarily one object.

Example HTML:

HTML

```


```

The browser may first retrieve:

1. base HTML object

Then discover:

2. a.jpg
3. b.jpg

So total:

3 objects

This distinction is crucial for RTT questions.

19. TIMING FOR ONE NON-PERSISTENT HTTP OBJECT

Assume:

- DNS already completed
- TCP used
- transmission time negligible
- no TLS
- ordinary HTTP request

RTT 1

TCP three-way handshake.

RTT 2

HTTP request goes to server and first bytes of HTTP response return.

Therefore:

$$\boxed{2RTT}$$

before object data transfer delay is considered.

20. WHY ONE RTT FOR HANDSHAKE?

TCP:

TEXT

```
C l i e n t ! '   S Y N
S e r v e r ! '   S Y N + A C K
C l i e n t ! '   A C K
```

At approximately one RTT, client has received SYN+ACK and can complete establishment/request as assumed by standard textbook timing.

For GATE simplified calculations:

$$\boxed{TCP \setminus \text{setup} = 1RTT}$$

21. WHY ANOTHER RTT FOR HTTP?

After connection:

TEXT

```
C l i e n t ! '   H T T P   G E T
S e r v e r ! '   H T T P   r e s p o n s e   b e g i n s
```

Request travels one way.

First response bytes travel back.

Total:

$$\boxed{1RTT}$$

So:

$$\boxed{1RTT + 1RTT = 2RTT}$$

22. INCLUDE TRANSMISSION TIME

If object size is:

$$\boxed{L}$$

and bottleneck rate:

$$\boxed{R}$$

object transmission time:

$$\lceil \frac{LR}{R} \rceil$$

Thus simplified non-persistent object retrieval:

$$\boxed{T = 2RTT + \frac{LR}{R}}$$

if no DNS and other delays are being counted.

23. NUMERICAL

RTT:

$$50\text{ms}$$

Object:

$$1\text{Mb}$$

Rate:

$$10\text{Mbps}$$

TCP + HTTP RTT delay:

$$2(50) = 100\text{ms}$$

Transmission:

$$\frac{1\text{Mb}}{10\text{Mbps}} = 0.1\text{s} = 100\text{ms}$$

Total:

$$100 + 100 = \boxed{200\text{ms}}$$

24. NON-PERSISTENT HTTP — MULTIPLE OBJECTS SEQUENTIALLY

Suppose page consists of:

- 1 base HTML object
- 4 images

Total:

$$5$$

If each object is fetched sequentially using its own TCP connection and transmission times ignored:

$$5 \times 2RTT$$

$$= \boxed{10RTT}$$

after DNS.

25. & μMPORTANT

Sometimes questions say:

Base HTML has already been fetched.

Then only embedded objects remain.

If 4 images remain:

$$4 \times 2RTT = 8RTT$$

Don't accidentally count HTML again.

26. FULL DNS + NON-PERSISTENT EXAMPLE

Suppose DNS requires:

$$3RTT$$

Then one webpage object requires:

$$2RTT$$

Total:

$$3RTT + 2RTT$$

$$= \boxed{5RTT}$$

ignoring transmission time.

This connects directly to our DNS chapter.

27. PAGE WITH EMBEDDED OBJECTS

Suppose:

- DNS = 3 RTT
- base HTML = 1 object
- HTML references 3 images
- non-persistent
- objects downloaded sequentially
- ignore transmission

Total HTTP objects:

$$1 + 3 = 4$$

HTTP time:

$$4 \times 2RTT = 8RTT$$

DNS:

$$3RTT$$

Total:

$$\boxed{11RTT}$$

PART B — PARALLEL NON-PERSISTENT CONNECTIONS

28. WHAT IF IMAGES ARE FETCHED IN PARALLEL?

Suppose after receiving base HTML, browser discovers 4 images.

Instead of:

TEXT

```
Image1
then Image2
then Image3
then Image4
```

it may open multiple TCP connections simultaneously.

Then the four image retrievals can overlap in time.

29. PARALLEL TIMING

Assume:

- all four connections start simultaneously
- same RTT
- negligible transmission delay
- unlimited parallelism for question

Then retrieving all 4 images can take approximately:

$\boxed{2RTT}$

instead of:

$\boxed{8RTT}$

because each connection needs 2 RTT, but they happen in parallel.

30. COMPLETE EXAMPLE

Base HTML via non-persistent connection:

$\boxed{2RTT}$

Then 4 images in parallel:

$\boxed{2RTT}$

Total after DNS:

$\boxed{4RTT}$

If DNS took:

$\boxed{3RTT}$

grand total:

$\boxed{7RTT}$

31. & pDEPENDENCY MATTERS

Images cannot usually be requested until browser first receives/parses the base HTML and discovers their URLs.

So don't parallelize:

HTML and unknown embedded resources

from time zero unless question says resource URLs are already known.

Typically:

```
TEXT
Get HTML first
    ! "
Discover images
    ! "
Fetch images
```

PART C — PERSISTENT HTTP

32. WHAT IS PERSISTENT HTTP?

Persistent HTTP allows:

Multiple HTTP requests/responses over the same TCP connection.

Instead of:

```
TEXT
O b j e c t 1   ! '   T C P   c o n n e c t i o n   1
O b j e c t 2   ! '   T C P   c o n n e c t i o n   2
O b j e c t 3   ! '   T C P   c o n n e c t i o n   3
```

USE:

```
TEXT
One TCP connection
    ! "
Object1
Object2
Object3
...
```

This saves repeated handshake overhead.

33. NON-PERSISTENT vs PERSISTENT

Non-Persistent	Persistent
One TCP connection per object	Reuse TCP connection
Repeated TCP handshakes	Fewer handshakes
More RTT overhead	Less RTT overhead

Non-Persistent	Persistent
More connection setup cost	Better efficiency

34. HTTP/1.1

HTTP/1.1 traditionally uses persistent connections by default.

You don't need detailed standards history for GATE, but know:

Persistent HTTP became the common model.

35. PERSISTENT HTTP — NO PIPELINING

Suppose one TCP connection already exists.

Client sends:

```
TEXT
GET object1
```

waits for response.

Then:

```
TEXT
GET object2
```

waits.

This is persistent but:

without pipelining.

Each subsequent request-response can still require roughly:

$\lceil 1\text{RTT} \rceil$

before response starts.

36. TIMING EXAMPLE — PERSISTENT WITHOUT PIPELINING

Suppose:

- base HTML + 3 images
- same server
- persistent TCP
- requests issued one after another
- ignore transmission

First:

TCP setup:

$\backslash[1RTT \backslash]$

Request base HTML:

$\backslash[1RTT \backslash]$

Then each of 3 objects:

$\backslash[1RTT \backslash]$

Thus:

$\backslash[1+1+3 \backslash]$

$\backslash[=\boxed{5RTT} \backslash]$

after DNS.

Compare non-persistent sequential:

$\backslash[4\times 2RTT=8RTT \backslash]$

Persistent saves time.

37. GENERAL PERSISTENT SEQUENTIAL FORM

For:

$\backslash[N \backslash]$

objects from same server over one persistent TCP connection:

- 1 RTT to establish TCP
- $\backslash(N\backslash)$ RTTs for $\backslash(N\backslash)$ sequential HTTP request/response-start exchanges

So:

$\backslash[\boxed{(N+1)RTT} \backslash]$

ignoring transmission.

This assumes no pipelining and one request-response at a time.

38. CHECK WITH 4 OBJECTS

$\backslash[N=4 \backslash]$

$\backslash[N+1=5 \backslash]$

So:

$\backslash[\boxed{5RTT} \backslash]$

Matches our previous example.

PART D — HTTP PIPELINING

39. WHAT IS PIPELINING?

With HTTP pipelining, a client can send multiple HTTP requests over the persistent connection without waiting for each previous response to complete.

Conceptually:

```
TEXT
GET image1
GET image2
GET image3
! "
Server responses
```

This can reduce RTT waiting.

40. PERSISTENT + PIPELINING TIMING

Suppose:

- base HTML first
- then 4 embedded objects
- same server
- persistent connection
- pipeline all four object requests
- ignore transmission time

Timeline:

1 RTT

Establish TCP.

1 RTT

Request/get base HTML start.

Now URLs known.

1 RTT

Send all four requests in pipeline; responses begin returning.

Total:

$\boxed{3RTT}$

after DNS.

$\text{Classic GATE formula.}$

41. COMPARE THREE CASES

Page:

- 1 HTML
- 4 embedded objects

Total:

5

Ignore DNS/transmission.

Non-persistent sequential

$5 \times 2 = 10RTT$

Persistent, no pipelining

$1 + 5 = 6RTT$

Wait—careful.

For $(N=5)$ total objects:

$1RTT \text{ TCP} + 5RTT \text{ requests} = 6RTT$

Persistent + pipelining embedded objects

$1RTT \text{ TCP} + 1RTT \text{ HTML} + 1RTT \text{ embedded}$

$= 3RTT$

42. WHY DID EARLIER 4-OBJECT EXAMPLE GIVE 5 RTT?

Earlier total objects were:

- base HTML
- 3 images

So:

$N=4$

Persistent sequential:

$1 + 4 = 5RTT$

Correct.

Always count total objects carefully.

43. HTTP PIPELINING vs NETWORK PIPELINING

Don't confuse:

Store-and-forward packet pipelining

Packets overlap across links.

HTTP pipelining

Multiple HTTP requests sent without waiting for each response.

Different concepts.

44. MODERN HTTP NOTE

Modern HTTP/2 and HTTP/3 use more advanced multiplexing rather than classic HTTP/1.1 pipelining.

But GATE networking questions traditionally focus heavily on:

- persistent
- non-persistent
- parallel connections
- RTT calculations

So keep the textbook model clear.

45. NON-PERSISTENT HTTP FORMULA

For N objects fetched sequentially with separate TCP connections:

$$T \approx 2N(RTT)$$

ignoring object transmission and DNS.

If individual object transmission times are T_i :

$$T = \sum_{i=1}^N (2RTT_i + T_i)$$

under the appropriate simplified model.

46. PERSISTENT SEQUENTIAL FORMULA

For N objects from the same server:

$$T \approx RTT_{TCP} + N(RTT_{HTTP})$$

If same RTT:

$$T \approx (N+1)RTT$$

ignoring transmission.

47. PERSISTENT PIPELINED FORMULA

If:

- one base HTML must first be retrieved
- it references any number of objects on same server
- all subsequent requests can be pipelined
- transmission ignored

Then:

$$T \approx 3RTT$$

after DNS:

1. TCP connection
2. base HTML request
3. all embedded requests

assuming one already-established TCP connection is reused.

48. ADD DNS

If DNS takes:

T_{DNS}

Then:

$T_{\text{total}} = T_{\text{DNS}} + T_{\text{HTTP}}$

Example DNS:

$2RTT$

persistent pipelined page:

$3RTT$

Total:

$5RTT$

49. FULL GATE NUMERICAL 1

A webpage contains:

- base HTML
- 5 small images

Assume:

- DNS already resolved
- non-persistent HTTP
- sequential retrieval
- same $RTT = 40 \text{ ms}$
- negligible transmission time

Total objects:

6

Each:

$2RTT = 80 \text{ ms}$

Total:

$6 \times 80 = 480 \text{ ms}$

50. SAME PAGE — PERSISTENT

One TCP connection.

Total objects:

6

TCP setup:

$[40\text{ms}]$

HTTP request-response waits:

$[6 \times 40 = 240\text{ms}]$

Total:

$[40 + 240 = \boxed{280\text{ms}}]$

51. SAME PAGE — PERSISTENT PIPELINING

TCP setup

$[40\text{ms}]$

Base HTML request

$[40\text{ms}]$

All 5 image requests pipelined

$[40\text{ms}]$

Total:

$[\boxed{120\text{ms}}]$

ignoring transmission.

52. ADD DNS TO SAME EXAMPLE

Suppose DNS resolution requires:

$[3]$

sequential DNS RTTs of:

$[20\text{ms}]$

DNS delay:

$[3 \times 20 = 60\text{ms}]$

Then persistent pipelined page:

$[60 + 120 = \boxed{180\text{ms}}]$

53. DIFFERENT DNS RTT AND HTTP RTT

Don't assume all RTTs equal unless question says so.

Suppose:

- DNS lookup = 70 ms total
- client-server RTT = 50 ms
- one non-persistent object

HTTP:

$$2 \times 50 = 100 \text{ ms}$$

Total:

$$70 + 100 = \boxed{170 \text{ ms}}$$

54. OBJECT TRANSMISSION TIME

Suppose object has size:

$$2 \text{ MB}$$

Rate:

$$20 \text{ Mbps}$$

Convert:

$$2 \text{ MB} = 16 \text{ Mb}$$

Transmission:

$$\frac{16 \text{ Mb}}{20 \text{ Mbps}} = 0.8 \text{ s}$$

If RTT:

$$100 \text{ ms} = 0.1 \text{ s}$$

non-persistent total:

$$2(0.1) + 0.8 = \boxed{1.0 \text{ s}}$$

55. & μBYTE vs BIT TRAP RETURNS

Object:

$$1 \text{ MB}$$

is not:

$$1 \text{ Mb}$$

Multiply bytes by:

$$8$$

when dividing by bps.

56. HTTP REQUEST SIZE

GATE timing questions often say:

Ignore HTTP request transmission time.

Why?

HTTP GET may be small relative to large object.

Then one RTT captures:

- request propagation
- response first-byte propagation

and object transmission is separately:

$\lfloor L/R \rfloor$

Read assumptions carefully.

57. BASE HTML TRANSMISSION

If question gives non-negligible HTML size:

include its transmission time too.

Don't treat every object as zero-size just because RTT formulas are familiar.

General principle:

$\lfloor \boxed{\text{Total time} = \text{waiting delays} + \text{transmission times}} \rfloor$

according to stated model.

58. PARALLEL CONNECTIONS AND TRANSMISSION RATE

A subtle trap:

If 4 objects are fetched in parallel over a single shared bottleneck, each may not receive the full link rate.

Do not simply say every object transfers at full $\lfloor R \rfloor$ unless question states independent/full rates.

If total bottleneck capacity is shared:

throughput sharing matters.

For many GATE RTT-only questions transmission time is ignored precisely to avoid this complexity.

59. PERSISTENT HTTP BENEFITS

Persistent connections provide:

- fewer TCP handshakes
- lower RTT overhead
- less repeated connection state
- better performance for pages with many objects

Also, an existing TCP connection may already have a larger `cwnd` than a brand-new connection, although this is beyond basic HTTP timing.

60. NON-PERSISTENT DISADVANTAGE

For each object:

TEXT

TCP setup
HTTP request
TCP close

Repeated setup wastes:

- RTT
- processing
- connection resources

particularly when many small objects exist.

61. HTTP CACHING

Browsers and intermediate caches can store web objects.

If object is cached:

browser may avoid downloading the full object again.

HTTP supports mechanisms for validating cached objects.

One important status:

`304 Not Modified`

62. CONDITIONAL GET — BASIC IDEA

Client has cached object.

It can ask server conceptually:

Has this object changed since my cached version?

If not:

TEXT

304 Not Modified

Then server need not resend the full object body.

Benefits:

- lower bandwidth
- reduced latency

63. DNS CACHE vs HTTP CACHE

Do not confuse them.

DNS cache

Stores:

name-resolution information.

Example:

TEXT

e x a m p l e . c o m ! ' I P

HTTP/browser cache

Stores:

web objects/content.

Example:

TEXT

logo.jpg
style.css

Different layers and purposes.

64. HTTP CONNECTION DOES NOT EQUAL TCP CONNECTION FOREVER

HTTP requests use TCP connections in the traditional model.

But HTTP persistence determines:

whether multiple HTTP transactions reuse the same TCP connection.

So:

- HTTP object "TCP connection
- except in the simple non-persistent one-object-per-connection case.

65. HTTP REQUEST vs TCP ACK

Don't confuse:

TCP ACK

Transport-layer reliability mechanism.

HTTP response

Application-layer response such as 200 OK.

A TCP ACK does not mean:

HTTP request succeeded.

It only confirms TCP bytes.

66. EXAMPLE

Client sends HTTP GET.

TCP ACK may acknowledge the GET bytes.

Server may later reply:

TEXT

404 Not Found

So:

Transport succeeded, application request resulted in an error response.

Different layers.

67. HTTP 404 DOES NOT MEAN DNS FAILED

If you receive:

\[404 \]

you probably successfully:

- resolved/connect to some server
- sent HTTP request
- got HTTP response

The server simply couldn't find the requested resource.

DNS failure is a different problem.

68. 5xx vs 4xx

4xx

Problem associated with client request/resource request.

Example:

\[404 \]

5xx

Server encountered failure while processing request.

Example:

\[500 \]

Memory:

4 = your/request side

5 = server side

69. HTTP CONNECTION TIMING — MASTER EXAMPLE

Suppose:

- DNS uses 2 RTTs
- page has base HTML + 3 images
- all from same server
- client-server RTT = 50 ms
- transmission ignored

Let's compare.

Non-persistent sequential

4 objects:

$4 \times 2RTT = 8RTT$

HTTP:

$8(50) = 400ms$

Suppose each DNS RTT also 50 ms:

$2RTT = 100ms$

Total:

$\boxed{500ms}$

70. PERSISTENT WITHOUT PIPELINING

Total objects:

4

HTTP:

$1RTT_{\{TCP\}} + 4RTT_{\{HTTP\}} = 5RTT$

$5(50) = 250ms$

DNS:

100ms

Total:

$\boxed{350ms}$

71. PERSISTENT WITH PIPELINING

HTTP:

$[1RTT \backslash TCP + 1RTT \backslash HTML + 1RTT \backslash all \ 3 \ image \ requests \]$

$[=3RTT \]$

$[=150ms \]$

DNS:

$[100ms \]$

Total:

$\boxed{250ms}$

72. PARALLEL NON-PERSISTENT

Base HTML:

$[2RTT \]$

Then 3 image connections simultaneously:

$[2RTT \]$

Total:

$[4RTT \]$

$[=200ms \]$

DNS:

$[100ms \]$

Total:

$\boxed{300ms}$

So in this simplified example:

Mode	Total
Non-persistent sequential	500 ms
Parallel non-persistent	300 ms
Persistent sequential	350 ms
Persistent pipelined	250 ms

This is exactly the kind of comparative reasoning GATE likes.

73. & pDO NOT MEMORIZE ONE FORMULA WITHOUT MODEL

Before any HTTP numerical ask:

1. How many objects?

2. Is base HTML included?
3. DNS cached?
4. Persistent or non-persistent?
5. Sequential or parallel?
6. Pipelining?
7. Same server?
8. Transmission negligible?
9. Same RTT for DNS and HTTP?
10. Is TCP connection already established?

0-1% This checklist prevents almost every error.

74. IF TCP CONNECTION ALREADY EXISTS

Suppose question says:

Persistent TCP connection is already established.

Then do **not** add:

$\backslash$ 1RTT $\backslash$

for TCP setup again.

Example 3 objects sequentially over existing persistent connection:

$\backslash$ $\boxed{3\text{RTT}}$ $\backslash$

rather than:

$\backslash$ 4RTT $\backslash$

75. IF DNS IS CACHED

Do not add DNS resolution delay.

Example:

DNS already cached, TCP not established, one non-persistent object.

Time:

$\backslash$ $\boxed{2\text{RTT}}$ $\backslash$

not:

$\backslash$ 3RTT $\backslash$

or more.

76. IF OBJECT IS ALREADY HTTP-CACHED AND VALID

The browser may not need to retrieve the object from the origin server at all.

If validation is required, timing depends on conditional request model.

Again:

only count network exchanges actually stated.

77. SAME SERVER vs DIFFERENT SERVERS

Persistent connection reuse only applies to requests that can use that relevant connection.

If embedded objects live on different servers/domains:

additional DNS resolutions and TCP connections may be required.

Example:

TEXT

```
H T M L ! '   w w w . s i t e . c o m
I m a g e ! '   i m a g e s . c d n . c o m
A d ! '   a d s . e x a m p l e . n e t
```

Do not assume one TCP connection serves all hosts.

78. DNS MAY BE REQUIRED FOR EACH DISTINCT HOSTNAME

If multiple object URLs use different hostnames and none are cached:

each hostname may require DNS resolution.

However DNS hierarchy results may also be partially cached.

Only model what question specifies.

79. HTTP/1.0 vs HTTP/1.1 — EXAM LEVEL

Simplified textbook association:

HTTP/1.0

Typically:

non-persistent by default.

HTTP/1.1

Typically:

persistent by default.

Do not overcomplicate unless question specifically asks version behavior.

80. HTTP DOES NOT PROVIDE TCP RELIABILITY ITSELF

HTTP relies on the transport below it in the traditional TCP model.

TCP handles:

- retransmission
- sequence numbers
- flow control
- congestion control

HTTP handles:

- request semantics
- response semantics
- resource representation

Layering again.

81. HTTP vs DNS — MASTER COMPARISON

DNS	HTTP
Resolves names	Transfers web resources
Application layer	Application layer
Often UDP/TCP port 53	Traditionally TCP port 80
N a m e ! ' I P	R e q u e s t ! ' R e s p o n s e
Root/TLD/Auth hierarchy	Client/server resource model

82. HOW GATE CAN ASK HTTP

Pattern 1 — Connection type

Persistent vs non-persistent.

Pattern 2 — RTT count

One or multiple objects.

Pattern 3 — Parallel connections

Calculate overlapped RTTs.

Pattern 4 — Persistent pipelining

Calculate reduced RTT count.

Pattern 5 — DNS + HTTP

Add DNS resolution delay.

Pattern 6 — Transmission time

$\lfloor L/R \rfloor$

Pattern 7 — Status codes

Identify 200/301/404/500.

Pattern 8 — Method

GET/POST/HEAD.

Pattern 9 — Cache

Conditional request / 304 concept.

83. & pGATE TRAPS

Trap 1

Non-persistent HTTP uses:

a separate TCP connection for each object.

Trap 2

One non-persistent object normally costs:

$\lfloor \boxed{2RTT} \rfloor$

before transmission time, assuming DNS already done.

Trap 3

Don't forget:

HTML itself is an object.

Trap 4

Embedded object URLs may not be known until base HTML is obtained.

Trap 5

Parallel object downloads overlap RTTs.

Do not sum them sequentially if question says parallel.

Trap 6

Persistent HTTP reuses TCP connection.

Do not add a handshake for every object.

Trap 7

If connection is already established:

remove TCP setup RTT.

Trap 8

If DNS cached:

remove DNS lookup RTTs.

Trap 9

HTTP status 404 is application-level resource failure, not necessarily TCP/DNS failure.

Trap 10

GET is not inherently secure and POST is not inherently secure.

HTTPS provides encryption/authentication properties, not method choice alone.

Trap 11

DNS cache and HTTP cache are different.

Trap 12

Bytes vs bits still matters when object transmission time is given.

84. MASTER HTTP MEMORY

For one non-persistent object:

C-R-D

C — Connect TCP

\[1RTT \]

R — Request/Response start

$\boxed{1RTT}$

D — Data transmission

$\boxed{L/R}$

So:

$\boxed{2RTT+L/R}$

85. PERSISTENT MEMORY

Connect once, request many.

For (N) sequential objects:

$\boxed{(N+1)RTT}$

ignoring transmission.

86. PIPELINING MEMORY

HTML first, then ask for all discovered objects together.

Under classic simplified assumptions:

$\boxed{3RTT}$

after DNS:

1. TCP
2. HTML
3. embedded objects

'p NOTEBOOK VERSION

HTTP

HTTP = **H**ypertext **T**ransfer **P**rotocol

Application-layer request-response protocol.

Traditional:

$\boxed{HTTP \rightarrow TCP}$

Default HTTP port:

$\boxed{80}$

HTTPS:

$\boxed{443}$

Request–Response

TEXT

```
C l i e n t ! '   H T T P   R e q u e s t
S e r v e r ! '   H T T P   R e s p o n s e
```

Common methods:

- GET
- POST
- HEAD

Status Codes

TEXT

```
2 0 0 ! '   O K
3 0 1 ! '   M o v e d   P e r m a n e n t l y
3 0 4 ! '   N o t   M o d i f i e d
4 0 4 ! '   N o t   F o u n d
5 0 0 ! '   I n t e r n a l   S e r v e r   E r r o r
```

Categories:

TEXT

```
2xx Success
3xx Redirect
4xx Client/request error
5xx Server error
```

Non-Persistent HTTP

One TCP connection per object.

One object:

$\boxed{2RTT + \text{TransmissionTime}}$

Ignoring transmission:

$\boxed{2RTT}$

For (N) sequential objects:

$\boxed{2NRTT}$

Persistent HTTP

Multiple objects reuse one TCP connection.

For (N) sequential objects:

$\boxed{(N+1)RTT}$

assuming connection not already established and transmission ignored.

Persistent + Pipelining

If:

- base HTML first
- all embedded objects then pipelined
- same server
- transmission ignored

then:

$\boxed{3RTT}$

after DNS.

DNS + HTTP

$\boxed{T_{\text{total}} = T_{\text{DNS}} + T_{\text{HTTP}}}$

Count only exchanges actually required.

87. APPLICATION LAYER — 5-MINUTE REVISION

DNS

$\boxed{\text{Name} \rightarrow \text{IP}}$

Hierarchy:

TEXT

R o o t ! ' T L D ! ' A u t h o r i t a t i v e

Records:

TEXT

A ! ' I P v 4
A A A A ! ' I P v 6
C N A M E ! ' A l i a s
M X ! ' M a i l
N S ! ' N a m e s e r v e r

Port:

$\boxed{53}$

HTTP

$\boxed{\text{Request} \rightarrow \text{Response}}$

Port:

[80]

One non-persistent object:

[2RTT]

Persistent:

reuse TCP.

88. 30-SECOND REVISION

Remember:

DNS finds the server. HTTP asks the server for the object.

For non-persistent HTTP:

[$\boxed{1RTT \text{ TCP} + 1RTT \text{ HTTP} = 2RTT/\text{object}}$]

Persistent:

Connect once, reuse connection.

Status:

TEXT

200 good
301 moved
304 cached unchanged
404 not found
500 server error

ACTIVE RECALL — DON'T LOOK BACK

Q1

At which layer does HTTP operate?

Q2

What is the normal request-response relationship between browser and web server?

Q3

What is the traditional well-known port for HTTP?

Q4

A single object is retrieved using non-persistent HTTP.

DNS is already complete and transmission time is ignored.

How many RTTs are required?

Q5 — Why?

Explain where those RTTs are spent.

Q6

A webpage consists of:

- 1 base HTML
- 4 images

Using sequential non-persistent HTTP and ignoring DNS/transmission, how many RTTs are needed?

Q7

For the same 5 objects, using one persistent connection without pipelining, how many RTTs are needed assuming the TCP connection must first be established?

Q8

The same page uses persistent HTTP with pipelining for all 4 images after the base HTML is received.

How many RTTs are needed after DNS?

Q9 — Parallel Connections

Base HTML requires 2 RTT using non-persistent HTTP.

Then 6 images are fetched using 6 parallel non-persistent TCP connections.

Ignoring transmission, how many total RTTs are required after DNS?

Q10 — DNS + HTTP

DNS takes:

\[3RTT \]

One non-persistent web object then needs:

\[2RTT \]

What is total?

Q11 — Transmission

RTT:

\[40ms \]

Object size:

\[2Mb \]

Rate:

\[20Mbps \]

Using non-persistent HTTP with DNS already complete, find total retrieval time.

Q12 — Status Codes

Match:

1. 200
2. 301
3. 304
4. 404
5. 500

with:

- Not Modified
- OK
- Not Found
- Internal Server Error
- Moved Permanently

Q13

What is the main difference between:

- non-persistent HTTP
- persistent HTTP

?

Q14 — GATE Trap

True or False:

A webpage containing one HTML file and five images contains only one HTTP object.

Explain.

NEXT !' COMPLETE COMPUTER NETWORKS GATE REVISION: Full Syllabus Checklist, Formula Sheet, High-Priority Traps, Confusing Concepts & Last-Day Mental Map

USER

next

ASSISTANT

COMPUTER NETWORKS — COMPLETE GATE REVISION

MASTER REVISION: Syllabus Checklist + Formula Sheet + Traps + Confusing Concepts + Last-Day Mental Map

' We have now completed the full syllabus you originally gave:

Principles of Layering; Basics of switching (circuit, packet and virtual circuit) and performance metrics; Data link layer: error detection, Medium Access Control, Ethernet; Distance vector and link state routing; IPv4 - Fragmentation, CIDR Notation, Network Address Translation; TCP - flow control and congestion control, socket API; DNS and HTTP.

This response is your **subject-level revision sheet**. Don't try to memorize every sentence. Focus on formulas, distinctions, traps, and the mental map.

1. COMPLETE SYLLABUS CHECKLIST

UNIT 1 — NETWORKING FOUNDATIONS

' Principles of layering ' Protocol vs service vs interface ' Encapsulation / decapsulation ' OSI model ' TCP/IP model ' PDU names ' Layer-wise addressing ' Router/switch/hub layer roles

UNIT 2 — PERFORMANCE METRICS

' Processing delay ' Queueing delay ' Transmission delay ' Propagation delay ' End-to-end delay ' Traffic intensity ' Bandwidth-delay product ' Store-and-forward ' Packet pipelining ' Throughput ' Bottleneck link ' Goodput / utilization

UNIT 3 — SWITCHING

' Circuit switching ' Packet switching ' Datagram network ' Virtual circuit ' FDM ' TDM ' Statistical multiplexing

UNIT 4 — DATALINK LAYER

' Framing ' Byte stuffing ' Bit stuffing ' MAC addressing ' Error-detection overview ' Parity ' Two-dimensional parity ' Checksum ' CRC ' Minimum Hamming distance

UNIT 5 — MEDIUM ACCESS CONTROL

' Pure ALOHA ' Slotted ALOHA ' CSMA ' 1-persistent ' Non-persistent ' p-persistent ' CSMA/CD ' Propagation effect ' Collision detection ' Binary exponential backoff

UNIT 6 — ETHERNET

' Ethernet frame ' Minimum/maximum frame sizes ' MTU ' FCS ' Switch learning ' Flooding/filtering ' Hub vs switch ' Collision domain ' Broadcast domain

UNIT 7 — ROUTING

' Routing vs forwarding ' Distance Vector ' Bellman-Ford ' Distance tables ' Count-to-infinity ' Split horizon ' Poison reverse ' Link State ' LSA ' Flooding ' LSDB ' Dijkstra ' DV vs LS

UNIT 8 — IPv4

' IPv4 addressing foundation ' Prefix/network/host bits ' Subnet masks ' CIDR ' Subnet ranges ' Route aggregation ' Longest Prefix Matching ' IPv4 fragmentation ' Identification ' DF/MF ' Fragment Offset ' 8-byte rule ' NAT ' Private IPv4 ranges ' PAT/NAPT

UNIT 9 — TCP

' Ports ' Sequence numbers ' ACKs ' Three-way handshake ' Reliable byte stream ' Flow control ' `rwnd` ' Sliding window ' Zero window ' Congestion control ' `cwnd` ' `ssthresh` ' Slow Start ' Congestion Avoidance ' Timeout ' Triple duplicate ACK ' Fast Retransmit ' Fast Recovery ' Tahoe vs Reno ' RTT/RTO basics ' Socket API

UNIT 10 — APPLICATION LAYER

' DNS ' DNS hierarchy ' Recursive vs iterative resolution ' DNS records ' Caching ' HTTP ' Persistent vs non-persistent HTTP ' HTTP RTT numericals ' Methods/status codes

2. MASTER FORMULA SHEET

This is the section you should revise repeatedly.

PERFORMANCE METRICS

Transmission Delay

$$\boxed{d_{\text{trans}} = \frac{L}{R}}$$

where:

- L = packet length in bits
- R = transmission rate in bits/s

Memory:

Packet size / Rate

Propagation Delay

$$\boxed{d_{\text{prop}} = \frac{D}{S}}$$

where:

- $\lambda(D)$ = distance
- $\lambda(S)$ = signal propagation speed

Memory:

Distance / Speed

Nodal Delay

$$\boxed{d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}}$$

Traffic Intensity

$$\boxed{\rho = \frac{\lambda R}{C}}$$

If:

$$\rho \rightarrow 1$$

queueing becomes severe.

Bandwidth-Delay Product

$$\boxed{BDP = \text{Bandwidth} \times \text{Delay}}$$

For TCP/end-to-end contexts frequently:

$$\boxed{BDP = \text{Bandwidth} \times RTT}$$

STORE-AND-FORWARD

One packet across N equal-rate links:

$$\boxed{T = N \frac{L}{R}}$$

ignoring other delays.

For P equal packets and N equal-rate links:

$$\boxed{T = (N+P-1) \frac{L}{R}}$$

Memory:

Links + Packets " 1

THROUGHPUT

Path:

$$R_1, R_2, \dots, R_n$$

Ideal throughput:

$$\boxed{\min(R_1, R_2, \dots, R_n)}$$

Memory:

Slowest link wins.

ALOHA

Pure ALOHA

$$\boxed{S = G e^{-2G}}$$

Maximum:

$$\boxed{S_{\max} = \frac{1}{2e} \approx 0.184}$$

Vulnerable time:

$$\boxed{2T}$$

Slotted ALOHA

$$\boxed{S = G e^{-G}}$$

Maximum:

$$\boxed{S_{\max} = \frac{1}{e} \approx 0.368}$$

Vulnerable time:

$$\boxed{T}$$

p-PERSISTENT CONTENTION

If n stations independently transmit with probability p :

Exactly one transmits:

$$\boxed{np(1-p)^{n-1}}$$

No station transmits:

$$\boxed{(1-p)^n}$$

Collision:

$$\boxed{1 - (1-p)^n - np(1-p)^{n-1}}$$

Simple optimum:

$$\boxed{p \approx \frac{1}{n}}$$

CSMA/CD

Minimum transmission time:

$$\boxed{T_{\text{frame}} \geq 2\tau}$$

Minimum frame size:

$$\boxed{L_{\text{min}} \geq 2\tau R}$$

where:

$$\tau = \text{one-way propagation delay}$$

If distance (D) and propagation speed (S) :

$$\boxed{L_{\text{min}} = \frac{2DR}{S}}$$

Classic Ethernet:

$$\boxed{64\text{B} = 512\text{bits}}$$

CRC

Generator length:

$$r+1$$

Generator degree:

$$r$$

CRC bits:

$$\boxed{r}$$

Append:

$$\boxed{r \text{ zeros}}$$

before division.

CRC codeword:

$$\boxed{\text{Data} \parallel \text{Remainder}}$$

Receiver:

Divide complete received codeword by generator.

Zero remainder:

No error detected.

HAMMING DISTANCE

Detection capability:

$$\boxed{d_{\text{min}} - 1}$$

Correction capability:

$$\boxed{\left\lfloor \frac{d_{\text{min}} - 1}{2} \right\rfloor}$$

DISTANCE VECTOR

Bellman-Ford:

$$\boxed{D_x(y) = \min_{v \in N(x)} [c(x,v) + D_v(y)]}$$

Memory:

Minimize ! ' Neighbour ! ' Destination wins.

DIJKSTRA

Relaxation:

$$\boxed{D(v) = \min [D(v), D(u) + c(u,v)]}$$

Memory:

Pick smallest ! ' Finalize ! ' Relax ! ' Repeat

IPv4 / CIDR

IPv4:

$$\boxed{32 \text{ bits}}$$

For n :

$$\boxed{\text{HostBits} = 32 - n}$$

Total addresses:

$$\boxed{2^{32-n}}$$

Traditional usable host addresses:

$$\boxed{2^{32-n} - 2}$$

Block size in relevant octet:

$$\boxed{256 - \text{Mask Value}}$$

Splitting p into q :

$$\boxed{2^{q-p}}$$

equal subnets.

IPv4 FRAGMENTATION

Original payload if total length given:

$$\boxed{\text{Data} = \text{TotalLength} - \text{Header}}$$

Maximum non-final fragment data:

$$\boxed{F = 8 \left\lfloor \frac{\text{MTU} - H}{8} \right\rfloor}$$

Fragment total length:

$$\boxed{H + \text{FragmentData}}$$

Fragment Offset:

$$\boxed{\text{Offset} = \frac{\text{original payload bytes preceding fragment}}{8}}$$

Number of fragments:

$$\boxed{N = \left\lceil \frac{\text{OriginalPayload}}{F} \right\rceil}$$

Additional header overhead:

$$\boxed{(N-1)H}$$

TCP

If TCP segment:

$$\boxed{\text{Seq} = S}$$

and contains (L) data bytes:

$$\boxed{\text{NextSeq} = S + L}$$

Last byte:

$$\boxed{S + L - 1}$$

ACK:

$$\boxed{\text{Next expected byte}}$$

TCP FLOW CONTROL

Conceptually:

$$\boxed{rwnd = \text{ReceiveBuffer} - \text{BufferedData}}$$

If:

$$\boxed{\text{ACK} = A, \quad rwnd = W}$$

allowed sequence window:

$$\boxed{A \text{ through } A + W - 1}$$

Effective sender limit:

$$\boxed{\min(rwnd, cwnd)}$$

Window-limited throughput:

$$\boxed{\frac{\text{Window}}{\text{RTT}}}$$

TCP SLOW START

Simplified:

$$\boxed{cwnd \approx 2 \times \text{per RTT}}$$

If initial (W_0) :

$$W_k = W_0 \cdot 2^k$$

while Slow Start continues.

CONGESTION AVOIDANCE

Simplified:

$$cwnd \leftarrow cwnd + 1MSS/RTT$$

TCP TIMEOUT

Classic textbook model:

$$ssthresh \leftarrow cwnd/2$$

then:

$$cwnd \leftarrow 1MSS$$

RENO — THREE DUPLICATE ACKs

Simplified GATE model:

$$ssthresh \leftarrow cwnd/2$$

After recovery:

$$cwnd \leftarrow ssthresh$$

then Congestion Avoidance.

RTT ESTIMATION

$$EstimatedRTT = (1-\alpha)Old + \alpha SampleRTT$$

Common:

$$\alpha = 0.125$$

Deviation:

$$DevRTT = (1-\beta)OldDev + \beta |SampleRTT - EstimatedRTT|$$

Common:

$$\beta = 0.25$$

RTO:

$$RTO = EstimatedRTT + 4DevRTT$$

HTTP

One non-persistent object:

$$\boxed{2RTT + \frac{LR}{R}}$$

ignoring DNS.

For (N) sequential non-persistent objects, negligible transmission:

$$\boxed{2NRTT}$$

Persistent, sequential, (N) objects:

$$\boxed{(N+1)RTT}$$

if TCP connection first needs establishment.

3. ~~0=10~~ MUST-MEMORIZE CONSTANTS

Item	Value
IPv4 size	32 bits
MAC address	48 bits
TCP/UDP port	16 bits
TCP seq number	32 bits
IPv4 basic header	20 B
TCP basic header	20 B
Ethernet MTU	1500 B
Ethernet minimum frame	64 B
Ethernet normal max untagged frame	1518 B
Ethernet FCS	4 B
Ethernet minimum payload/data field	46 B
DNS port	53
HTTP port	80
HTTPS port	443

4. ETHERNET FRAME — MUST REMEMBER

Standard untagged MAC frame:

TEXT

Destination	Source	Type/Length	Data+Pad	FCS
6 B	6 B	2 B	46–1500	4 B

Therefore:

$\boxed{6-6-2-46 \rightarrow 1500-4}$

Minimum:

$\boxed{64B}$

Maximum:

$\boxed{1518B}$

Ethernet MTU:

$\boxed{1500B}$

& pMTU is not full Ethernet-frame size.

5. PRIVATE IPv4 — MUST MEMORIZE

$\boxed{10.0.0.0/8}$

$\boxed{172.16.0.0/12}$

which means approximately:

TEXT

1 7 2 . 1 6 . x . x !' 1 7 2 . 3 1 . x . x

$\boxed{192.168.0.0/16}$

Memory:

10 — 172.16–31 — 192.168

6. SUBNET MASK MINI-TABLE

Memorize:

Prefix	Mask value	Block	Addresses
/25	128	128	128
/26	192	64	64
/27	224	32	32
/28	240	16	16
/29	248	8	8
/30	252	4	4
/31	254	2	2
/32	255	1	1

Memory mask sequence:

7. TOP CONFUSING PAIRS

1. Transmission vs Propagation

Transmission

$$\frac{LR}{}$$

Depends on:

- packet length
- rate

Propagation

$$\frac{DS}{}$$

Depends on:

- distance
- signal speed

Memory:

Load packet = transmission. Travel = propagation.

2. Bandwidth vs Throughput

Bandwidth:

Maximum potential capacity.

Throughput:

Actual achieved rate.

3. Circuit vs Packet Switching

Circuit:

reserve resources first.

Packet:

dynamically share network.

4. Datagram vs Virtual Circuit

Datagram:

each packet independently routed.

VC:

logical path established first.

5. Error Detection vs Error Correction

Detection:

know error occurred.

Correction:

reconstruct correct data.

6. MAC vs IP

MAC:

local link / next hop.

IP:

network-layer destination across networks.

Memory:

IP = final goal. MAC = next step.

7. Routing vs Forwarding

Routing:

Calculate paths.

Forwarding:

Send packet to next hop.

Memory:

Routing decides. Forwarding sends.

8. Distance Vector vs Link State

Distance Vector:

Neighbours give distances.

Bellman-Ford.

Link State:

Everyone builds topology map.

Dijkstra.

Memory:

DV asks directions. LS gets map.

9. Network Address vs Broadcast Address

Network:

host bits all 0.

Broadcast:

host bits all 1.

10. MTU vs MSS

MTU:

maximum IP packet on link.

MSS:

maximum TCP payload.

Typical:

$\backslash$ 1500-20-20=1460B $\backslash$

11. Fragment Total Length vs Fragment Data

\[Total=Header+Data\]

Offsets are calculated using:

DATAonly.

12. DF vs MF

DF:

Don't Fragment.

MF:

More Fragments.

Memory:

D = Don't

M = More

13. rwnd vs cwnd

rwnd :

receiver capacity.

cwnd :

network congestion estimate.

Memory:

R = Receiver

C = Congestion

14. Sequence Number vs ACK

Sequence:

first byte in this segment.

ACK:

next byte expected.

15. Slow Start vs Congestion Avoidance

Slow Start:

\[1,2,4,8 \]

exponential.

Congestion Avoidance:

\[8,9,10,11 \]

linear.

16. Fast Retransmit vs Fast Recovery

Fast Retransmit:

resend lost segment after 3 duplicate ACKs.

Fast Recovery:

Reno window-management mechanism after that loss.

17. Tahoe vs Reno

Tahoe:

any loss !' cwnd=1 .

Reno:

timeout !' 1 triple duplicate ACK !' halve/recover.

18. Recursive vs Iterative DNS

Recursive:

Find final answer for me.

Iterative:

Tell me who to ask next.

19. Persistent vs Non-Persistent HTTP

Non-persistent:

TCP connection per object.

Persistent:

reuse TCP connection.

8. 0=TOP GATE TRAPS ACROSS THE SUBJECT

& pTrap 1

$\backslash [1\text{Byte}=8\text{bits}]$

Always check units.

& pTrap 2

Mbps usually means:

$\backslash [10^6\text{bps}]$

unless question specifies otherwise.

& pTrap 3

Transmission delay depends on rate.

Propagation delay does **not**.

& pTrap 4

Path throughput is:

$\backslash [\min(R_i)]$

not average.

& pTrap 5

Pure ALOHA maximum is:

$\backslash [18.4\%]$

Slotted:

$\backslash [36.8\%]$

Do not swap.

& ⚠Trap 6

CSMA doesn't eliminate collisions.

Propagation delay still allows them.

& ⚠Trap 7

CSMA/CD worst-case detection requires:

2τ

not τ .

& ⚠Trap 8

Switch learns:

SOURCE MAC.

It forwards using:

DESTINATION MAC.

Memory:

Learn Source, Forward Destination.

& ⚠Trap 9

Router:

L3 / IP.

Switch:

L2 / MAC.

Hub:

L1.

& ⚠Trap 10

DV does not have complete topology.

LS does.

& pTrap 11

Shortest path means:

minimum total metric.

Not necessarily fewest hops.

& pTrap 12

Explicit CIDR prefix overrides old class assumptions.

& pTrap 13

Longest Prefix Match:

greatest prefix length among matching entries wins.

& pTrap 14

Fragment Offset is:

$\lfloor \boxed{\text{8-byte units}} \rfloor$

not bytes.

& pTrap 15

Offset uses original:

payload position,

not fragment total length.

& pTrap 16

All non-last fragments need payload divisible by 8.

Last fragment doesn't.

& pTrap 17

All fragments of one original datagram share Identification.

& pTrap 18

NAT " routing.

NAT " firewall.

& pTrap 19

TCP sequence numbers count:

bytes,

not segments.

& pTrap 20

ACK is next expected byte.

Not last received byte.

& pTrap 21

SYN consumes 1 sequence number.

FIN consumes 1.

Pure ACK consumes 0.

& pTrap 22

Zero receive window does not mean connection closed.

& pTrap 23

Slow Start is exponential despite its name.

& pTrap 24

Reno:

timeout !' 1.

Triple duplicate ACK !' Fast Retransmit/Recovery.

& pTrap 25

`accept()` returns a new connected socket.

Listening socket remains.

& pTrap 26

DNS does not resolve IP !' MAC.

DNS resolves:

`\[ Name\to IP \]`

& pTrap 27

DNS does not always use UDP.

It can use UDP or TCP.

Port:

$\lfloor 53 \rfloor$

& ¶Trap 28

Non-persistent HTTP:

$\lfloor 2RTT/object \rfloor$

only under assumptions such as:

- DNS already done
 - negligible transmission
 - TCP not already established.
-

& ¶Trap 29

HTML itself is an HTTP object.

& ¶Trap 30

Parallel downloads should not be added sequentially.

9. ~~0=0%~~ ULTRA-HIGH-PRIORITY NUMERICAL AREAS

If time becomes extremely limited, prioritize these.

Priority 1 — Delays

Practice:

$\lfloor L/R, \text{quad } D/S \rfloor$

and total nodal delay.

Priority 2 — Store-and-Forward

$\lfloor (N+P-1)L/R \rfloor$

Priority 3 — CRC

Practice:

- append zeros
 - XOR division
 - remainder
 - codeword
-

Priority 4 — ALOHA

Memorize:

TEXT

Pure:

$S = Ge^{-2G}$

Max = 18.4%

Slotted:

$S = Ge^{-G}$

Max = 36.8%

Priority 5 — CSMA/CD

$\lceil L_{\min} = 2\tau R \rceil$

Priority 6 — Ethernet

Know:

TEXT

64 B minimum

1518 B max untagged

1500 B MTU

46 B minimum data

Priority 7 — Distance Vector

$\lceil D_x(y) = \min[c(x,v) + D_v(y)] \rceil$

Priority 8 — Dijkstra

Practice complete table iterations.

Priority 9 — CIDR

Practice:

- network address
- broadcast
- block size
- host count
- aggregation
- LPM

Priority 10 — Fragmentation

$\lceil \frac{L}{P} \rceil$ specially practice complete tables:

Fragment	Data	Total	Offset	MF
----------	------	-------	--------	----

Priority 11 — TCP Sequence/ACK

Practice:

$\backslash$ NextSeq=Seq+Length $\backslash$

$\backslash$ ACK=NextExpected $\backslash$

Priority 12 — TCP Congestion Window Graphs

Practice:

TEXT

```

S l o w   S t a r t   ! '   e x p o n e n t i a l
C A   ! '   l i n e a r
T i m e o u t   ! '   1
R e n o   t r i p l e   d u p   ! '   h a l f

```

Priority 13 — HTTP RTT

Practice:

- non-persistent sequential
- parallel
- persistent sequential
- pipelining
- DNS + HTTP

10. GATE NUMERICAL SOLVING TEMPLATES

Delay Question

Write:

GIVEN !' FIND !' COMPONENTS

$\backslash$ $d_{\text{trans}}=L/R$ $\backslash$

$\backslash$ $d_{\text{prop}}=D/S$ $\backslash$

Then add only requested components.

DV Question

Make:

Neighbour	Local cost	Advertised cost	Total
-----------	------------	-----------------	-------

Choose minimum.

Dijkstra Question

Make:

Step	Finalized	A	B	C	D
------	-----------	---	---	---	---

Always:

Pick smallest !' Relax neighbours.

CIDR Question

Use:

1. Find mask.
2. Find interesting octet.
3. Block:

\[256-Mask \]

4. Locate IP.
5. First = network.
6. Last = broadcast.

Fragmentation Question

Always write:

1. Original payload.
2. Max payload/fragment.
3. Round down to multiple of 8.
4. Split.
5. Add header.
6. Calculate offsets.
7. Set MF.

TCP **cwnd** Question

Create:

RTT	Phase	cwnd	Event
-----	-------	------	-------

This prevents mistakes.

HTTP Question

First write:

TEXT

DNS?
Number of objects?
Persistent?
Parallel?
Pipelined?
TCP already open?
Transmission ignored?

Only then calculate RTT.

11. ONE-LINE MEMORY SHEET

Read this repeatedly.

Layering

Provide upward, use downward, protocol with peer.

Encapsulation

Down add, up remove.

PDU

Data !' Segment !' Packet !' Frame !' Bits.

Delay

Load using rate; travel using speed.

Throughput

Slowest link wins.

Circuit

Reserve first.

Datagram

Decide each packet.

VC

Set path first.

CRC

Append !' Divide !' XOR !' Remainder.

ALOHA

Pure 18.4; Slotted 36.8.

CSMA

Listen first, but collision still possible.

CSMA/CD

Must transmit for 2 τ .

Ethernet switch

Learn source; forward destination.

DV

Ask neighbours.

LS

Build map.

Routing

Decides.

Forwarding

Sends.

IPv4

32 bits.

CIDR

Prefix + Host = 32.

Network

Host bits 0.

Broadcast

Host bits 1.

LPM

Most specific route wins.

Fragmentation

Offset = payload position / 8.

NAT

Private inside, public outside, table connects.

TCP sequence

First byte.

TCP ACK

Next expected byte.

Flow Control

Protect receiver — rwnd.

Congestion Control

Protect network — cwnd.

Slow Start

Multiply.

Congestion Avoidance

Add.

Timeout

Go to 1.

Reno duplicate ACKs

Half + recover.

Socket server

Socket !' Bind !' Listen !' Accept.

DNS

Name !' IP.

DNS hierarchy

Root !' TLD !' Authoritative.

HTTP non-persistent

2 RTT per object.

Persistent HTTP

Connect once, request many.

12. COMPLETE SUBJECT MENTAL MAP

TEXT

COMPUTER NETWORKS

```
%  
% % %   W H Y   L A Y E R S ?  
%       % % %   P r o t o c o l  
%       % % %   S e r v i c e  
%       % % %   I n t e r f a c e  
%       % % %   E n c a p s u l a t i o n  
%  
% % %   H O W   L O N G   D O E S   D A T A   T A K E ?  
%       % % %   P r o c e s s i n g  
%       % % %   Q u e u e i n g  
%       % % %   T r a n s m i s s i o n   =   L / R  
%       % % %   P r o p a g a t i o n   =   D / S  
%       % % %   T h r o u g h p u t   /   B D P  
%  
% % %   H O W   I S   N E T W O R K   S H A R E D ?  
%       % % %   C i r c u i t  
%       % % %   P a c k e t  
%       % % %   D a t a g r a m  
%       % % %   V i r t u a l   C i r c u i t  
%
```

```

% % %   H O W   D O E S   O N E   L I N K   W O R K ?
%       % % %   F r a m e s
%       % % %   M A C
%       % % %   E r r o r   D e t e c t i o n
%       %       % % %   P a r i t y
%       %       % % %   C h e c k s u m
%       %       % % %   C R C
%       % % %   A L O H A
%       % % %   C S M A
%       % % %   E t h e r n e t
%
% % %   H O W   D O   R O U T E R S   F I N D   P A T H S ?
%       % % %   D i s t a n c e   V e c t o r
%       %       % % %   B e l l m a n - F o r d
%       %       % % %   C o u n t - t o - I n f i n i t y
%       % % %   L i n k   S t a t e
%       %       % % %   D i j k s t r a
%
% % %   H O W   A R E   H O S T S   A D D R E S S E D ?
%       % % %   I P v 4
%       % % %   C I D R
%       % % %   L o n g e s t   P r e f i x   M a t c h
%       % % %   F r a g m e n t a t i o n
%       % % %   N A T
%
% % %   H O W   D O   P R O C E S S E S   C O M M U N I C A T E   R E L I A B L Y ?
%       % % %   T C P
%       % % %   P o r t s
%       % % %   S e q   /   A C K
%       % % %   H a n d s h a k e
%       % % %   F l o w   C o n t r o l   ! '   r w n d
%       % % %   C o n g e s t i o n   ! '   c w n d
%       % % %   R e l i a b i l i t y   /   R T O
%       % % %   S o c k e t   A P I
%
% % %   H O W   D O   A P P L I C A T I O N S   U S E   N E T W O R K ?
%       % % %   D N S
%       %       % % %   N a m e   ! '   I P
%       % % %   H T T P
%       %       % % %   R e q u e s t   ! '   R e s p o n s e

```

13. 0-5 MINUTE PRE-EXAM REVISION

If your exam starts in five minutes, revise only this:

Performance

$\lambda_d = L/R$

$\lambda_d = D/S$

$\lambda_{Throughput} = \min R_i$

$\lambda_{StoreForward} = (N+P-1)L/R$

MAC

TEXT

Pure ALOHA !' 18.4 %
Slotted !' 36.8 %

Ethernet

TEXT

MAC = 48 bits
 Min frame = 64 B
 MTU = 1500 B
 Max untagged = 1518 B

Switch:

Learn source; forward destination.

Routing

DV:

$[D_x(y) = \min[c(x, v) + D_v(y)]]$

LS:

Dijkstra.

CIDR

$[Addresses = 2^{\{32 - prefix\}}]$

$[Block = 256 - Mask]$

Longest matching prefix wins.

Fragmentation

$[Payload = Total - Header]$

$[MaxFragData = 8 \left\lfloor \frac{MTU - H}{8} \right\rfloor]$

$[Offset = \frac{\text{previous payload}}{8}]$

MF:

1 except last.

TCP

$[NextSeq = Seq + Bytes]$

$[ACK = NextExpected]$

TEXT

```
r w n d ! '   r e c e i v e r  
c w n d ! '   n e t w o r k
```

TEXT

```
S l o w   S t a r t ! '   × 2  
C A ! '   + 1  
T i m e o u t ! '   c w n d = 1  
R e n o   3   d u p   A C K ! '   h a l f / r e c o v e r
```

Server socket:

TEXT

```
s o c k e t ! '   b i n d ! '   l i s t e n ! '   a c c e p t
```

DNS

TEXT

```
N a m e ! '   I P  
R o o t ! '   T L D ! '   A u t h o r i t a t i v e  
P o r t 53
```

HTTP

Non-persistent:

\[2RTT/object \]

Persistent:

reuse TCP.

14. ~~0=30%~~SECOND LAST-DAYREVISION

If you remember nothing else:

Transmission = L/R. Propagation = D/S.

Throughput = bottleneck.

Pure ALOHA 18.4%, Slotted 36.8%.

CSMA/CD = 2 Ä .

Switch learns source MAC.

DV = Bellman-Ford. LS = Dijkstra.

IPv4 = 32 bits. CIDR host bits = 32 - prefix.

Fragment offset is in 8-byte units.

TCP Seq = first byte; ACK = next expected byte.

rwnd protects receiver; cwnd protects network.

Slow Start doubles; Congestion Avoidance adds.

Timeout !' 1. Reno 3 dup ACK !' halve/recover.

Server socket = socket-bind-listen-accept.

DNS = Name !' IP !' TLD !' Authoritative .

Non-persistent HTTP = 2 RTT/object.

15. SUBJECT STATUS

' COMPUTER NETWORKS SYLLABUS: COMPLETE

Covered from:

foundation level

through:

GATE numerical/problem-solving level

including:

- formulas
- shortcuts
- common traps
- conceptual distinctions
- numerical patterns
- revision notes

The best next step is **not more theory**. It is to convert this knowledge into GATE marks by solving mixed questions.

NEXT !' COMPUTER NETWORKS GATE TEST — Level 1: 15 Mixed Questions Across the Complete Syllabus

USER

next

ASSISTANT

COMPUTER NETWORKS — GATE TEST

LEVEL 1 — 15 Mixed Questions Across the Complete Syllabus

This test is meant to check whether the concepts are actually usable, not just familiar.

Rules

- Try without looking at notes.
- For numericals, write the formula and units.
- For MCQs, answer with the option letter + one-line reason.
- Do not worry about perfect wording.
- Send me your answers as 1, 2, 3...15.
- I will grade each as ' / & p' 'L and explain only what needs correction.

Q1 — Delay Numerical

A packet of size:

12000 bits

is transmitted over a link of rate:

3 Mbps

The distance is:

600 km

and propagation speed is:

$2 \times 10^8 \text{ m/s}$

Ignore processing and queueing.

Find:

1. Transmission delay
2. Propagation delay
3. Total delay for the complete packet to reach the other end

Q2 — Throughput

A path contains links of capacities:

$[20\text{Mbps}, 50\text{Mbps}, 8\text{Mbps}, 100\text{Mbps}]$

Assuming no other limitation, maximum end-to-end throughput is:

A. 44.5 Mbps B. 100 Mbps C. 8 Mbps D. 20 Mbps

Q3 — ALOHA

Which statement is correct?

A. Pure ALOHA maximum efficiency is approximately 36.8%. B. Slotted ALOHA vulnerable period is $(2T)$. C. Pure ALOHA vulnerable period is $(2T)$. D. Slotted ALOHA completely eliminates collisions.

Q4 — CSMA/CD Numerical

Two stations are connected through a shared Ethernet network.

Maximum one-way propagation delay:

$25\mu\text{s}$

Link rate:

20Mbps

What is the minimum frame size required for collision detection?

Give answer in:

- bits
- bytes

Use:

$L_{\min} = 2\tau R$

Q5 — Ethernet Switch

A switch receives a frame on port 3:

TEXT

Source MAC = AA
Destination MAC = BB

Its current table is:

MAC	Port
AA	1
CC	4

What should the switch do?

A. Learn B B !and forward to port 1 B. Learn A A !and flood the frame except port 3 C. Learn A A !and 1 discard the frame D. Learn B B !and forward to port 4

Q6 — CRC Concept

A generator polynomial is represented by:

TEXT
101101

Answer:

1. What is the degree of the generator?
2. How many CRC bits will be appended?
3. How many zero bits are appended to the original data before modulo-2 division?

Q7 — Distance Vector

Router X has two neighbours A and B.

$c(X,A)=2$

$c(X,B)=5$

Their advertised distances to destination D are:

$D_A(D)=8$

$D_B(D)=3$

Find:

1. Cost through A
2. Cost through B
3. X's final distance to D
4. Next hop

Q8 — Distance Vector Failure

After a link failure, two Distance Vector routers repeatedly advertise:

TEXT
3 !' 4 !' 5 !' 6 !' 7 !' . . .

for an unreachable destination.

What is this phenomenon called?

Also name **one** technique used to reduce/prevent the simple two-router version of this problem.

Q9 — Dijkstra

Network edges:

$\{ A-B=2 \}$

$\{ A-C=6 \}$

$\{ B-C=1 \}$

$\{ B-D=5 \}$

$\{ C-D=2 \}$

Starting from A, find:

1. Shortest distance A! B
2. Shortest distance A! C
3. Shortest distance A! D
4. Next hop from A toward D

Q10 — CIDR

Given:

TEXT

192.168.10.173/27

Find:

1. Subnet mask
2. Block size
3. Network address
4. Broadcast address
5. Traditional usable host range
6. Total addresses in the block

Q11 — Longest Prefix Match

Router table:

Prefix	Interface
0.0.0.0/0	A
10.0.0.0/8	B
10.20.0.0/16	C
10.20.32.0/19	D
10.20.40.0/21	E

Destination:

TEXT

10.20.45.7

Which interface is selected?

Show briefly why.

Q12 — IPv4 Fragmentation Ø=Ý%

An IPv4 datagram has a **total length** of:

\[4000B \]

IPv4 header:

\[20B \]

Outgoing-link MTU:

\[1500B \]

Find for every fragment:

Fragment	Data Size	Total Length	Fragment Offset	MF
----------	-----------	--------------	-----------------	----

Also state the Identification relationship among the fragments.

Q13 — TCP Sequence & ACK

A TCP sender sends a segment with:

\[Seq=7000 \]

and:

\[1200B \]

of application data.

Find:

1. Sequence number of first byte
2. Sequence number of last byte
3. Next sequence number
4. ACK number expected if receiver gets all bytes correctly

Q14 — TCP Congestion Control Ø=Ý%

TCP Reno currently has:

\[cwnd=24MSS \]

Case A

A retransmission timeout occurs.

Find:

1. new ssthresh
2. new cwnd

3. next phase

Case B

Instead, three duplicate ACKs occur.

Using the simplified GATE Reno model, find:

1. new `sssthresh`
2. `cwnd` after recovery
3. next phase

Q15 — DNS + HTTP Ø=Ý%

A browser needs a webpage containing:

- 1 base HTML object
- 4 embedded images

Assume:

- nothing is cached
- DNS resolution requires 3 sequential RTTs
- all RTTs are equal
- all objects are on the same web server
- transmission times are negligible
- HTTP uses one persistent TCP connection
- after receiving the base HTML, the browser pipelines all 4 image requests

How many total RTTs are required from the beginning of DNS resolution until the image responses begin arriving?

A. 5 RTT B. 6 RTT C. 7 RTT D. 13 RTT

ANSWER FORMAT

Reply like this:

TEXT

```
1. Transmission = ..., Propagation = ..., Total = ...
2. C = ...
3. ...
...
15. ...
```

For anything you don't know, write:

TEXT

I don't know

Do **not** guess just to look correct. That actually helps me identify where your revision should focus.

NEXT !' Send Your 15 Answers — I'll Grade Them ' / & p 'Land Build Your Weak-Area Revision List